KingbaseES 数据库快速上手

金仓数据库管理系统 KingbaseES

文档版本：V9（V009R001C010）

发布日期：2025 年 12 月 3 日

中电科金仓（北京）科技股份有限公司

目　录

第 1 章　安装 KingbaseES　1
1.1　Docker 快速体验　1
1.1.1　环境准备　1
1.1.1.1　硬件环境要求　1
1.1.1.2　软件环境要求　1
1.1.1.3　创建安装路径　1
1.1.1.4　导入镜像　2
1.1.2　启动实例　3
1.1.3　安装后检查　3
1.1.4　命令行界面　3
1.2　ISO 包快速体验　4
1.2.1　硬件环境要求　4
1.2.2　创建安装用户　5
1.2.3　创建安装目录　5
1.2.4　安装包准备　5
1.2.5　安装包的挂载　6
1.2.6　安装 KingbaseES　6

第 2 章　访问 KingbaseES　14
2.1　客户端工具访问　14
2.1.1　ksql　14
2.1.1.1　关于 ksql　14
2.1.1.2　使用 ksql 连接到数据库　14
2.1.1.3　在 ksql 中断开数据库连接　15
2.2　编程接口-Java　15
2.2.1　JDBC　15
2.2.1.1　前提条件　15
2.2.1.2　JDBC 包　16
2.2.1.2.1　开发源程序　16
2.2.2　SpringBoot　27
2.2.2.1　前提条件　27

2.2.2.2 操作步骤 27
2.2.2.2.1 步骤一：获取 Kingbase 数据库连接信息 28
2.2.2.2.2 步骤二: 导入 java-kingbase-springboot 项目到 IDEA 中 28
2.2.2.2.3 步骤三: 修改 java-kingbase-springboot 项目中的数据库连接信息 29
2.2.2.2.4 步骤四: 运行 java-kingbase-springboot 项目 29
2.2.2.3 项目代码介绍 30
2.2.2.3.1 application.properties 代码介绍 34
2.2.2.3.2 UserDaoImpl.java 文件介绍 34
2.2.2.3.3 UserDao.java 文件介绍 37
2.2.2.3.4 User.java 文件介绍 37
2.2.2.3.5 TestSpringbootApplication.java 文件介绍 39
2.2.2.3.6 TestSpringbootApplicationTests.java 文件介绍 39
2.2.2.3.7 完整的代码展示 42
2.2.3 SpringBatch 47
2.2.3.1 前提条件 47
2.2.3.2 操作步骤 47
2.2.3.2.1 步骤一：获取 Kingbase 数据库连接信息 48
2.2.3.2.2 步骤二: 导入 java-kingbase-springbatch 项目到 IDEA 中 48
2.2.3.2.3 步骤三: 修改 java-kingbase-springbatch 项目中的数据库连接信息 49
2.2.3.2.4 步骤四: 导入方言包到项目 49
2.2.3.2.5 步骤五: 运行 java-kingbase-springbatch 项目 49
2.2.3.3 项目代码介绍 50
2.2.3.3.1 pom.xml 代码介绍 52
2.2.3.3.2 application.properties 文件介绍 55
2.2.3.3.3 BatchApplication.java 文件介绍 56
2.2.3.3.4 BatchConfig.java 文件介绍 56
2.2.3.3.5 People.java 文件介绍 60
2.2.3.3.6 PeopleDESC.java 文件介绍 61
2.2.3.3.7 AddPeopleDescProcessor.java 文件介绍 62
2.2.3.3.8 AddDescPeopleWriter.java 文件介绍 63
2.2.3.3.9 AddPeopleWriter.java 文件介绍 65
2.2.3.3.10 BatchConfigTest.java 文件介绍 66
2.2.3.3.11 AddPeopleDescProcessorTest.java 文件介绍 68
2.2.3.3.12 AddDescPeopleWriterTest.java 文件介绍 69
2.2.3.3.13 AddPeopleWriterTest.java 文件介绍 72
2.2.3.3.14 完整的代码展示 74
2.2.3.4 相关文档 81
2.2.4 SpringJDBC 81
2.2.4.1 前提条件 82
2.2.4.2 操作步骤 82
2.2.4.2.1 步骤一：将 spring-jdbc-kingbase-client 项目导入到 IDEA 中 82
2.2.4.2.2 步骤二: 获取 KingBase 数据库 URL 83

2.2.4.2.3 步骤三: 修改 spring-jdbc-kingbase-client 项目中的数据库连接信息 83
2.2.4.2.4 步骤四: 运行 spring-jdbc-kingbase-client 项目 84
2.2.4.3 项目代码介绍 84
2.2.4.3.1 pom.xml 代码介绍 85
2.2.4.3.2 Main.java 代码介绍 87
2.2.4.3.3 完整的代码展示 91
2.2.4.4 相关文档 93
2.2.5 MyBatis 93
2.2.5.1 前提条件 93
2.2.5.2 操作步骤 93
2.2.5.2.1 步骤一：获取 Kingbase 数据库连接信息 94
2.2.5.2.2 步骤二: 导入 mybatis-kingbase 项目到 IDEA 中 94
2.2.5.2.3 步骤三: 修改 mybatis-kingbase 项目中的数据库连接信息 95
2.2.5.2.4 步骤四: 运行 mybatis-kingbase 项目 95
2.2.5.3 项目代码介绍 97
2.2.5.3.1 pom.xml 代码介绍 98
2.2.5.3.2 jdbc.properties 文件介绍 101
2.2.5.3.3 mybatis-config.xml 介绍 101
2.2.5.3.4 IUserMapper.xml 代码介绍 104
2.2.5.3.5 IAppMapper.java 代码介绍 106
2.2.5.3.6 IUserMapper.java 代码介绍 108
2.2.5.3.7 App.java 代码介绍 108
2.2.5.3.8 User.java 代码介绍 110
2.2.5.3.9 TestMybatis.java 文件介绍 111
2.2.5.3.10 完整的代码展示 117
2.2.5.4 相关文档 125
2.2.6 Hibernate 126
2.2.6.1 前提条件 126
2.2.6.2 操作步骤 126
2.2.6.2.1 步骤一：获取 Kingbase 数据库连接信息 126
2.2.6.2.2 步骤二: 导入 jdbc-kingbase-hibernate 项目到 IDEA 中 127
2.2.6.2.3 步骤三: 修改 jdbc-kingbase-hibernate 项目中的数据库连接信息 127
2.2.6.2.4 步骤四: 运行 jdbc-kingbase-hibernate 项目 128
2.2.6.3 项目代码介绍 132
2.2.6.3.1 pom.xml 代码介绍 133
2.2.6.3.2 User.hbm.xml 代码介绍 135
2.2.6.3.3 hibernate.cfg.xml 代码介绍 136
2.2.6.3.4 UserDao.java 代码介绍 139
2.2.6.3.5 User.java 代码介绍 142
2.2.6.3.6 TestHibernate.java 代码介绍 144
2.2.6.3.7 完整的代码展示 146
2.2.6.4 相关文档 151

2.2.7 SpringDataJPA . . . 151
2.2.7.1 前提条件 . . . 151
2.2.7.2 操作步骤 . . . 152
2.2.7.2.1 步骤一：获取 Kingbase 数据库连接信息 . . . 152
2.2.7.2.2 步骤二: 导入 java-kingbase-springdatajpa 项目到 IDEA 中 . . . 153
2.2.7.2.3 步骤三: 修改 java-kingbase-springdatajpa 项目中的数据库连接信息 . . . 153
2.2.7.2.4 步骤四: 运行 java-kingbase-springdatajpa 项目 . . . 153
2.2.7.3 项目代码介绍 . . . 158
2.2.7.3.1 pom.xml 代码介绍 . . . 160
2.2.7.3.2 application.yml 代码介绍 . . . 163
2.2.7.3.3 userRepository.java 文件介绍 . . . 164
2.2.7.3.4 User.java 文件介绍 . . . 165
2.2.7.3.5 UserServiceImpl.java 文件介绍 . . . 167
2.2.7.3.6 UserService.java 文件介绍 . . . 170
2.2.7.3.7 TestSpringDataJpaApplication.java 文件介绍 . . . 171
2.2.7.3.8 TestSpringDataJpaApplicationTests.java 文件介绍 . . . 172
2.2.7.3.9 完整的代码展示 . . . 174
2.2.7.4 相关文档 . . . 181
2.3 编程接口-C/C++ . . . 181
2.3.1 Libkci . . . 181
2.3.1.1 前提条件 . . . 181
2.3.1.2 操作实例 . . . 181
2.3.1.2.1 下载并解压驱动包 . . . 182
2.3.1.2.2 开发源程序 . . . 182
2.3.1.2.3 编译 . . . 185
2.3.1.2.4 运行 . . . 186
2.3.1.3 常见问题 . . . 186
2.3.2 Dci . . . 186
2.3.2.1 前提条件 . . . 186
2.3.2.2 Dci 和 Oci . . . 187
2.3.2.3 操作实例 . . . 187
2.3.2.3.1 创建测试程序目录 . . . 187
2.3.2.3.2 开发源程序 . . . 188
2.3.2.3.3 编译程序 . . . 195
2.3.2.3.4 运行程序 . . . 196
2.3.2.4 常见问题 . . . 196
2.3.3 Odbc . . . 197
2.3.3.1 前提条件 . . . 197
2.3.3.2 操作实例 - Linux . . . 197
2.3.3.2.1 安装并部署驱动 . . . 197
2.3.3.2.2 开发源程序 . . . 199
2.3.3.2.3 编译 . . . 204

2.3.3.2.4 运行 204
2.3.3.3 操作实例 - Windows 205
2.3.3.3.1 安装并部署驱动 205
2.3.3.4 常见问题 212
2.3.4 Oledb 212
2.3.4.1 前提条件 212
2.3.4.2 操作实例 212
2.3.4.3 常见问题 219
2.4 编程接口-Python 220
2.4.1 Ksycopg2 220
2.4.1.1 前提条件 220
2.4.1.2 操作实例 221
2.4.1.2.1 安装部署驱动 221
2.4.1.2.2 开发源程序 223
2.4.1.2.3 运行程序 226
2.4.1.3 常见问题 226
2.5 编程接口-Go 227
2.5.1 Gokb 227
2.5.1.1 前提条件 227
2.5.1.2 操作实例 227
2.5.1.2.1 下载和安装 Go 227
2.5.1.2.2 使用 GOPATH 管理 Gokb 228
2.5.1.2.3 使用 GOMODULE 管理 Gokb 229
2.5.1.2.4 示例文件 230
2.6 编程接口-其他 234
2.6.1 Ado.net 234
2.6.1.1 环境准备 234
2.6.1.2 支持的.NET 版本说明 234
2.6.1.3 支持的 ORM 框架说明 235
2.6.1.4 操作实例 235
2.6.1.4.1 新建项目 236
2.6.1.4.2 添加驱动引用 236
2.6.1.4.3 程序编写 237
2.6.1.4.4 运行程序 240
2.6.1.5 常见问题 240
2.6.2 Pdo_kdb 241
2.6.2.1 前提条件 241
2.6.2.2 操作实例 241
2.6.2.2.1 下载合适 PHP 驱动 242
2.6.2.2.2 安装部署 PHP 驱动 242
2.6.2.2.3 测试示例 247
2.6.2.3 常见问题 249

第 3 章 使用 KingbaseES 251
3.1 SQL 基础 251
3.1.1 连接和断开数据库 251
3.1.1.1 关于 ksql 251
3.1.1.2 使用 ksql 连接到数据库 252
3.1.1.3 在 ksql 中断开数据库连接 252
3.1.2 创建和管理数据库 252
3.1.2.1 创建数据库 253
3.1.2.2 查看数据库列表 253
3.1.2.3 使用数据库 254
3.1.2.4 删除数据库 254
3.1.3 创建和管理表空间 254
3.1.3.1 创建表空间 254
3.1.3.2 使用表空间 255
3.1.3.3 查询表空间 255
3.1.3.4 查询表空间使用率 255
3.1.3.5 修改表空间 256
3.1.3.6 删除表空间 256
3.1.4 创建和管理模式 257
3.1.4.1 创建模式 257
3.1.4.2 查看模式列表 258
3.1.4.3 模式的搜索路径 258
3.1.4.4 模式的权限控制 258
3.1.4.5 使用模式 259
3.1.4.6 更改模式 260
3.1.4.7 删除模式 260
3.1.5 创建和管理表 260
3.1.5.1 创建表 261
3.1.5.2 修改表结构 261
3.1.5.3 查看表结构 261
3.1.5.4 插入表数据 262
3.1.5.5 更新表数据 262
3.1.5.6 删除表数据 263
3.1.5.7 查询表数据 263
3.1.5.8 删除表 264
3.1.6 创建和管理索引 264
3.1.6.1 创建索引 264
3.1.6.2 修改索引 265
3.1.6.3 查看索引 266
3.1.6.4 重建索引 266
3.1.6.5 删除索引 266
3.1.7 创建和管理视图 267

3.1.7.1 创建视图 267
3.1.7.2 查询视图定义 267
3.1.7.3 查询视图 268
3.1.7.4 修改视图 268
3.1.7.5 删除视图 268
3.1.8 创建和管理用户 269
3.1.8.1 创建用户 269
3.1.8.2 修改用户 269
3.1.8.3 查看用户 270
3.1.8.4 授予权限 270
3.1.8.5 撤销权限 270
3.1.8.6 删除用户 271
3.1.9 启动和管理事务 271
3.1.9.1 事务控制命令 271
3.1.9.2 事务示例 272

版权声明 274

服务周期承诺 275

第1章　安装 KingbaseES

1.1　Docker 快速体验

本节将讲解在 Linux 系统中使用 docker 命令行快速安装 KingbaseES。

1.1.1　环境准备

1.1.1.1　硬件环境要求

KingabaseES Docker 软件包支持通用 X86_64、龙芯、飞腾、鲲鹏等国产 CPU 硬件体系架构。

1.1.1.2　软件环境要求

软件包名称	版本说明
Docker	20.10.0 及以上版本

1.1.1.3　创建安装路径

使用 root 用户执行创建数据库持久化存储路径，在宿主机执行如下命令：

```
mkdir -p /opt/kingbase
mkdir -p /opt/kingbase/data
chmod -R 755 /opt/kingbase/data
```

1.1.1.4　导入镜像

可以通过电科金仓官网下载对应平台的安装程序，也可以通过电科金仓的销售人员、售后支持人员或代理商获取安装程序。

获取对应金仓数据库镜像，例如 kingbase.tar，导入 tar 包到本机 docker 镜像仓库中。

查询软件包目录如下：

```
cd /opt/kingbase
ls -al
```

执行示例

```
[root@kingbase /opt/kingbase]# cd /opt/kingbase # 进入软件安装包目录
[root@kingbase /opt/kingbase]# ls -al # 查看容器镜像
-rw-r--r--  1 root root   137014730 Sep  1 18:56 kingbase.tar
drwxr-xr-x  2 root root        6    Sep  1 18:56 data
```

导入金仓数据库镜像

使用 root 用户执行导入镜像至 Docker 镜像库

```
docker load -i /opt/kingbase/kingbase.tar
```

导入镜像示例:

```
[root@kingbase /opt/kingbase]# docker load -i /opt/kingbase/kingbase.tar
96723fcc60ed: Loading layer [==================================>]  309.9MB/309.9MB
472038ee8f3a: Loading layer [==================================>]  22.53kB/22.53kB
c47ca06c1f31: Loading layer [==================================>]  7.168kB/7.168kB
4b96eaf3f5a9: Loading layer [==================================>]  8.192kB/8.192kB
80e74e42a456: Loading layer [==================================>]  7.168kB/7.168kB
2736b91ba7fd: Loading layer [==================================>]    497MB/497MB
9c1c6592e2c1: Loading layer [==================================>]  7.168kB/7.168kB
b677a0e279f7: Loading layer [==================================>]  7.168kB/7.168kB
62f44c97360a: Loading layer [==================================>]  4.096kB/4.096kB
Loaded image: kingbase:v1
[root@kingbase /opt/kingbase]# docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
kingbase            v1                  84c4c61cc40e        3 weeks ago         794MB
```

信息:

当使用 `docker load -i kingbase.tar` 命令报错时，可使用 `docker import kingbase.tar` 命令，尝试重新导包。

1.1.2 启动实例

最小启动命令示例

```
[root@kingbase /home/kingbase]# docker run -tid --privileged \
-p 54321:54321 \
--name kingbase  \
kingbase:v1 /usr/sbin/init
# docker 返回生成的容器 ID
c8784e857a4a08a9a810bf0591b1b78e3659f4f8ea552d5b3023cea963a216ca
```

金仓数据库服务端使用 54321TCP 端口。使用 `-p 54321:54321` 端口将数据库 54321 端口映射到本机提供服务。当前部署方式未对存储数据进行持久化。如果需要将数据持久化到本机的文件夹中进行存储，则执行以下命令进行创建:

```
[root@kingbase /home/kingbase]# docker run -tid --privileged \
-p 54321:54321 \
--name kingbase  \
-v /opt/kingbase/data:/home/kingbase/userdata \
kingbase:v1 /usr/sbin/init
```

信息:

/home/kingbase/userdata: 金仓数据库默认数据文件目录。

/opt/kingbase/data: 本机存储持久化目录, 可以通过修改此目录来符合您本机的存储目录规划。

生产环境部署数据库 Docker 容器的说明请参照 docker 软件包安装。

1.1.3 安装后检查

```
[root@kingbase /home/kingbase]# docker ps
CONTAINER ID        IMAGE               COMMAND                  CREATED             STATUS
PORTS                     NAMES
c8784e857a4a        kingbase:v1         "/bin/bash /home/kin…"   3 seconds ago       Up 2 seconds
                       kingbase
```

1.1.4 命令行界面

进入容器并执行 `bash`

```
docker exec -it kingbase /bin/bash
```

执行 ksql 访问命令行界面:

```
[kingbase@df874e55bd7f ~]$ ksql -Ukingbase -d test
ksql (V9.0)
Type "help" for help.

test=#
```

1.2　ISO 包快速体验

KingbaseES iso 安装主要分为以下几个步骤：

- 硬件环境要求
- 创建安装用户
- 创建安装目录
- 安装包准备
- 安装包的挂载
- 安装并启动 KingbaseES
- 安装后检查

1.2.1　硬件环境要求

KingbaseES 支持通用 X86_64、龙芯、飞腾、鲲鹏等国产 CPU 硬件体系架构。

表 1.2.1: 硬件要求

版本	要求
标准版/企业版/专业版/开发版	CPU：X86、龙芯、飞腾、鲲鹏内存：512MB 以上硬盘：11GB 以上空闲空间

备注:

/tmp 目录需要至少 10G 空间。如果安装过程中出现存储空间不足的情况，请先释放足够的磁盘空间，再执行安

装程序。如果硬件配置不满足要求，需要更换满足要求的硬件设备再进行安装。

1.2.2　创建安装用户

使用 root 用户运行如下命令创建 kingbase 用户：

```
useradd kingbase
或
groupadd kingbase
```

备注:

创建安装系统用户 kingbase 之后，如无特殊说明，本指南后续的操作默认使用 kingbase 用户进行操作。

使用 root 用户运行如下命令设置 kingbase 用户密码：

```
passwd kingbase
```

备注:

密码需要输入两次，保证两次输入的密码相同。

1.2.3　创建安装目录

KingbaseES 默认的安装目录是/opt/Kingbase/ES/V9，使用 root 用户先创建该目录。

```
mkdir -p /opt/Kingbase/ES/V9
chmod o+rwx /opt/Kingbase/ES/V9
```

1.2.4　安装包准备

获取安装包

可以通过电科金仓官网下载对应平台的安装程序，也可以通过电科金仓的销售人员、售后支持人员或代理商获取安装程序。

上传安装包

将上述安装包软件包上传至服务器到存放软件包的路径（本文示例：/opt/software/）

使用 root 用户创建存放软件路径

```
mkdir -p /opt/software/
mkdir -p /opt/software/KingbaseESV9
```

1.2.5　安装包的挂载

iso 格式的安装程序包需要先挂载才能使用, 需要使用 root 用户挂载 iso 文件，命令如下：

```
cd /opt/software/
mount KingbaseES_V009R001C010_Lin64_install.iso ./KingbaseESV9
```

1.2.6　安装 KingbaseES

1. 使用 kingbase 用户执行数据库安装命令

```
cd /opt/software/KingbaseESV9
sh setup.sh -i console
```

操作日志

```
# 执行安装命令
[kingbase@localhost /]$ /opt/software/KingbaseESV9
[kingbase@localhost KingbaseESV9]$ sh setup.sh -i console
Now launch installer...
Command line arguments: -console -language chn

===============================================================================
选择安装类型
----

  1- 安装新实例
  2- 修改已有实例
请输入 [1]安装新实例，[2]修改已有实例，[Q]退出当前修改程序  [默认： 1]  # 输入 1，然后按 Enter 键

===============================================================================
欢迎使用 KingbaseES 安装程序
----

欢迎使用本安装程序！

本安装程序旨在为您提供方便快捷的安装体验。
在开始安装之前，请注意以下几点：
```

```
退出应用程序:
    在安装过程中，我们建议退出所有正在运行的应用程序，
    以确保安装程序顺利进行，您可以在安装完成后重新打开这些应用程序。

保存文档:
    在继续本次安装之前，请确保您已保存并关闭所有打开的文档。
    这可以防止因安装过程中需要重启电脑而导致数据丢失。

可能需要重启:
    本次安装可能需要重启您的电脑以完成安装过程。
    请不要担心，这是为了确保安装的完整性和稳定性。
    安装程序会在需要重启时提醒您。

安装前准备:
    在继续安装之前，建议您查看安装程序提供的系统要求和注意事项，
    这将有助于您顺利完成安装并获得最佳使用体验。

谢谢您使用 KingbaseES，
如果您在安装过程中遇到任何问题或需要帮助，请随时联系我们的技术支持团队。
祝您使用愉快!

按下 [ENTER] 下一步， [Q] 退出程序 [默认: <ENTER>]  # 按 Enter 键

==============================================================================
许可协议
----

中电科金仓（北京）科技股份有限公司
“软件最终用户许可协议”

    重要须知，请认真阅读：本《最终用户许可协议》（以下称《协议》）是您（个人
或单一实体）与中电科金仓（北京）科技股份有限公司（以下简称“电科金仓”）之间有关
上述电科金仓软件产品的法律协议。本“软件产品”包括计算机软件，并可能包括相关媒体
、印刷材料和联机文档（“软件产品”）。本“软件产品”还包括对电科金仓提供给您的原
“软件产品”的任何更新和补充资料。任何与本“软件产品”一同提供给您的并与单独一份
软件许可证相关的软件产品是根据本《协议》中的条款而授予您。您一旦安装、复制、下载
、访问或以其它方式使用“软件产品”，即表示您同意接受本《协议》各项条款的约束。如
您不同意本《协议》中的条款，请不要安装、复制或使用“软件产品”。

    软件产品许可证

    本“软件产品”受著作权法及国际著作权条约和其它知识产权法和条约的保护。
    本“软件产品”只许可使用，而不出售。

1、许可证的授予。只要您遵守本《协议》，电科金仓将授予您下列非独占性的不可转让的
```

权利：

应用软件。本软件的使用应在相应合同上规定的地点上使用。使用应受到购买的数量和许可种类（如合同中所约定）的使用限制所制约。如果许可种类没有特殊说明，则您只能在单一一台计算机、工作站、手持式计算机、智能电话或其他数字电子仪器（“计算机”）上安装、使用、访问、显示、运行或以其他方式互相作用于（“运行”）本“软件产品”的一份副本

保留权利。除本协议中具体的规定外，未明示授予的一切其它权利均为电科金仓所有。电科金仓保留对本协议内容的解释权。

2、其它权利和限制的说明。

限制转让。在任何情况下，没有电科金仓的事先书面同意，您不得转让本协议或本协议项下授予的任何使用许可。

非再销售性软件。如果本“软件产品”标明“非再销售品”，则无论本《协议》其它条款有如何说明，您对本“软件产品”的使用只限于演示、测试或评价目的，而您不得再销售或以其它方式有偿转让本“软件产品”。

对反向工程、反向编译、反汇编的限制。您不得对本“软件产品”进行反向工程、反向编译或反汇编；尽管有此限制，但如果适用法律明示允许上述活动，并仅在适用法律明示允许的范围内从事上述活动，则不在此限。

商标。本《协议》不授予您任何有关电科金仓商标或服务商标的权利。

出租限制。不得出租、租赁或出借本“软件产品”。

出口限制。您承认本“软件产品”受中国出口法律管辖。您同意遵守所有适用于本“软件产品”的国际法和国内法，包括《中华人民共和国对外贸易法》，以及中国政府颁布的相关软件出口的限制和规定。

支持服务。电科金仓为您提供与“软件产品”有关的支持服务（“支持服务”），但支持服务的具体内容受相关合同的制约，电科金仓保留对支持服务进行收费的权利。支持服务的使用受用户手册、联机文档和/或其它电科金仓提供的材料中所述的各项政策和计划的制约。提供给您作为支持服务的一部份的任何附加软件代码应被视为本“软件产品”的一部分，并须符合本《协议》中的各项条款和条件。至于您提供给电科金仓作为支持服务的一部分的技术信息，电科金仓可将其用于商业用途，包括产品支持和开发。除了在为您提供支持时必须的情况外，电科金仓在使用这些技术信息时不会以个人形式提及您。
--更多(按下'空格'进行翻页)-- # 按空格键

协议终止。如您未遵守本《协议》的各项条款和条件，在不损害其它权利的情况下，电科金仓可终止本《协议》。如此类情况发生，您必须销毁“软件产品”的所有副本及其所有组成部分。

3、升级版本。如本“软件产品”标明为升级版本，您必须获取由电科金仓标明为合格使用升级版本的产品的许可证方可使用本“软件产品”。电科金仓公司保留在任何时候为您提供本“软件产品”的升级版本或升级收取费用的权利。标明为升级版本的“软件产品”替换和/或补充使您有资格使用升级版本的基础的产品，您只可根据本《协议》的条款使用所产生的升级产品。如本“软件产品”是您获得许可作为单一产品使用的一套软件程序包中一个组件的升级版本，则本“软件产品”只可作为该单一产品包的一部分而使用和转让，并且不可将其分开使用在一台以上的计算机上。

4、知识产权。本“软件产品”（包括但不限于本“软件产品”中所含的任何图像、照片、动画、录像、录音、音乐、文字和附加程序）、随附的印刷材料、及本“软件产品”的任何副本的产权、著作权和其他知识产权，均由电科金仓或其供应商拥有。通过使用“软件产品”可访问的内容的一切所有权和知识产权均属于各自内容所有者拥有，并可能受著作权法或其它知识产权法和条约的保护。本《协议》不授予您使用这些内容的权利。如果这份“软件产品”包括只以电子形式提供的文档，您可以打印一份该电子文档。您不可以从本软件中去掉版权声明，并保证为本“软件产品”复制品（全部或部分）复制版权声明，您同意制止以任何形式非法复制本软件及文档。您不可复制本“软件产品”随附的印刷材料。

5、多重媒体软件。您可以由多种媒体获得“软件产品” 。无论您收到何种类型或大小的媒体，您只可使用适用于您单一的计算机的一种媒体。您不可在另一台计算机上运行另一种媒体。除作为“软件产品”永久转让的一部分外（如上文所述），您不得将另一种媒体出借、出租或租赁或以其他方式转移给另一用户。

6、备份副本。在按照本《协议》安装一份本“软件产品”副本后，您可以保留电科金仓用以提供给您本“软件产品”的原媒体，仅用于备份或存档之用。如果需要原媒体方可在计算机上使用“软件产品”，您可以复制一份“软件产品”副本仅用于备份或存档之用。除本《协议》中明文规定外，您不可复制本“软件产品”或随附本“软件产品”的印刷材料。经允许的复制件应以安全的方式予以保存。

7、保密。“保密信息”包括程序（含程序中使用的方法和概念）和电科金仓识别为专有的或保密的一切信息。除非本协议明确允许，您不得将保密信息透露或者以其他方式提供给第三方和您没有必要授权其在业务中使用程序的任何雇员。您应采取一切合理、必要的步骤，保证不将本软件产品的程序或任何部分透露或以其他方式提供给第三方。

如您是在中华人民共和国取得此电科金仓产品，下列有限保证适用于您。

1、有限保证

只要您有有效的许可，电科金仓保证：（1）“软件产品”自收货之日起九十天内基本符合随附书面材料所述的功能。（2）电科金仓提供的支持服务应基本与有关书面材料中所述相符，并且电科金仓支持工程师将在商业所允许的范围内尽力解决任何问题。如果本产品不符合本保证，电科金仓将修理、更换本产品或退回您已支付的价款，但您应将该“软件产品”连同您所持有的发票一同退还给电科金仓。如果本产品的故障是由事故、滥用或误用所造成，则本保证无效。退换的产品享受原保证期剩余时间，或三十天的保证（以时间较长者为准）。在适用法律允许的最大范围内，并且除上述保证外，电科金仓否认一切明示或默示的保证、条件和其他条款。不能予以排除的任何默示保证仅限于九十天或适用法律允许的最短期间（以时间较长者为准）内有效。

```
2、有限责任
    在适用法律所允许的最大范围内，除上述保证规定者外，电科金仓或其供应商在任
何情况下绝不就因使用或不能使用“软件产品”或因提供或未提供支持服务所发生的任何特
殊的、意外的、非直接的或间接的损失（包括但不限于营业利润损失、营业中断、商业信息
的遗失或任何其他金钱上的损失）承担赔偿责任，即使电科金仓或其任何供应商事先被告知
该损害发生的可能性。不论任何情况，电科金仓在本《协议》任何条款下所承担的全部责任
，以您就“软件产品”实际已付的价款或人民币 10.00 元为限，以两者之间较高额为准
--更多(按下'空格'进行翻页)--   # 按空格键

。但是，如果您已经与电科金仓达成支持服务协议，电科金仓就支持服务的全部赔偿责任应
以该支持服务协议条款为准。

管辖法律
    本协议受中华人民共和国法律管辖（包括但不限于《中华人民共和国著作权法》、
《中华人民共和国计算机软件保护条例》、《中华人民共和国商标法》、《中华人民共和国
专利法》、《中华人民共和国反不正当竞争法》等法律法规）。因本协议或其违约引起的或
与之有关的任何争论或索赔，应由电科金仓所在地人民法院管辖。电科金仓的知识产权如被
侵犯，上述规定不限制电科金仓就此向具有合法管辖权的法院请求做出强制性的补救措施。

电科金仓的联系方式如下：

地址：北京市朝阳区容达路 7 号中国电科太极信息产业园 E 座二层
邮编：100102
电话：86-10-5885 1118
网址：www.kingbase.com.cn
全国服务热线：400-601-1188
技术支持邮箱：support@kingbase.com.cn

请输入 [1]接受，[2]拒绝，[3]重新展示：    # 输入 1，然后按 Enter 键

================================================================================
添加 Licence
----

不选择授权文件，则使用软件自带试用版授权
提示：请在有效期内及时更换正时授权文件

输入授权文件地址：[默认：  ]    # 按 Enter 键

输入字母 N 进入下一步，输入字母 P 返回上一步，输入字母 Q 退出修改程序 [默认： N]  # 按 Enter 键

================================================================================
选择安装路径
----
```

```
请选择一个安装目录。
您想在哪一个位置安装？
缺省安装文件夹：/opt/Kingbase/ES/V9

输入一个绝对路径，或按 ENTER 键以接受缺省路径 [默认： /opt/Kingbase/ES/V9]    # 按 Enter 键

-----------------------------------------------------
安装文件夹为：/opt/Kingbase/ES/V9
  是否正确？(Y/N)

请选择
-----------------------------------------------------
输入 Y 是，N 否： [默认： Y]    # 按 Enter 键

输入字母 N 进入下一步，输入字母 P 返回上一步，输入字母 Q 退出修改程序 [默认： N]  # 按 Enter 键

===============================================================================
选择安装集
----

  1- 完全安装
     最常用的应用程序功能组件。建议大多数用户采用此选项。

  2- 服务器安装
     只安装数据库服务。

  3- 定制安装
     选择此选项以定制要安装的功能部件。

输入“安装集”的号码，或按 ENTER 键以接受缺省值 [默认： 1]  # 按 Enter 键

===============================================================================
安装预览
----

安装目录：
    /opt/Kingbase/ES/V9

已安装组件：
    引导组件
    产品手册
    数据库运维工具
    数据库服务器
    高可用组件
    接口
```

```
    数据库集群部署工具
    数据库迁移工具
    数据库开发工具(CS)

未安装组件:



磁盘空间信息(用于安装目标)
  所需空间: 2,992.35 MB, 可用空间: 3,992.74 GB

输入字母 N 进入下一步, 输入字母 P 返回上一步, 输入字母 Q 退出修改程序 [默认: N]  # 按 Enter 键



================================================================================
安装进度
----

====
开始安装
架构: 6.0.0-SNAPSHOT-1461c (kInstaller)
平台: linux,version=3.10.0-862.el7.x86_64,arch=x64,symbolicName=null,javaVersion=1.8.0_262
[ Starting to unpack ]
[ Processing package: install (1/9) ]
[ Processing package: doc (2/9) ]
[ Processing package: Suptools (3/9) ]
[ Processing package: Server (4/9) ]
[ Processing package: KingbaseHA (5/9) ]
[ Processing package: Interface (6/9) ]
[ Processing package: DeployTool (7/9) ]
[ Processing package: KDTS (8/9) ]
[ Processing package: KStudio (9/9) ]
[ Unpacking finished ]
安装完成
[ Starting processing ]
Starting process modifyexecute (1/3)
Starting process modifyFiles (2/3)
The beginning of Repair File Contents.
Running modifyFilesValue.sh...
Repair File Contents Finish.



Starting process packtools (3/3)
安装成功 100%

================================================================================
```

```
快捷方式
----


------
创建快捷方式
------
输入 Y 是，N 否： [默认： Y]                          # 按 Enter 键
创建快捷方式属主：1：当前用户;2：所有用户 [默认： 1]      # 按 Enter 键
选择快捷方式的程序组：[默认： KingbaseESV009R001]     # 按 Enter 键

==============================================================================
恭喜您！安装完成
----



  恭喜您！安装完成

  安装目录：
    /opt/Kingbase/ES/V9



    引导组件:/opt/Kingbase/ES/V9/KESRealPro/V009R001/install
    产品手册:/opt/Kingbase/ES/V9/KESRealPro/V009R001/doc
    数据库运维工具:/opt/Kingbase/ES/V9/KESRealPro/V009R001/SupTools
    数据库服务器:/opt/Kingbase/ES/V9/KESRealPro/V009R001/Server
    高可用组件:/opt/Kingbase/ES/V9/KESRealPro/V009R001/KingbaseHA
    接口:/opt/Kingbase/ES/V9/KESRealPro/V009R001/Interface
    数据库集群部署工具:/opt/Kingbase/ES/V9/KESRealPro/V009R001/ClientTools/guitools/DeployTools
    数据库迁移工具:/opt/Kingbase/ES/V9/KESRealPro/V009R001/ClientTools/guitools/KDts
    数据库开发工具(CS):/opt/Kingbase/ES/V9/KESRealPro/V009R001/ClientTools/guitools/KStudio


如需初始化数据库，请启动 Kconsole:
/opt/Kingbase/ES/V9/Server/bin/kconsole.sh
手动初始化数据库:
/opt/Kingbase/ES/V9/KESRealPro/V009R001/Server/bin/initdb -U "system" -x "12345678ab" -E "UTF-8" --lc-ctype=
"zh_CN.UTF-8" --lc-collate="zh_CN.UTF-8" -D "/opt/Kingbase/ES/V9/data"
[ Writing the uninstaller data ... ]
[ 命令行安装完成 ]
```

第2章　访问 KingbaseES

2.1　客户端工具访问

2.1.1　ksql

2.1.1.1　关于 ksql

ksql 是 KingbaseES 数据库的命令行连接工具，支持本地和远程两种连接方式。

- 本地连接: ksql 连接到同一台计算机运行的 KingbaseES 数据库。
- 远程连接: ksql 通过网络访问另一台计算机运行的 KingbaseES 数据库。

2.1.1.2　使用 ksql 连接到数据库

ksql 连接到数据库需要两步：获取服务端口、ksql 连接数据库。下述为每一步操作细节：

Step1：获取 KingbaseES 数据库服务端口

若使用默认端口号（注：KingbaseES 数据库的默认端口号为 54321），请直接执行Step2 操作。

- Windows 平台下，执行以下命令获取端口号。

```
findstr /R "^\s*port" C:\kb_install\data\kingbase.conf
```

其中 `C:\kb_install\data\kingbase.conf` 为数据库的数据文件路径，请根据实际情况进行替换。

- Linux 平台下，执行以下命令获取端口号。

```
cat /test/kb_install/data/kingbase.conf | grep port
```

其中 `/test/kb_install/data` 为数据库的数据文件路径，请根据实际情况进行替换。

Step2：使用 ksql 连接 KingbaseES 数据库

- 指定端口号的方式连接数据库。

```
ksql -h 10.11.0.11 -p 54321 -d kingbase -U Alice
```

其中 10.11.0.11 为数据库服务器的 IP 地址，kingbase 为要连接的数据库名称，54321 是数据库服务端口，Alice 为数据库用户。若使用本地连接方式，可以省略-h 选项。

- 若 KingbaseES 数据库使用默认端口号，可以省略端口号进行连接。

```
ksql -h localhost -d kingbase -U Alice
```

其中 localhost 代表 ksql 将连接到当前服务器的 KingbaseES 数据库，kingbase 代表要连接的数据库名称，Alice 为数据库用户。

2.1.1.3　在 ksql 中断开数据库连接

可直接在 ksql 中输入 `\q` 或 QUIT 命令断开数据库连接。

提示:

关于 ksql 的更多使用方式，请参考 ksql 命令参考中的详细说明。

2.2　编程接口-Java

2.2.1　JDBC

本文提供使用 JDBC 连接 Kingbase 数据库的连接代码示例。

2.2.1.1　前提条件

- 确保已设置基本的应用开发环境。
- 确保已安装与驱动对应版本的数据库，且数据库连接可用。
- 确保当前支持 JDBC 开发：
  - Linux：自带 JDK
  - Windows：确保已安装 JDK
- 确保 JDBC 支持开发环境：
  - 系统架构：

    Linux：x86_64、arm、loongarch、mips、sw

Windows：V9 版本提供 64 位支持

– JDK 版本：

支持 1.6、1.7、1.8。

2.2.1.2　JDBC 包

kingbase 提供两种 JDBCjar 包：postgresql-42.2.9.jar 和 kingbase8-x.x.x.jar，两种 jar 包功能一致。

以 kingbase8 为例跟 JDK 对应关系为:

驱动包	支持最低版本
kingbase8-9.0.0.jre6.jar	JDK1.6
kingbase8-9.0.0.jre7.jar	JDK1.7
kingbase8-9.0.0.jar	JDK1.8

可通过已发布版本的数据库安装包、官网下载页的接口侧、或联系技服人员获取对应版本架构的 JDBC 驱动。

- postgresql-42.2.9.jar：主类名为"org.postgresql.Driver"，数据库连接的 url 前缀为"jdbc:postgresql"，该驱动包与 PostgreSQL 保持兼容，其中类名、类结构与 PostgreSQL 驱动完全一致，曾云星宇 PostgreSQL 的应用程序可以直接移植到当前系统使用。
- kingbase8-x.x.x.jar：主类名为" com.kingbase8.Driver"，数据库连接的 url 前缀为" jdbc:kingbase8"，当同一 JVM 进程内需要同时访问 PostgreSQL 及 kingbase 时，使用 kingbase8-x.x.x.jar 可以避免类名冲突。

2.2.1.2.1　开发源程序

以下是一个插入并执行查询的具体示例。

1. 加载驱动：

   在创建数据库连接之前，需要加载数据库驱动类，驱动类不同包位置不同。kingbase jdbc 驱动为" com.kingbase8.Driver"，其中 url 前缀为"jdbc:kingbase8"。

   可以在 maven 项目的 pom.xml 文件中添加依赖, 根据需求选择需要的版本：

```
<!-- 最低可支持 JDK1.8 -->
<dependency>
<groupId>cn.com.kingbase</groupId>
<artifactId>kingbase8</artifactId>
<version>9.0.0</version>
</dependency>
<!-- 最低可支持 JDK1.7 -->
```

```
<dependency>
<groupId>cn.com.kingbase</groupId>
<artifactId>kingbase8</artifactId>
<version>9.0.0.jre7</version>
</dependency>
<!-- 最低可支持 JDK1.6 -->
<dependency>
<groupId>cn.com.kingbase</groupId>
<artifactId>kingbase8</artifactId>
<version>9.0.0.jre6</version>
</dependency>
```

2. 连接数据库

   参数说明

   - host 是数据库服务器的地址，包括域名、主机名、主机的 IP 地址等。缺省是”localhost”。
   - port 是数据库服务器监听的端口号。KingbaseES 服务器的缺省端口号：54321。
   - database 是数据库名。
   - userID 是连接数据库的用户名。
   - myPassword 是连接数据库用户的密码。

   JDBC 提供了三个方法，用于创建数据库连接：

   - DriverManager.getConnection(String url);

     此连接方法需要在 URL 上面拼接用户名密码。

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Properties;

public class JdbcConn {
  public static void main(String[] args) {
        getConnect();
  }
  public static Connection getConnect() {
        String driver = "com.kingbase8.Driver";
        String sourceURL = "jdbc:kingbase8://host:port/database?user=userID&password=myPassword";
        Connection conn = null;
        try {
```

```
            Class.forName(driver);
        } catch (Exception e) {
            e.printStackTrace();
            return null;
        }
        try {
            conn = DriverManager.getConnection(sourceURL);
            System.out.println("连接成功！ ");
            return conn;
        } catch (Exception e) {
            e.printStackTrace();
            return null;
        }
    }
}
```

- DriverManager.getConnection(String url，Properties info);

  此连接方法的用户名密码等参数均在 Properties 对象的实例中，可通过 setProperty 添加。

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Properties;

public class JdbcConn {
  public static void main(String[] args) {
        getConnect();
  }
 public static Connection getConnect() {
        String driver = "com.kingbase8.Driver";
        String sourceURL = "jdbc:kingbase8://host:port/database";
        Properties info = new Properties();
        info.setProperty("user","userId");
        info.setProperty("password","myPassword");
        Connection conn = null;
        try {
            Class.forName(driver);
        } catch (Exception e) {
            e.printStackTrace();
            return null;
        }
        try {
```

```
            conn = DriverManager.getConnection(sourceURL, info);
            System.out.println("连接成功！");
            return conn;
        } catch (Exception e) {
            e.printStackTrace();
            return null;
        }
    }
}
```

- DriverManager.getConnection(String url，String user，String password);

  此连接方法需要将用户名和密码作为变量输入。

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Properties;

public class JdbcConn {
  public static void main(String[] args) {
        getConnect();
  }
 public static Connection getConnect() {
        String driver = "com.kingbase8.Driver";
        String sourceURL = "jdbc:kingbase8://host:port/database";
        String username="userID";
        String passwd="myPassword";
        Connection conn = null;
        try {
            Class.forName(driver);
        } catch (Exception e) {
            e.printStackTrace();
            return null;
        }
        try {
            conn = DriverManager.getConnection(sourceURL, username, passwd);
            System.out.println("连接成功！");
            return conn;
        } catch (Exception e) {
            e.printStackTrace();
            return null;
        }
```

```
    }
}
```

3. Statement 示例

   通过 statement 执行增删改查操作。

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class testStatement {
  protected String Driver = "com.kingbase8.Driver";
  protected static String Url = "jdbc:kingbase8://localhost:54321/test";
  protected Connection connection = null;
  protected Statement statement = null;
  protected String create_table = "create table orders(ORDER_ID SERIAL, "
                                        + "ISBN int, CUSTOMERID varchar(20))";

  /* 加载驱动 */
  protected void Driver() throws ClassNotFoundException {
    Class.forName(Driver);
  }

  public static void main(String[] args) throws ClassNotFoundException,
                                                    SQLException, InterruptedException {
    testStatement test = new testStatement();
    test.Driver();
    test.connection(Url);
    test.statement();
    test.table();
    test.getGeneratedKeys();
    test.close();
  }

  /* 建立连接 */
  protected void connection(String url) throws SQLException,
                                                  InterruptedException {
    connection = DriverManager.getConnection(url, "system", "manager");
    if (connection != null) {
      System.out.println("connection sucessful!");
    } else {
      System.out.println("connection fail!");
    }
```

```
}

/* 建立语句对象 */
protected void statement() throws SQLException {
  statement = connection.createStatement();
}

/* 执行建表语句 */
protected void table() throws SQLException {
  statement.executeUpdate(create_table);
}

protected void getGeneratedKeys() throws SQLException {
  /* 执行 SQL 语句并返回所有列 */
  int rows = statement.executeUpdate("insert into orders (ISBN," +
                                          "CUSTOMERID) VALUES (195123018,'BILLING')"
          , Statement.RETURN_GENERATED_KEYS);
  System.out.println("rows:" + rows);
  ResultSet rs = null;
  rs = statement.getGeneratedKeys();
  boolean b = rs.next();
  if (b) {
    System.out.println(rs.getString(1));
  }
  rs.close();
  /* 执行 SQL 语句并返回数组中的指定列 */
  statement.executeUpdate("delete from orders");
  boolean result = statement.execute("insert into orders (ISBN," +
                                          "CUSTOMERID) VALUES (195123018,'BILLING')"
          , Statement.RETURN_GENERATED_KEYS);
  System.out.println("result:" + result);
  rs = statement.getGeneratedKeys();
  boolean c = rs.next();
  if (c) {
    System.out.println(rs.getString(1));
  }
  rs.close();
  statement.executeUpdate("delete from orders");
  String keyColumn[] = {"order_id"};
  int row = statement.executeUpdate("insert into orders (ISBN," +
                                         "CUSTOMERID) VALUES (195123018,'BILLING')", keyColumn);
  System.out.println("row:" + row);
  rs = statement.getGeneratedKeys();
  boolean d = rs.next();
  if (d) {
```

```
      System.out.println(rs.getString(1));
    }
    rs.close();
    statement.executeUpdate("delete from orders");
    String keyColumn1[] = {"order_id"};
    boolean result1 = statement.execute("insert into orders (ISBN," +
                                                "CUSTOMERID) VALUES (195123018,'BILLING')",
keyColumn1);
    System.out.println("result1:" + result1);
    rs = statement.getGeneratedKeys();
    boolean e = rs.next();
    if (e) {
      System.out.println(rs.getString(1));
    }
    rs.close();
  }

  /* 关闭语句、连接对象 */
  protected void close() throws SQLException {
    statement.executeUpdate("drop table orders");
    if (statement != null) {
      statement.close();
    }
    if (connection != null) {
      connection.close();
    }
  }
}
```

4. PreparedStatement 示例

   获取参数元信息、获取主键。

```
import java.math.BigDecimal;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ParameterMetaData;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class testPreparedStatement {
  protected String Driver = "com.kingbase8.Driver";
  protected static String Url = "jdbc:kingbase8://localhost:54321/test";
```

```
protected Connection connection = null;
protected Statement statement = null;
protected enumn[] employee = new enumn[5];

protected void Driver() throws ClassNotFoundException {
  Class.forName(Driver);
}

public static void main(String[] args) throws SQLException,
                                                    ClassNotFoundException, InterruptedException {
  testPreparedStatement test = new testPreparedStatement();
  test.initEmployee();
  test.Driver();
  test.connection(Url);
  test.statement();
  test.table();
  test.preparedStatement();
  test.getParameterMetaData();
  test.close();
}

/* 初始化数据 */
protected void initEmployee() {
  for (int i = 0; i < employee.length; i++) {
    employee[i] = new enumn();
    employee[i].id = i;
    employee[i].salary = new BigDecimal("100." + i);
  }
}

/* 建立连接 */
protected void connection(String url) throws SQLException,
                                                  InterruptedException {
  connection = DriverManager.getConnection(url, "system", "manager");
  if (connection != null) {
    System.out.println("connection sucessful!");
  } else {
    System.out.println("connection fail!");
  }
}

/* 建立语句对象 */
protected void statement() throws SQLException {
  statement = connection.createStatement();
}
```

```
/* 执行建表和插入语句 */
protected void table() throws SQLException {
  statement.executeUpdate("create table employees (ID int not null " +
                                  "primary key, SALARY numeric(9,5))");
  for (int i = 0; i < 5; i++) {
    statement.executeUpdate("insert into employees values (" + i +
                                    "," + "100.10" + ")");
  }
}

protected void preparedStatement() throws SQLException {
  /* 使用 PreparedStatement 更新语句 */
  PreparedStatement preparedStatement = connection.prepareStatement(
          "UPDATE employees SET SALARY = ? WHERE ID = ?");
  for (int i = 0; i < employee.length; i++) {
    preparedStatement.setBigDecimal(1, employee[i].salary);
    preparedStatement.setInt(2, employee[i].id);
    preparedStatement.executeUpdate();
  }
  /* 查询数据 */
  ResultSet rs = statement.executeQuery("select SALARY from employees");
  while (rs.next()) {
    System.out.println(rs.getBigDecimal(1));
  }
  rs.close();
  preparedStatement.close();
}

/* 获取参数元信息 */
protected void getParameterMetaData() throws SQLException {
  statement.executeUpdate("delete from employees");
  PreparedStatement preparedStatement = connection.prepareStatement(
          "insert into employees (ID, SALARY) values (?,?)");
  ParameterMetaData pmd = preparedStatement.getParameterMetaData();
  int parameterCount = pmd.getParameterCount();
  System.out.println(parameterCount);
  preparedStatement.close();
}

/* 删除表，关闭语句、连接对象 */
protected void close() throws SQLException {
  statement.executeUpdate("drop table employees");
  if (statement != null) {
    statement.close();
```

```
    }
    if (connection != null) {
      connection.close();
    }
  }
}

class enumn {
  BigDecimal salary;
  int id;
}
```

5. ResultSet 示例

   查询打印结果集、获取结果集元信息、可更新结果集的使用。

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.ResultSetMetaData;
import java.sql.SQLException;
import java.sql.Statement;

public class testResultSet {
  protected String Driver = "com.kingbase8.Driver";
  protected static String Url = "jdbc:kingbase8://localhost:54321/test";
  protected Connection connection = null;
  protected Statement statement = null;

  protected void Driver() throws ClassNotFoundException {
    Class.forName(Driver);
  }

  protected void connection(String url) throws SQLException,
                                                InterruptedException {
    connection = DriverManager.getConnection(url, "system", "manager");
    if (connection != null) {
      System.out.println("connection sucessful!");
    } else {
      System.out.println("connection fail!");
    }
  }

  protected void statement() throws SQLException {
    statement = connection.createStatement();
  }
```

```
protected void table() throws SQLException {
  statement.executeUpdate("create table table1(id int primary key," +
                                  "name char(10))");
  statement.executeUpdate("insert into table1 values (1,'KingbaseES')");
}

public static void main(String[] args) throws ClassNotFoundException,
                                                    SQLException, InterruptedException {
  testResultSet test = new testResultSet();
  test.Driver();
  test.connection(Url);
  test.statement();
  test.table();
  test.resultSet();
  test.updataResultSet();
  test.close();
}

protected void resultSet() throws SQLException {
  /* 查询并返回结果集 */
  ResultSet rs = statement.executeQuery("select * from table1");
  while (rs.next()) {
    System.out.println(rs.getInt(1));
  }
  /* 获取结果集元信息对象 */
  ResultSetMetaData rsmd = rs.getMetaData();
  /* 获取第一列的类型名称 */
  System.out.println(rsmd.getColumnTypeName(1));
  rs.close();
}

/* 通过结果集更新数据 */
protected void updataResultSet() throws SQLException {
  Statement stmt = connection.createStatement(
          ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_UPDATABLE);
  ResultSet rs = stmt.executeQuery("select * from table1");
  rs.next();
  System.out.println("update before: " + rs.getInt(1));
  rs.updateInt(1, 21);
  rs.updateRow();
  System.out.println("update after: " + rs.getInt(1));
  rs.close();
  stmt.close();
}
```

```
  protected void close() throws SQLException {
    statement.executeUpdate("drop table table1");
    if (statement != null) {
      statement.close();
    }
    if (connection != null) {
      connection.close();
    }
  }
}
```

JDBC 更多示例请参考 JDBC 产品手册的程序示例。

2.2.2　SpringBoot

本文将介绍如何使用 SpringBoot 框架和 kingbase 数据库构建一个应用程序，实现创建表、插入数据和查询数据等基本操作。

点击下载 java-kingbase-springboot 示例工程。

2.2.2.1　前提条件

- 您已安装 Kingbase 数据库。
- 您已安装 JDK 1.8 和 Maven。
- 您已安装 IDE 工具。

备注:

本文档运行代码使用的工具是 IntelliJ IDEA 2021.3.2 (Community Edition) 版本，您也可以根据个人喜好选择适合自己的工具运行示例代码。

2.2.2.2　操作步骤

备注:

本文中给出的操作步骤是基于 Windows 环境生成。如果您使用的是其他操作系统环境或编译器，那么操作步骤可能会略有不同。

---

1. 获取 Kingbase 数据库连接信息。
2. 导入 java-kingbase-springboot 项目到 IDEA 中。
3. 修改 java-kingbase-springboot 项目中的数据库连接信息。
4. 运行 java-kingbase-springboot 项目。

2.2.2.2.1　步骤一：获取 Kingbase 数据库连接信息

1. 联系 Kingbase 数据库部署人员或者管理员获取相应的数据库连接信息。

```
./ksql -U system -d test -h xx.xx.xx.xx -p 54321
```

2. 根据已部署的 Kingbase 数据库填写下面的 URL 的对应信息。

备注:

在 application.properties 文件中需要这里的 URL 信息。

```
jdbc:kingbase8://host:port/database?user=$user_name&password=$password
```

参数说明：

- host：提供 Kingbase 数据库连接 IP。直连方式使用的是 KESServer 节点的 IP 地址。
- port：提供 Kingbase 数据库连接端口。直连方式默认是 54321，在部署 kingbase 数据库时可自定义。
- database：需要访问的数据库名称。
- user_name：通过 -u 参数指定，管理员用户为 system。
- password：提供账户密码。

更多 URL 参数说明信息请参考 Jdbc 连接属性。

2.2.2.2.2　步骤二: 导入 java-kingbase-springboot 项目到 IDEA 中

1. 打开 `IntelliJ IDEA`，选择 `File > Open...` 选项。
2. 在弹出的 `Open File or Project` 窗口中，选择对应的项目文件，单击 `OK` 完成项目文件导入。
3. `IntelliJ IDEA` 将会自动识别项目中的各类文件，并在 `Project` 工具窗口中，可以查看项目的目录结构、文件列表、模块列表、依赖关系等信息。

   `Project` 工具窗口通常位于 IntelliJ IDEA 界面的最左侧，默认情况下是打开的。如果 `Project` 工具窗口被关闭了，可以通过点击菜单栏中的 `View > Tool Windows > Project` 或者使用快捷键 `Alt + 1` 来重新打开它。

备注:

当使用 IntelliJ IDEA 导入项目时，IntelliJ IDEA 会自动检测项目中的 pom.xml 文件，并根据文件中描述的依赖关系自动下载所需的依赖库，并将它们添加到项目中。

4. 查看项目情况。

2.2.2.2.3　步骤三: 修改 java-kingbase-springboot 项目中的数据库连接信息

根据步骤一：获取 Kingbase 数据库连接信息中的信息修改 application.properties 文件中的数据库连接信息。

示例如下:

- 数据库驱动名称为:com.kingbase8.Driver
- KES Server 节点的 IP 地址为 127.0.0.1。
- 访问端口使用的是 54321。
- 需要访问的 database 名称为 test。
- 账户是 system。
- 密码是 ******。

示例代码如下：

```
spring.datasource.driverClassName=com.kingbase8.Driver
spring.datasource.url=jdbc:kingbase8://localhost:54321/test
spring.datasource.username=system
spring.datasource.password=******
```

2.2.2.2.4　步骤四: 运行 java-kingbase-springboot 项目

运行路径：

1. 在项目结构中找到 `src > test > java` 中找到 TestSpringbootApplicationTests.java 文件。
2. 在工具菜单栏中选择 运行 (U) > 运行 > `TestSpringbootApplicationTests`，或直接单击右上角绿色三角形运行。
3. 通过 IDEA 的控制台来查看项目的日志信息和输出结果。

运行结果：

```
test_springboot delete successfully!
test_springboot create successfully!
user = User{id=2, name='update'}
User{id=2, name='update'}
User{id=3, name='insert3'}
User{id=4, name='insert4'}
User{id=5, name='insert5'}
```

```
User{id=6, name='insert6'}
User{id=7, name='insert7'}
User{id=8, name='insert8'}
User{id=9, name='insert9'}
User{id=10, name='insert10'}
```

2.2.2.3　项目代码介绍

点击下载 java-kingbase-springboot，这是一个名为 java-kingbase-springboot 的压缩包。

解压后，得到一个名为 java-kingbase-springboot 的文件夹。目录结构如下所示：

```
--pom.xml

 src
   main
      java
        com
            kingbase
                testspringboot
                    --TestSpringbootApplication.java

                     dao
                       --UserDao.java

                       impl
                           --UserDaoImpl.java

                     entity
                         --User.java

      resources
          --application.properties

   test
       java
           com
               kingbase
                   testspringboot
                       --TestSpringbootApplicationTests.java

 target
```

文件说明:

- pom.xml：Maven 项目的配置文件，包含了项目的依赖、插件、构建等信息。
- src：通常用于表示项目中存放源代码的目录。
- main: 存放主要的源代码和资源文件的目录。
- java: 存放 Java 源代码的目录。
- com: 存放 Java 包的根目录。
- kingbase: 存放项目的根目录。
- testspringboot：Java 包的根目录，包含了项目的所有 Java 类。
- TestSpringbootApplication.java：项目的主类，包含了 main 方法。
- dao: 存放数据访问对象（Data Access Object）包，用于访问数据库或其他数据存储服务。
- UserDao.java：用户数据访问对象，用于操作用户数据的增删改查等操作。
- impl：DAO 接口的实现目录。
- UserDaoImpl.java：用户 DAO 接口的实现类。
- entity：实体类的目录，用于存放与数据库表对应的 Java 类。
- User.java：用户持久化对象，用于映射用户数据表的字段。
- resources: 存放资源文件的目录，如配置文件、SQL 文件等。
- application.properties：项目的配置文件，用于配置应用程序的属性和参数。
- test: 存放测试代码和资源文件的目录。
- TestSpringbootApplicationTests.java: 存放测试 springboot 的 Java 类。
- target: 存放编译后的 Class 文件、Jar 包等文件的目录。

pom.xml 代码介绍:

备注:

如果您只是想验证示例，那么请使用默认代码，无需修改。您也可以按照以下讲解，根据自己的需求修改 pom.xml 文件。

---

pom.xml 配置文件内容如下:

1. 文件声明语句。

   声明本文件是一个 XML 文件，使用的 XML 版本是 1.0，字符编码方式是 UTF-8。

   代码如下:

```
<?xml version="1.0" encoding="UTF-8"?>
```

2. 配置 POM 的命名空间和 POM 模型版本。

- 通过 xmlns 指定 POM 的命名空间为 `http://maven.apache.org/POM/4.0.0`。
- 通过 xmlns:xsi 指定 XML 命名空间为 `http://www.w3.org/2001/XMLSchema-instance`。
- 通过 xsi:schemaLocation 指定 POM 的命名空间为 `http://maven.apache.org/POM/4.0.0`，并指定 POM 的 XSD 文件的位置为 `https://maven.apache.org/xsd/maven-4.0.0.xsd`。
- 通过 modelVersion 元素指定该 POM 文件使用的 POM 模型版本为 `4.0.0`。

代码如下:

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance
"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
</project>
```

3. 配置父项信息。
   - 通过 groupId 指定父项标识为 org.springframework.boot。
   - 通过 artifactId 指定父项依赖为 spring-boot-starter-parent。
   - 通过 version 指定父项的版本号为 2.7.11。
   - 通过 relativePath 表示父项的路径为空。

   代码如下:

```
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.7.11</version>
<relativePath/>
```

4. 配置基本信息。
   - 通过 groupId 指定项目标识为 com.kingbase。
   - 通过 artifactId 指定项目依赖为 java-kingbase-springboot。
   - 通过 version 指定项目的版本号为 0.0.1-SNAPSHOT。
   - 通过 description 介绍项目信息为 Demo project for Spring Boot。

   代码如下:

```
<groupId>com.kingbase</groupId>
<artifactId>java-kingbase-springboot</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>java-kingbase-springboot</name>
<description>Demo project for Spring Boot</description>
```

5. 配置 java 版本。

指定项目使用的 Java 版本为 1.8。

代码如下：

```
<properties>
    <java.version>1.8</java.version>
</properties>
```

6. 配置核心依赖。

   - 指定依赖项所属的组织为 org.springframework.boot，名称为 spring-boot-starter，该依赖库包含了 Spring Boot 默认支持的组件依赖，支持 Web、数据处理、安全、Test 等功能。

     代码如下:

     ```
     <dependency>
         <groupId>org.springframework.boot</groupId>
         <artifactId>spring-boot-starter</artifactId>
     </dependency>
     ```

   - 指定依赖项所属的组织为 org.springframework.boot，名称为 spring-boot-starter-jdbc，通过该依赖可以使用 Spring Boot 提供的 JDBC 相关功能，如连接池、数据源配置等。

     代码如下:

     ```
     <dependency>
         <groupId>org.springframework.boot</groupId>
         <artifactId>spring-boot-starter-jdbc</artifactId>
     </dependency>
     ```

   - 指定依赖项所属的组织为 org.springframework.boot，名称为 spring-boot-starter-test，作用范围为 test，通过该依赖可以使用 Spring Boot 提供的测试框架和工具，如 JUnit、Mockito、Hamcrest 等。

     代码如下:

     ```
     <dependency>
         <groupId>org.springframework.boot</groupId>
         <artifactId>spring-boot-starter-test</artifactId>
         <scope>test</scope>
     </dependency>
     ```

   - 指定依赖项所属的组织为 cn.com.kingbase，名称为 kingbase8，版本号为 9.0.0，通过该依赖可以使用 Kingbase 提供的客户端功能，如连接、查询、事务等。

     代码如下:

     ```
     <dependency>
         <groupId>cn.com.kingbase</groupId>
         <artifactId>kingbase8</artifactId>
     ```

```
    <version>9.0.0</version>
</dependency>
```

7. 配置 Maven 插件。

   指定依赖项所属的组织为 org.springframework.boot，名称为 spring-boot-maven-plugin，该插件用于将 Spring Boot 应用程序打包成可执行的 JAR 包或 WAR 包，并且可以直接运行。

   代码如下:

```
<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
```

2.2.2.3.1　application.properties 代码介绍

application.properties 文件配置了 Spring Boot 应用程序的数据源，指定了连接 Kingbase 数据库所需的驱动类名、URL、用户名和密码。通过这些配置，应用程序可以连接并操作 Kingbase 数据库。

- 通过 spring.datasource.driverClassName 指定数据库驱动程序为 com.kingbase8.Driver，用于与 Kingbase 数据库建立连接。
- 通过 spring.datasource.url 指定连接数据库的 URL。
- 通过 spring.datasource.username 指定连接数据库的用户名。
- 通过 spring.datasource.password 指定连接数据库的密码。

代码如下:

```
spring.datasource.driverClassName=com.kingbase8.Driver
spring.datasource.url=jdbc:kingbase8://localhost:54321/test
spring.datasource.username=system
spring.datasource.password=******
```

2.2.2.3.2　UserDaoImpl.java 文件介绍

UserDaoImpl.java 文件通过使用 JdbcTemplate 对象执行 SQL 语句，实现了对 User 表的插入、删除、更新和查询操作。

UserDaoImpl.java 文件的代码主要包括以下几个部分：

1. 引用其他类和接口。

   声明当前文件所属的包名为 com.kingbase.testspringboot.dao.impl，包含以下接口和类：

   - UserDao 接口：用于实现了 UserDao 接口中定义的方法。
   - User 类：用于传递和存储用户数据。
   - Autowired 注解：将 JdbcTemplate 对象注入到该类中，用于执行 SQL 语句。
   - BeanPropertyRowMapper 类：用于将数据库查询结果映射为 Java 对象。
   - JdbcTemplate 类：用于执行 SQL 语句和处理数据库访问。
   - Repository 注解：标记 Spring 的仓库组件，用于数据访问。
   - List 接口：用于操作查询结果集合。

   代码如下:

```
import com.kingbase.testspringboot.dao.UserDao;
import com.kingbase.testspringboot.entity.User;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.BeanPropertyRowMapper;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;
```

2. 定义 UserDaoImpl 类。

   通过使用 Repository 注解指定该类的 Bean 名称为 userDao，并使用 JdbcTemplate 对象执行 SQL 语句，实现了对 User 表的插入、删除、更新和查询操作。

   - 插入用户信息。

     通过调用 JdbcTemplate 对象的 insert 方法，将用户的 ID 和姓名插入到数据库中的 test_springboot 表中，并根据插入操作的结果返回相应的布尔值。

     代码如下:

```
@Autowired
    private JdbcTemplate jdbcTemplate; //Jdbc connection tool class
    @Override
    public boolean insertUser(User user) {
        String sql = "insert into test_springboot(id,name)values(?,?)";
        Object[] params = {user.getId(), user.getName()};
        return jdbcTemplate.update(sql, params) > 0;
    }
```

   - 删除用户信息。

     通过调用 JdbcTemplate 对象的 delete 方法，根据用户的 ID 删除数据库中的用户信息，并根据删除操作的结果返回相应的布尔值。

代码如下:

```
@Override
public boolean deleteById(Long id) {
    String sql = "delete from test_springboot where id=?";
    Object[] params = {id};
    return jdbcTemplate.update(sql, params) > 0;
}
```

- 更新用户信息。

  通过调用 JdbcTemplate 对象的 update 方法，将用户的姓名更新到数据库中的 test_springboot 表中指定 ID 的记录，并根据更新操作的结果返回相应的布尔值。

  代码如下:

```
@Override
public boolean updateUser(User user) {
    String sql = "update test_springboot set name=? where id=?";
    Object[] params = {user.getName(), user.getId()};
    return jdbcTemplate.update(sql, params) > 0;
}
```

- 查询用户信息。

  通过使用 JdbcTemplate 对象执行 SQL 查询语句，从名为 test_springboot 的表中查询指定 ID 的用户记录，并使用 BeanPropertyRowMapper 进行结果映射。返回查询结果对象。

  代码如下:

```
@Override
public User selectUserById(Long id) {
    String sql = "select * from test_springboot where id=?";
    Object[] params = new Object[]{id};
    return jdbcTemplate.queryForObject(
            sql,
            params,
            new BeanPropertyRowMapper<>(User.class));
}
```

- 查询所有用户信息

  通过调用 JdbcTemplate 对象的 query 方法，查询数据库中的所有用户信息，并将查询结果映射为 User 类型的对象列表返回。

  代码如下:

```
@Override
public List<User> selectAllUsers() {
```

```
    String sql = "select * from test_springboot";
    return jdbcTemplate.query(sql, new BeanPropertyRowMapper<User>(User.class));
}
```

2.2.2.3.3　UserDao.java 文件介绍

UserDao.java 文件通过 UserDao 接口用于定义对用户数据的操作方法。

UserDao.java 文件的代码主要包括以下几个部分：

1. 引用其他类和接口。

   声明当前文件所属的包名为 package com.kingbase.testspringboot.dao，包含以下接口和类:

   - User 类: 用于传递和存储用户数据。
   - List 接口: 用于操作查询结果集合。

   代码如下:

```
import com.kingbase.testspringboot.entity.User;
import java.util.List;
```

2. 定义 UserDao 接口。

   通过使用 UserDao 接口，定义对用户数据的操作方法。包括插入用户信息、删除用户信息、更新用户信息、根据用户 ID 查询用户信息以及查询所有用户信息等方法。

   代码如下:

```
public interface UserDao {
    boolean insertUser(User user);
    boolean deleteById(Long id);
    boolean updateUser(User user);
    User selectUserById(Long id);
    List<User> selectAllUsers();
}
```

2.2.2.3.4　User.java 文件介绍

User.java 文件通过定义 User 类用于表示用户对象。

1. 声明 User 对象。

   声明 User 类包含 id 和 name 两个私有字段，并提供了一个无参构造方法。通过设置和获取 id 和 name 字段的值来操作用户信息。

   代码如下:

```
private Long id;
private String name;

public User() {
}
```

2. 创建 User 对象。

   定义 User 类的带参构造方法，用于创建一个具有指定 id 和 name 的 User 对象。

   代码如下:

```
public User(Long id, String name) {
    this.id = id;
    this.name = name;
}
```

3. 获取和设置 id 和 name 值。

   定义 User 类中的四个方法，用于获取和设置 id 和 name 属性的值。

   - getId 方法用于获取 id 值。
   - setId 方法用于设置 id 值。
   - getName 方法用于获取用户名 name 的值。
   - setName 方法用于设置用户名 name 的值。

   代码如下:

```
public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}
```

4. 重写 User 类中的 toString 方法，用于返回 User 对象的字符串表示形式。

定义 @Override 覆盖父类中的同名方法。定义 toString 方法，用于返回 User 对象的字符串表示形式。通过字符串拼接，将 id 和 name 属性的值格式化为一个字符串，返回给调用者 User。

代码如下:

```
public String toString() {
    return "User{" +
            "id=" + id +
            ", name='" + name + '\'' +
            '}';
}
```

2.2.2.3.5　TestSpringbootApplication.java 文件介绍

TestSpringbootApplication.java 文件通过 UserDao 接口用于启动 Spring Boot 应用程序。

TestSpringbootApplication.java 文件的代码主要包括以下几个部分：

1. 定义类和接口。

   声明当前文件所属的包名为 com.kingbase.testspringboot，包含以下接口和类：

   - SpringApplication 类：用于启动 Spring Boot 应用程序。
   - @SpringBootApplication 注解：用于指示该类是 Spring Boot 应用程序的入口点。

   代码如下:

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

2. 定义 TestSpringbootApplication 类。

   使用 SpringBootApplication 注解标识该类是一个 Spring Boot 应用程序的入口类。通过调用 SpringApplication 类的 run 方法，可以启动 Spring Boot 应用程序。

   代码如下:

```
@SpringBootApplication
public class TestSpringbootApplication {

    public static void main(String[] args) {
        SpringApplication.run(TestSpringbootApplication.class, args);
    }
}
```

2.2.2.3.6　TestSpringbootApplicationTests.java 文件介绍

TestSpringbootApplicationTests.java 文件通过 UserDao 接口用于启动 Spring Boot 应用程序。

TestSpringbootApplicationTests.java 文件的代码主要包括以下几个部分：

1. 引用其他类和接口。

   声明当前文件所属的包名为 com.kingbase.testspringboot，包含以下接口和类：

   - UserDao 接口：用于实现了 UserDao 接口中定义的方法。
   - User 类：用于传递和存储用户数据。
   - Test 注解：用于标识测试方法。
   - Autowired 注解：将 JdbcTemplate 对象注入到该类中，用于执行 SQL 语句。
   - SpringBootTest 注解：用于标识该类是一个 Spring Boot 的测试类。
   - JdbcTemplate 类：用于执行 SQL 语句和处理数据库访问。
   - List 接口：用于操作查询结果集合。

   代码如下:

```
import com.kingbase.testspringboot.dao.UserDao;
import com.kingbase.testspringboot.entity.User;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.jdbc.core.JdbcTemplate;
import java.util.List;
```

2. 定义 TestSpringbootApplicationTests 类。

   使用 SpringBootTest 注解标识测试 Spring Boot 应用程序中的功能。通过使用 UserDao 和 JdbcTemplate 对象，对用户数据进行增删改查操作，并输出结果。

   - 定义对象。

     使用 Autowired 注解自动注入 UserDao 和 JdbcTemplate 对象。

     代码如下:

```
@Autowired
private UserDao userDao;
@Autowired
private JdbcTemplate jdbcTemplate;
```

   - 定义 contextLoads 方法。

     使用 contextLoads 方法作为测试方法的具体实现。

   - 删除 test_springboot 表。

     使用 jdbcTemplate.execute 方法执行 SQL 语句 drop table test_springboot，删除名为 test_springboot 的表。

代码如下:

```
try {
jdbcTemplate.execute("drop table test_springboot");
System.out.println("test_springboot delete successfully!");
}
```

- 创建 test_springboot 表。

  使用 jdbcTemplate.execute 方法执行 SQL 语句 create table test_springboot (id int primary key, name varchar(50))，创建名为 test_springboot 的表，包含 id 和 name 两个字段。

  代码如下:

```
catch (Exception ignore) {

    } finally {
        jdbcTemplate.execute("create table test_springboot (" +
                "id int primary key," +
                "name varchar(50))");
        System.out.println("test_springboot create successfully!");
    }
```

- 插入数据。

  使用 userDao.insertUser 方法向 test_springboot 表中插入 10 条用户数据，id 从 1 到 10，name 为“insert”+ i。

  代码如下:

```
for (int i = 1; i <= 10; i++) {
userDao.insertUser(new User((long) i, "insert" + i));
}
```

- 删除数据。

  使用 userDao.deleteById 方法删除 id 为 1 的用户数据。

  代码如下:

```
userDao.deleteById(1L);
```

- 更新数据。

  使用 userDao.updateUser 方法更新 id 为 2 的用户数据，将 name 修改为 update。

  代码如下:

```
userDao.updateUser(new User(2L, "update"));
```

- 查询数据。

  使用 userDao.selectUserById 方法查询 id 为 2 的用户数据，并将结果打印输出。

  代码如下:

```
User user = userDao.selectUserById(2L);
System.out.println("user = " + user);
```

- 查询所有数据。

  使用 userDao.selectAllUsers 方法查询 test_springboot 表中的所有用户数据，并将结果打印输出。代码如下:

```
List<User> userList = userDao.selectAllUsers();
userList.forEach(System.out::println);
```

2.2.2.3.7　完整的代码展示

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.7.11</version>
<relativePath/> <!-- lookup parent from repository -->
</parent>
<groupId>com.kingbase</groupId>
<artifactId>java-kingbase-springboot</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>java-kingbase-springboot</name>
<description>Demo project for Spring Boot</description>
<properties>
<java.version>1.8</java.version>
</properties>
<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter</artifactId>
</dependency>
<dependency>
<groupId>cn.com.kingbase</groupId>
```

```
<artifactId>kingbase8</artifactId>
<version>9.0.0</version>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-test</artifactId>
<scope>test</scope>
</dependency>
<dependency>
<groupId>org.slf4j</groupId>
<artifactId>slf4j-log4j12</artifactId>
</dependency>
<dependency>
<groupId>org.junit.platform</groupId>
<artifactId>junit-platform-launcher</artifactId>
<version>1.8.2</version>
<scope>test</scope>
</dependency>
</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

application.properties

```
spring.datasource.driverClassName=com.kingbase8.Driver
spring.datasource.url=jdbc:kingbase8://localhost:54321/test
spring.datasource.username=system
spring.datasource.password=******
```

UserDaoImpl.java

```
package com.kingbase.testspringboot.dao.impl;
@Repository("userDao")
public class UserDaoImpl implements UserDao {
@Autowired
private JdbcTemplate jdbcTemplate; //Jdbc connection tool class
@Override
public boolean insertUser(User user) {
String sql = "insert into test_springboot(id,name)values(?,?)";
Object[] params = {user.getId(), user.getName()};
return jdbcTemplate.update(sql, params) > 0;
}
@Override
public boolean deleteById(Long id) {
String sql = "delete from test_springboot where id=?";
Object[] params = {id};
return jdbcTemplate.update(sql, params) > 0;
}
@Override
public boolean updateUser(User user) {
String sql = "update test_springboot set name=? where id=?";
Object[] params = {user.getName(), user.getId()};
return jdbcTemplate.update(sql, params) > 0;
}
@Override
public User selectUserById(Long id) {
String sql = "select * from test_springboot where id=?";
Object[] params = new Object[]{id};
return jdbcTemplate.queryForObject(
sql,
params,
new BeanPropertyRowMapper<>(User.class));
}
@Override
public List<User> selectAllUsers() {
String sql = "select * from test_springboot";
return jdbcTemplate.query(sql, new BeanPropertyRowMapper<User>(User.class));
}
}
```

UserDao.java

```
package com.kingbase.testspringboot.dao;
public interface UserDao {
boolean insertUser(User user);
boolean deleteById(Long id);
boolean updateUser(User user);
User selectUserById(Long id);
List<User> selectAllUsers();
}
```

User.java

```
package com.kingbase.testspringboot.entity;
public class User {
private Long id;
private String name;
public User() {
}
public User(Long id, String name) {
this.id = id;
this.name = name;
}
public Long getId() {
return id;
}
public void setId(Long id) {
this.id = id;
}
public String getName() {
return name;
}
public void setName(String name) {
this.name = name;
}
@Override
public String toString() {
return "User{" +
"id=" + id +
", name='" + name + '\'' +
'}';
}
}
```

TestSpringbootApplication.java

```
package com.kingbase.testspringboot;
@SpringBootApplication
public class TestSpringbootApplication {
public static void main(String[] args) {
SpringApplication.run(TestSpringbootApplication.class, args);
}
}
```

TestSpringbootApplicationTests.java

```
package com.kingbase.testspringboot;
@SpringBootTest(classes = TestSpringbootApplication.class)
class TestSpringbootApplicationTests {
@Autowired
private UserDao userDao;
@Autowired
private JdbcTemplate jdbcTemplate; //Jdbc connection tool class
@Test
void contextLoads() {
try {
// Use the execute() method to execute SQL statements and delete the user table test_springboot
jdbcTemplate.execute("drop table test_springboot");
System.out.println("test_springboot delete successfully!");
} catch (Exception ignore) {
} finally {
// Use the execute() method to execute SQL statements and create user table tests_ user
jdbcTemplate.execute("create table test_springboot (" +
"id int primary key," +
"name varchar(50))");
System.out.println("test_springboot create successfully!");
}
//UserDao userDao=new UserDaoImpl();
//ApplicationContext ioc=new ApplicationContext("/appli");`
//add
for (int i = 1; i <= 10; i++) {
userDao.insertUser(new User((long) i, "insert" + i));
}
//delete
userDao.deleteById(1L);
//update
userDao.updateUser(new User(2L, "update"));
//selectUserById
```

```
User user = userDao.selectUserById(2L);
System.out.println("user = " + user);
//query all users
List<User> userList = userDao.selectAllUsers();
userList.forEach(System.out::println);
}
}
```

Jdbc 更多示例请参考 Jdbc 产品手册的程序示例。

2.2.3　SpringBatch

本文将介绍如何使用 SpringBatch 框架和 Kingbase 数据库构建一个应用程序，实现创建表、插入数据和查询数据等基本操作。

点击下载 java-kingbase-springbatch 示例工程。

2.2.3.1　前提条件

- 您已安装 Kingbase 数据库。
- 您已安装 JDK 1.8 和 Maven。
- 您已安装 IDE 工具。

备注:

本文档运行代码使用的工具是 IntelliJ IDEA 2023.2.8 (Community Edition) 版本，您也可以根据个人喜好选择适合自己的工具运行示例代码。

2.2.3.2　操作步骤

1. 获取 Kingbase 数据库连接信息。
2. 导入 java-kingbase-springbatch 项目到 IDEA 中。
3. 修改 java-kingbase-springbatch 项目中的数据库连接信息。
4. 运行 java-kingbase-springbatch 项目。

备注:

本文中给出的操作步骤是基于 Windows 环境生成。如果您使用的是其他操作系统环境或编译器，那么操作步骤可能会略有不同。

2.2.3.2.1　步骤一：获取 Kingbase 数据库连接信息

1. 联系 Kingbase 数据库部署人员或者管理员获取相应的数据库连接信息。

```
./ksql -U system -d test -h xx.xx.xx.xx -p 54321
```

2. 根据已部署的 Kingbase 数据库填写下面的 URL 的对应信息

备注:

在 application.properties 文件中需要这里的 URL 信息。

```
jdbc:kingbase8://host:port/database?user=$user_name&password=$password
```

参数说明：

- host：提供 Kingbase 数据库连接 IP。直连方式使用的是 KESServer 节点的 IP 地址。
- port：提供 Kingbase 数据库连接端口。直连方式默认是 54321，在部署 kingbase 数据库时可自定义。
- database：需要访问的数据库名称。
- user_name：通过 -u 参数指定，管理员用户为 system。
- password：提供账户密码。更多 URL 参数说明信息请参考 Jdbc 连接属性。

2.2.3.2.2　步骤二: 导入 java-kingbase-springbatch 项目到 IDEA 中

1. 打开 IntelliJ IDEA，选择 File > Open…选项。
2. 在弹出的 Open File or Project 窗口中，选择对应的项目文件，单击 OK 完成项目文件导入。
3. IntelliJ IDEA 将会自动识别项目中的各类文件，并在 Project 工具窗口中，可以查看项目的目录结构、文件列表、模块列表、依赖关系等信息。Project 工具窗口通常位于 IntelliJ IDEA 界面的最左侧，默认情况下是打开的。如果 Project 工具窗口被关闭了，可以通过点击菜单栏中的 View > Tool Windows > Project 或者使用快捷键 Alt + 1 来重新打开它。

   备注:

   当使用 IntelliJ IDEA 导入项目时，IntelliJ IDEA 会自动检测项目中的 pom.xml 文件，并根据文件中描述的依赖关系自动下载所需的依赖库，并将它们添加到项目中。

2.2.3.2.3　步骤三: 修改 java-kingbase-springbatch 项目中的数据库连接信息

根据步骤一：获取 Kingbase 数据库连接信息中的信息修改 application.properties 文件中的数据库连接信息。示例如下

- 数据库驱动名称为 `com.kingbase8.Driver`。
- KES Server 节点的 IP 地址为 `192.168.xx.xxx`。
- 访问端口使用的是 `7901`。
- 需要访问的 `database` 名称为 `test`。
- 账户是 `system`。
- 密码是 `******`。

配置示例如下:

```
spring.datasource.driverClassName=com.kingbase8.Driver
spring.datasource.url=jdbc:kingbase8://192.168.xx.xxx:7901/test
spring.datasource.username=system
spring.datasource.password=******

# JPA
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.Kingbase8Dialect

# Spring Batch
spring.batch.job.enabled=false

# Logger
logging.level.org.springframework=INFO
logging.level.com.example=DEBUG
```

2.2.3.2.4　步骤四: 导入方言包到项目

执行路径:

1. 在导航栏中选择文件 > 项目结构 > 项目设置 > 库。
2. 点击库上方的 + 号，选中 java，选择 java-kingbase-springbatch>lib 目录下的方言包 hibernate-4.dialect.jar，即可完成方言包的加载。

2.2.3.2.5　步骤五: 运行 java-kingbase-springbatch 项目

运行路径:

1. 在项目结构中找到 src > test > java 中找到 AddDescPeopleWriterTest.java 文件。

2. 在工具菜单栏中选择运行 (U) > 运行 > AddDescPeopleWriterTest，或直接单击右上角绿色三角形运行（快捷键 Shift+F10）。
3. 通过 IDEA 的控制台来查看项目的日志信息和输出结果。

   运行结果

```
people_desc 表中的数据:
PeopleDESC [name=John, age=25, gender=male, desc=This is John with age 25]
PeopleDESC [name=Alice, age=30, gender=female, desc=This is Alice with age 30]
PeopleDESC [name=Lucy, age=26, gender=female, desc=This is Lucy with age 26]
Batch Job execution completed.
```

2.2.3.3　项目代码介绍

点击下载 java-kingbase-springbatch，这是一个名为 java-kingbase-springbatch 的压缩包。解压后，得到一个名为 java-kingbase-springbatch 的文件夹。目录结构如下所示：

```
  pom.xml

 src
   main
     java
       com
           kingbase
               example
                   batch
                       BatchApplication.java

                       config
                          BatchConfig.java

                       model
                          People.java
                          PeopleDESC.java

                       processor
                          AddPeopleDescProcessor.java

                       writer
                           AddDescPeopleWriter.java
                           AddPeopleWriter.java

     resources
         application.properties
```

```
  test
      java
          com
              kingbase
                  example
                      batch
                          config
                              BatchConfigTest.java

                          processor
                              AddPeopleDescProcessorTest.java

                          writer
                              AddDescPeopleWriterTest.java
                              AddPeopleWriterTest.java

target

lib
   hibernate-4.dialect.jar
```

文件说明:

- pom.xml：Maven 项目的配置文件，包含了项目的依赖、插件、构建等信息。
- src：通常用于表示项目中存放源代码的目录。
- main: 存放主要的源代码和资源文件的目录。
- java: 存放 Java 源代码的目录。
- com.kingbase.example.batch: 包名。
- BatchApplication.java：这是应用程序的入口类，包含了应用程序的主方法。
- config：这是配置类文件夹，包含了应用程序的配置类。
- BatchConfig.java：这是应用程序的配置类，用于配置应用程序的一些属性和行为。
- model：这是模型类文件夹，包含了应用程序的数据模型类。
- People.java：这是一个人员数据模型类。
- PeopleDESC.java：这是一个人员 DESC 数据模型类。
- processor：这是处理器类文件夹，包含了应用程序的处理器类。
- AddPeopleDescProcessor.java：这是一个添加人员 DESC 信息的处理器类。
- writer：这是写入器类文件夹，包含了应用程序的写入器类。
- AddDescPeopleWriter.java：这是一个写入人员 DESC 信息的写入器类。

- AddPeopleWriter.java：这是一个写入人员信息的写入器类。
- resources：这是资源文件夹，包含了应用程序的配置文件和其他静态资源文件。
- application.properties：这是应用程序的配置文件，用于配置应用程序的属性。
- test: 存放测试代码和资源文件的目录。
- BatchConfigTest.java：这是应用程序配置类的测试类。
- AddPeopleDescProcessorTest.java：这是添加人员 DESC 处理器的测试类。
- AddDescPeopleWriterTest.java：这是写入人员 DESC 信息的写入器的测试类。
- AddPeopleWriterTest.java：这是写入人员信息的写入器的测试类。
- target: 存放编译后的 Class 文件、Jar 包等文件的目录。
- lib：这是一个方言包文件夹
- hibernate-4.dialect.jar：这是一个方言包

2.2.3.3.1　pom.xml 代码介绍

备注:

如果您只是想验证示例，那么请使用默认代码，无需修改。您也可以按照以下讲解，根据自己的需求修改 pom.xml 文件。

---

pom.xml 配置文件内容如下

1. 文件声明语句。声明本文件是一个 XML 文件，使用的 XML 版本是 1.0，字符编码方式是 UTF-8。代码如下:

```
<?xml version="1.0" encoding="UTF-8"?>
```

2. 配置 POM 的命名空间和 POM 模型版本。

- 通过 xmlns 指定 POM 的命名空间为 `http://maven.apache.org/POM/4.0.0`。
- 通过 xmlns:xsi 指定 XML 命名空间为 `http://www.w3.org/2001/XMLSchema-instance`。
- 通过 xsi:schemaLocation 指定 POM 的命名空间为 `http://maven.apache.org/POM/4.0.0`，并指定 POM 的 XSD 文件的位置为 `https://maven.apache.org/xsd/maven-4.0.0.xsd`。
- 通过 modelVersion 元素指定该 POM 文件使用的 POM 模型版本为 `4.0.0`。

代码如下:

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
 <modelVersion>4.0.0</modelVersion>
</project>
```

3. 配置父项信息。

- 通过 groupId 指定父项标识为 org.springframework.boot。
- 通过 artifactId 指定父项依赖为 spring-boot-starter-parent。
- 通过 version 指定父项的版本号为 2.7.11。
- 通过 relativePath 表示父项的路径为空。

代码如下:

```
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>2.7.11</version>
        <relativePath/>
```

4. 配置基本信息。

- 通过 groupId 指定项目标识为 com.kingbase。
- 通过 artifactId 指定项目依赖为 java-kingbase-springboot。
- 通过 version 指定项目的版本号为 0.0.1-SNAPSHOT。
- 通过 description 介绍项目信息为 Demo project for Spring Batch。代码如下:

```
        <groupId>com.kingbase</groupId>
        <artifactId>java-kingbase-springboot</artifactId>
        <version>0.0.1-SNAPSHOT</version>
        <name>java-kingbase-springbatch</name>
        <description>Demo project for Spring Batch</description>
```

5. 配置 java 版本。

指定项目使用的 Java 版本为 1.8。

代码如下：

```
<properties>
    <java.version>1.8</java.version>
</properties>
```

6. 配置核心依赖。

- 指定依赖项所属的组织为 org.springframework.boot，名称为 spring-boot-starter，该依赖库包含了 Spring Boot 默认支持的组件依赖，支持 Web、数据处理、安全、Test 等功能。

代码如下:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter</artifactId>
</dependency>
```

- 指定依赖项所属的组织为 org.springframework.boot，名称为 spring-boot-starter-jdbc，通过该依赖可以使用 Spring Boot 提供的 JDBC 相关功能，如连接池、数据源配置等。

  代码如下:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
```

- 指定依赖项所属的组织为 org.springframework.boot，名称为 spring-boot-starter-test，作用范围为 test，通过该依赖可以使用 Spring Boot 提供的测试框架和工具，如 JUnit、Mockito、Hamcrest 等。

  代码如下:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

- 指定依赖项所属的组织为 cn.com.kingbase，名称为 kingbase8，版本号为 9.0.0，通过该依赖可以使用 Kingbase 提供的客户端功能，如连接、查询、事务等。

  代码如下：

```
<dependency>
    <groupId>cn.com.kingbase</groupId>
    <artifactId>kingbase8</artifactId>
    <version>9.0.0</version>
</dependency>
```

7. 配置 Maven 插件。指定依赖项所属的组织为 org.springframework.boot，名称为 spring-boot-maven-plugin，该插件用于将 Spring Boot 应用程序打包成可执行的 JAR 包或 WAR 包，并且可以直接运行。

   代码如下:

```
<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
```

```
</build>
```

2.2.3.3.2　application.properties 文件介绍

application.properties 文件用于配置数据库连接和其他相关的配置项。如数据库驱动程序、连接 URL、用户名和密码等。还包括了一些关于 JPA（Java Persistence API）和 Spring Batch 的配置，以及日志级别的设置。

1. 数据库连接配置：

- 通过 spring.datasource.driverClassName 指定数据库驱动程序为 com.kingbase8.Driver，用于与 Kingbase 数据库建立连接。
- 通过 spring.datasource.url 指定连接数据库的 URL。
- 通过 spring.datasource.username 指定连接数据库的用户名。
- 通过 spring.datasource.password 指定连接数据库的密码。

代码如下:

```
spring.datasource.driverClassName=com.kingbase8.Driver
spring.datasource.url=jdbc:kingbase8://192.168.xx.xxx:7901/test
spring.datasource.username=system
spring.datasource.password=******
```

2. JPA 配置:

- 通过 spring.jpa.show-sql 指定是否在日志中显示 SQL 语句，设置为 true，表示显示 SQL 语句。
- 通过 spring.jpa.hibernate.ddl-auto 指定 Hibernate 的 DDL 操作行为，这里设置为 update，表示在应用程序启动时自动更新数据库结构。
- 通过 spring.jpa.properties.hibernate.dialect 指定 hibernate 应该使用的方言 dialect，这里设置为 org.hibernate.dialect.Kingbase8Dialect，表示 hibernate 将生成与 Kingbase8 相兼容的 SQL 语句。

代码如下:

```
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.Kingbase8Dialect
```

3. Spring Batch 配置：

- 通过 spring.batch.job.enabled 指定是否启用 Spring Batch 作业，这里设置为 false ，表示禁用 Spring Batch 作业。

代码如下：

```
spring.batch.job.enabled=false
```

4. 日志配置：

- 通过 logging.level.org.springframework 指定 Spring 框架的日志级别为 INFO。
- 通过 logging.level.com.example 指定应用程序自定义代码的日志级别为 DEBUG。

代码如下：

```
logging.level.org.springframework=INFO
logging.level.com.example=DEBUG
```

2.2.3.3.3　BatchApplication.java 文件介绍

BatchApplication.java 文件是 Spring Boot 应用程序的入口文件。BatchApplication.java 文件的代码主要包括以下几个部分：

1. 引用其他类和接口。声明当前文件包含以下接口和类：

- SpringApplication 类：用于启动 Spring Boot 应用程序。
- SpringBootApplication 注解：用于标记该类为 Spring Boot 应用程序的入口。

代码如下：

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

2. 定义 BatchApplication 类。

使用 @SpringBootApplication 注解标记 BatchApplication 类为 Spring Boot 应用程序的入口。在 BatchApplication 类定义了一个静态的 main 方法作为应用程序的入口点。在该方法中，使用 SpringApplication.run 方法启动 Spring Boot 应用程序。再定义了一个名为 runBatchJob 的方法，用于运行批处理作业。代码如下：

```
@SpringBootApplication
public class BatchApplication {
    public static void main(String[] args) {
        SpringApplication.run(BatchApplication.class, args);
    }

    public void runBatchJob() {
    }
}
```

2.2.3.3.4　BatchConfig.java 文件介绍

BatchConfig.java 文件用于配置批处理作业的步骤、读取器、处理器和写入器等组件。BatchConfig.java 文件的代码主要包括以下几个部分：

1. 引用其他类和接口。声明当前文件包含以下接口和类：

- People 类：用于存储从数据库中读取的人员信息。
- PeopleDESC 类：用于存储对人员信息进行转换或处理后的描述信息。
- AddPeopleDescProcessor 类：将读取到的 People 对象转换为 PeopleDESC 对象，
- ItemProcessor 接口的实现类。
- AddDescPeopleWriter 类：将 PeopleDESC 对象写入到目标位置，ItemWriter 接口的实现类。
- Job 接口：表示一个批处理作业。
- Step 接口：表示作业中的一个步骤。
- EnableBatchProcessing 注解：Spring Batch 的配置注解，用于启用和配置 Spring Batch 处理功能。
- JobBuilderFactory 类：用于创建和配置作业。
- StepBuilderFactory 类：用于创建和配置步骤。
- RunIdIncrementer 类：Spring Batch 的运行 ID（Run ID）自增器，用于在每次运行作业时增加运行的 ID。
- ItemProcessor 接口：用于对读取到的项进行处理或转换。
- ItemReader 接口：用于从数据源中读取项。
- ItemWriter 接口：用于将处理或转换后的项写入到指定的目标位置。
- JdbcCursorItemReader 类：用于从数据库中读取数据并返回游标结果集。
- Autowired 注解：用于进行依赖注入。
- Bean 注解：用于创建和配置 Bean。
- ComponentScan 注解：用于指定要进行组件扫描的包或类。
- Configuration 注解：用于将类标记为配置类。
- EnableAutoConfiguration 注解：用于启用 Spring Boot 的自动配置。
- SpringBootApplication 注解：用于标记该类为 Spring Boot 应用程序的入口。
- DataSource 接口：用于表示数据库的连接

代码如下：

```
import com.kingbase.example.batch.model.People;
import com.kingbase.example.batch.model.PeopleDESC;
import com.kingbase.example.batch.processor.AddPeopleDescProcessor;
import com.kingbase.example.batch.writer.AddDescPeopleWriter;
import org.springframework.batch.core.Job;
import org.springframework.batch.core.Step;
import org.springframework.batch.core.configuration.annotation.EnableBatchProcessing;
import org.springframework.batch.core.configuration.annotation.JobBuilderFactory;
import org.springframework.batch.core.configuration.annotation.StepBuilderFactory;
import org.springframework.batch.core.launch.support.RunIdIncrementer;
import org.springframework.batch.item.ItemProcessor;
```

```
import org.springframework.batch.item.ItemReader;
import org.springframework.batch.item.ItemWriter;
import org.springframework.batch.item.database.JdbcCursorItemReader;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.autoconfigure.EnableAutoConfiguration;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.BeanPropertyRowMapper;

import javax.sql.DataSource;
```

2. 定义 BatchConfig 类。这是一个简单的 Spring Batch 批处理作业。它定义了数据的读取、处理和写入方式，并将这些步骤封装为一个作业。使用 Spring Batch 的注解和自动配置功能，通过配置类中的各个 @Bean 方法，可以创建相应的组件实例，并在 step1 中使用这些组件完成数据的读取、处理和写入。

- 通过 @Configuration 表示这个类是一个配置类。
- 通过 @EnableBatchProcessing 启用 Spring Batch 的处理功能，该注解会自动创建必要的 bean，如 JobRepository、JobLauncher 等。
- 通过 @SpringBootApplication 用于 Spring Boot 应用的主类注解，是 Spring Boot 应用的启动点。
- 通过 @ComponentScan 指定要进行组件扫描的包，告诉 Spring 在这个包和子包中扫描和注册所有的组件。
- 通过 @EnableAutoConfiguration 自动配置 Spring Boot 应用程序的基础设施。

代码如下：

```
@Configuration
@EnableBatchProcessing
@SpringBootApplication
@ComponentScan("com.kingbase.example.batch.writer")
@EnableAutoConfiguration
public class BatchConfig {
}
```

a. 定义 @Autowired 注解。

通过 @Autowired 注解将 JobBuilderFactory 、StepBuilderFactory 和 DataSource 注入到 BatchConfig 类中的成员变量中。JobBuilderFactory 用于创建和配置作业（Job）的工厂类，StepBuilderFactory 用于创建和配置步骤（Step）的工厂类，DataSource 用于获取数据库连接的接口。

代码如下：

```
    @Autowired
    private JobBuilderFactory jobBuilderFactory;
```

```
    @Autowired
    private StepBuilderFactory stepBuilderFactory;

    @Autowired
    private DataSource dataSource;// 使用 Spring Boot 自动配置提供的默认 dataSource
```

b. 定义 @Bean 注解。

使用 @Bean 注解定义了几个方法，用于创建批处理作业的读取器、处理器、写入器、步骤和作业。

- 通过 peopleReader 方法创建一个 ItemReader 组件实例，该组件使用 JdbcCursorItemReader 来读取数据库中的 People 对象数据。设置数据源 dataSource 、设置 RowMapper 来将数据库行映射为 People 对象，设置 SQL 查询语句为 SELECT * FROM people 。
- 通过 addPeopleDescProcessor 方法创建一个 ItemProcessor 组件实例，该组件使用 AddPeopleDescProcessor 来处理 People 对象，返回转换为 PeopleDESC 对象。
- 通过 addDescPeopleWriter 方法创建一个 ItemWriter 组件实例，该组件使用 AddDescPeopleWriter 来将 PeopleDESC 对象写入到目标位置。
- 通过 step1 方法创建一个 Step 组件实例，该步骤的名称为 step1 ，通过 stepBuilderFactory.get 获取步骤构建器，设置读取器为 ItemReader 组件，设置处理器为 ItemProcessor 组件，设置写入器为 ItemWriter 组件，设置 chunk 大小为 10 ，最后调用 build 构建并返回配置完成的 Step 。
- 通过 importJob 方法创建一个 Job 组件实例，该作业的名称为 importJob ，通过 jobBuilderFactory.get 获取作业构建器，设置增量器为 RunIdIncrementer，设定作业 flow 的最初步骤为 Step ，最后调用 build 构建并返回配置完成的 job 。

代码如下：

```
@Bean
    public ItemReader<People> peopleReader() {
        JdbcCursorItemReader<People> reader = new JdbcCursorItemReader<>();
        reader.setDataSource((javax.sql.DataSource) dataSource);
        reader.setRowMapper(new BeanPropertyRowMapper<>(People.class));
        reader.setSql("SELECT * FROM people");
        return reader;
    }

    @Bean
    public ItemProcessor<People, PeopleDESC> addPeopleDescProcessor() {
        return new AddPeopleDescProcessor();
    }

//    @Bean
    public ItemWriter<PeopleDESC> addDescPeopleWriter() {
        return new AddDescPeopleWriter();
    }
```

```
    @Bean
    public Step step1(ItemReader<People> reader, ItemProcessor<People, PeopleDESC> processor,
                      ItemWriter<PeopleDESC> writer) {
        return stepBuilderFactory.get("step1")
                .<People, PeopleDESC>chunk(10)
                .reader(reader)
                .processor(processor)
                .writer(writer)
                .build();
    }

    @Bean
    public Job importJob(Step step1) {
        return jobBuilderFactory.get("importJob")
                .incrementer(new RunIdIncrementer())
                .flow(step1)
                .end()
                .build();
    }
```

2.2.3.3.5　People.java 文件介绍

People.java 文件是创建一个 People 类的数据模型，表示一个人的信息。该类包含了三个私有成员变量 name，age 以及 gender ，以及相应的 getter 和 setter 方法。最后重写 toString 方法用于打印对象的信息。其中 name 表示人的姓名，age 表示人的年龄，gender 表示人的性别。通过 getter 和 setter 方法可以获取和设置这些属性的值。该类的作用是为批处理程序的输入和输出提供一种存储和传递数据的方式。在批处理的读取和写入操作中，使用 People 对象来存储数据，通过 setter 方法设置数据，通过 getter 方法获取数据。

代码如下：

```
public class People {
    private String name;
    private int age;
    private String gender;

    // getters and setters

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
```

```
    public String getGender() {
        return gender;
    }
    public void setGender(String gender) {
        this.gender = gender;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }

    @Override
    public String toString() {
        return "People [name=" + name + ", age=" + age + ", gender="  + gender +"]";
    }
    // Getters and setters
}
```

2.2.3.3.6　PeopleDESC.java 文件介绍

PeopleDESC.java 文件是创建一个 PeopleDESC 类的数据模型，用于表示人员的信息，PeopleDESC 类有五个属性：name 、age 、desc 、id 和 gender ，分别表示人的姓名、年龄、描述、标识和性别。该类包含了相应的 getter 和 setter 方法来访问和设置属性的值。重写 toString 方法用于返回类的字符串表示形式，包含姓名、年龄和描述。

与 People 类类似，PeopleDESC 类也用于在批处理程序的输入和输出中存储和传递数据。代码如下：

```
public class PeopleDESC {
    private String name;
    private int age;
    private String desc;
    private int id;
    private String gender ;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
```

```
    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }

    public String getDesc() {
        return desc;
    }

    public void setDesc(String desc) {
        this.desc = desc;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getGender() {
        return gender;
    }

    public void setGender(String gender) {
        this.gender = gender;
    }

    @Override
    public String toString() {
        return "PeopleDESC [name=" + name + ", age=" + age+ ", gender=" + gender + ", desc=" + desc + "]";
    }
}
```

2.2.3.3.7　AddPeopleDescProcessor.java 文件介绍

AddPeopleDescProcessor.java 文件定义一个名为 AddPeopleDescProcessor 的类，实现 ItemProcessor 接口，用于将 People 对象转换为 PeopleDESC 对象。AddPeopleDescProcessor.java 文件的代码主要包括以下几个部分：

1. 引用其他类和接口。声明当前文件包含以下接口和类：

- People 类：用于存储从数据库中读取的人员信息。
- PeopleDESC 类：用于存储对人员信息进行转换或处理后的描述信息。
- ItemProcessor 接口：用于对读取到的项进行处理或转换。

代码如下：

```
import com.kingbase.example.batch.model.People;
import com.kingbase.example.batch.model.PeopleDESC;
import org.springframework.batch.item.ItemProcessor;
```

2. 定义 AddPeopleDescProcessor 类。

ItemProcessor 接口的 AddPeopleDescProcessor 类用于将 People 对象转换为 PeopleDESC 对象，实现在批处理过程中对输入的数据的处理逻辑。在该类的 process 方法中，首先创建一个 PeopleDESC 对象 desc ，然后通过 item 参数获取 People 对象的属性（name 和 age ），将这些属性设置到 desc 对象中。同时，也为 desc 对象的 desc 属性赋值，赋值的逻辑为根据 People 对象的属性生成一段描述信息。最后，返回通过处理后的 PeopleDESC 对象。

代码如下：

```
public class AddPeopleDescProcessor implements ItemProcessor<People, PeopleDESC> {
    @Override
    public PeopleDESC process(People item) throws Exception {
        PeopleDESC desc = new PeopleDESC();
        desc.setName(item.getName());
        desc.setAge(item.getAge());
        desc.setGender(item.getGender());
        desc.setDesc("This is " + item.getName() + " with age " + item.getAge() + "and gender is"  + item.
getGender());
        return desc;
    }
}
```

2.2.3.3.8　AddDescPeopleWriter.java 文件介绍

AddDescPeopleWriter.java 文件实现了 ItemWriter 接口的 AddDescPeopleWriter 类，用于将 People 对象写入到数据库。AddDescPeopleWriter.java 文件的代码主要包括以下几个部分：

1. 引用其他类和接口。

声明当前文件包含以下接口和类：

- PeopleDESC 类：用于存储对人员信息进行转换或处理后的描述信息。
- ItemWriter 接口：用于将处理或转换后的项写入到指定的目标位置。
- Autowired 注解：用于进行依赖注入。
- JdbcTemplate 类：提供执行 SQL 语句的方法。

- List 接口：用于操作查询结果集合。

代码如下：

```
import com.kingbase.example.batch.model.PeopleDESC;
import org.springframework.batch.item.ItemWriter;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;

import java.util.List;
```

2. 定义 AddDescPeopleWriter 类。

a. 使用 @Autowired 注解将 JdbcTemplate 实例自动注入进来，在写入数据时使用该实例执行数据库操作。

代码如下：

```
@Autowired
private JdbcTemplate jdbcTemplate;
```

b. 在 write 方法中，对传入的 List< ? extends PeopleDESC> 进行遍历，依次取出每个 PeopleDESC 对象。首先，执行 SQL 语句 DROP TABLE people_desc ，用于删除可能已存在的名为 people_desc 的表。然后，执行 SQL 语句 CREATE TABLE people_desc (id INT, name VARCHAR(255), age INT, gender VARCHAR(255), description VARCHAR(255)) ，用于创建一个名为 people_desc 的表，表中包含了 id 、name 、age 、gender 和 description 五个列。接下来，使用 SQL 语句 INSERT INTO people_desc (id, name, age, gender，description) VALUES (?, ?, ?, ?, ?) ，将每个 PeopleDESC 对象的属性值分别插入到 people_desc 表中。

代码如下：

```
@Override
    public void write(List<? extends PeopleDESC> items) throws Exception {
        // 先删除可能存在的表
        jdbcTemplate.execute("DROP TABLE IF EXISTS people_desc");
        // 建表语句
        String createTableSql = "CREATE TABLE people_desc (id INT, name VARCHAR(255), age INT, gender
VARCHAR(255), description VARCHAR(255))";
        jdbcTemplate.execute(createTableSql);
        for (PeopleDESC item : items) {
            String sql = "INSERT INTO people_desc (id, name, age, gender,description) VALUES (?, ?, ? ,?, ?)
";
            jdbcTemplate.update(sql, item.getId(), item.getName(), item.getAge(), item.getGender(), item.
getDesc());
        }
    }
```

2.2.3.3.9　AddPeopleWriter.java 文件介绍

AddPeopleWriter.java 文件实现了 ItemWriter 接口的 AddDescPeopleWriter 类，用于将 PeopleDESC 对象写入到数据库

AddPeopleWriter.java 文件的代码主要包括以下几个部分：

1. 引用其他类和接口。

   声明当前文件包含以下接口和类：

- People 类：用于存储从数据库中读取的人员信息。
- ItemWriter 接口：用于将处理或转换后的项写入到指定的目标位置。
- Autowired 注解：用于进行依赖注入。
- JdbcTemplate 类：提供执行 SQL 语句的方法。
- Component 注解：用于将该类标记为 Spring 组件。
- List 接口：用于操作查询结果集合。

代码如下：

```
import com.kingbase.example.batch.model.People;
import org.springframework.batch.item.ItemWriter;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;

import java.util.List;
```

2. 定义 AddPeopleWriter 类。

   a. 使用 @Autowired 注解将 JdbcTemplate 实例自动注入进来，在写入数据时使用该实例执行数据库操作。

代码如下：

```
@Autowired
private JdbcTemplate jdbcTemplate;
```

b. 在 write 方法中，对传入的 List< ? extends People> 进行遍历，依次取出每个 People 对象。首先，执行 SQL 语句 DROP TABLE people ，用于删除可能已存在的名为 people 的表。然后，执行 SQL 语句 CREATE TABLE people (name VARCHAR(255), age INT, gender varchar(255))，用于创建一个名为 people 的表，表中包含了 name、age 和 gender 三个列。接下来，使用 SQL 语句 INSERT INTO people (name, age,gender) VALUES (?, ?, ?) ，将每个 People 对象的属性值分别插入到 people 表中。

代码如下：

```
@Override
    public void write(List<? extends People> items) throws Exception {
```

```
        // 先删除可能存在的表
        jdbcTemplate.execute("DROP TABLE IF EXISTS people");
        // 建表语句
        String createTableSql = "CREATE TABLE people (name VARCHAR(255), age INT, gender varchar(255))";
        jdbcTemplate.execute(createTableSql);
        for (People item : items) {
            String sql = "INSERT INTO people (name, age,gender) VALUES (?, ?,?)";
            jdbcTemplate.update(sql, item.getName(), item.getAge(),item.getGender());
        }
    }
```

2.2.3.3.10　BatchConfigTest.java 文件介绍

BatchConfigTest.java 文件是一个使用 JUnit 进行测试的类，用于测试 Spring Batch 的作业配置。BatchConfigTest.java 文件的代码主要包括以下几个部分：

1. 引用其他类和接口。

   声明当前文件包含以下接口和类：

- Assert 类：用于断言测试结果。
- Test 注解：用于标记测试方法。
- RunWith 注解：用于指定测试运行器。
- Job 接口：表示一个批处理作业。
- JobExecution 类：用于表示批处理作业的执行。
- JobParameters 类：用于表示批处理作业的参数。
- JobParametersBuilder 类：用于构建批处理作业的参数。
- JobLauncher 接口：用于启动批处理作业。
- Autowired 注解：用于进行依赖注入。
- SpringBootTest 注解：用于指定测试类为 Spring Boot 测试。
- SpringRunner 类：用于指定测试运行器为 SpringRunner。

代码如下：

```
import org.junit.Assert;
import org.junit.jupiter.api.Test;
import org.junit.runner.RunWith;
import org.springframework.batch.core.Job;
import org.springframework.batch.core.JobExecution;
import org.springframework.batch.core.JobParameters;
import org.springframework.batch.core.JobParametersBuilder;
```

```
import org.springframework.batch.core.launch.JobLauncher;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.junit4.SpringRunner;

import javax.batch.runtime.BatchStatus;
import java.util.UUID;
```

2. 定义 BatchConfigTest 类。

   通过使用 SpringBootTest 注解和 SpringRunner 运行器，可以进行 Spring Boot 的集成测试。在 testJob 方法中，使用 JobLauncherTestUtils 辅助类启动批处理作业，并使用断言来验证作业的执行状态。

   a. 使用 @Autowired 注解自动注入 JobLauncherTestUtils 实例。

   代码如下：

```
@Autowired
    private JobLauncher jobLauncher;
```

   b. 使用 @Test 注解标记 testJob 方法为测试方法。在该方法中，首先创建 JobParameters 对象，然后使用 jobLauncherTestUtils.launchJob 方法启动批处理作业，并使用 Assert.assertEquals 方法断言作业的执行状态为 COMPLETED 。

```
@Test
    public void testJob() throws Exception {
        JobParameters jobParameters = new JobParametersBuilder()
                .addString("jobParam", UUID.randomUUID().toString())
                .toJobParameters();

        JobLauncherTestUtils jobLauncherTestUtils = new JobLauncherTestUtils();
        JobExecution jobExecution = jobLauncherTestUtils.launchJob(jobParameters);
        Assert.assertEquals(BatchStatus.COMPLETED.toString(), jobExecution.getStatus().toString());
    }
```

   c. 使用 @Autowired 注解自动注入 JobLauncher 实例。

   代码如下：

```
@Autowired
    private JobLauncher jobLauncher;
```

   d. 使用 @Autowired 注解自动注入 Job 实例。

   代码如下：

```
@Autowired
private Job job;
```

e. 定义了一个名为 JobLauncherTestUtils 的内部类，用于辅助启动批处理作业。在该类中定义了一个 launchJob 方法，用于启动批处理作业。在该方法中，使用 jobLauncher.run 方法启动作业，并返回作业的执行结果。

代码如下：

```
private class JobLauncherTestUtils {

    public JobExecution launchJob(JobParameters jobParameters) throws Exception {
        return jobLauncher.run(job, jobParameters);
    }
}
```

2.2.3.3.11　AddPeopleDescProcessorTest.java 文件介绍

AddPeopleDescProcessorTest.java 文件是一个使用 JUnit 进行测试的类，用于测试 Spring Batch 的作业配置。AddPeopleDescProcessorTest.java 文件的代码主要包括以下几个部分：

1. 引用其他类和接口。

   声明当前文件包含以下接口和类：

- People 类：用于存储从数据库中读取的人员信息。
- PeopleDESC 类：用于存储对人员信息进行转换或处理后的描述信息。
- Test 注解：用于标记测试方法。
- RunWith 注解：用于指定测试运行器。
- Autowired 注解：用于进行依赖注入。
- SpringBootTest 注解：用于指定测试类为 Spring Boot 测试。
- SpringRunner 类：用于指定测试运行器为 SpringRunner。

代码如下：

```
import com.kingbase.example.batch.model.People;
import com.kingbase.example.batch.model.PeopleDESC;
import org.junit.Assert;
import org.junit.jupiter.api.Test;
import org.junit.runner.RunWith;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.junit4.SpringRunner;
```

2. 定义 AddPeopleDescProcessorTest 类。

   通过使用 SpringBootTest 注解和 SpringRunner 运行器，进行 Spring Boot 的集成测试。

   a. 使用 @Autowired 注解自动注入 AddPeopleDescProcessor 实例。

代码如下：

```
@Autowired
    private AddPeopleDescProcessor processor;
```

b. 使用 @Test 注解标记 testProcess 方法为测试方法。在该方法中，首先创建一个 People 对象，然后使用 processor.process 方法处理该对象，并将结果赋值给一个 PeopleDESC 对象。

代码如下：

```
@Test
    public void testProcess() throws Exception {
        People people = new People();
        people.setName("John");
        people.setAge(25);
        people.setGender("male");

        PeopleDESC desc = processor.process(people);

        Assert.assertEquals("John", desc.getName());
        Assert.assertEquals(25, desc.getAge());
        Assert.assertEquals("male", desc.getGender());
        Assert.assertEquals("This is John with age 25", desc.getDesc());
    }
```

2.2.3.3.12　AddDescPeopleWriterTest.java 文件介绍

AddDescPeopleWriterTest.java 文件是一个使用 JUnit 进行测试的类，用于测试 AddDescPeopleWriter 的写入逻辑。AddDescPeopleWriterTest.java 文件的代码主要包括以下几个部分：

1. 引用其他类和接口。声明当前文件包含以下接口和类：

- PeopleDESC 类：用于存储对人员信息进行转换或处理后的描述信息。
- Assert 类：用于断言测试结果。
- Test 注解：用于标记测试方法。
- RunWith 注解：用于指定测试运行器。
- Autowired 注解：用于进行依赖注入。
- SpringBootTest 注解：用于指定测试类为 Spring Boot 测试。
- JdbcTemplate 类：提供执行 SQL 语句的方法。
- SpringRunner 类：用于指定测试运行器为 SpringRunner。
- ArrayList 类，用于创建一个空的列表。
- List 接口：用于操作查询结果集合。

代码如下：

```
import com.kingbase.example.batch.model.PeopleDESC;
import org.junit.Assert;
import org.junit.jupiter.api.Test;
import org.junit.runner.RunWith;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.test.context.junit4.SpringRunner;

import java.util.ArrayList;
import java.util.List;
```

2. 定义 AddDescPeopleWriterTest 类。通过使用 SpringBootTest 注解和 SpringRunner 运行器，进行 Spring Boot 的集成测试。

a. 使用 @Autowired 注入实例。使用 @Autowired 注解自动注入 AddPeopleDescProcessor 和 JdbcTemplate 实例。

代码如下：

```
@Autowired
private AddDescPeopleWriter writer;
@Autowired
private JdbcTemplate jdbcTemplate;
```

b. 使用 @Test 测试数据的插入和输出。使用 @Test 注解标记 testWrite 方法为测试方法。在该方法中，首先创建一个空的 peopleDescList 列表，并向列表中添加三个 PeopleDESC 对象。然后使用 writer.write 方法将列表中的数据写入到数据库中。接着使用 jdbcTemplate 执行查询语句，获取 people_desc 表中的数据，并使用断言语句验证数据的正确性。最后，将查询结果输出到控制台，并输出作业执行完成的信息。

a. 插入数据到 people_desc 表。首先创建了一个空的 PeopleDESC 对象列表 peopleDescList 。其次创建了三个 PeopleDESC 对象 desc1 ，desc2 和 desc3 ，并分别设置了它们的属性值。将 desc1，desc2 和 desc3 添加到 peopleDescList 列表中。接下来调用了 writer 的 write 方法，将 peopleDescList 中的对象写入数据库中的 people_desc 表。然后使用 JdbcTemplate 执行了一条查询语句 SELECT COUNT(*) FROM people_desc ，获取了 people_desc 表中的记录数，并将结果赋值给变量 count 。最后使用 Assert.assertEquals 方法进行断言，验证 count 的值是否等于 3。

代码如下：

```
// 插入数据到 people_desc 表
        List<PeopleDESC> peopleDescList = new ArrayList<>();
        PeopleDESC desc1 = new PeopleDESC();
        desc1.setId(1);
        desc1.setName("John");
        desc1.setAge(25);
```

```
        desc1.setGender("male");

        desc1.setDesc("This is John with age 25");
        peopleDescList.add(desc1);
        PeopleDESC desc2 = new PeopleDESC();
        desc2.setId(2);
        desc2.setName("Alice");
        desc2.setAge(30);
        desc2.setGender("female");
        desc2.setDesc("This is Alice with age 30");
        peopleDescList.add(desc2);

        PeopleDESC desc3 = new PeopleDESC();
        desc3.setId(3);
        desc3.setName("Lucy");
        desc3.setAge(26);
        desc3.setGender("female");
        desc3.setDesc("This is Lucy with age 26");
        peopleDescList.add(desc3);
        writer.write(peopleDescList);

        String selectSql = "SELECT COUNT(*) FROM people_desc";
        int count = jdbcTemplate.queryForObject(selectSql, Integer.class);
        Assert.assertEquals(3, count);
```

b. 输出 people_desc 表中的数据。首先使用 JdbcTemplate 执行了一条查询语句 SELECT * FROM people_desc ，并使用 lambda 表达式处理查询结果。在 lambda 表达式中，使用 rs.getInt 、rs.getString 等方法获取查询结果集中的字段值，并将字段值设置到一个新创建的 PeopleDESC 对象中。将每个新创建的 PeopleDESC 对象添加到一个结果列表 resultDesc 中。之后打印了一行提示信息 people_desc 表中的数据：，然后使用 for 循环遍历 resultDesc 列表中的每个 PeopleDESC 对象，并使用 System.out.println 打印输出每个对象的内容。最后打印了一条作业执行完成的信息。

代码如下：

```
// 输出 people_desc 表中的数据
        List<PeopleDESC> resultDesc = jdbcTemplate.query("SELECT * FROM people_desc", (rs, rowNum) -> {
            PeopleDESC desc = new PeopleDESC();

            desc.setId(rs.getInt("id"));
            desc.setName(rs.getString("name"));
            desc.setAge(rs.getInt("age"));
            desc.setGender(rs.getString("gender"));
            desc.setDesc(rs.getString("description"));

            return desc;
```

```
        });

        System.out.println("people_desc 表中的数据: ");
        for (PeopleDESC desc : resultDesc) {
            System.out.println(desc);
        }

        // 输出作业执行完成后的信息
        System.out.println("Batch Job execution completed.");
```

2.2.3.3.13　AddPeopleWriterTest.java 文件介绍

AddPeopleWriterTest.java 文件是一个使用 JUnit 进行测试的类，用于测试 AddPeopleWriterTest 的写入逻辑。AddPeopleWriterTest.java 文件的代码主要包括以下几个部分：

1. 引用其他类和接口。

声明当前文件包含以下接口和类：

- People 类：用于存储从数据库中读取的人员信息。
- Test 注解：用于标记测试方法。
- RunWith 注解：用于指定测试运行器。
- Autowired 注解：用于进行依赖注入。
- SpringBootApplication 注解：用于标记该类为 Spring Boot 应用程序的入口。
- SpringBootTest 注解：用于指定测试类为 Spring Boot 测试。
- ComponentScan 注解：用于指定要进行组件扫描的包或类。
- JdbcTemplate 类：提供执行 SQL 语句的方法。
- SpringRunner 类：用于指定测试运行器为 SpringRunner。
- ArrayList 类，用于创建一个空的列表。
- List 接口：用于操作查询结果集合。

代码如下：

```
import com.kingbase.example.batch.model.People;
import org.junit.jupiter.api.Test;
import org.junit.runner.RunWith;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.jdbc.core.JdbcTemplate;
```

```
import org.springframework.test.context.junit4.SpringRunner;

import java.util.ArrayList;
import java.util.List;
```

2. 定义 AddPeopleWriterTest 类。通过使用 SpringBootTest 注解和 SpringRunner 运行器，进行 Spring Boot 的集成测试，并使用 @ComponentScan 注解指定要扫描的包路径。

   a. 使用 @Autowired 注入实例。使用 @Autowired 注解自动注入 addPeopleWriter 和 JdbcTemplate 实例。

   代码如下：

```
@Autowired
    private AddPeopleWriter addPeopleWriter;
    @Autowired
    private JdbcTemplate jdbcTemplate;
```

```
        b. 使用 @Test 测试数据的插入和输出。
                a. 插入数据到 people 表。 首先，创建一个空的 People 对象列表 peopleList 。然后，创建三个 People 对象 person1，person2 和 person3 ，并设置它们的名称，年龄和性别属性。接着，将这三个 People 对象添加到 peopleList 列表中之后，调用 addPeopleWriter 的 write 方法，将 peopleList 作为参数传递给该方法，用于将这些 People 对象写入数据库。
```

   代码如下：

```
// 插入数据到 people 表
        List<People> peopleList = new ArrayList<>();
        People person1 = new People();
        person1.setName("Tom");
        person1.setAge(25);
        person1.setGender("male");

        peopleList.add(person1);
        People person2 = new People();
        person2.setName("Jack");
        person2.setAge(24);
        person2.setGender("male");

        peopleList.add(person2);
        People person3 = new People();
        person3.setName("Olivia");
        person3.setAge(23);
        person3.setGender("female");

        peopleList.add(person3);
        addPeopleWriter.write(peopleList);
```

b. 输出 people 表中的数据。 首先使用 JdbcTemplate 执行了一条查询语句 SELECT * FROM people ，并使用 lambda 表达式处理查询结果。在 lambda 表达式中，使用 rs.getString 和 rs.getInt 方法获取查询结果集中的字段值，并将字段值设置到一个新创建的 People 对象中。将每个新创建的 People 对象添加到一个结果列表 result 中。接下来打印了一行提示信息 people 表中的数据：然后使用 for 循环遍历 result 列表中的每个 People 对象，并使用 System.out.println 打印输出每个对象的内容。最后打印了一条作业执行完成的信息。

代码如下：

```
// 查询并输出结果
        List<People> result = jdbcTemplate.query("SELECT * FROM people", (rs, rowNum) -> {
            People person = new People();
            person.setName(rs.getString("name"));
            person.setAge(rs.getInt("age"));
            person.setGender(rs.getString("Gender"));
            return person;
        });

        System.out.println("people 表中的数据: ");
        for (People person : result) {
            System.out.println(person);
        }
```

2.2.3.3.14　完整的代码展示

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.7.11</version>
<relativePath/> <!-- lookup parent from repository -->
</parent>
<groupId>com.kingbase</groupId>
<artifactId>java-kingbase-springboot</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>java-kingbase-springbatch</name>
<description>Demo project for Spring Batch</description>
<properties>
<java.version>1.8</java.version>
</properties>
```

```
<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
<groupId>org.slf4j</groupId>
<artifactId>slf4j-log4j12</artifactId>
</dependency>
<dependency>
<groupId>cn.com.kingbase</groupId>
<artifactId>kingbase8</artifactId>
<version>9.0.0</version>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-test</artifactId>
<scope>test</scope>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-batch</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
<groupId>org.apache.tomcat</groupId>
<artifactId>tomcat-jdbc</artifactId>
</dependency>
<dependency>
<groupId>junit</groupId>
<artifactId>junit</artifactId>
<version>4.10</version>
<scope>test</scope>
</dependency>
```

```
<dependency>
<groupId>javax.activation</groupId>
<artifactId>javax.activation-api</artifactId>
<version>1.2.0</version>
</dependency>
<dependency>
<groupId>jakarta.persistence</groupId>
<artifactId>jakarta.persistence-api</artifactId>
<version>2.2.3</version>
</dependency>
</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

application.properties

```
spring.datasource.driverClassName=com.kingbase8.Driver
spring.datasource.url=jdbc:kingbase8://192.168.xx.xxx:7901/test
spring.datasource.username=system
spring.datasource.password=123456
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.Kingbase8Dialect
spring.batch.job.enabled=false
logging.level.org.springframework=INFO
logging.level.com.example=DEBUG
```

BatchConfig.java

```
package com.kingbase.example.batch.config;
@Configuration
@EnableBatchProcessing
@SpringBootApplication
@ComponentScan("com.kingbase.example.batch.writer")
@EnableAutoConfiguration
```

```
public class BatchConfig {
@Autowired
private JobBuilderFactory jobBuilderFactory;
@Autowired
private StepBuilderFactory stepBuilderFactory;

@Autowired
private DataSource dataSource;// 使用 Spring Boot 自动配置提供的默认 dataSource
@Bean
public ItemReader<People> peopleReader() {
JdbcCursorItemReader<People> reader = new JdbcCursorItemReader<>();
reader.setDataSource((javax.sql.DataSource) dataSource);
reader.setRowMapper(new BeanPropertyRowMapper<>(People.class));
reader.setSql("SELECT * FROM people");
return reader;
}

@Bean
public ItemProcessor<People, PeopleDESC> addPeopleDescProcessor() {
return new AddPeopleDescProcessor();
}
//    @Bean
public ItemWriter<PeopleDESC> addDescPeopleWriter() {
return new AddDescPeopleWriter();
}
@Bean
public Step step1(ItemReader<People> reader, ItemProcessor<People, PeopleDESC> processor,
ItemWriter<PeopleDESC> writer) {
return stepBuilderFactory.get("step1")
.<People, PeopleDESC>chunk(10)
.reader(reader)
.processor(processor)
.writer(writer)
.build();
}

@Bean
.incrementer(new RunIdIncrementer())
.flow(step1)
.end()
.build();
}
}
```

People.java

```
package com.kingbase.example.batch.model;
public class People {
private String name;
private int age;
private String gender;
// getters and setters

public String getName() {
return name;
}

public void setName(String name) {
this.name = name;
}

public String getGender() {
return gender;
}
public void setGender(String gender) {
this.gender = gender;
}

public int getAge() {
return age;
}

public void setAge(int age) {
this.age = age;
}

@Override
public String toString() {
return "People [name=" + name + ", age=" + age + ", gender="  + gender +"]";
}
// Getters and setters
}
```

PeopleDESC.java

```
package com.kingbase.example.batch.model;
public class PeopleDESC {
private String name;
```

```
private int age;
private String desc;
private int id;
private String gender ;
public String getName() {
return name;
}

public void setName(String name) {
this.name = name;
}

public int getAge() {
return age;
}

public void setAge(int age) {
this.age = age;
}

public String getDesc() {
return desc;
}

public void setDesc(String desc) {
this.desc = desc;
}

public int getId() {
return id;
}

public void setId(int id) {
this.id = id;
}

public String getGender() {
return gender;
}

public void setGender(String gender) {
this.gender = gender;
}

@Override
```

```
public String toString() {
return "PeopleDESC [name=" + name + ", age=" + age+ ", gender=" + gender + ", desc=" + desc + "]";
}
}
```

AddPeopleDescProcessor.java

```
package com.kingbase.example.batch.processor;
public class AddPeopleDescProcessor implements ItemProcessor<People, PeopleDESC> {
@Override
public PeopleDESC process(People item) throws Exception {
PeopleDESC desc = new PeopleDESC();
desc.setName(item.getName());
desc.setAge(item.getAge());
desc.setGender(item.getGender());
desc.setDesc("This is " + item.getName() + " with age " + item.getAge() + "and gender is"  + item.
getGender());
return desc;
}
}
```

AddDescPeopleWriter.java

```
package com.kingbase.example.batch.writer;
@Component
public class AddDescPeopleWriter implements ItemWriter<PeopleDESC> {
@Autowired
private JdbcTemplate jdbcTemplate;
@Override
public void write(List<? extends PeopleDESC> items) throws Exception {
// 先删除可能存在的表
jdbcTemplate.execute("DROP TABLE IF EXISTS people_desc");
// 建表语句
String createTableSql = "CREATE TABLE people_desc (id INT, name VARCHAR(255), age INT, gender VARCHAR(255),
description VARCHAR(255))";
jdbcTemplate.execute(createTableSql);
for (PeopleDESC item : items) {
String sql = "INSERT INTO people_desc (id, name, age, gender,description) VALUES (?, ?, ? ,?, ?)";
jdbcTemplate.update(sql, item.getId(), item.getName(), item.getAge(), item.getGender(), item.getDesc());
}
}
}
```

AddPeopleWriter.java

```
package com.kingbase.example.batch.writer;
@Component
public class AddPeopleWriter implements ItemWriter<People> {
@Autowired
private JdbcTemplate jdbcTemplate;
@Override
public void write(List<? extends People> items) throws Exception {
// 先删除可能存在的表
jdbcTemplate.execute("DROP TABLE IF EXISTS people");
// 建表语句
String createTableSql = "CREATE TABLE people (name VARCHAR(255), age INT, gender varchar(255))";
jdbcTemplate.execute(createTableSql);
for (People item : items) {
String sql = "INSERT INTO people (name, age,gender) VALUES (?, ?,?)";
jdbcTemplate.update(sql, item.getName(), item.getAge(),item.getGender());
}
}
}
```

BatchApplication.java

```
package com.kingbase.example.batch;
@SpringBootApplication
public class BatchApplication {
public static void main(String[] args) {
SpringApplication.run(BatchApplication.class, args);
}
public void runBatchJob() {
}
}
```

2.2.3.4　相关文档

Jdbc 更多示例请参考 Jdbc 产品手册的程序示例。

2.2.4　SpringJDBC

本文将介绍如何使用 Spring JDBC 框架和 kingbase 数据库构建一个应用程序，实现创建表、插入数据、查询数据，更新数据、删除数据和删除表等基本操作。

点击下载 spring-jdbc-kingbase-client 示例工程。

2.2.4.1　前提条件

- 您已安装 Kingbase 数据库。
- 您已安装 JDK 1.8 和 Maven。
- 您已安装 IDE 工具，比如 IDEA。

备注:

本文档运行代码使用的工具是 IntelliJ IDEA 2023.2.8 (Community Edition) 版本，您也可以根据个人喜好选择适合自己的工具运行示例代码。

2.2.4.2　操作步骤

备注:

本文中给出的操作步骤是基于 Windows 环境生成。如果您使用的是其他操作系统环境或编译器，那么操作步骤可能会略有不同。

- 将 spring-jdbc-kingbase-client 项目导入到 IDEA 中。
- 获取 KingBase 数据库 URL。
- 修改 spring-jdbc-kingbase-client 项目中的数据库连接信息。
- 运行 spring-jdbc-kingbase-client 项目。

2.2.4.2.1　步骤一：将 spring-jdbc-kingbase-client 项目导入到 IDEA 中

1. 打开 IDEA，在菜单栏上选择 File->Open。在弹出的对话框中，点击 Directory 按钮选择项目所在的 spring-jdbc-kingbase-client 目录，然后点击确定完成导入。
2. IntelliJ IDEA 将会自动识别项目中的各类文件，并在 Project 工具窗口中，可以查看项目的目录结构、文件列表、模块列表、依赖关系等信息。Project 工具窗口通常位于 IntelliJ IDEA 界面的最左侧，默认情况下是打开的。如果 Project 工具窗口被关闭了，可以通过点击菜单栏中的 View > Tool Windows > Project 或者使用快捷键 Alt + 1 来重新打开它。
3. 根据已部署的 Kingbase 数据库填写下面的 URL 的对应信息

备注:

当使用 Eclipse 导入 Maven 项目时，Eclipse 会自动检测项目中的 pom.xml 文件，并根据文件中描述的依赖关系自动下载所需的依赖库，并将它们添加到项目中。

2.2.4.2.2　步骤二: 获取 KingBase 数据库 URL

1. 联系 Kingbase 数据库部署人员或者管理员获取相应的数据库连接串。

   示例如下：

   ```
   ./ksql -U system -d test -h xx.xx.xx.xx -p 54321
   ```

2. 根据 Kingbase 数据库连接串信息填写下面 URL 的对应信息。

   ```
   jdbc:kingbase8://$host:$port/$database_name?user=$user_name&password=$password
   ```

   参数说明：

   - $host ：提供 Kingbase 数据库连接 IP。
   - $port ：提供 Kingbase 数据库连接端口。默认是 54321 ，在部署 Kingbase 数据库时可自定义端口号。
   - $database_name：需要访问的 database 名称。

   备注:

   连接租户的用户需要拥有该 database 的 CREATE TABLE 、DROP TABLE 、INSERT 、DELETE 、UPDATE 和 SELECT 权限。

   示例如下：

   ```
   jdbc:kingbase8://192.168.xx.xxx:7901/test?
   user=system&password=******
   ```

2.2.4.2.3　步骤三: 修改 spring-jdbc-kingbase-client 项目中的数据库连接信息

根据步骤二：获取 Kingbase 数据库 URL 中获取的信息修改文件 spring-jdbc-kingbase-client/src/main/java/com/example/Main.java 中的数据库连接信息。

示例如下：

```
// Database Connect Information
String url = "jdbc:kingbase8://192.168.xx.xxx:7901/test";
String username = "system";
String password = "******";
```

- Kingbase 节点的 IP 地址为 `192.168.xx.xxx`。

- 访问端口使用的是 `7901`。
- 需要访问的 database 名称为 `test`。
- 连接账户是 `system`。`system` 是 kingbase 数据库中默认创建的一个管理员。
- 密码是 `******`。

2.2.4.2.4　步骤四: 运行 spring-jdbc-kingbase-client 项目

运行路径

1. 在项目结构中找到 src/main/java/com/example/中找到 Main.java 文件。
2. 在工具菜单栏中选择运行 (U) > 运行 > Main(快捷键 Ctrl+Shift+F10)，或直接单击右上角绿色三角形 (快捷键 Shift+F10) 运行。
3. 通过 IDEA 的控制台来查看项目的日志信息和输出结果。

运行结果：

```
Create table successfully.
Insert data successfully.
Update data successfully.
Delete data successfully.
id: 30, name: CC
id: 10, name: A1
Query data successfully.
Table drop successfully.
```

2.2.4.3　项目代码介绍

点击下载 spring-jdbc-kingbase-client，这是一个名为 spring-jdbc-kingbase-client 的压缩包。

解压后，得到一个名为 java-kingbase-springboot 的文件夹。目录结构如下所示：

```
spring-jdbc-kingbase-client
  src
    main
    java
    com
    example
    Main.java
  pom.xml
```

文件说明：

- src：源代码根目录。

- main：主代码目录，包含应用程序的主要逻辑。
- java：Java 源代码目录。
- com：Java 包目录。
- example：示例项目的包目录。
- Main.java：主类，包含创建表和插入数据等逻辑。
- pom.xml：Maven 项目的配置文件，用于管理项目的依赖和构建设置。

2.2.4.3.1　pom.xml 代码介绍

pom.xml 文件是 Maven 项目的配置文件，定义了项目的依赖项、插件和构建规则等信息。Maven 是一个 Java 项目管理工具，可以自动下载依赖项、编译和打包项目等操作。本文 pom.xml 文件的代码主要包括以下几个部分：

1. 文件声明语句。

   声明本文件是一个 XML 文件，使用的 XML 版本是 1.0 ，字符编码方式是 UTF-8 。

   代码如下：

```
<?xml version="1.0" encoding="UTF-8"?>
```

2. 配置 POM 的命名空间和 POM 模型版本。

   - 通过 xmlns 指定 POM 的命名空间为 `http://maven.apache.org/POM/4.0.0`。
   - 通过 xmlns:xsi 指定 XML 命名空间为 `http://www.w3.org/2001/XMLSchema-instance`。
   - 通过 xsi:schemaLocation 指定 POM 的命名空间为 `http://maven.apache.org/POM/4.0.0`，并指定 POM 的 XSD 文件的位置为 `https://maven.apache.org/xsd/maven-4.0.0.xsd`。
   - 通过 modelVersion 元素指定该 POM 文件使用的 POM 模型版本为 `4.0.0`。

   代码如下:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
                xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
                xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/
maven-4.0.0.xsd">
        <modelVersion>4.0.0</modelVersion>

<!-- 其他配置 -->

</project>
```

3. 配置基本信息。

   - 通过 groupId 指定项目标识为 com.kingbase。
   - 通过 artifactId 指定项目依赖为 spring-jdbc-kingbase-client。

- 通过 version 指定项目的版本号为 0.0.1-SNAPSHOT。

代码如下:

```
<groupId>com.example</groupId>
<artifactId>spring-jdbc-kingbase-client</artifactId>
<version>1.0-SNAPSHOT</version>
```

4. 配置项目源文件的属性。

   指定 Maven 的编译器插件为 maven-compiler-plugin ，并设置了源代码和目标 Java 版本都为 8。这意味着项目的源代码使用 Java 8 特性编写，且编译后的字节码也将兼容 Java 8 运行时环境。这样设置可以确保项目在编译和运行时能够正确地处理 Java 8 的语法和特性。

   代码如下：

```
<build>
                <plugins>
                        <plugin>
                                <groupId>org.apache.maven.plugins</groupId>
                                <artifactId>maven-compiler-plugin</artifactId>
                                <configuration>
                                        <source>8</source>
                                        <target>8</target>
                                </configuration>
                        </plugin>
                </plugins>
        </build>
```

5. 配置核心依赖。

   - 添加 spring-jdbc 依赖库：
   - 通过 groupId 指定依赖项所属的组织为 org.springframework。
   - 通过 artifactId 指定依赖项的名称为 spring-jdbc。
   - 通过 version 指定依赖项的版本号为 5.2.0.RELEASE。

   代码如下：

```
<dependency>
                        <groupId>org.springframework</groupId>
                        <artifactId>spring-jdbc</artifactId>
                        <version>5.2.0.RELEASE</version>
                </dependency>
```

   - 指定依赖项所属的组织为 cn.com.kingbase，名称为 kingbase8，版本号为 9.0.0，通过该依赖可以使用 Kingbase 提供的客户端功能，如连接、查询、事务等。代码如下:

```
<dependency>
        <groupId>cn.com.kingbase</groupId>
        <artifactId>kingbase8</artifactId>
        <version>9.0.0</version>
</dependency>
```

2.2.4.3.2　Main.java 代码介绍

Main.java 文件是示例程序的一部分，代码演示了如何使用 Spring JDBC 框架执行数据库操作。它首先配置了数据库连接信息，然后创建了一个 JdbcTemplate 对象来执行数据库操作。代码中还包括了创建表、插入数据和查询数据的示例。本文 Main.java 文件的代码主要包括以下几个部分：

1. 定义包和导入必要的类。

   a. 声明当前代码所属的包名为 com.example。

   b. 导入 JdbcTemplate 类，它是 Spring JDBC 框架中的核心类，用于执行数据库操作。

   c. 导入 DriverManagerDataSource 类，它是 Spring 提供的一个实现了 DataSource 接口的数据源类，用于配置数据库连接信息。

   代码如下：

```
package com.example;

import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
```

2. 创建一个 Main 类，并定义了 main 方法。

   a. 定义一个名为 Main 的公共类，作为程序的入口点。类名需要与文件名保持一致。

   b. 定义一个公共静态方法 main，作为程序的起始执行点。

   c. 其他数据库操作。

   代码如下：

```
public class Main {
public static void main(String[] args) {
// 数据库连接信息
// 创建数据源
// 创建 JdbcTemplate 对象
// 创建表
// 插入数据
// 更新数据
// 删除数据
// 查询数据
// 删除表
```

```
}
}
```

3. 定义数据库连接信息。

   定义用于连接数据库的 URL、用户名和密码。您需要将 *host*port 、*database*$_n$*ame*user_name 和 $password 替换为实际的数据库连接信息。

   代码如下：

```
String url = "jdbc:kingbase8://$host:$port/$database_name";
String username = "$user_name";
String password = "$password";
```

   参数解释：

   - $host ：提供 KingBase 数据库连接 IP。
   - $port ：提供 KingBase 数据库连接端口。默认是 54321，在部署在部署 KingBase 数据库时可自定义。
   - $database_name：需要访问的 database 名称。
   - $user_name ：提供的连接账户。
   - $password ：提供账户密码。

   示例如下：

```
// Database Connect Information
String url = "jdbc:kingbase8://192.168.xx.xxx:7901/test";
String username = "system";
String password = "******";
```

4. 创建数据源。

   创建一个 DriverManagerDataSource 对象，并配置了数据库连接的相关信息，包括驱动类名、数据库 URL、用户名和密码。具体步骤如下：

   - 创建一个 DriverManagerDataSource 对象，用于配置数据库连接的信息。
   - 设置驱动类名：通过调用 setDriverClassName 方法，设置数据库驱动类名为 com.kingbase8.Driver。这是用于连接 kingbase 数据库的驱动类。
   - 设置数据库 URL：通过调用 setUrl 方法，设置数据库的 URL。
   - 设置用户名：通过调用 setUsername 方法，设置数据库的用户名。
   - 设置密码：通过调用 setPassword 方法，设置数据库的密码。

   代码如下：

```
// create data source
DriverManagerDataSource dataSource = new DriverManagerDataSource();
dataSource.setDriverClassName("com.kingbase8.Driver");
dataSource.setUrl(url);
dataSource.setUsername(username);
dataSource.setPassword(password);
```

5. 创建 JdbcTemplate 对象。

   创建一个 JdbcTemplate 对象，它是 Spring JDBC 框架的核心类之一。JdbcTemplate 提供了执行数据库操作的方法，如执行 SQL 语句、更新数据以及查询数据等。

   代码如下：

```
// Create a JdbcTemplate object
JdbcTemplate jdbcTemplate = new JdbcTemplate(dataSource);
```

6. 创建表。

   - 定义一个创建表的 SQL 语句，SQL 语句中指定表名为 table_springjdbc，并定义了两个列，一个是名为 id 的 NUMBER 类型列，另一个是名为 name 的 VARCHAR2(20) 类型列。
   - 执行创建表的 SQL 语句，jdbcTemplate 对象通过调用 execute 方法，执行了上述定义的创建表的 SQL 语句。
   - 在控制台输出一条成功创建表的消息。

   代码如下:

```
// Create table
String createTableSql = "CREATE TABLE table_springjdbc (id INT, name VARCHAR(20))";
jdbcTemplate.execute(createTableSql);
System.out.println("Create table successfully.");
```

7. 插入数据。

   定义插入数据的 SQL 语句，SQL 语句中指定了要插入的表为 table_springjdbc，列为 id 和 name，插入了两行数据，分别是 (10,‘AA’)、(20,‘BB’) 和 (30,‘CC’)。执行插入数据的 SQL 语句，jdbcTemplate 对象通过调用 update 方法，执行了上述定义的插入数据的 SQL 语句。在控制台输出一条成功插入数据的消息。

   代码如下：

```
// Insert table
String insertDataSql = "INSERT INTO table_springjdbc (id,name) VALUES (10,'AA'),(20,'BB'), (30,'CC')";
jdbcTemplate.update(insertDataSql);
System.out.println("Insert data successfully.");
```

8. 更新数据。

   a. 定义一个 SQL 语句，用于更新数据库表 test_springjdbc 中 id 为 10 的记录的 name 字段的值为 A1。

b. 通过 jdbcTemplate 对象执行 SQL 语句，将更新操作应用于数据库。

c. 在控制台输出一条成功更新数据的消息。

代码如下：

```
// Update data
String updateDataSql = "UPDATE table_springjdbc SET name = 'A1' WHERE id = 10";
jdbcTemplate.update(updateDataSql);
System.out.println("Update data successfully.");
```

9. 删除数据。

a. 定义一个 SQL 语句，用于从数据库表 table_springjdbc 中删除 id 为 20 的记录。

b. 通过 jdbcTemplate 对象执行 SQL 语句，将删除操作应用于数据库。

c. 在控制台输出一条成功删除数据的消息。

```
// Delete data
String deleteDataSql = "DELETE FROM table_springjdbc WHERE id = 20";
jdbcTemplate.update(deleteDataSql);
System.out.println("Delete data successfully.");
```

10. 查询数据。

a. 定义查询数据的 SQL 语句，SQL 语句中使用了 SELECT * 来选择所有列，从 test_springjdbc 表中查询数据。

b. 执行查询数据的 SQL 语句，jdbcTemplate 对象通过调用 query 方法，执行了上述定义的查询数据的 SQL 语句，并接收一个回调函数作为参数。

c. 回调函数中的代码使用 ResultSet 对象 (rs) 获取查询结果中的每一行数据，并进行处理。回调函数使用 getInt 和 getString 方法从 ResultSet 对象中获取 id 和 name 列的值，并将其打印到控制台。最后，回调函数返回 null。

d. 在控制台输出一条成功查询数据的消息。

代码如下：

```
// Query data
String selectDataSql = "SELECT * FROM table_springjdbc";
jdbcTemplate.query(selectDataSql, (rs, rowNum) -> {
        int id = rs.getInt("id");
        String name = rs.getString("name");
        System.out.println("id: " + id + ", name: " + name);
        return null;
});
System.out.println("Query data successfully.");
```

11. 删除表。

a. 定义一个 SQL 语句，用于删除数据库中的表 table_springjdbc。

b. 通过 jdbcTemplate 对象执行 SQL 语句，将删除表的操作应用于数据库。

c. 输出一条成功删除表的消息。

代码如下：

```
// Delete table
String deleteTableSql = "DROP TABLE table_springjdbc";
jdbcTemplate.execute(deleteTableSql);
System.out.println("Table drop successfully.");
```

2.2.4.3.3　完整的代码展示

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
<groupId>com.example</groupId>
<artifactId>spring-jdbc-kingbase-client</artifactId>
<version>1.0-SNAPSHOT</version>
<build>
<plugins>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<configuration>
<source>8</source>
<target>8</target>
</configuration>
</plugin>
</plugins>
</build>
<dependencies>
<dependency>
<groupId>cn.com.kingbase</groupId>
<artifactId>kingbase8</artifactId>
<version>9.0.0</version>
</dependency>
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-jdbc</artifactId>
```

```
<version>5.2.0.RELEASE</version>
</dependency>
</dependencies>
</project>
```

main.java

```
package com.example;
public class Main {
public static void main(String[] args) {
// Database Connect Information
String url = "jdbc:kingbase8://$host:$port/$database_name";
String username = "$user_name";
String password = "$password";
// create data source
DriverManagerDataSource dataSource = new DriverManagerDataSource();
dataSource.setDriverClassName("com.kingbase8.Driver");
dataSource.setUrl(url);
dataSource.setUsername(username);
dataSource.setPassword(password);
// Create a JdbcTemplate object
JdbcTemplate jdbcTemplate = new JdbcTemplate(dataSource);
// Create table
String createTableSql = "CREATE TABLE table_springjdbc (id INT, name VARCHAR(20))";
jdbcTemplate.execute(createTableSql);
System.out.println("Create table successfully.");
// Insert data
String insertDataSql = "INSERT INTO table_springjdbc (id,name) VALUES (10,'AA'),(20,'BB'), (30,'CC')";
jdbcTemplate.update(insertDataSql);
System.out.println("Insert data successfully.");
// Update data
String updateDataSql = "UPDATE table_springjdbc SET name = 'A1' WHERE id = 10";
jdbcTemplate.update(updateDataSql);
System.out.println("Update data successfully.");
// Delete data
String deleteDataSql = "DELETE FROM table_springjdbc WHERE id = 20";
jdbcTemplate.update(deleteDataSql);
System.out.println("Delete data successfully.");
// Query data
String selectDataSql = "SELECT * FROM table_springjdbc";
jdbcTemplate.query(selectDataSql, (rs, rowNum) -> {
int id = rs.getInt("id");
String name = rs.getString("name");
```

```
System.out.println("id: " + id + ", name: " + name);
return null;
});
System.out.println("Query data successfully.");

// Delete table
String deleteTableSql = "DROP TABLE table_springjdbc";
jdbcTemplate.execute(deleteTableSql);
System.out.println("Table drop successfully.");
}
}
```

2.2.4.4　相关文档

Jdbc 更多示例请参考 Jdbc 产品手册的程序示例。

2.2.5　MyBatis

本文将介绍如何使用 MyBatis 框架和 Kingbase 数据库构建一个应用程序，实现创建表、插入数据和查询数据等基本操作。

点击下载 mybatis-kingbase 示例工程。

2.2.5.1　前提条件

- 您已安装 Kingbase 数据库。
- 您已安装 JDK 1.8 和 Maven。
- 您已安装 IDE 工具。

备注:

本文档运行代码使用的工具是 IntelliJ IDEA 2023.2.8 (Community Edition) 版本，您也可以根据个人喜好选择适合自己的工具运行示例代码。

2.2.5.2　操作步骤

备注:

本文中给出的操作步骤是基于 Windows 环境生成。如果您使用的是其他操作系统环境或编译器，那么操作步骤可能会略有不同。

1. 获取 Kingbase 数据库连接信息。
2. 导入 mybatis-kingbase 项目到 IDEA 中。
3. 修改 mybatis-kingbase 项目中的数据库连接信息。
4. 运行 mybatis-kingbase 项目。

2.2.5.2.1　步骤一：获取 Kingbase 数据库连接信息

1. 联系 Kingbase 数据库部署人员或者管理员获取相应的数据库连接信息。

```
./ksql -U system -d test -h xx.xx.xx.xx -p 54321
```

2. 根据已部署的 Kingbase 数据库填写下面的 URL 的对应信息。

备注:

在 jdbc.properties 文件中需要这里的 URL 信息。

```
jdbc.url=jdbc:kingbase8://host:port/database?user=$user_name&password=$password
```

参数说明:

- host：提供 Kingbase 数据库连接 IP。直连方式使用的是 KESServer 节点的 IP 地址。
- port：提供 Kingbase 数据库连接端口。直连方式默认是 54321，在部署 kingbase 数据库时可自定义。
- database：需要访问的数据库名称。
- user_name：通过 -u 参数指定，管理员用户为 system。
- password：提供账户密码。

更多 URL 参数说明信息请参考 Jdbc 连接属性。

2.2.5.2.2　步骤二: 导入 mybatis-kingbase 项目到 IDEA 中

1. 打开 IntelliJ IDEA，选择 File > Open…选项。
2. 在弹出的 Open File or Project 窗口中，选择对应的 mybatis-kingbase 项目文件夹，单击 OK 完成项目导入。
3. IntelliJ IDEA 将会自动识别项目中的各类文件，并在 Project 工具窗口中，可以查看项目的目录结构、文件列表、模块列表、依赖关系等信息。Project 工具窗口通常位于 IntelliJ IDEA 界面的最左侧，默认情况下是打开的。如果 Project 工具窗口被关闭了，可以通过点击菜单栏中的 View > Tool Windows > Project 或者使用快捷键 Alt + 1 来重新打开它。

备注:

当使用 IntelliJ IDEA 导入项目时，IntelliJ IDEA 会自动检测项目中的 pom.xml 文件，并根据文件中描述的依赖关系自动下载所需的依赖库，并将它们添加到项目中。

2.2.5.2.3　步骤三: 修改 mybatis-kingbase 项目中的数据库连接信息

根据步骤一：获取 Kingbase 数据库连接信息中的信息修改 application.properties 文件中的数据库连接信息。

示例如下:

- 数据库驱动名称为 `com.kingbase8.Driver`。
- KES Server 节点的 IP 地址为 `192.168.xx.xxx`。
- 访问端口使用的是 `7901`。
- 需要访问的 database 名称为 `test`。
- 设置 URL 额外的连接属性为 `useServerPrepStmts=true&rewriteBatchedStatements=true`。
- 账户是 `system`。
- 密码是 `******`。

配置示例如下

```
jdbc.driver=com.kingbase8.Driver
jdbc.url=jdbc:kingbase8://192.168.xx.xxx:7901/test?useServerPrepStmts=true&rewriteBatchedStatements=true
jdbc.username=system
jdbc.password=******
```

2.2.5.2.4　步骤四: 运行 mybatis-kingbase 项目

运行路径：

1. 在项目结构中找到 src > test > java 中找到 TestMybatis.java 文件。
2. 在工具菜单栏中选择运行 (U) > 运行 > TestMybatis(快捷键 Shift+F10)，或直接单击右上角绿色三角形运行 (快捷键 Shift+F10)。
3. 通过 IDEA 的控制台来查看项目的日志信息和输出结果。

运行 testUserMapper 方法输出结果如下：

```
User{id=2, name='update'}
User{id=3, name='insert'}
User{id=4, name='insert'}
User{id=5, name='insert'}
```

```
User{id=6, name='insert'}
User{id=7, name='insert'}
User{id=8, name='insert'}
User{id=9, name='insert'}
User{id=10, name='insert'}
usersByPage = [User{id=5, name='insert'}, User{id=6, name='insert'}, User{id=7,
name='insert'}]
```

运行 testSqlSession 方法输出结果如下：

```
User{id=2, name='update'}
User{id=3, name='insert'}
User{id=4, name='insert'}
User{id=5, name='insert'}
User{id=6, name='insert'}
User{id=7, name='insert'}
User{id=8, name='insert'}
User{id=9, name='insert'}
User{id=10, name='insert'}
```

运行 testAppMapper 方法输出结果如下：

```
App{id=3, name='insert3'}
App{id=4, name='insert4'}
App{id=5, name='insert5'}
App{id=6, name='insert6'}
App{id=7, name='insert7'}
App{id=8, name='insert8'}
App{id=9, name='insert9'}
App{id=10, name='insert10'}
App{id=2, name='update'}
pageList = Page{count=true, pageNum=2, pageSize=3, startRow=3, endRow=6, total=9, pages=3, reasonable=false,
pageSizeZero=true}[App{id=6, name='insert6'}, App{id=7, name='insert7'}, App{id=8, name='insert8'}]
```

完整的输出结果如下：

```
App{id=3, name='insert3'}
App{id=4, name='insert4'}
App{id=5, name='insert5'}
App{id=6, name='insert6'}
App{id=7, name='insert7'}
App{id=8, name='insert8'}
App{id=9, name='insert9'}
App{id=10, name='insert10'}
App{id=2, name='update'}
pageList = Page{count=true, pageNum=2, pageSize=3, startRow=3, endRow=6, total=9, pages=3, reasonable=false,
```

```
pageSizeZero=true}[App{id=6, name='insert6'}, App{id=7, name='insert7'}, App{id=8, name='insert8'}]
User{id=3, name='insert'}
User{id=4, name='insert'}
User{id=5, name='insert'}
User{id=6, name='insert'}
User{id=7, name='insert'}
User{id=8, name='insert'}
User{id=9, name='insert'}
User{id=10, name='insert'}
User{id=2, name='update'}
usersByPage = [User{id=3, name='insert'}, User{id=4, name='insert'}, User{id=5, name='insert'}]
User{id=3, name='insert'}
User{id=4, name='insert'}
User{id=5, name='insert'}
User{id=6, name='insert'}
User{id=7, name='insert'}
User{id=8, name='insert'}
User{id=9, name='insert'}
User{id=10, name='insert'}
User{id=2, name='update'}
```

2.2.5.3　项目代码介绍

点击下载 mybatis-kingbase 示例工程，这是一个名为 mybatis-kingbase 的压缩包。解压后，得到一个名为 mybatis-kingbase 的文件夹。目录结构如下所示：

```
--pom.xml

 src

  main
    java
        com
           kingbase
               mapper
                  ------IAppMapper.java
                   -----IUserMapper.java

               pojo
                  ---App.java
                   --User.java

    resources
        --jdbc.properties
```

```
      --mybatis-config.xml
      -IUserMapper.xml

  test
      java
          ---TestMybatis.java

target
```

文件说明:

- pom.xml：Maven 项目的配置文件，包含了项目的依赖、插件、构建等信息。
- src：通常用于表示项目中存放源代码的目录。
- main: 存放主要的源代码和资源文件的目录。
- java: 存放 Java 源代码的目录。
- com: 存放 Java 包的根目录。
- kingbase: 存放项目的根目录。
- mapper: 存放 MyBatis 的 Mapper 接口和 XML 文件。
- IAppMapper.java：存放应用数据访问层接口。
- IUserMapper.java：存放用户数据访问层接口。
- pojo: 存放实体类的目录。
- App.java：存放应用实体类。
- User.java：存放用户实体类。
- resources: 存放资源文件的目录，如配置文件、SQL 文件等。
- jdbc.properties: 存放数据库连接信息的配置文件。
- mybatis-config.xml: 存放 MyBatis 的配置文件。
- IUserMapper.xml：存放用户数据访问层的 XML 配置文件。
- test: 存放测试代码和资源文件的目录。
- TestMybatis.java: 存放测试 MyBatis 的 Java 类。
- target: 存放编译后的 Class 文件、Jar 包等文件的目录

2.2.5.3.1　pom.xml 代码介绍

如果您只是想验证示例，那么请使用默认代码，无需修改。您也可以按照以下讲解，根据自己的需求修改 pom.xml 文件。

pom.xml 配置文件内容如下

1. 文件声明语句。声明本文件是一个 XML 文件，使用的 XML 版本是 1.0 ，字符编码方式是 UTF-8 。代码如下：

```
<?xml version="1.0" encoding="UTF-8"?>
```

2. 配置 POM 的命名空间和 POM 模型版本。
   - 通过 xmlns 指定 POM 的命名空间为 `http://maven.apache.org/POM/4.0.0`。
   - 通过 xmlns:xsi 指定 XML 命名空间为 `http://www.w3.org/2001/XMLSchema-instance`。
   - 通过 xsi:schemaLocation 指定 POM 的命名空间为 `http://maven.apache.org/POM/4.0.0`，并指定 POM 的 XSD 文件的位置为 `https://maven.apache.org/xsd/maven-4.0.0.xsd`。
   - 通过 modelVersion 元素指定该 POM 文件使用的 POM 模型版本为 `4.0.0`。

   代码如下:

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance
"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
</project>
```

3. 配置基本信息。
   - 通过 groupId 指定项目标识为 com.kingbase。
   - 通过 artifactId 指定项目依赖为 java-kingbase-springboot。
   - 通过 version 指定项目的版本号为 0.0.1-SNAPSHOT。

   代码如下:

```
<groupId>com.kingbase.example</groupId>
<artifactId>java-kingbase-mybatis</artifactId>
<version>1.0-SNAPSHOT</version>
```

4. 通过 build 定义项目的构建过程。
   - 通过 plugins 指定项目中配置的插件。
   - 通过 plugin 指定项目中配置一个插件。
   - 通过 groupId 指定项目标识为 org.apache.maven.plugins。
   - 通过 artifactId 指定项目依赖为 maven-compiler-plugin。
   - 通过 configuration 指定配置的插件的参数。
   - 通过 source 指定编译器的源代码版本为 8。
   - 通过 target 指定编译器的源代码版本为 8。

代码如下：

```
<build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <configuration>
                    <source>8</source>
                    <target>8</target>
                </configuration>
            </plugin>
        </plugins>
</build>
```

5. 通过 dependencies 定义项目所依赖组件。

   - 指定依赖项所属的组织为 cn.com.kingbase ， 名称为 kingbase8，版本号为 9.0.0 。

   代码如下:

```
<dependency>
            <groupId>cn.com.kingbase</groupId>
            <artifactId>kingbase8</artifactId>
            <version>9.0.0</version>
</dependency>
```

   - 指定依赖项所属测试架构为 junit ， 名称为 junit ， 版本号为 4.10 。

   代码如下:

```
<dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.10</version>
</dependency>
```

   - 指定依赖项所属架构为 org.mybatis ， 名称为 mybatis ， 版本号为 3.5.9 。

   代码如下:

```
<dependency>
            <groupId>org.mybatis</groupId>
            <artifactId>mybatis</artifactId>
            <version>3.5.9</version>
</dependency>
```

   - 指定依赖项所属插件为 com.github.pagehelper ， 名称为 pagehelper ， 版本号为 5.3.0

代码如下：

```
<dependency>
            <groupId>com.github.pagehelper</groupId>
            <artifactId>pagehelper</artifactId>
            <version>5.3.0</version>
</dependency>
```

2.2.5.3.2　jdbc.properties 文件介绍

jdbc.properties 是一个属性文件，用于存储与数据库连接相关的配置信息，包括数据库的 URL、用户名、密码等。代码包含的内容如下：

代码中各字段的值从步骤一：获取 Kingbase 数据库连接串中直接获取。

- jdbc.driver：数据库驱动程序的类名，用于加载数据库驱动程序。
- jdbc.url：数据库的 URL 地址，用于指定连接的数据库。
- jdbc.username：数据库的用户名，用于验证数据库连接。
- jdbc.password：数据库的密码，用于验证数据库连接。

代码如下:

```
jdbc.driver=com.kingbase8.Driver
jdbc.url=jdbc:kingbase8://192.168.xx.xxx:7901/test
jdbc.username=system
jdbc.password=******
```

2.2.5.3.3　mybatis-config.xml 介绍

mybatis-config.xml 文件用于配置 MyBatis 框架的全局属性和插件等。

mybatis-config.xml 文件的代码主要包括以下几个部分：

1. 文件声明。

   声明本文件是一个 XML 文件，这个声明表示当前文档是一个 MyBatis 配置文件，使用的 XML 版本是 1.0 ，字符编码方式是 UTF-8 。版本号为 3.0，语言为英语，使用了 MyBatis 官方网站提供的 DTD（Document Type Definition）文件进行验证。

   - version：指定 XML 文件的版本。
   - encoding：指定 XML 文件的编码方式。
   - DOCTYPE：声明类型为文档类型声明。
   - configuration：指定文档类型为 MyBatis 配置文件。
   - PUBLIC：指定文档类型为公共文档类型。

- mybatis.org：表示 MyBatis 官方网站。
- DTD Config 3.0：表示 MyBatis 配置文件的版本号。
- EN：表示文档类型的语言为英语。

代码如下:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE configuration PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-config.dtd">
```

2. 指定要加载的属性文件的路径和文件名。

   代码如下：

```
<properties resource="jdbc.properties"></properties>
```

3. 配置 MyBatis 全局设置。

   MyBatis 全局设置的内容包含以下几个部分：

   - setting：用于配置单个设置项，例如缓存、日志等。
   - cacheEnabled：用于启用或禁用缓存功能，设置为 true。
   - lazyLoadingEnabled：用于启用或禁用延迟加载功能，设置为 true。
   - aggressiveLazyLoading：用于启用或禁用积极延迟加载功能，设置为 true。
   - multipleResultSetsEnabled：用于配置是否启用多结果集的支持，设置为 true。
   - useColumnLabel：用于配置否使用列标签作为结果集中的列名，设置为 true。
   - useGeneratedKeys：用于配置是否使用自动生成的主键，设置为 true。
   - autoMappingBehavior：，用于配置对自动映射的处理行为设置为 PARTIAL。
   - defaultExecutorType：用于配置默认的执行器类型，设置为 SIMPLE。
   - mapUnderscoreToCamelCase：用于配置是否将数据库列名中的下划线转换为 Java 对象属性名中的驼峰命名法，设置为 true。
   - localCacheScope：用于配置本地缓存的作用域，设置为 SESSION。
   - jdbcTypeForNull：用于配置在处理空值时使用的 JDBC 类型，设置为 NULL。

   settings 元素中的子元素是可选的，可以根据实际情况进行添加或删除。

   代码如下：

```
<settings>
        <setting name="cacheEnabled" value="true"/>
        <setting name="lazyLoadingEnabled" value="true"/>
        <setting name="aggressiveLazyLoading" value="true"/>
```

```
        <setting name="multipleResultSetsEnabled" value="true"/>
        <setting name="useColumnLabel" value="true"/>
        <setting name="useGeneratedKeys" value="true"/>
        <setting name="autoMappingBehavior" value="PARTIAL"/>
        <setting name="defaultExecutorType" value="SIMPLE"/>
        <setting name="mapUnderscoreToCamelCase" value="true"/>
        <setting name="localCacheScope" value="SESSION"/>
        <setting name="jdbcTypeForNull" value="NULL"/>
</settings>
```

4. 配置 MyBatis 的插件。

   MyBatis 的插件内容包含以下几部分：

   - plugin：用于配置单个插件。
   - property：用于指定插件的属性。
   - interceptor：用于指定插件的实现类。
   - helperDialect：用于指定所选的数据库。
   - offsetAsPageNum：表示是否将 offset 参数作为 pageNum 参数使用。
   - rowBoundsWithCount：表示是否进行 count 查询。
   - pageSizeZero：表示是否支持 pageSize 为 0 的查询。
   - reasonable：表示是否启用合理化查询。
   - params：表示用于传递分页参数的参数名和参数值的映射关系。
   - supportMethodsArguments：表示是否支持使用方法参数来传递分页参数。
   - returnPageInfo：表示返回值类型。

   代码如下：

```
<plugins>
        <plugin interceptor="com.github.pagehelper.PageInterceptor">
            <property name="helperDialect" value="mysql"/>
            <property name="offsetAsPageNum" value="true"/>
            <property name="rowBoundsWithCount" value="true"/>
            <property name="pageSizeZero" value="true"/>
            <property name="reasonable" value="false"/>
            <property name="params" value="pageNum=start;pageSize=limit;"/>
            <property name="supportMethodsArguments" value="true"/>
            <property name="returnPageInfo" value="check"/>
        </plugin>
</plugins>
```

5. 配置 MyBatis 的环境。MyBatis 的环境内容包含以下几个部分：

- environment：用于配置单个环境。
- transactionManager：用于指定事务管理器的实现类。
- dataSource：用于指定数据源的实现类。
- property：指定数据库驱动程序的类名。

在 MyBatis 的其他配置元素中，可以使用 ${} 占位符来引用环境中配置的数据源、事务管理器和执行器等。

代码如下：

```
<environments default="development">
        <environment id="development">
            <transactionManager type="JDBC"/>
            <dataSource type="POOLED">
                <property name="driver" value="${jdbc.driver}"/>
                <property name="url" value="${jdbc.url}"/>
                <property name="username" value="${jdbc.username}"/>
                <property name="password" value="${jdbc.password}"/>
            </dataSource>
        </environment>
</environments>
```

6. 配置 MyBatis 的映射器。

   MyBatis 的映射器常用元素如下：

   - resource：用于指定映射器的 XML 配置文件路径和文件名。
   - class：用于指定映射器的 Java 类名。

   如果使用 XML 配置文件，需要在 mapper 元素中指定 XML 文件的路径和文件名；如果使用 Java 接口，需要在 mapper 元素中指定 Java 类的全限定名。

   代码如下：

```
<mappers>
        <mapper resource="IUserMapper.xml"></mapper>
        <mapper class="com.kingbase.mapper.IAppMapper"></mapper>
</mappers>
```

2.2.5.3.4　IUserMapper.xml 代码介绍

IUserMapper.xml 文件是个映射文件，用于定义与用户对象相关的 SQL 语句。该文件中定义了一个名为 IUserMapper 的命名空间，用于存放与用户对象相关的 SQL 语句。如果您只是想验证示例，那么请使用默认代码，无需修改。若需要修改，请根据实际情修改。

IUserMapper.xml 文件的代码主要包括以下几个部分：

1. 文件声明语句。XML 文件的声明部分，用于指定 XML 文件的版本和编码方式。DTD 文件的声明部分，用于引入 MyBatis 的 DTD 文件。

   代码如下：

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper
PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
```

2. 配置与 IUserMapper.java 的映射关系。

   - 定义 Mapper 接口的命名空间，用于与 Java 代码中的 Mapper 接口进行对应。
   - 创建名为 insertUser 的 SQL 语句，向 test_user 表中插入一条记录，包括 id 和 name 两个字段，值分别为 #{id} 和 #{name}。parameterType 属性指定了传入的参数类型为 com.kingbase.pojo.User。
   - 创建名为 deleteUser 的 SQL 语句，从 test_user 表中删除一条记录，根据 id 字段的值匹配相应的记录，值为 #{id}。
   - 创建名为 updateUser 的 SQL 语句，用于更新 test_user 表中的一条记录，将 name 字段的值更新为 #{name}，根据 id 字段的值匹配相应的记录，值为 #{id}。
   - 创建名为 selectUsers 的 SQL 语句，用于查询 test_user 表中的所有用户记录。
   - 创建名为 selectUserByPage 的 SQL 语句，从 test_user 表中查询指定页数的用户对象。使用子查询和 rownum 字段进行分页查询，其中 #{pageNum} 和 #{pageSize} 表示当前页码和每页的记录数。

   代码如下：

```
<mapper namespace="com.kingbase.mapper.IUserMapper">
    <insert id="insertUser" parameterType="com.kingbase.pojo.User">
        INSERT INTO test_user (id,name) VALUES (#{id},#{name})
    </insert>
    <delete id="deleteUser" parameterType="long">
        DELETE FROM test_user WHERE id = #{id}
    </delete>
    <update id="updateUser" parameterType="com.kingbase.pojo.User">
        UPDATE test_user SET name = #{name} WHERE id = #{id}
    </update>
    <select id="selectUsers" resultType="com.kingbase.pojo.User">
        SELECT id,name FROM test_user
    </select>
    <!--    There are two ways to paginate queries: 1. Use the pagehelper plugin; 2. Use SQL statements
to paginate  -->
    <!-- SQL statement pagination: Oracle mode does not support the limit keyword, and instead uses the
unique field 'rownum'-->
    <select id="selectUserByPage" resultType="com.kingbase.pojo.User">
        select id,name from test_user limit #{pageNum},#{pageSize}
```

```
    </select>
</mapper>
```

2.2.5.3.5　IAppMapper.java 代码介绍

IAppMapper.java 文件用于定义 SQL 映射关系。

IAppMapper.java 文件的代码主要包括以下几个部分：

1. 定义 Mapper 包。

   声明当前文件所属的包名为 com.kingbase.mapper 。

   Mapper 包中的以下接口和类：

   - App 接口：表示与数据库表进行映射，实现数据的读写操作。
   - org.apache.ibatis.annotations.*：用于导入 MyBatis 的注解类。
   - java.util.List：用于导入 java.util 包中的 List 类。代码如下：

   代码如下：

```
package com.kingbase.mapper;
import com.kingbase.pojo.App;
import org.apache.ibatis.annotations.*;
import java.util.List;
```

2. 定义 IAppMapper 接口。

   IAppMapper 接口用于定义 MyBatis 的 Mapper 接口。Mapper 接口用于定义 SQL 映射关系，实现对数据库的增删改查操作。具体来说，IAppMapper 接口定义了对 test_app 表进行增删改查的方法，包括：

   - @Insert 方法：用于向数据库中插入数据。
   - @Update 方法：用于更新数据库中的数据。
   - @Delete 方法：用于删除数据库中的数据。
   - @Select 方法：用于查询数据库中的数据。
   - @Results 方法：表示查询结果的映射关系，用于将查询结果中的字段映射到 Java 对象的属性上。

   对数据库的增删改查操作如下：

   - 插入数据：将 App 对象映射到 test_app 表中的一条记录，App 对象中的属性值使用 #{属性名} 的方式进行占位符表示。

     Integer 类型的返回值表示 SQL 语句插入数据后返回的自动生成的 ID，即 test_app 表中的 ID 字段对应的值。

```
@Insert("insert into test_app(id,name) values(#{id},#{name})")
Integer insertApp(App app);
```

- 删除数据：从 test_app 表中删除 id 等于 #{id} 的数据。

```
@Delete("delete from test_app where id =#{id}")
Integer deleteApp(Long id);
```

- 更新数据：对 test_app 表中的记录进行修改需要进行 Update 操作。

```
@Update("update test_app set name= #{name} where id = #{id}")
Integer updateApp(App user);
```

- 查询数据并映射数据：查询 test_app 表中的所有数据。同时，使用 @Results 注解和 @Result 注解将查询结果映射到 App 对象中的 id 和 name 属性中。最终返回一个 App 对象的列表。

```
@Select("select * from test_app")
@Results({
        @Result(id = true, column = "id", property = "id"),
        @Result(column = "name", property = "name")
    })
List<App> selectApps();
```

完整代码如下：

```
package com.kingbase.mapper;
import com.kingbase.pojo.App;
import org.apache.ibatis.annotations.*;
import java.util.List;

//using annotations
public interface IAppMapper {
    @Insert("insert into test_app(id,name) values(#{id},#{name})")
    Integer insertApp(App app);

    @Delete("delete from test_app  where id =#{id}")
    Integer deleteApp(Long id);

    @Update("update test_app set name= #{name} where id = #{id}")
    Integer updateApp(App user);

    @Select("select * from test_app")
    @Results({
            @Result(id = true, column = "id", property = "id"),
            @Result(column = "name", property = "name")
    })
    List<App> selectApps();
}
```

2.2.5.3.6　IUserMapper.java 代码介绍

IUserMapper.java 文件用于定义数据库操作的方法。

IUserMapper.java 文件的代码主要包括以下几个部分：

1. 引用其他类和接口。声明当前文件包含的接口和类：
   - User 类：表示用户对象。
   - org.apache.ibatis.annotations.Param 类：MyBatis 框架中的参数注解。
   - List 接口：表示列表类型。

   代码如下：

```
package com.kingbase.mapper;
import com.kingbase.pojo.User;
import org.apache.ibatis.annotations.Param;
import java.util.List;
```

2. 定义 IUserMapper 接口。

   定义了一个名为 IUserMapper 的接口，其中定义了插入、删除、更新和查询用户数据的方法，以及分页查询用户数据的方法。同时使用了 JavaDoc 注释和 @Param 注解来提高代码的可读性和可维护性。

   代码如下：

```
//using XML
public interface IUserMapper {
    Integer insertUser(User user);
    Integer deleteUser(Long id);
    Integer updateUser(User user);
    List<User> selectUsers();
    public List<User> selectUserByPage(@Param("user") User user, @Param("pageNum") Integer pageNum,
                                       @Param("pageSize") Integer pageSize);
}
```

2.2.5.3.7　App.java 代码介绍

App.java 文件用于表示 App 应用程序对象。其中包括了 id 和 name 两个属性。类中还定义了属性的访问方法和构造方法，以及将对象转化为字符串的 toString 方法。

App.java 文件的代码主要包括以下几个部分：

- 定义 pojo 包。声明当前文件所属的包名为 com.kingbase.pojo。
- 定义 App 类。

- 定义了一个类 App，其中包含了两个私有属性 id 和 name，以及一个无参构造方法 public App(){}。
- 定义一个带参数的构造方法 public App(Long id, String name)，用于创建一个具有指定 id 和 name 的应用程序对象。
- 定义 getId 方法、setId 方法、getName 方法和 setName 方法，用于获取和设置应用程序名称，通过 setName 方法返回设置应用程序的名称。
- 通过重写 toString 方法，将应用程序对象转换为字符串表示，便于输出和调试。

代码如下：

```
package com.kingbase.pojo;

public class App {
    private Long id;
    private String name;

    public App() {
    }

    public App(Long id, String name) {
        this.id = id;
        this.name = name;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    @Override
    public String toString() {
        return "App{" +
                "id=" + id +
                ", name='" + name + '\'' +
                '}';
```

```
    }
}
```

2.2.5.3.8　User.java 代码介绍

User.java 文件用于表示用户对象。其中包括了 id 和 name 两个属性。类中还定义了属性的访问方法和构造方法，以及将对象转化为字符串的 toString 方法。该类的创建同上面 App.java 代码的创建类似。

代码如下：

```
package com.kingbase.pojo;

public class User {
    private Long id;
    private String name;

    public User() {
    }

    public User(Long id, String name) {
        this.id = id;
        this.name = name;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    @Override
    public String toString() {
        return "User{" +
                "id=" + id +
                ", name='" + name + '\'' +
```

```
                '}';
    }
}
```

2.2.5.3.9　TestMybatis.java 文件介绍

TestMybatis.java 用于展示如何使用 MyBatis 进行数据操作。测试 MyBatis 框架的基本功能和 SQL 语句的执行情况，以及 Mapper 接口的调用方式和 SQL 语句的参数和返回值等。

1. 引用其他类和接口。

   用于导入需要使用的类和接口如下：

   - IAppMapper 接口：用于定义与 App 对象相关的 SQL 语句。
   - IUserMapper 接口：用于定义与 User 对象相关的 SQL 语句。
   - App 类：App 对象，用于测试 SQL 语句的执行情况。
   - User 类：User 对象，用于测试 SQL 语句的执行情况。
   - PageHelper 插件：用于实现分页查询功能。
   - PageInfo 插件：用于封装分页查询结果。
   - Resources 类：用于加载 MyBatis 配置文件。
   - SqlSession 类：用于执行 SQL 语句和管理事务。
   - SqlSessionFactory 类：用于创建 SqlSession 对象。
   - SqlSessionFactoryBuilder 类：用于创建 SqlSessionFactory 对象。
   - org.junit.Test：JUnit 测试框架的注解，用于标记测试方法。
   - IOException 类：用于表示输入输出操作中的错误。
   - SQLException 类：用于表示 SQL 操作中的错误。
   - Statement 接口：用于执行 SQL 语句并返回结果。
   - java.util.List 接口：用于表示一个有序的集合，其中的元素可以重复。

   代码如下：

```
import com.kingbase.mapper.IAppMapper;
import com.kingbase.mapper.IUserMapper;
import com.kingbase.pojo.App;
import com.kingbase.pojo.User;
import com.github.pagehelper.PageHelper;
import com.github.pagehelper.PageInfo;
import org.apache.ibatis.io.Resources;
import org.apache.ibatis.session.SqlSession;
```

```
import org.apache.ibatis.session.SqlSessionFactory;
import org.apache.ibatis.session.SqlSessionFactoryBuilder;
import org.junit.Test;

import java.io.IOException;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.List;
```

2. 定义 testUserMapper 方法。

testUserMapper 方法用于测试 User 对象的 Mapper 接口中定义的 SQL 语句的执行情况。

- 使用 SqlSessionFactoryBuilder 类的 build() 方法，创建一个 SqlSessionFactory 实例。用来管理 SqlSession 实例的创建和销毁。
- 使用 SqlSessionFactory 的 openSession() 方法，创建一个 SqlSession 实例。可以执行各种 SQL 语句，包括查询、插入、更新和删除等操作。
- 使用 SqlSession 实例的 getConnection() 方法，获取一个 Connection 实例。可以执行各种数据库操作。
- 使用 Connection 实例的 createStatement() 方法，创建一个 Statement 实例。依次执行 SQL 语句。
- 使用 Statement 对象的 execute() 方法，执行删除名为 test_user 表的 SQL 语句。
- 使用 Statement 对象的 execute() 方法，创建名为 test_user 表。该表包含两个字段，一个是 id，类型为 number(20)，作为主键；另一个是 name，类型为 varchar2(100)。
- 使用 SqlSession 实例的 getMapper() 方法，获取 IUserMapper 接口的实例。定义各种数据库操作方法。
- 使用 for 循环插入 10 条数据到 test_user 表，在每次循环中，创建一个新的 User 对象，使用 Mapper 接口的 insertUser() 方法，执行插入操作。将执行插入操作结果会存储在 insertResult 变量中。
- 对 test_user 表中的数据进行删除、更新、查询，最后使用 forEach() 方法打印出该用户列表中的每个用户的信息。
- 创建名为 insert 的用户，使用 selectUserByPage() 方法查询 test_user 表中所有名称为“insert”的用户数据，并返回指定页面的用户列表。查询的是第 2 页，每页显示 3 条数据。并将这些用户数据打印到控制台上。同时关闭 Statement 和 SqlSession 实例，并提交事务，释放资源。

代码如下：

```
public void testUserMapper() throws SQLException, IOException {
        //mybatis xml usecases
        SqlSessionFactory sqlSessionFactory = new SqlSessionFactoryBuilder().build(Resources.
getResourceAsStream("mybatis-config.xml"));
        SqlSession sqlSession = sqlSessionFactory.openSession();
        Statement statement = sqlSession.getConnection().createStatement();
        try {
            statement.execute("drop table if exists test_user");
```

```
        } catch (SQLException ex) {
        } finally {
            statement.execute("create table test_user(id number(20) primary key,name varchar(100))");
        }
        IUserMapper mapper = sqlSession.getMapper(IUserMapper.class);
        //insert 10 users
        for (int i = 1; i <= 10; i++) {
            User user = new User((long) i, "insert");
            Integer insertResult = mapper.insertUser(user);
        }
        //delete id==1
        Integer deleteResult = mapper.deleteUser(1L);
        //update id==2L name=update
        User updateUser = new User(2L, "update");
        Integer updateResult = mapper.updateUser(updateUser);
        //selectUsers query all
        List<User> userList = mapper.selectUsers();
        userList.forEach(System.out::println);
        //selectUsersByPage: use the rownum keyword in SQL statements to manually perform pagination
queries,
        // example: data on page 2 (3 items per page)
        User user = new User();
        user.setName("insert");

        List<User> usersByPage = mapper.selectUserByPage(user, 0, 3);
        System.out.println("usersByPage = " + usersByPage);
        statement.close();
        sqlSession.commit();
        sqlSession.close();
    }
```

3. 定义 testSqlSession 方法。

   testSqlSession 方法用于测试 SqlSession 对象的基本功能，包括执行 SQL 语句、提交事务和关闭 SqlSession 对象等。

   - 使用 SqlSessionFactoryBuilder 类的 build() 方法，创建一个 SqlSessionFactory 实例。用来管理 SqlSession 实例的创建和销毁。
   - 使用 SqlSessionFactory 的 openSession() 方法，创建一个 SqlSession 实例。可以执行各种 SQL 语句，包括查询、插入、更新和删除等操作。
   - 使用 SqlSession 实例的 getConnection() 方法，获取一个 Connection 实例。可以执行各种数据库操作。
   - 使用 Connection 实例的 createStatement() 方法，创建一个 Statement 实例。依次执行 SQL 语句。
   - 使用 Statement 对象的 execute() 方法，执行删除名为 test_user 表的 SQL 语句。

- 使用 Statement 对象的 execute() 方法，创建名为 test_user 表。该表包含两个字段，一个是 id，类型为 number(20)，作为主键；另一个是 name，类型为 varchar2(100)。
- 使用 for 循环插入 10 条数据到 test_user 表，在每次循环中，创建一个新的 User 对象，使用 Mapper 接口的 insertUser() 方法，执行插入操作。将执行插入操作结果会存储在 insertResult 变量中。
- 使用 sqlSession 的 delete() 方法，执行删除操作。通过传入的参数 1L 来设置删除条件。将删除操作的结果存储在 deleteResult 变量中。
- 使用 sqlSession 对象执行了数据库更新操作。创建一个 User 对象，调用 update 方法传入 SQL 语句的标识符和参数对象，完成更新操作。具体的 SQL 语句和参数映射关系可以在“com.kingbase.mapper.IUserMapper”接口中的 XML 配置文件中找到。更新操作的结果存储在 updateResult 变量中。
- 使用 SqlSessionFactory 的 openSession() 方法创建了一个 SqlSession 实例，通过该实例执行了查询操作，并将查询结果存储在 userList 变量中。最后，使用 forEach 方法遍历 userList 并输出到控制台。
- 通过 statement.close() 关闭数据库连接中的 Statement 对象。然后，通过 sqlSession.commit() 提交事务，将所有的修改持久化到数据库中。最后，通过 sqlSession.close() 关闭 SqlSession 对象，释放相关的资源并关闭与数据库的连接。

代码如下：

```
@Test
    public void testSqlSession() throws SQLException, IOException {
        //SqlSession usecases
        SqlSessionFactory sqlSessionFactory = new SqlSessionFactoryBuilder().build(Resources.
getResourceAsStream("mybatis-config.xml"));
        SqlSession sqlSession = sqlSessionFactory.openSession();
        Statement statement = sqlSession.getConnection().createStatement();
        try {
            statement.execute("drop table if exists test_user");
        } catch (SQLException ex) {
        } finally {
            statement.execute("create table test_user(id number(20) primary key,name varchar(100))");
        }
        //insert
        for (int i = 1; i <= 10; i++) {
            User user = new User((long) i, "insert");
            //Integer insertResult = mapper.insertUser(user);
            int insertResult = sqlSession.insert("com.kingbase.mapper.IUserMapper.insertUser", user);
        }
        //delete
        int deleteResult = sqlSession.delete("com.kingbase.mapper.IUserMapper.deleteUser", 1L);
        //update
        User updateUser = new User(2L, "update");
        int updateResult = sqlSession.update("com.kingbase.mapper.IUserMapper.updateUser", updateUser);
        //selectUsers
```

```
        List<User> userList = sqlSession.selectList("com.kingbase.mapper.IUserMapper.selectUsers",
null);
        userList.forEach(System.out::println);
        //System.out.println("userList = " + userList);
        statement.close();
        sqlSession.commit();
        sqlSession.close();
    }
```

4. 定义 testAppMapper 方法。

   testAppMapper 方法用于测试 AppMapper 的功能。

   - 使用 SqlSessionFactoryBuilder 类的 build() 方法，创建一个 SqlSessionFactory 实例。用来管理 SqlSession 实例的创建和销毁。
   - 使用 SqlSessionFactory 的 openSession() 方法，创建一个 SqlSession 实例。可以执行各种 SQL 语句，包括查询、插入、更新和删除等操作。
   - 使用 SqlSession 实例的 getConnection() 方法，获取一个 Connection 实例。可以执行各种数据库操作。
   - 使用 Connection 实例的 createStatement() 方法，创建一个 Statement 实例。依次执行 SQL 语句。
   - 使用 Statement 对象的 execute() 方法，执行删除名为 test_app 表的 SQL 语句。
   - 使用 Statement 对象的 execute() 方法，创建名为 test_app 表。该表包含两个字段，一个是 id，类型为 number(20)，作为主键；另一个是 name，类型为 varchar2(100)。
   - 使用 SqlSession 实例的 getMapper() 方法，获取 IAppMapper 接口的实例。定义各种数据库操作方法。
   - 使用 for 循环插入 10 条数据到 test_app 表，在每次循环中，创建一个新的 App 对象，使用 Mapper 接口的 insertApp() 方法，执行插入操作。将执行插入操作结果会存储在 insertResult 变量中。
   - 使用 mapper 的 delete() 方法，执行删除操作。通过传入的参数 1L 来设置删除条件。将删除操作的结果存储在 deleteResult 变量中。
   - 使用 mapper 对象执行了数据库更新操作。创建一个 App 对象，调用 update 方法传入 SQL 语句的标识符和参数对象，完成更新操作。
   - 创建一个 App 对象 updateApp，包含 id 为 2L 和 name 为“update”。
   - 调用 mapper 的 updateApp 方法，并传入 updateApp 对象，执行更新操作。
   - 调用 sqlSession 的 commit 方法，提交数据库事务。
   - 调用 mapper 的 selectApps 方法，查询所有 App 对象。使用 forEach 方法遍历 userList 并输出到控制台。
   - 调用 PageHelper 的 startPage 方法，并传入页码和每页数据条数，设置分页参数。查询所有 App 对象，并返回一个包含所有 App 对象的 List 对象。
   - 调用 PageInfo 的 getList 方法，获取分页后的 App 对象列表，使用 System.out.println 方法将分页后的 App 对象列表打印输出。

- 通过 sqlSession.close() 关闭 SqlSession 对象，释放相关的资源。

代码如下：

```
@Test
    public void testAppMapper() throws SQLException, IOException {
        //mybatis annotation usecases
        SqlSessionFactory sqlSessionFactory = new SqlSessionFactoryBuilder().build(Resources.
getResourceAsStream("mybatis-config.xml"));
        SqlSession sqlSession = sqlSessionFactory.openSession();
        Statement statement = sqlSession.getConnection().createStatement();
        try {
            statement.execute("drop table if exists test_app");
        } catch (SQLException ex) {
        } finally {
            statement.execute("create table test_app(id number(20) primary key,name varchar(100))");
        }

        IAppMapper mapper = sqlSession.getMapper(IAppMapper.class);

        //insert
        for (int i = 1; i <= 10; i++) {
            App app = new App((long) i, "insert" + i);
            Integer insertResult = mapper.insertApp(app);
        }

        //delete
        Integer deleteResult = mapper.deleteApp(1L);
        //update
        App updateApp = new App(2L, "update");
        Integer updateResult = mapper.updateApp(updateApp);
        //commit
        sqlSession.commit();

        //selectApps
        List<App> appList = mapper.selectApps();
        appList.forEach(System.out::println);

        //selectbyPage
        //set page parameters
        PageHelper.startPage(2, 3);
        //selectApps
        List<App> appList1 = mapper.selectApps();
        //get pageList
        PageInfo pageInfo = new PageInfo(appList1);
        List<App> pageList = pageInfo.getList();
```

```
        System.out.println("pageList = " + pageList);

        sqlSession.close();
    }
```

2.2.5.3.10　完整的代码展示

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
<groupId>com.kingbase.example</groupId>
<artifactId>java-kingbase-mybatis</artifactId>
<version>1.0-SNAPSHOT</version>
<build>
<plugins>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<configuration>
<source>8</source>
<target>8</target>
</configuration>
</plugin>
</plugins>
</build>
<dependencies>
<dependency>
<groupId>cn.com.kingbase</groupId>
<artifactId>kingbase8</artifactId>
<version>9.0.0</version>
</dependency>
<dependency>
<groupId>junit</groupId>
<artifactId>junit</artifactId>
<version>4.10</version>
</dependency>
<dependency>
<groupId>org.mybatis</groupId>
<artifactId>mybatis</artifactId>
<version>3.5.9</version>
```

```
</dependency>
<!--  pagehelper plug-in -->
<dependency>
<groupId>com.github.pagehelper</groupId>
<artifactId>pagehelper</artifactId>
<version>5.3.0</version>
</dependency>
</dependencies>
</project>
```

jdbc.properties

```
jdbc.driver=com.kingbase8.Driver
jdbc.url=jdbc:kingbase8://192.168.xx.xxx:7901/test
jdbc.username=system
jdbc.password=123456
```

mybatis-config.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE configuration PUBLIC "-//mybatis.org//DTD Config 3.0//EN" "http://mybatis.org/dtd/mybatis-3-
config.dtd">
<configuration>
<properties resource="jdbc.properties"></properties>
<settings>
<setting name="cacheEnabled" value="true"/>
<setting name="lazyLoadingEnabled" value="true"/>
<setting name="aggressiveLazyLoading" value="true"/>
<setting name="multipleResultSetsEnabled" value="true"/>
<setting name="useColumnLabel" value="true"/>
<setting name="useGeneratedKeys" value="true"/>
<setting name="autoMappingBehavior" value="PARTIAL"/>
<setting name="defaultExecutorType" value="SIMPLE"/>
<setting name="mapUnderscoreToCamelCase" value="true"/>
<setting name="localCacheScope" value="SESSION"/>
<setting name="jdbcTypeForNull" value="NULL"/>
</settings>
<plugins>
<plugin interceptor="com.github.pagehelper.PageInterceptor">

<property name="helperDialect" value="mysql"/>
```

```
<property name="offsetAsPageNum" value="true"/>
<property name="rowBoundsWithCount" value="true"/>
<property name="pageSizeZero" value="true"/>

<property name="reasonable" value="false"/>

<property name="params" value="pageNum=start;pageSize=limit;"/>
<property name="supportMethodsArguments" value="true"/>
<property name="returnPageInfo" value="check"/>
</plugin>
</plugins>
<environments default="development">
<environment id="development">
<transactionManager type="JDBC"/>
<dataSource type="POOLED">
<property name="driver" value="${jdbc.driver}"/>
<property name="url" value="${jdbc.url}"/>
<property name="username" value="${jdbc.username}"/>
<property name="password" value="${jdbc.password}"/>
</dataSource>
</environment>
</environments>
<mappers>
<mapper resource="IUserMapper.xml"></mapper>
<mapper class="com.kingbase.mapper.IAppMapper"></mapper>
</mappers>
</configuration>
```

IUserMapper.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper
PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="com.kingbase.mapper.IUserMapper">
<insert id="insertUser" parameterType="com.kingbase.pojo.User">
INSERT INTO test_user (id,name) VALUES (#{id},#{name})
</insert>
<delete id="deleteUser" parameterType="long">
DELETE FROM test_user WHERE id = #{id}
</delete>
<update id="updateUser" parameterType="com.kingbase.pojo.User">
UPDATE test_user SET name = #{name} WHERE id = #{id}
```

```
</update>
<select id="selectUsers" resultType="com.kingbase.pojo.User">
SELECT id,name FROM test_user
</select>
<!--    There are two ways to paginate queries: 1. Use the pagehelper plugin; 2. Use SQL statements to
paginate  -->
<!-- SQL statement pagination: Oracle mode does not support the limit keyword, and instead uses the unique
field 'rownum'-->
<select id="selectUserByPage" resultType="com.kingbase.pojo.User">
select id,name from test_user limit #{pageNum},#{pageSize}
</select>
</mapper>
```

IUserMapper.java

```
package com.kingbase.mapper;

//using XML
public interface IUserMapper {
Integer insertUser(User user);
Integer deleteUser(Long id);
Integer updateUser(User user);
List<User> selectUsers();
public List<User> selectUserByPage(@Param("user") User user, @Param("pageNum") Integer pageNum,
@Param("pageSize") Integer pageSize);
}
```

IAppMapper.java

```
package com.kingbase.mapper;



//using annotations
public interface IAppMapper {
@Insert("insert into test_app(id,name) values(#{id},#{name})")
Integer insertApp(App app);

@Delete("delete from test_app  where id =#{id}")
Integer deleteApp(Long id);

@Update("update test_app set name= #{name} where id = #{id}")
Integer updateApp(App user);
```

```
@Select("select * from test_app")
@Results({
@Result(id = true, column = "id", property = "id"),
@Result(column = "name", property = "name")
})
List<App> selectApps();
}
```

app.java

```
package com.kingbase.pojo;
public class App {
private Long id;
private String name;

public App() {
}

public App(Long id, String name) {
this.id = id;
this.name = name;
}

public Long getId() {
return id;
}

public void setId(Long id) {
this.id = id;
}

public String getName() {
return name;
}

public void setName(String name) {
this.name = name;
}

@Override
public String toString() {
return "App{" +
```

```
"id=" + id +
", name='" + name + '\'' +
'}';
}
}
```

user.java

```
package com.kingbase.pojo;
public class User {
private Long id;
private String name;

public User() {
}

public User(Long id, String name) {
this.id = id;
this.name = name;
}

public Long getId() {
return id;
}

public void setId(Long id) {
this.id = id;
}

public String getName() {
return name;
}

public void setName(String name) {
this.name = name;
}

@Override
public String toString() {
return "User{" +
"id=" + id +
", name='" + name + '\'' +
'}';
```

```
}
}
```

testMybatis.java

```
public class TestMybatis {
@Test
public void testUserMapper() throws SQLException, IOException {
//mybatis xml usecases
SqlSessionFactory sqlSessionFactory = new SqlSessionFactoryBuilder().build(Resources.getResourceAsStream(
"mybatis-config.xml"));
SqlSession sqlSession = sqlSessionFactory.openSession();
Statement statement = sqlSession.getConnection().createStatement();
try {
statement.execute("drop table if exists test_user");
} catch (SQLException ex) {
} finally {
statement.execute("create table test_user(id number(20) primary key,name varchar(100))");
}
IUserMapper mapper = sqlSession.getMapper(IUserMapper.class);
//insert 10 users
for (int i = 1; i <= 10; i++) {
User user = new User((long) i, "insert");
Integer insertResult = mapper.insertUser(user);
}
//delete id==1
Integer deleteResult = mapper.deleteUser(1L);
//update id==2L name=update
User updateUser = new User(2L, "update");
Integer updateResult = mapper.updateUser(updateUser);
//selectUsers query all
List<User> userList = mapper.selectUsers();
userList.forEach(System.out::println);
//selectUsersByPage: use the rownum keyword in SQL statements to manually perform pagination queries,
// example: data on page 2 (3 items per page)
User user = new User();
user.setName("insert");

List<User> usersByPage = mapper.selectUserByPage(user, 0, 3);
System.out.println("usersByPage = " + usersByPage);
statement.close();
```

```
sqlSession.commit();
sqlSession.close();
}

@Test
public void testSqlSession() throws SQLException, IOException {
//SqlSession usecases
SqlSessionFactory sqlSessionFactory = new SqlSessionFactoryBuilder().build(Resources.getResourceAsStream(
"mybatis-config.xml"));
SqlSession sqlSession = sqlSessionFactory.openSession();
Statement statement = sqlSession.getConnection().createStatement();
try {
statement.execute("drop table if exists test_user");
} catch (SQLException ex) {
} finally {
statement.execute("create table test_user(id number(20) primary key,name varchar(100))");
}
//insert
for (int i = 1; i <= 10; i++) {
User user = new User((long) i, "insert");
//Integer insertResult = mapper.insertUser(user);
int insertResult = sqlSession.insert("com.kingbase.mapper.IUserMapper.insertUser", user);
}
//delete
int deleteResult = sqlSession.delete("com.kingbase.mapper.IUserMapper.deleteUser", 1L);
//update
User updateUser = new User(2L, "update");
int updateResult = sqlSession.update("com.kingbase.mapper.IUserMapper.updateUser", updateUser);
//selectUsers
List<User> userList = sqlSession.selectList("com.kingbase.mapper.IUserMapper.selectUsers", null);
userList.forEach(System.out::println);
//System.out.println("userList = " + userList);
statement.close();
sqlSession.commit();
sqlSession.close();
}

@Test
public void testAppMapper() throws SQLException, IOException {
//mybatis annotation usecases
SqlSessionFactory sqlSessionFactory = new SqlSessionFactoryBuilder().build(Resources.getResourceAsStream(
"mybatis-config.xml"));
SqlSession sqlSession = sqlSessionFactory.openSession();
Statement statement = sqlSession.getConnection().createStatement();
try {
```

```
statement.execute("drop table if exists test_app");
} catch (SQLException ex) {
} finally {
statement.execute("create table test_app(id number(20) primary key,name varchar(100))");
}

IAppMapper mapper = sqlSession.getMapper(IAppMapper.class);

//insert
for (int i = 1; i <= 10; i++) {
App app = new App((long) i, "insert" + i);
Integer insertResult = mapper.insertApp(app);
}

//delete
Integer deleteResult = mapper.deleteApp(1L);
//update
App updateApp = new App(2L, "update");
Integer updateResult = mapper.updateApp(updateApp);
//commit
sqlSession.commit();

//selectApps
List<App> appList = mapper.selectApps();
appList.forEach(System.out::println);

//selectbyPage
//set page parameters
PageHelper.startPage(2, 3);
//selectApps
List<App> appList1 = mapper.selectApps();
//get pageList
PageInfo pageInfo = new PageInfo(appList1);
List<App> pageList = pageInfo.getList();
System.out.println("pageList = " + pageList);

sqlSession.close();
}
}
```

2.2.5.4　相关文档

Jdbc 更多示例请参考 Jdbc 产品手册的程序示例。

2.2.6 Hibernate

本文将介绍如何使用 Hibernate 框架和 Kingbase 数据库构建一个应用程序，实现创建表、插入数据和查询数据等基本操作。

点击下载 jdbc-kingbase-hibernate 示例工程。

2.2.6.1 前提条件

- 您已安装 Kingbase 数据库。
- 您已安装 JDK 1.8 和 Maven。
- 您已安装 IDE 工具。

备注:

本文档运行代码使用的工具是 IntelliJ IDEA 2023.2.8 (Community Edition) 版本，您也可以根据个人喜好选择适合自己的工具运行示例代码。

2.2.6.2 操作步骤

备注:

本文中给出的操作步骤是基于 Windows 环境生成。如果您使用的是其他操作系统环境或编译器，那么操作步骤可能会略有不同。

---

1. 获取 Kingbase 数据库连接信息。
2. 导入 jdbc-kingbase-hibernate 项目到 IDEA 中。
3. 修改 jdbc-kingbase-hibernate 项目中的数据库连接信息。
4. 运行 jdbc-kingbase-hibernate 项目。

2.2.6.2.1 步骤一：获取 Kingbase 数据库连接信息

1. 联系 Kingbase 数据库部署人员或者管理员获取相应的数据库连接信息。

```
./ksql -U system -d test -h xx.xx.xx.xx -p 54321
```

2. 根据已部署的 Kingbase 数据库填写下面的 URL 的对应信息。

备注:

在 jdbc.properties 文件中需要这里的 URL 信息。

```
jdbc.url=jdbc:kingbase8://host:port/database?user=$user_name&password=$password
```

参数说明:

- host：提供 Kingbase 数据库连接 IP。直连方式使用的是 KESServer 节点的 IP 地址。
- port：提供 Kingbase 数据库连接端口。直连方式默认是 54321，在部署 kingbase 数据库时可自定义。
- database：需要访问的数据库名称。
- user_name：通过 -u 参数指定，管理员用户为 system。
- password：提供账户密码。

更多 URL 参数说明信息请参考 Jdbc 连接属性。

2.2.6.2.2　步骤二: 导入 jdbc-kingbase-hibernate 项目到 IDEA 中

1. 打开 IntelliJ IDEA，选择 File > Open…选项。
2. 在弹出的 Open File or Project 窗口中，选择对应的 jdbc-kingbase-hibernate 项目文件夹，单击 OK 完成项目导入。
3. IntelliJ IDEA 将会自动识别项目中的各类文件，并在 Project 工具窗口中，可以查看项目的目录结构、文件列表、模块列表、依赖关系等信息。

   Project 工具窗口通常位于 IntelliJ IDEA 界面的最左侧，默认情况下是打开的。如果 Project 工具窗口被关闭了，可以通过点击菜单栏中的 View > Tool Windows > Project 或者使用快捷键 Alt + 1 来重新打开它。

备注:

当使用 IntelliJ IDEA 导入项目时，IntelliJ IDEA 会自动检测项目中的 pom.xml 文件，并根据文件中描述的依赖关系自动下载所需的依赖库，并将它们添加到项目中。

2.2.6.2.3　步骤三: 修改 jdbc-kingbase-hibernate 项目中的数据库连接信息

根据步骤一：获取 Kingbase 数据库连接信息中的信息修改 application.properties 文件中的数据库连接信息。

示例如下：

- 数据库驱动名称：`com.kingbase8.Driver`
- KES Server 节点的 IP 地址: `192.168.xx.xxx`。
- 访问端口: `54321`。
- 需要访问的 database 名称: `test`。

- 账户: system。
- 密码: **********。

配置示例如下:

```
<property name="hibernate.connection.driver_class">com.kingbase8.Driver</property>
<property name="hibernate.connection.url">jdbc:kingbase8://192.168.xx.xxx:54321/test</property>
<property name="hibernate.connection.username">system</property>
<property name="hibernate.connection.password">******</property>
```

2.2.6.2.4　步骤四: 运行 jdbc-kingbase-hibernate 项目

运行路径:

1. 在项目结构中找到 src > test > java 中找到 TestHibernate.java 文件。
2. 在工具菜单栏中选择运行 (U) > 运行 > TestHibernate(快捷键 Shift+F10)，或直接单击右上角绿色三角形运行 (快捷键 Shift+F10)。
3. 通过 IDEA 的控制台来查看项目的日志信息和输出结果。

运行结果:

```
User{id=3, name='user_insert3'}
User{id=4, name='user_insert4'}
User{id=5, name='user_insert5'}
User{id=2, name='update'}
```

完整的输出结果如下:

```
一月 20, 2025 5:33:53 下午 org.hibernate.Version logVersion
INFO: HHH000412: Hibernate Core {5.2.17.Final}
一月 20, 2025 5:33:53 下午 org.hibernate.cfg.Environment <clinit>
INFO: HHH000206: hibernate.properties not found
一月 20, 2025 5:33:53 下午 org.hibernate.annotations.common.reflection.java.JavaReflectionManager <clinit>
INFO: HCANN000001: Hibernate Commons Annotations {5.0.1.Final}
一月 20, 2025 5:33:54 下午 org.hibernate.engine.jdbc.connections.internal.ConnectionProviderInitiator
initiateService
INFO: HHH000130: Instantiating explicit connection provider: org.hibernate.c3p0.internal.
C3P0ConnectionProvider
一月 20, 2025 5:33:54 下午 org.hibernate.c3p0.internal.C3P0ConnectionProvider configure
INFO: HHH010002: C3P0 using driver: com.kingbase8.Driver at URL: jdbc:kingbase8://192.168.xx.xxx:54321/test
一月 20, 2025 5:33:54 下午 org.hibernate.c3p0.internal.C3P0ConnectionProvider configure
INFO: HHH10001001: Connection properties: {user=system, password=****}
一月 20, 2025 5:33:54 下午 org.hibernate.c3p0.internal.C3P0ConnectionProvider configure
INFO: HHH10001003: Autocommit mode: false
一月 20, 2025 5:33:54 下午 com.mchange.v2.log.MLog
```

```
信息: MLog clients using java 1.4+ standard logging.
一月 20, 2025 5:33:54 下午 com.mchange.v2.c3p0.C3P0Registry
信息: Initializing c3p0-0.9.5.2 [built 08-December-2015 22:06:04 -0800; debug? true; trace: 10]
一月 20, 2025 5:33:55 下午 org.hibernate.c3p0.internal.C3P0ConnectionProvider configure
INFO: HHH10001007: JDBC isolation level: <unknown>
一月 20, 2025 5:33:55 下午 com.mchange.v2.c3p0.impl.AbstractPoolBackedDataSource
信息: Initializing c3p0 pool... com.mchange.v2.c3p0.PoolBackedDataSource@85f1f3fe [ connectionPoolDataSource
-> com.mchange.v2.c3p0.WrapperConnectionPoolDataSource@6673fda1 [ acquireIncrement -> 3,
acquireRetryAttempts -> 30, acquireRetryDelay -> 1000, autoCommitOnClose -> false, automaticTestTable ->
null, breakAfterAcquireFailure -> false, checkoutTimeout -> 30000, connectionCustomizerClassName -> null,
connectionTesterClassName -> com.mchange.v2.c3p0.impl.DefaultConnectionTester, contextClassLoaderSource ->
caller, debugUnreturnedConnectionStackTraces -> false, factoryClassLocation -> null,
forceIgnoreUnresolvedTransactions -> false, forceSynchronousCheckins -> false, identityToken ->
1hgep1nb810d4x2du7uhz8|1ef6d34c, idleConnectionTestPeriod -> 3000, initialPoolSize -> 1,
maxAdministrativeTaskTime -> 0, maxConnectionAge -> 0, maxIdleTime -> 20000, maxIdleTimeExcessConnections ->
0, maxPoolSize -> 1, maxStatements -> 100, maxStatementsPerConnection -> 0, minPoolSize -> 1,
nestedDataSource -> com.mchange.v2.c3p0.DriverManagerDataSource@6f696b47 [ description -> null, driverClass
-> null, factoryClassLocation -> null, forceUseNamedDriverClass -> false, identityToken ->
1hgep1nb810d4x2du7uhz8|2cb2fc20, jdbcUrl -> jdbc:kingbase8://192.168.xx.xxx:54321/test, properties ->
{user=******, password=******} ], preferredTestQuery -> null, privilegeSpawnedThreads -> false,
propertyCycle -> 0, statementCacheNumDeferredCloseThreads -> 0, testConnectionOnCheckin -> false,
testConnectionOnCheckout -> false, unreturnedConnectionTimeout -> 0, usesTraditionalReflectiveProxies ->
false; userOverrides: {} ], dataSourceName -> null, extensions -> {}, factoryClassLocation -> null,
identityToken -> 1hgep1nb810d4x2du7uhz8|27adc16e, numHelperThreads -> 3 ]
一月 20, 2025 5:33:55 下午 org.hibernate.dialect.Dialect <init>
INFO: HHH000400: Using dialect: org.hibernate.dialect.Kingbase8Dialect
一月 20, 2025 5:33:55 下午 org.hibernate.type.BasicTypeRegistry register
INFO: HHH000270: Type registration [java.util.UUID] overrides previous : org.hibernate.type.
UUIDBinaryType@4cf8b2dc
Hibernate:

    drop table if exists test_hibernate_oracle cascade
一月 20, 2025 5:33:56 下午 org.hibernate.resource.transaction.backend.jdbc.internal.
DdlTransactionIsolatorNonJtaImpl getIsolatedConnection
INFO: HHH10001501: Connection obtained from JdbcConnectionAccess [org.hibernate.engine.jdbc.env.internal.
JdbcEnvironmentInitiator$ConnectionProviderJdbcConnectionAccess@5b989dc7] for (non-JTA) DDL execution was
not in auto-commit mode; the Connection 'local transaction' will be committed and the Connection will be set
into auto-commit mode.
Hibernate:

    drop sequence if exists hibernate_sequence
Hibernate: create sequence hibernate_sequence start 1 increment 1
一月 20, 2025 5:33:56 下午 org.hibernate.resource.transaction.backend.jdbc.internal.
DdlTransactionIsolatorNonJtaImpl getIsolatedConnection
INFO: HHH10001501: Connection obtained from JdbcConnectionAccess [org.hibernate.engine.jdbc.env.internal.
```

```
JdbcEnvironmentInitiator$ConnectionProviderJdbcConnectionAccess@6826c41e] for (non-JTA) DDL execution was
not in auto-commit mode; the Connection 'local transaction' will be committed and the Connection will be set
into auto-commit mode.
Hibernate:

    create table test_hibernate_oracle (
       USER_ID int4 not null,
        USER_NAME varchar(255),
        primary key (USER_ID)
    )
一月 20, 2025 5:33:56 下午 org.hibernate.tool.schema.internal.SchemaCreatorImpl applyImportSources
INFO: HHH000476: Executing import script 'org.hibernate.tool.schema.internal.exec.
ScriptSourceInputNonExistentImpl@41c89d2f'
Hibernate:
    select
        nextval ('hibernate_sequence')
Hibernate:
    insert
    into
        test_hibernate_oracle
        (USER_NAME, USER_ID)
    values
        (?, ?)
Hibernate:
    select
        nextval ('hibernate_sequence')
Hibernate:
    insert
    into
        test_hibernate_oracle
        (USER_NAME, USER_ID)
    values
        (?, ?)
Hibernate:
    select
        nextval ('hibernate_sequence')
Hibernate:
    insert
    into
        test_hibernate_oracle
        (USER_NAME, USER_ID)
    values
        (?, ?)
Hibernate:
    select
```

```
        nextval ('hibernate_sequence')
Hibernate:
    insert
    into
        test_hibernate_oracle
        (USER_NAME, USER_ID)
    values
        (?, ?)
Hibernate:
    select
        nextval ('hibernate_sequence')
Hibernate:
    insert
    into
        test_hibernate_oracle
        (USER_NAME, USER_ID)
    values
        (?, ?)
Hibernate:
    delete
    from
        test_hibernate_oracle
    where
        USER_ID=?
Hibernate:
    update
        test_hibernate_oracle
    set
        USER_NAME=?
    where
        USER_ID=?
Hibernate:
    SELECT
        *
    FROM
        test_hibernate_oracle
User{id=3, name='user_insert3'}
User{id=4, name='user_insert4'}
User{id=5, name='user_insert5'}
User{id=2, name='update'}
```

2.2.6.3　项目代码介绍

点击下载 jdbc-kingbase-hibernate 示例工程，这是一个名为 jdbc-kingbase-hibernate 的压缩包。解压后，得到一个名为 jdbc-kingbase-hibernate 的文件夹。目录结构如下所示：

```
--pom.xml

 lib
   hibernate-4.dialect.jar
 src
  main
    java
      com
         kingbase
              dao
                --UserDao.java

              pojo
                  --User.java

    resources
         --hibernate.cfg.xml

          com
             kingbase
                  pojo
                      --User.hbm.xml

  test
      java
          --TestHibernate.java

 target
```

文件说明:

- pom.xml：Maven 项目的配置文件，包含了项目的依赖、插件、构建等信息。
- lib: 存放项目方言包的根目录。
- hibernate-4.dialect.jar：方言包文件
- src：通常用于表示项目中存放源代码的目录。
- main: 存放主要的源代码和资源文件的目录。
- java: 存放 Java 源代码的目录。
- com: 存放 Java 包的根目录。

- kingbase: 存放项目的根目录。
- dao: 存放数据访问对象（Data Access Object）包，用于访问数据库或其他数据存储服务。
- UserDao.java：用户数据访问对象，用于操作用户数据的增删改查等操作。
- User.java：用户持久化对象，用于映射用户数据表的字段。
- pojo: 持久化对象（Plain Old Java Object），存放与数据库表对应的 Java 类。用于映射数据库表或其他数据存储结构。
- resources: 存放资源文件的目录，如配置文件、SQL 文件等。
- hibernate.cfg.xml: 存放 Hibernate 的配置文件，用于配置 Hibernate 的基本参数和设置数据源等信息。
- User.hbm.xml: 存放用户持久化对象的映射文件，用于定义用户持久化对象和用户数据表之间的映射关系。
- test: 存放测试代码和资源文件的目录。
- TestHibernate.java: 存放测试 Hibernate 的 Java 类。
- target: 存放编译后的 Class 文件、Jar 包等文件的目录。

2.2.6.3.1　pom.xml 代码介绍

如果您只是想验证示例，那么请使用默认代码，无需修改。您也可以按照以下讲解，根据自己的需求修改 pom.xml 文件。

pom.xml 配置文件内容如下

1. 文件声明语句。

   声明本文件是一个 XML 文件，使用的 XML 版本是 1.0 ，字符编码方式是 UTF-8 。

   代码如下：

```
<?xml version="1.0" encoding="UTF-8"?>
```

2. 配置 POM 的命名空间和 POM 模型版本。

   - 通过 xmlns 指定 POM 的命名空间为 `http://maven.apache.org/POM/4.0.0`。
   - 通过 xmlns:xsi 指定 XML 命名空间为 `http://www.w3.org/2001/XMLSchema-instance`。
   - 通过 xsi:schemaLocation 指定 POM 的命名空间为 `http://maven.apache.org/POM/4.0.0`，并指定 POM 的 XSD 文件的位置为 `https://maven.apache.org/xsd/maven-4.0.0.xsd`。
   - 通过 modelVersion 元素指定该 POM 文件使用的 POM 模型版本为 `4.0.0`。

   代码如下:

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance
"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
</project>
```

3. 配置基本信息。
   - 通过 groupId 指定项目标识为 com.kingbase。
   - 通过 artifactId 指定项目依赖为 java-kingbase-springboot。
   - 通过 version 指定项目的版本号为 0.0.1-SNAPSHOT。

   代码如下:

```
<groupId>com.kingbase</groupId>
<artifactId>jdbc-kingbase-hibernate</artifactId>
<version>1.0-SNAPSHOT</version>
```

4. 通过 <build> 定义项目的构建过程。
   - 通过 <plugins> 指定项目中配置的插件。
   - 通过 <plugin> 指定项目中配置一个插件。
   - 通过 <groupId> 指定项目标识为 org.apache.maven.plugins。
   - 通过 <artifactId> 指定项目依赖为 maven-compiler-plugin。
   - 通过 <configuration> 指定配置的插件的参数。
   - 通过 <source> 指定编译器的源代码版本为 8。
   - 通过 <target> 指定编译器的源代码版本为 8。

   代码如下：

```
<build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <configuration>
                    <source>8</source>
                    <target>8</target>
                </configuration>
            </plugin>
        </plugins>
</build>
```

5. 通过 <dependencies> 定义项目所依赖组件。
   - 指定依赖项所属的组织为 cn.com.kingbase ， 名称为 kingbase8，版本号为 9.0.0 。

     代码如下:

```
<dependency>
            <groupId>cn.com.kingbase</groupId>
            <artifactId>kingbase8</artifactId>
            <version>9.0.0</version>
</dependency>
```

- 指定依赖项所属测试架构为 junit ， 名称为 junit ， 版本号为 4.13。

  代码如下:

```
<dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13</version>
</dependency>
```

- 指定依赖项所属架构为 org.hibernate ， 核心库为 hibernate-core ， 版本号为 5.2.17.Final 。

  代码如下:

```
<dependency>
            <groupId>org.hibernate</groupId>
            <artifactId>hibernate-core</artifactId>
            <version>5.2.17.Final</version>
</dependency>
```

- 指定依赖项所属架构为 org.hibernate ， 数据源库为 hibernate-c3p0 ， 版本号为 5.2.17.Final

  代码如下：

```
<dependency>
            <groupId>org.hibernate</groupId>
            <artifactId>hibernate-c3p0</artifactId>
            <version>5.2.17.Final</version>
</dependency>
```

2.2.6.3.2　User.hbm.xml 代码介绍

User.hbm.xml 文件是一个 Hibernate 的映射文件，用于将 Java 对象映射到数据库中的表。User.hbm.xml 文件的代码主要包括以下几个部分：

- jdbc.driver：数据库驱动程序的类名，用于加载数据库驱动程序。
- jdbc.url：数据库的 URL 地址，用于指定连接的数据库。
- jdbc.username：数据库的用户名，用于验证数据库连接。
- jdbc.password：数据库的密码，用于验证数据库连接。

1. 文件声明语句。

   声明本文件是一个 XML 文件，使用的 XML 版本是 1.0 ，字符编码方式是 UTF-8 。同时指定了 Hibernate 映射文件的 DTD 版本和位置，指定 DTD 文档类型为 hibernate-mapping ，版本为 3.0，语言为 EN，文件所在的 URL 为 http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd

   代码如下:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
```

2. 配置映射文件。

   定义一个 Hibernate 映射文件，将名为 User 的实体类映射到名为 test_hibernate_oracle 的数据库表。

   - 通过 package 属性指定映射文件中的 Java 包为 com.kingbase.pojo。
   - 通过 class 标签映射数据表中的数据，name 属性指定 Java 类名为 User，table 属性指定数据库表名为 test_hibernate_oracle。
   - 通过 id 定义主键属性，name 属性指定 Java 类 User 的成员变量为 id，column 属性指定数据库字段名称为 USER_ID。
   - 通过 generator 定义主键生成策略，class 属性指定主键生成器的类型为 sequence，param 元素指定数据库中的序列名称为 SQ_USER，name 属性指定该属性对应数据库表中的 USER_NAME 列，并指定该属性的数据类型为字符串。

   代码如下:

```
<hibernate-mapping package="com.kingbase.pojo">
    <class name="User" table="test_hibernate_oracle">
        <!-- Configure primary key generation strategy -->
        <id name="id" column="USER_ID">
            <generator class="sequence">
                <param name="sequence">SQ_USER</param>
            </generator>
        </id>
        <!-- Configuration Tables and Properties -->
        <property name="name" column="USER_NAME" type="string"/>
    </class>
</hibernate-mapping>
```

2.2.6.3.3　hibernate.cfg.xml 代码介绍

hibernate.cfg.xml 文件是一个 Hibernate 的配置文件，用于配置 Hibernate 的运行环境和连接数据库的参数。

hibernate.cfg.xml 文件的代码主要包括以下几个部分：

1. 文件声明语句。

   声明本文件是一个 XML 文件，使用的 XML 版本是 1.0 ，字符编码方式是 UTF-8 。同时指定了 Hibernate 配置文件的 DTD 版本和位置，指定 DTD 文档类型为 hibernate-configuration ，版本为 3.0，语言为 EN，文件所在的 URL 为 http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd

   代码如下：

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
```

2. 配置 configuration 参数。

   根目录元素中通过元素设置 Hibernate 的会话工厂，用于创建和管理会话对象。会话工厂包括了连接数据库的驱动程序类名、URL、用户名和密码，连接池的相关参数，数据库方言、SQL 输出和格式化、自动创建数据库表、当前会话上下文、批量处理和二级缓存等配置。

   - 配置数据库信息。

     配置 Hibernate 与连接数据库的参数，包括了连接数据库的驱动程序类名、URL、用户名和密码。

     – 通过 hibernate.connection.driver_class 指定数据库驱动程序为 com.kingbase8.Driver，用于与 Kingbase 数据库建立连接。

     – 通过 hibernate.connection.url 元素指定数据库的 URL，包括数据库的地址、端口号、数据库名称等信息。

     – 通过 hibernate.connection.username 元素指定连接数据库的用户名。

     – 通过 hibernate.connection.password 元素指定连接数据库的密码。

     代码如下：

```
<property name="hibernate.connection.driver_class">com.kingbase8.Driver</property>
<property name="hibernate.connection.url">jdbc:kingbase8://192.168.xx.xxx:54321/test</property>
<property name="hibernate.connection.username">system</property>
<property name="hibernate.connection.password">******</property>
```

   - 配置连接池信息。

     配置 Hibernate 使用的连接池的相关参数。指定连接池为 C3P0，参数的设置包括连接池的最大和最小连接数、超时时间、连接的最大空闲时间、缓存的 Statement 对象数量、连接的空闲测试时间、一次性获取的连接数以及连接的验证方式。

     备注:

     以下参数仅供参考，并非完整支持的参数列表，若需要配置其他参数，请参考相关文档以获取更详细的参数信息。

- 通过 hibernate.connection.provider_class 指定连接池为 C3P0，用于管理数据库连接的创建和释放。
- 通过 hibernate.c3p0.max_size 指定连接池的最大连接数为 60。
- 通过 hibernate.c3p0.min_size 指定连接池的最小连接数为 30。
- 通过 hibernate.c3p0.checkoutTimeout 指定连接池获取连接的超时时间为 30000 毫秒。
- 通过 hibernate.c3p0.timeout 指定连接池的最大空闲时间为 2000 毫秒，超过该时间将被关闭。
- 通过 hibernate.c3p0.max_statements 指定连接池中缓存的最大 SQL 语句数为 100 条。
- 通过 hibernate.c3p0.idle_test_period 指定连接池中空闲连接的检测周期为 3000 毫秒。
- 通过 hibernate.c3p0.acquire_increment 指定连接池动态增长时的连接数增量为 3 条。
- 通过 hibernate.c3p0.validate 指定连接池连接时验证为 true。

代码如下：

```
<property name="hibernate.connection.provider_class">org.hibernate.c3p0.internal.
C3P0ConnectionProvider</property>
<property name="hibernate.c3p0.max_size">60</property>
<property name="hibernate.c3p0.min_size">30</property>

<property name="hibernate.c3p0.checkoutTimeout">30000</property>
<property name="hibernate.c3p0.timeout">20000</property>

<property name="hibernate.c3p0.max_statements">100</property>
<property name="hibernate.c3p0.idle_test_period">3000</property>
<property name="hibernate.c3p0.acquire_increment">3</property>
<property name="hibernate.c3p0.validate">true</property>
```

- 配置 SQL 交互信息。

  配置 Hibernate 与数据库连接的相关信息。包括数据库方言、SQL 打印和格式化、表的自动创建、当前会话上下文、批处理大小、缓存使用以及加载映射文件。

  - 通过 dialect 指定使用的数据库方言为 Kingbase8Dialect，用于以确保与特定数据库的兼容性。
  - 通过 hibernate.show_sql 设置是否在控制台上打印 Hibernate 生成的 SQL 语句。
  - 通过 hibernate.format_sql 设置是否格式化打印的 SQL 语句。
  - 通过 hbm2ddl.auto 设置自动创建表格。
  - 通过 current_session_context_class 指定使用线程级别的当前会话上下文。
  - 通过 hibernate.jdbc.batch_size 设置批处理的大小。
  - 通过 hibernate.cache.use_second_level_cache 设置是否启用二级缓存。
  - 指定加载映射文件为 com/kingbase/pojo/User.hbm.xml。

  代码如下：

```
<property name="dialect">org.hibernate.dialect.Kingbase8Dialect</property>
<property name="hibernate.show_sql">true</property>
<property name="hibernate.format_sql">true</property>
<property name="hbm2ddl.auto">create</property>
<property name="current_session_context_class">thread</property>
<property name="hibernate.jdbc.batch_size">10</property>
<property name="hibernate.cache.use_second_level_cache">false</property>
<mapping resource="com/kingbase/pojo/User.hbm.xml"/>
```

2.2.6.3.4　UserDao.java 代码介绍

UserDao.java 文件通过用户数据访问对象（DAO）类，使用 Session 对象实现了用户数据的增删改查操作。UserDao.java 文件的代码主要包括以下几个部分：

1. 引用其他类和接口。

   声明当前文件包含以下接口和类：

   - User 类：用于操作用户对象。
   - Session 类：用于与数据库进行交互。
   - SessionFactory 类：用于创建 Session 实例。
   - Transaction 类：用于管理数据库事务。
   - Configuration 类：用于加载 Hibernate 的配置文件。
   - Query 类：用于执行查询操作。
   - List 接口：用于操作查询结果集合。

   代码如下：

```
import com.kingbase.pojo.User;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
import org.hibernate.query.Query;

import java.util.List;
```

2. 定义 UserDao 类。

   通过为 UserDao 的 Java 类，作为数据访问对象，用于对用户对象进行持久化操作。该类封装了一系列与数据库交互相关的方法，对用户对象进行增删改查等操作。通过调用 UserDao 类中的方法，可以方便地与数据库进行交互，实现持久化操作。

   1. 对 UserDao 类实例进行初始化操作。

使用对象初始化块的方式，在类的实例化过程中初始化 Session 对象。首先创建一个 Configuration 对象，并调用 configure 方法来读取 Hibernate 的配置文件（hibernate.cfg.xml ）。然后，通过调用 buildSessionFactory 方法创建一个 SessionFactory 对象。最后，通过调用 openSession 方法创建一个 Session 对象，并将其赋值给 session 变量。

代码如下：

```
private Session session;
    {
        Configuration cfg = new Configuration().configure();
        // Create SessionFactory
        SessionFactory sessionFactory = cfg.buildSessionFactory();
        session = sessionFactory.openSession();
    } //Read the hibernate.cfg.xml file
```

2. 设定 ID 查询对应的用户信息。

   通过 selectUserById 方法，提供的 ID 参数查询对应的用户信息。使用 session 的 get 方法，通过传入的 User.class 和 ID 参数，从数据库中获取对应 ID 的 User 记录，并将其存储在 user 变量中。

   代码如下：

```
public User selectUserById(int ID) {
        User user = session.get(User.class, ID);
        return user;
    }
```

3. 根据用户名查询用户信息。

   通过 selectUserbyName 方法，根据用户名查询用户信息。首先定义 HQL（Hibernate Query Language）语句，用于查询指定名称的用户数据。其中表名为 test_hibernate_oracle ，表别名为 u ，查询字段名为 name 。通过使用 Session 对象的 createQuery 方法创建一个 Query 对象，并将 HQL 语句和实体类作为参数传入。使用 Query 对象的 setParameter 方法设置查询参数，其中 0 表示参数的位置，name 表示参数的值。使用 Query 对象的 list 方法执行查询，并将查询结果转换为 User 对象列表。

   代码如下：

```
public List<User> selectUserbyName(String name) {
        String sql = "FROM test_hibernate_oracle u WHERE u.name =?";
        Query<User> query = session.createQuery(sql, User.class);
        query.setParameter(0, name);
        List<User> users = query.list();
        return users;
    }
```

4. 查询所有用户数据。

   通过 selectUser 方法，实现了查询所有用户数据的方法。定义一个 SQL 语句，用于查询所有用户数据。使用 Session 对象的 createNativeQuery 方法创建一个 Query 对象，并将 SQL 语句作为参数传入。然后，使

用 addEntity 方法将实体类 User 添加到查询中，以便将查询结果转换为 User 对象列表。使用 Query 对象的 getResultList 方法执行查询，并将查询结果转换为 User 对象列表。

代码如下：

```
public List<User> selectUser() {
        String sql = "SELECT * FROM test_hibernate_oracle";
        Query<User> query = session.createNativeQuery(sql).addEntity(User.class);
        List<User> users = query.getResultList();
        return users;
    }
```

5. 插入用户数据。

通过 insertUser 方法，插入用户信息。指定一个 User 类型的参数 user ，用于传入要插入的用户对象。通过调用 Hibernate Session 的 beginTransaction 方法创建一个事务对象，并将其保存在变量 beginTransaction 中。使用 save 方法将用户对象保存到数据库中。通过调用 Session 的 getTransaction 方法获取当前事务，并调用 commit 方法提交事务，将之前的操作持久化到数据库，并返回插入操作的结果。

代码如下：

```
public int insertUser(User user) {
        // open transaction
        Transaction beginTransaction = session.beginTransaction();
        session.save(user);
        session.getTransaction().commit();
        return 1;
    }
```

6. 删除用户数据。

通过 deleteUserById 方法，从数据库中删除传入的 ID 所对应的用户记录。指定一个整型参数 id ，用于传入要删除的用户 ID。通过调用 Hibernate Session 的 beginTransaction 方法开始一个新的事务。使用 get 方法根据指定的用户 ID 和实体类类型（User.class）获取对应的用户对象。使用 delete 方法删除用户对象。通过调用 Session 的 getTransaction 方法获取当前事务，并调用 commit 方法提交事务，删除用户信息持久化到数据库，并返回删除操作的结果。

代码如下：

```
public int deleteUserById(int id) {
        // open transaction
        session.beginTransaction();
        User user = session.get(User.class, id);
        session.delete(user);
        session.getTransaction().commit();
        return 1;
    }
```

7. 修改用户数据。

   通过 updateUserById 方法，更新对应的用户信息。指定一个 User 类型的参数 user，用于指定要更新的用户数据。使用 Session 对象的 beginTransaction 方法开启一个事务。使用 Session 对象的 get 方法根据用户 ID 获取对应的 User 对象。使用 Session 对象的 merge 方法将传入的 User 对象与获取到的 User 对象进行合并，将传入的用户数据更新到数据库中。使用 Session 对象的 getTransaction 方法获取当前事务，并调用 commit 方法提交事务，并返回修改操作的结果。

   代码如下：

```
public int updateUserById(User user) {
        // open transaction
        session.beginTransaction();
        User user1 = session.get(User.class, user.getId());
        session.merge(user);
        session.getTransaction().commit();
        return 1;
    }
```

2.2.6.3.5　User.java 代码介绍

User.java 文件通过用户数据访问对象（DAO）类，通过 Session 对象实现了用户数据的增删改查操作。

User.java 文件的代码主要包括以下几个部分：

1. 引用其他类。

   声明当前文件使用 JPA 注解配置了相关类与数据库表的映射关系：

   - Column 注解：用于标识实体类中的属性与数据库表中的列的对应关系。
   - Entity 注解：用于映射数据库中的表。
   - GeneratedValue 注解：用于表示该属性的值是自动生成的。
   - GenerationType 注解：用于指定主键生成策略。
   - Id 注解：用于指定唯一标识属性。
   - Table 注解：用于表示实体类对应的表名。

   代码如下：

```
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;
```

2. 映射数据库中的表。

   映射数据库表的实体类，并指定实体类对应的表名为 test_hibernate_oracle。

   代码如下：

```
@Entity
@Table(name = "test_hibernate_oracle")
```

3. 定义 User 类。

   用于与数据库中的一个名为 test_hibernate_oracle 的表格进行映射。

   1. 定义两个属性 id 和 name 。使用注解标识了 User 类中的 id 和 name 属性与数据库表中的列的对应关系。通过 @Id 注解标识 id 属性为主键，@GeneratedValue 注解指定了主键的生成策略为序列，@Column 注解指定了属性与数据库表中的列的映射关系。id 属性用于存储用户 ID ，name 属性用于存储用户名。

      代码如下：

```
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE)
@Column(name = "user_id")
private int id;

@Column(name = "user_name")
private String name;
```

   2. 创建 User 对象。

      定义 User 类的两个构造方法，用于创建 User 对象。定义一个无参构造方法用于 Hibernate 查询操作。定义了一个有参构造方法，用于初始化 id 和 name 属性。

      代码如下：

```
public User() {
    }

    public User(int id, String name) {
        this.id = id;
        this.name = name;
    }
```

   3. 获取和设置 id 和 name 值。

      定义 User 类中的四个方法，用于获取和设置 id 和 name 属性的值。getId 方法，用于获取 id 值，setId 方法，用于设置 id 值。getName 方法，用于获取用户名 name 的值。setName 方法，用于设置用户名 name 的值。

      代码如下：

```
public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
```

4. 返回 User 对象的字符串表示形式。

   重写 User 类中的 toString 方法，用于返回 User 对象的字符串表示形式。定义 @Override 覆盖父类中的同名方法。定义 toString 方法，用于返回 User 对象的字符串表示形式。通过字符串拼接，将 id 和 name 属性的值格式化为一个字符串，返回给调用者 User 。

   代码如下：

```
@Override
    public String toString() {
        return "User{" +
                "id=" + id +
                ", name='" + name + '\'' +
                '}';
    }
```

2.2.6.3.6　TestHibernate.java 代码介绍

TestHibernate.java 文件通过对 UserDao 对象执行插入、删除、更新和查询操作，展示 Hibernate 的基本数据库操作功能。TestHibernate.java 文件的代码主要包括以下几个部分：

1. 引用其他类和接口。

   声明当前文件相关的类和接口，包括 UserDao 、User 、Session 、SessionFactory、Configuration 等。

   - UserDao 类：用于执行与用户相关的数据库操作。
   - User 类：用于操作用户对象。
   - Session 类：用于与数据库进行会话操作。
   - SessionFactory 类：用于创建 Session 对象。

- Configuration 类：用于配置 Hibernate。
- SessionImpl 类：用于获取底层的 JDBC 连接。
- Test 注解：用于标记测试方法。
- IOException 类：用于处理 IO 异常。
- Connection 类：用于获取 JDBC 连接。
- SQLException 类：用于处理 SQL 异常。
- List 类：用于存储查询结果集。
- UUID 类：用于生成唯一标识符。

代码如下：

```
import com.kingbase.dao.UserDao;
import com.kingbase.pojo.User;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
import org.hibernate.internal.SessionImpl;
import org.junit.Test;

import java.io.IOException;
import java.sql.Connection;
import java.sql.SQLException;
import java.util.List;
import java.util.UUID;
```

2. 定义 testHibernate 方法。

   testHibernate 方法包含了插入、删除、更新和查询等基本操作。创建 UserDao 对象，循环插入 5 条用户数据。调用 insertUser 方法插入用户数据。调用 deleteUserById 方法删除 id 为 1 的用户数据。调用 updateUserById 方法更新 id 为 2 的用户数据。调用 selectUser 方法查询所有用户数据。遍历查询结果并打印到控制台。

   代码如下：

```
public class TestHibernate {
    @Test
    public void testHibernate() throws SQLException, IOException {
        UserDao userDao = new UserDao();
        for (int i = 1; i <= 5; i++) {
            userDao.insertUser(new User(i, "user_insert" + i));
        }
        int deleteUserById = userDao.deleteUserById(1);
        int update = userDao.updateUserById(new User(2, "update"));
        List<User> users = userDao.selectUser();
        users.forEach(System.out::println);
```

```
    }
}
```

2.2.6.3.7　完整的代码展示

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
<groupId>com.kingbase</groupId>
<artifactId>jdbc-kingbase-hibernate</artifactId>
<version>1.0-SNAPSHOT</version>
<build>
<plugins>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<configuration>
<source>8</source>
<target>8</target>
</configuration>
</plugin>
</plugins>
</build>
<dependencies>
<dependency>
<groupId>junit</groupId>
<artifactId>junit</artifactId>
<version>4.13</version>
</dependency>
<dependency>
<groupId>cn.com.kingbase</groupId>
<artifactId>kingbase8</artifactId>
<version>9.0.0</version>
</dependency>
<dependency>
<groupId>org.hibernate</groupId>
<artifactId>hibernate-core</artifactId>
<version>5.2.17.Final</version>
</dependency>
<!-- Hibernate c3p0 connection pool-->
```

```
<dependency>
<groupId>org.hibernate</groupId>
<artifactId>hibernate-c3p0</artifactId>
<version>5.2.17.Final</version>
</dependency>
</dependencies>
</project>
```

User.hbm.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping package="com.kingbase.pojo">
<class name="User" table="test_hibernate_oracle">
<!-- Configure primary key generation strategy -->
<id name="id" column="USER_ID">
<generator class="sequence">
<param name="sequence">SQ_USER</param>
</generator>
</id>
<!-- Configuration Tables and Properties -->
<property name="name" column="USER_NAME" type="string"/>
</class>
</hibernate-mapping>
```

hibernate.cfg.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
<session-factory>
<property name="hibernate.connection.driver_class">com.kingbase8.Driver</property>
<property name="hibernate.connection.url">jdbc:kingbase8://192.168.xx.xxx:54321/test</property>
<property name="hibernate.connection.username">system</property>
<property name="hibernate.connection.password">******</property>
<property name="hibernate.connection.provider_class">org.hibernate.c3p0.internal.C3P0ConnectionProvider</
property>
<property name="hibernate.c3p0.max_size">1</property>
```

```
<property name="hibernate.c3p0.min_size">1</property>

<property name="hibernate.c3p0.checkoutTimeout">30000</property>
<property name="hibernate.c3p0.timeout">20000</property>

<property name="hibernate.c3p0.max_statements">100</property>
<property name="hibernate.c3p0.idle_test_period">3000</property>
<property name="hibernate.c3p0.acquire_increment">3</property>
<property name="hibernate.c3p0.validate">true</property>

<property name="dialect">org.hibernate.dialect.Kingbase8Dialect</property>
<property name="hibernate.show_sql">true</property>
<property name="hibernate.format_sql">true</property>
<property name="hbm2ddl.auto">create</property>
<property name="current_session_context_class">thread</property>
<property name="hibernate.jdbc.batch_size">10</property>
<property name="hibernate.cache.use_second_level_cache">false</property>
<mapping resource="com/kingbase/pojo/User.hbm.xml"/>
</session-factory>
</hibernate-configuration>
```

UserDao.java

```
package com.kingbase.dao;

public class UserDao {
private Session session;
{
Configuration cfg = new Configuration().configure();
// Create SessionFactory
SessionFactory sessionFactory = cfg.buildSessionFactory();
session = sessionFactory.openSession();
} //Read the hibernate.cfg.xml file
//    private Session session = HibernateUtil.getSession();

public User selectUserById(int ID) {
User user = session.get(User.class, ID);
return user;
}

public List<User> selectUserbyName(String name) {
String sql = "FROM test_hibernate_oracle u WHERE u.name =?";
Query<User> query = session.createQuery(sql, User.class);
```

```
query.setParameter(0, name);
List<User> users = query.list();
return users;
}

public List<User> selectUser() {
String sql = "SELECT * FROM test_hibernate_oracle";
Query<User> query = session.createNativeQuery(sql).addEntity(User.class);
List<User> users = query.getResultList();
return users;
}

public int insertUser(User user) {
// open transaction
Transaction beginTransaction = session.beginTransaction();
session.save(user);
session.getTransaction().commit();
return 1;
}

public int deleteUserById(int id) {
// open transaction
session.beginTransaction();
User user = session.get(User.class, id);
session.delete(user);
session.getTransaction().commit();
return 1;
}

public int updateUserById(User user) {
// open transaction
session.beginTransaction();
User user1 = session.get(User.class, user.getId());
session.merge(user);
session.getTransaction().commit();
return 1;
}

}
```

User.java

```
package com.kingbase.pojo;

@Entity
@Table(name = "test_hibernate_oracle")
public class User {
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE)
@Column(name = "user_id")
private int id;

@Column(name = "user_name")
private String name;

public User() {
}

public User(int id, String name) {
this.id = id;
this.name = name;
}

public int getId() {
return id;
}

public void setId(int id) {
this.id = id;
}

public String getName() {
return name;
}

public void setName(String name) {
this.name = name;
}

@Override
public String toString() {
return "User{" +
"id=" + id +
", name='" + name + '\'' +
'}';
```

```
}
}
```

TestHibernate.java

```
public class TestHibernate {
@Test
public void testHibernate() throws SQLException, IOException {
UserDao userDao = new UserDao();
for (int i = 1; i <= 5; i++) {
userDao.insertUser(new User(i, "user_insert" + i));
}
int deleteUserById = userDao.deleteUserById(1);
int update = userDao.updateUserById(new User(2, "update"));
List<User> users = userDao.selectUser();
users.forEach(System.out::println);
}
}
```

2.2.6.4　相关文档

Jdbc 更多示例请参考 Jdbc 产品手册的程序示例。

2.2.7　SpringDataJPA

本文将介绍如何使用 SpringDataJPA 框架和 Kingbase 数据库构建一个应用程序，实现创建表、插入数据和查询数据等基本操作。

点击下载 java-kingbase-springdatajpa 示例工程。

2.2.7.1　前提条件

- 您已安装 Kingbase 数据库。
- 您已安装 JDK 1.8 和 Maven。
- 您已安装 IDE 工具。

备注:

本文档运行代码使用的工具是 IntelliJ IDEA 2023.2.8 (Community Edition) 版本，您也可以根据个人喜好选择适合自己的工具运行示例代码。

2.2.7.2　操作步骤

备注:

本文中给出的操作步骤是基于 Windows 环境生成。如果您使用的是其他操作系统环境或编译器，那么操作步骤可能会略有不同。

1. 获取 Kingbase 数据库连接信息。
2. 导入 java-kingbase-springdatajpa 项目到 IDEA 中。
3. 修改 java-kingbase-springdatajpa 项目中的数据库连接信息。
4. 运行 java-kingbase-springdatajpa 项目。

2.2.7.2.1　步骤一：获取 Kingbase 数据库连接信息

1. 联系 Kingbase 数据库部署人员或者管理员获取相应的数据库连接信息。

```
./ksql -U system -d test -h xx.xx.xx.xx -p 54321
```

2. 根据已部署的 Kingbase 数据库填写下面的 URL 的对应信息

备注:

在 jdbc.properties 文件中需要这里的 URL 信息。

```
jdbc.url=jdbc:kingbase8://host:port/database?user=$user_name&password=$password
```

参数说明:

- host：提供 Kingbase 数据库连接 IP。直连方式使用的是 KESServer 节点的 IP 地址。
- port：提供 Kingbase 数据库连接端口。直连方式默认是 54321，在部署 kingbase 数据库时可自定义。
- database：需要访问的数据库名称。
- user_name：通过 -u 参数指定，管理员用户为 system。
- password：提供账户密码。

更多 URL 参数说明信息请参考 Jdbc 连接属性。

2.2.7.2.2　步骤二: 导入 java-kingbase-springdatajpa 项目到 IDEA 中

1. 打开 IntelliJ IDEA，选择 File > Open…选项。
2. 在弹出的 Open File or Project 窗口中，选择对应的 java-kingbase-springdatajpa 项目文件夹，单击 OK 完成项目导入。
3. IntelliJ IDEA 将会自动识别项目中的各类文件，并在 Project 工具窗口中，可以查看项目的目录结构、文件列表、模块列表、依赖关系等信息。Project 工具窗口通常位于 IntelliJ IDEA 界面的最左侧，默认情况下是打开的。如果 Project 工具窗口被关闭了，可以通过点击菜单栏中的 View > Tool Windows > Project 或者使用快捷键 Alt + 1 来重新打开它。

备注:

当使用 IntelliJ IDEA 导入项目时，IntelliJ IDEA 会自动检测项目中的 pom.xml 文件，并根据文件中描述的依赖关系自动下载所需的依赖库，并将它们添加到项目中。

2.2.7.2.3　步骤三: 修改 java-kingbase-springdatajpa 项目中的数据库连接信息

根据步骤一：获取 Kingbase 数据库连接信息中的信息修改 application.yml 文件中的数据库连接信息。示例如下

- 数据库驱动名称为 `com.kingbase8.Driver`。
- KES Server 节点的 IP 地址为 `192.168.xx.xxx`。
- 访问端口使用的是 `54321`。
- 需要访问的 database 名称为 `test`。
- 账户是 `system`。
- 密码是 `******`。

配置示例如下

```yaml
spring:
  datasource:
    driver-class-name: com.kingbase8.Driver
    url: jdbc:kingbase8://192.168.xx.xxx:54321/test
    username: system
    password: ******
    type: com.alibaba.druid.pool.DruidDataSource
```

2.2.7.2.4　步骤四: 运行 java-kingbase-springdatajpa 项目

运行路径：

1. 在项目结构中找到 src > test > java 中找到 TestSpringDataJpaApplicationTests.java 文件。

2. 在工具菜单栏中选择运行 (U) > 运行 > TestSpringDataJpaApplicationTests(快捷键 Shift+F10)，或直接单击右上角绿色三角形运行 (快捷键 Shift+F10)。
3. 通过 IDEA 的控制台来查看项目的日志信息和输出结果。

运行结果：

```
User{id=1, username='insert1'}
User{id=2, username='update'}
User{id=3, username='insert3'}
User{id=4, username='insert4'}
User{id=5, username='insert5'}
```

完整的运行结果：

```
20:23:23.158 [main] DEBUG org.springframework.test.context.BootstrapUtils - Instantiating
CacheAwareContextLoaderDelegate from class [org.springframework.test.context.cache.
DefaultCacheAwareContextLoaderDelegate]
20:23:23.173 [main] DEBUG org.springframework.test.context.BootstrapUtils - Instantiating BootstrapContext
using constructor [public org.springframework.test.context.support.DefaultBootstrapContext(java.lang.Class,
org.springframework.test.context.CacheAwareContextLoaderDelegate)]
20:23:23.227 [main] DEBUG org.springframework.test.context.BootstrapUtils - Instantiating
TestContextBootstrapper for test class [com.kingbase.testspringdatajpa.TestSpringDataJpaApplicationTests]
from class [org.springframework.boot.test.context.SpringBootTestContextBootstrapper]
20:23:23.240 [main] INFO org.springframework.boot.test.context.SpringBootTestContextBootstrapper - Neither
@ContextConfiguration nor @ContextHierarchy found for test class [com.kingbase.testspringdatajpa.
TestSpringDataJpaApplicationTests], using SpringBootContextLoader
20:23:23.245 [main] DEBUG org.springframework.test.context.support.AbstractContextLoader - Did not detect
default resource location for test class [com.kingbase.testspringdatajpa.TestSpringDataJpaApplicationTests]:
class path resource [com/kingbase/testspringdatajpa/TestSpringDataJpaApplicationTests-context.xml] does not
exist
20:23:23.246 [main] DEBUG org.springframework.test.context.support.AbstractContextLoader - Did not detect
default resource location for test class [com.kingbase.testspringdatajpa.TestSpringDataJpaApplicationTests]:
class path resource [com/kingbase/testspringdatajpa/TestSpringDataJpaApplicationTestsContext.groovy] does
not exist
20:23:23.246 [main] INFO org.springframework.test.context.support.AbstractContextLoader - Could not detect
default resource locations for test class [com.kingbase.testspringdatajpa.
TestSpringDataJpaApplicationTests]: no resource found for suffixes {-context.xml, Context.groovy}.
20:23:23.247 [main] INFO org.springframework.test.context.support.AnnotationConfigContextLoaderUtils - Could
not detect default configuration classes for test class [com.kingbase.testspringdatajpa.
TestSpringDataJpaApplicationTests]: TestSpringDataJpaApplicationTests does not declare any static, non-
private, non-final, nested classes annotated with @Configuration.
20:23:23.298 [main] DEBUG org.springframework.test.context.support.ActiveProfilesUtils - Could not find an
'annotation declaring class' for annotation type [org.springframework.test.context.ActiveProfiles] and class
[com.kingbase.testspringdatajpa.TestSpringDataJpaApplicationTests]
20:23:23.387 [main] DEBUG org.springframework.context.annotation.ClassPathScanningCandidateComponentProvider
- Identified candidate component class: file [D:\工作目录\个人目录\JDBC\project\java-kingbase-springdatajpa\
```

```
target\classes\com\kingbase\testspringdatajpa\TestSpringDataJpaApplication.class]
20:23:23.388 [main] INFO org.springframework.boot.test.context.SpringBootTestContextBootstrapper - Found
@SpringBootConfiguration com.kingbase.testspringdatajpa.TestSpringDataJpaApplication for test class com.
kingbase.testspringdatajpa.TestSpringDataJpaApplicationTests
20:23:23.500 [main] DEBUG org.springframework.boot.test.context.SpringBootTestContextBootstrapper -
@TestExecutionListeners is not present for class [com.kingbase.testspringdatajpa.
TestSpringDataJpaApplicationTests]: using defaults.
20:23:23.501 [main] INFO org.springframework.boot.test.context.SpringBootTestContextBootstrapper - Loaded
default TestExecutionListener class names from location [META-INF/spring.factories]: [org.springframework.
boot.test.mock.mockito.MockitoTestExecutionListener, org.springframework.boot.test.mock.mockito.
ResetMocksTestExecutionListener, org.springframework.boot.test.autoconfigure.restdocs.
RestDocsTestExecutionListener, org.springframework.boot.test.autoconfigure.web.client.
MockRestServiceServerResetTestExecutionListener, org.springframework.boot.test.autoconfigure.web.servlet.
MockMvcPrintOnlyOnFailureTestExecutionListener, org.springframework.boot.test.autoconfigure.web.servlet.
WebDriverTestExecutionListener, org.springframework.boot.test.autoconfigure.webservices.client.
MockWebServiceServerTestExecutionListener, org.springframework.test.context.web.
ServletTestExecutionListener, org.springframework.test.context.support.
DirtiesContextBeforeModesTestExecutionListener, org.springframework.test.context.event.
ApplicationEventsTestExecutionListener, org.springframework.test.context.support.
DependencyInjectionTestExecutionListener, org.springframework.test.context.support.
DirtiesContextTestExecutionListener, org.springframework.test.context.transaction.
TransactionalTestExecutionListener, org.springframework.test.context.jdbc.SqlScriptsTestExecutionListener,
org.springframework.test.context.event.EventPublishingTestExecutionListener]
20:23:23.518 [main] INFO org.springframework.boot.test.context.SpringBootTestContextBootstrapper - Using
TestExecutionListeners: [org.springframework.test.context.web.ServletTestExecutionListener@6caf0677, org.
springframework.test.context.support.DirtiesContextBeforeModesTestExecutionListener@413d1baf, org.
springframework.test.context.event.ApplicationEventsTestExecutionListener@16eb3ea3, org.springframework.
boot.test.mock.mockito.MockitoTestExecutionListener@273444fe, org.springframework.boot.test.autoconfigure.
SpringBootDependencyInjectionTestExecutionListener@33bc72d1, org.springframework.test.context.support.
DirtiesContextTestExecutionListener@1a75e76a, org.springframework.test.context.transaction.
TransactionalTestExecutionListener@5524cca1, org.springframework.test.context.jdbc.
SqlScriptsTestExecutionListener@5032714f, org.springframework.test.context.event.
EventPublishingTestExecutionListener@48bb62, org.springframework.boot.test.mock.mockito.
ResetMocksTestExecutionListener@353352b6, org.springframework.boot.test.autoconfigure.restdocs.
RestDocsTestExecutionListener@4681c175, org.springframework.boot.test.autoconfigure.web.client.
MockRestServiceServerResetTestExecutionListener@57a78e3, org.springframework.boot.test.autoconfigure.web.
servlet.MockMvcPrintOnlyOnFailureTestExecutionListener@402c4085, org.springframework.boot.test.
autoconfigure.web.servlet.WebDriverTestExecutionListener@6fa51cd4, org.springframework.boot.test.
autoconfigure.webservices.client.MockWebServiceServerTestExecutionListener@2767e23c]
20:23:23.522 [main] DEBUG org.springframework.test.context.support.
AbstractDirtiesContextTestExecutionListener - Before test class: context [DefaultTestContext@183ec003
testClass = TestSpringDataJpaApplicationTests, testInstance = [null], testMethod = [null], testException =
[null], mergedContextConfiguration = [WebMergedContextConfiguration@7d9d0818 testClass =
TestSpringDataJpaApplicationTests, locations = '{}', classes = '{class com.kingbase.testspringdatajpa.
TestSpringDataJpaApplication}', contextInitializerClasses = '[]', activeProfiles = '{}',
```

```
propertySourceLocations = '{}', propertySourceProperties = '{org.springframework.boot.test.context.
SpringBootTestContextBootstrapper=true}', contextCustomizers = set[org.springframework.boot.test.context.
filter.ExcludeFilterContextCustomizer@795cd85e, org.springframework.boot.test.json.
DuplicateJsonObjectContextCustomizerFactory$DuplicateJsonObjectContextCustomizer@384ad17b, org.
springframework.boot.test.mock.mockito.MockitoContextCustomizer@0, org.springframework.boot.test.web.client.
TestRestTemplateContextCustomizer@694abbdc, org.springframework.boot.test.autoconfigure.actuate.metrics.
MetricsExportContextCustomizerFactory$DisableMetricExportContextCustomizer@1fa268de, org.springframework.
boot.test.autoconfigure.properties.PropertyMappingContextCustomizer@0, org.springframework.boot.test.
autoconfigure.web.servlet.WebDriverContextCustomizerFactory$Customizer@222545dc, org.springframework.boot.
test.context.SpringBootTestArgs@1, org.springframework.boot.test.context.
SpringBootTestWebEnvironment@7c0e2abd], resourceBasePath = 'src/main/webapp', contextLoader = 'org.
springframework.boot.test.context.SpringBootContextLoader', parent = [null]], attributes = map['org.
springframework.test.context.web.ServletTestExecutionListener.activateListener' -> true]], class annotated
with @DirtiesContext [false] with mode [null].


  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v2.7.0)

2025-01-21 20:23:23.876  INFO 21408 --- [           main] c.k.t.TestSpringDataJpaApplicationTests  :
Starting TestSpringDataJpaApplicationTests using Java 1.8.0_152 on GBJ-0916 with PID 21408 (started by
Administrator in D:\工作目录\个人目录\JDBC\project\java-kingbase-springdatajpa)
2025-01-21 20:23:23.877  INFO 21408 --- [           main] c.k.t.TestSpringDataJpaApplicationTests  : No
active profile set, falling back to 1 default profile: "default"
2025-01-21 20:23:24.463  INFO 21408 --- [           main] .s.d.r.c.RepositoryConfigurationDelegate :
Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2025-01-21 20:23:24.519  INFO 21408 --- [           main] .s.d.r.c.RepositoryConfigurationDelegate :
Finished Spring Data repository scanning in 47 ms. Found 1 JPA repository interfaces.
2025-01-21 20:23:25.072  INFO 21408 --- [           main] o.hibernate.jpa.internal.util.LogHelper  :
HHH000204: Processing PersistenceUnitInfo [name: default]
2025-01-21 20:23:25.128  INFO 21408 --- [           main] org.hibernate.Version                    :
HHH000412: Hibernate ORM core version 5.6.9.Final
2025-01-21 20:23:25.355  INFO 21408 --- [           main] o.hibernate.annotations.common.Version   :
HCANN000001: Hibernate Commons Annotations {5.1.2.Final}
2025-01-21 20:23:25.904 ERROR 21408 --- [           main] com.alibaba.druid.pool.DruidDataSource   :
testWhileIdle is true, validationQuery not set
2025-01-21 20:23:25.912  INFO 21408 --- [           main] com.alibaba.druid.pool.DruidDataSource   :
{dataSource-1} inited
2025-01-21 20:23:26.021  INFO 21408 --- [           main] org.hibernate.dialect.Dialect            :
HHH000400: Using dialect: org.hibernate.dialect.Kingbase8Dialect
2025-01-21 20:23:26.169  INFO 21408 --- [           main] o.hibernate.id.enhanced.TableGenerator   :
```

```
HHH000398: Explicit segment value for id generator [hibernate_sequences.sequence_name] suggested; using
default [default]
2025-01-21 20:23:26.695  INFO 21408 --- [           main] o.h.e.t.j.p.i.JtaPlatformInitiator       :
HHH000490: Using JtaPlatform implementation: [org.hibernate.engine.transaction.jta.platform.internal.
NoJtaPlatform]
2025-01-21 20:23:26.704  INFO 21408 --- [           main] j.LocalContainerEntityManagerFactoryBean :
Initialized JPA EntityManagerFactory for persistence unit 'default'
2025-01-21 20:23:28.163  INFO 21408 --- [           main] c.k.t.TestSpringDataJpaApplicationTests  : Started
TestSpringDataJpaApplicationTests in 4.598 seconds (JVM running for 5.574)
Hibernate: select user0_.id as id1_0_0_, user0_.username as username2_0_0_ from test_springdatajpa_2 user0_
where user0_.id=?
Hibernate: select user0_.id as id1_0_0_, user0_.username as username2_0_0_ from test_springdatajpa_2 user0_
where user0_.id=?
Hibernate: update test_springdatajpa_2 set username=? where id=?
Hibernate: select user0_.id as id1_0_0_, user0_.username as username2_0_0_ from test_springdatajpa_2 user0_
where user0_.id=?
Hibernate: select user0_.id as id1_0_0_, user0_.username as username2_0_0_ from test_springdatajpa_2 user0_
where user0_.id=?
Hibernate: select user0_.id as id1_0_0_, user0_.username as username2_0_0_ from test_springdatajpa_2 user0_
where user0_.id=?
Hibernate: select user0_.id as id1_0_0_, user0_.username as username2_0_0_ from test_springdatajpa_2 user0_
where user0_.id=?
Hibernate: select user0_.id as id1_0_0_, user0_.username as username2_0_0_ from test_springdatajpa_2 user0_
where user0_.id=?
Hibernate: select user0_.id as id1_0_0_, user0_.username as username2_0_0_ from test_springdatajpa_2 user0_
where user0_.id=?
Hibernate: select user0_.id as id1_0_0_, user0_.username as username2_0_0_ from test_springdatajpa_2 user0_
where user0_.id=?
Hibernate: select user0_.id as id1_0_0_, user0_.username as username2_0_0_ from test_springdatajpa_2 user0_
where user0_.id=?
Hibernate: select count(*) as col_0_0_ from test_springdatajpa_2 user0_ where user0_.id=?
Hibernate: select user0_.id as id1_0_0_, user0_.username as username2_0_0_ from test_springdatajpa_2 user0_
where user0_.id=?
Hibernate: update test_springdatajpa_2 set username=? where id=?
Hibernate: select user0_.id as id1_0_0_, user0_.username as username2_0_0_ from test_springdatajpa_2 user0_
where user0_.id=?
user = User{id=2, username='update'}
Hibernate: select user0_.id as id1_0_, user0_.username as username2_0_ from test_springdatajpa_2 user0_
where user0_.username=?
userByName = null
Hibernate: select user0_.id as id1_0_, user0_.username as username2_0_ from test_springdatajpa_2 user0_
order by user0_.id asc limit ?
Hibernate: select count(user0_.id) as col_0_0_ from test_springdatajpa_2 user0_
User{id=1, username='insert1'}
User{id=2, username='update'}
```

```
User{id=3, username='insert3'}
User{id=4, username='insert4'}
User{id=5, username='insert5'}
2025-01-21 20:23:28.438  INFO 21408 --- [ionShutdownHook] j.LocalContainerEntityManagerFactoryBean : Closing
JPA EntityManagerFactory for persistence unit 'default'
2025-01-21 20:23:28.439  INFO 21408 --- [ionShutdownHook] com.alibaba.druid.pool.DruidDataSource   :
{dataSource-1} closing ...
2025-01-21 20:23:28.442  INFO 21408 --- [ionShutdownHook] com.alibaba.druid.pool.DruidDataSource   :
{dataSource-1} closed
```

2.2.7.3　项目代码介绍

点击下载java-kingbase-springdatajpa 示例工程，这是一个名为 java-kingbase-springdatajpa 的压缩包。解压后，得到一个名为 java-kingbase-springdatajpa 的文件夹。目录结构如下所示：

```
--pom.xml

 lib
   hibernate-4.dialect.jar

 src
  main
    java
        com
           kingbase
                testspringdatajpa
                          --TestSpringDataJpaApplication.java

                           dao
                              UserRepository.java

                           entity
                               User.java

                           service
                               --UserService.java

                               impl
                                   --UserServiceImpl.java

    resources
          --application.yml
          --static
           templates
```

```
  test
     java
         com
            kingbase
                 testspringdatajpa
                            --TestSpringDataJpaApplicationTests.java

target
```

文件说明:

- pom.xml：Maven 项目的配置文件，包含了项目的依赖、插件、构建等信息。
- lib: 存放项目方言包的根目录。
- hibernate-4.dialect.jar：方言包文件
- src：通常用于表示项目中存放源代码的目录。
- main: 存放主要的源代码和资源文件的目录。
- java: 存放 Java 源代码的目录。
- com: 存放 Java 包的根目录。
- kingbase: 存放项目的根目录。
- testspringdatajpa：Java 包的根目录，包含了项目的所有 Java 类。
- TestSpringDataJpaApplication.java：Spring Boot 应用程序的入口类。
- dao: 存放数据访问对象（Data Access Object）包，用于访问数据库或其他数据存储服务。
- UserRepository.java：用户数据访问接口，定义了对用户数据的增删改查操作。
- entity：实体类的目录，用于存放与数据库表对应的 Java 类。
- User.java：用户持久化对象，用于映射用户数据表的字段。
- service：服务接口目录，用于定义业务逻辑接口。
- UserService.java：用户服务接口，定义了对用户数据的操作方法。
- impl：服务实现目录，用于存放业务逻辑的实现类。
- UserServiceImpl.java：用户服务的实现类，实现了 UserService 接口中定义的方法。
- resources: 存放资源文件的目录，如配置文件、SQL 文件等。
- application.yml：应用程序的配置文件，用于配置数据库连接等信息。
- static：存放静态文件，如 CSS、JavaScript 等。
- templates：存放模板文件，如 HTML 模板。
- test: 存放测试代码和资源文件的目录。

- TestSpringDataJpaApplicationTests.java: 存放用于执行和验证与 Spring Data JPA 相关的功能的类。
- target: 存放编译后的 Class 文件、Jar 包等文件的目录。

2.2.7.3.1　pom.xml 代码介绍

如果您只是想验证示例，那么请使用默认代码，无需修改。您也可以按照以下讲解，根据自己的需求修改 pom.xml 文件。

pom.xml 配置文件内容如下

1. 文件声明语句。声明本文件是一个 XML 文件，使用的 XML 版本是 1.0 ，字符编码方式是 UTF-8 。

   代码如下：

```
<?xml version="1.0" encoding="UTF-8"?>
```

2. 配置 POM 的命名空间和 POM 模型版本。

- 通过 xmlns 指定 POM 的命名空间为 `http://maven.apache.org/POM/4.0.0`。
- 通过 xmlns:xsi 指定 XML 命名空间为 `http://www.w3.org/2001/XMLSchema-instance`。
- 通过 xsi:schemaLocation 指定 POM 的命名空间为 `http://maven.apache.org/POM/4.0.0`，并指定 POM 的 XSD 文件的位置为 `https://maven.apache.org/xsd/maven-4.0.0.xsd`。
- 通过 modelVersion 元素指定该 POM 文件使用的 POM 模型版本为 `4.0.0`。

代码如下:

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
 <modelVersion>4.0.0</modelVersion>
</project>
```

3. 配置父项信息。

- 通过 <groupId> 指定父项标识为 org.springframework.boot。
- 通过 <artifactId> 指定父项依赖为 spring-boot-starter-parent。
- 通过 <version> 指定父项的版本号为 2.7.0。
- 通过 relativePath 表示父项的路径为空。

代码如下:

```
<parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>2.7.0</version>
        <relativePath/> <!-- lookup parent from repository -->
</parent>
```

4. 配置基本信息。

- 通过 <groupId> 指定项目标识为 com.kingbase。
- 通过 <artifactId> 指定项目依赖为 java-kingbase-springdatajpa。
- 通过 <version> 指定项目的版本号为 0.0.1-SNAPSHOT。
- 通过 description 介绍项目信息为 Demo project for Spring Boot。

代码如下：

```
<groupId>com.kingbase</groupId>
<artifactId>java-kingbase-springdatajpa</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>java-kingbase-springdatajpa</name>
<description>Demo project for Spring Boot</description>
```

5. 配置 java 版本。

指定项目使用的 Java 版本为 1.8。

代码如下:

```
<properties>
        <java.version>1.8</java.version>
</properties>
```

- 指定依赖项所属测试架构为 junit ， 名称为 junit ， 版本号为 4.13。

6. 配置核心依赖。

- a. 指定依赖项所属的组织为 org.springframework.boot ， 名称为 spring-boot-starter-data-jpa ， 该依赖库包含了使用 JPA 进行数据访问的必要依赖和配置，Spring Boot Starter Data JPA 是一个 Spring Boot 的启动器。

代码如下:

```
<dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

- b. 指定依赖项所属的组织为 org.springframework.boot ， 名称为 spring-boot-starter-web ， 该依赖库包含了使用 Spring Boot 进行 Web 开发所需的依赖和配置，它是一个 Spring Boot 的启动器。

代码如下：

```
<dependency>
            <groupId>org.springframework.boot</groupId>
```

```
            <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

- c. 指定依赖项所属的组织为 org.springframework.boot ，名称为 spring-boot-starter-test ，作用范围为 test ，通过该依赖可以使用 Spring Boot 提供的测试框架和工具，如 JUnit、Mockito、Hamcrest 等。

  代码如下：

```
<dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
</dependency>
```

- d. 指定依赖项所属的组织为 com.alibaba ，名称为 druid ，版本号为 1.2.8 ，通过该依赖可以使用 Druid 库，管理和优化数据库连接的获取和释放。

  代码如下：

```
<!-- alibaba druid -->
<dependency>
            <groupId>com.alibaba</groupId>
            <artifactId>druid</artifactId>
            <version>1.2.8</version>
</dependency>
```

- e. 指定依赖项所属的组织为 cn.com.kingbase ，名称为 kingbase8 ，版本号为 9.0.0 ，通过该依赖可以使用 Kingbase 提供的客户端功能，如连接、查询、事务等。

  代码如下：

```
<dependency>
            <groupId>cn.com.kingbase</groupId>
            <artifactId>kingbase8</artifactId>
            <version>9.0.0</version>
</dependency>
```

7. 配置 Maven 插件。

   指定依赖项所属的组织为 org.springframework.boot ，名称为 spring-boot-maven-plugin ，该插件用于将 Spring Boot 应用程序打包成可执行的 JAR 包或 WAR 包，并且可以直接运行。以为了避免与其他插件或工具产生冲突，在构建的过程中 Lombok 依赖。

   代码如下：

```
<build>
        <plugins>
            <plugin>
```

```
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <configuration>
                    <excludes>
                        <exclude>
                            <groupId>org.projectlombok</groupId>
                            <artifactId>lombok</artifactId>
                        </exclude>
                    </excludes>
                </configuration>
            </plugin>
        </plugins>
</build>
```

2.2.7.3.2　application.yml 代码介绍

application.yml 文件是一个 Spring Boot 应用程序的配置文件，其中包含了一些关键配置项。它配置了应用程序的上下文路径、监听端口号，以及数据库连接信息和 JPA 相关的属性。application.yml 文件的代码主要包括以下几个部分：

- Server 部分
  - servlet：用于配置 Servlet 相关的属性。
  - context-path：指定应用程序的上下文路径为 testspringdatajpa。
  - port：指定应用程序监听的端口号为 8890。

代码如下:

```
server:
servlet:
context-path: /testspringdatajpa
port: 8890
```

- Spring 部分
  - datasource ：用于配置数据源相关的属性，即数据库连接信息。
  - driver-class-name：指定数据库驱动的类名为 com.kingbase8.Driver。
  - url：指定数据库的连接 URL，包括数据库的地址、端口号、数据库名称等信息。
  - username：指定连接数据库的用户名。
  - password：指定连接数据库的密码。
  - type：指定使用 Druid 连接池作为数据源。

代码如下:

```
spring:
  datasource:
    driver-class-name: com.kingbase8.Driver
    url: jdbc:kingbase8://192.168.xx.xxx:54321/test
    username: system
    password: ******
    type: com.alibaba.druid.pool.DruidDataSource
```

- JPA 部分
  - database-platform：指定 hibernate 方言类名为，使用 kingbase 方言
  - hibernate：用于配置 Hibernate 相关的属性。
  - ddl-auto：指定 Hibernate 在启动时自动更新数据库结构为 update。
  - show-sql：指定是否在控制台打印 SQL 语句，值为 true。
  - format-sql：指定是否格式化打印的 SQL 语句，值为 true。
  - open-in-view：指定是否开启 Open-in-View 模式，值为 false。

  代码如下:

```
jpa:
    database-platform: org.hibernate.dialect.Kingbase8Dialect
    hibernate:
    ddl-auto: update
    show-sql: true
    format-sql: true
    open-in-view: false
```

2.2.7.3.3　userRepository.java 文件介绍

userRepository.java 文件通过 DAO 接口用于定义对用户实体（User）进行数据库操作的方法。它包含了自定义的更新和查询方法，使用 SQL 语句和 HQL 语句进行操作。userRepository.java 文件的代码主要包括以下几个部分：

1. 引用其他类和接口。

   声明当前文件包含以下接口和类：

   - User 类：用于在服务类中操作和处理用户对象。
   - JpaRepository 接口：用于继承该接口以获得基本的 CRUD 操作方法。
   - Modifying 注解：用于标记修改操作的方法。
   - Query 注解，用于定义自定义查询方法。
   - Param 注解，用于将方法参数与查询语句中的参数进行映射。
   - Repository 注解，用于将 DAO 接口标记为 Spring 的仓库组件。

代码如下：

```
import com.kingbase.testspringdatajpa.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;
```

2. 定义 UserRepository 接口。

   UserRepository 接口用于访问用户数据。标记为一个 Spring 的 Repository 组件，并指定 bean 的名称为 userRepository 。

   - 通过 @Query 注解指定 SQL 语句，并使用 nativeQuery = true 使用原生 SQL 语句。
   - 通过 @Modifying 注解自定义的更新方法，使用原生 SQL 语句进行更新操作。
   - 通过 @Query 注解指定 HQL（Hibernate Query Language）语句，并使用 @Param 注解将方法参数与 HQL 语句中的参数进行映射。

   代码如下：

```
@Repository("userRepository")
public interface UserRepository extends JpaRepository<User, Integer> {
    //Custom repository handwritten SQL, placeholder value transfer form
    @Query(value = "update test_springdatajpa_2 set username=?1 where id=?2", nativeQuery = true)
    @Modifying
    int updateById(String name, String id);

    //SPEL expression, hql syntax
    @Query("SELECT u FROM User u WHERE u.username = :username")
    User findUser(@Param("username") String username);//Mapping parameter username to database field
username
}
```

2.2.7.3.4　User.java 文件介绍

User.java 文件通过定义 User 类用于表示用户对象。

User.java 文件的代码主要包括以下几个部分：

1. 引入其他类。javax.persistence.* ：用于引入 javax.persistence 包中的所有类。javax.persistence 是 Java Persistence API（JPA）的标准包，提供了一组用于进行对象关系映射（ORM）的接口和注解。代码如下：

```
import javax.persistence.*;
```

2. 定义 User 对象。使用 JPA 注解来标识实体类和表的映射关系，包括表名、主键、字段名等。通过 @Entity 注解标记该类为一个实体类，@Table 注解指定了对应的数据库表名，@Id 注解标记了实体类的主键，@Generat-

edValue 注解指定了主键的生成策略，@Column 注解用于指定字段名和约束条件。该类具有 id 和 username 两个属性，分别表示用户的唯一标识和用户名。还提供了一个默认构造方法，用于创建空的 User 对象。

代码如下：

```
@Entity
@Table(name = "test_springdatajpa_2")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.TABLE)
    // JPA provides four standard uses:
    //GenerationType.TABLE: use a specific database table to save the primary key.
    //GenerationType.SEQUENCE: generate primary keys based on the sequence of the underlying database,
provided that the database supports sequences.
    //GenerationType.IDENTITY: the primary key is automatically generated by the database (mainly automatic
growth type)
    //GenerationType.AUTO: the primary key is controlled by the program.
    @Column(name = "id", nullable = false)
    private Integer id;

    @Column(name = "username")
    private String username;

    public User() {
    }
}
```

3. 获取和设置 id 和 name 值。该实体类提供了一个带参构造方法，用于创建实体类对象并初始化 id 和 username 属性。定义四个方法，用于获取和设置 id 和 name 属性的值。

- getId 方法用于获取 id 值。
- setId 方法用于设置 id 值。
- getName 方法用于获取用户名 name 的值。
- setName 方法用于设置用户名 name 的值。

代码如下：

```
public User(Integer id, String username) {
        this.id = id;
        this.username = username;
    }

    public Integer getId() {
        return id;
    }
```

```
    public void setId(Integer id) {
        this.id = id;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }
```

4. 返回 User 对象的字符串表示形式。

   重写 User 类中的 toString 方法，用于返回 User 对象的字符串表示形式。定义 @Override 覆盖父类中的同名方法。定义 toString 方法，用于返回 User 对象的字符串表示形式。通过字符串拼接，将 id 和 name 属性的值格式化为一个字符串，返回给调用者 User 。

   代码如下：

```
@Override
    public String toString() {
        return "User{" +
                "id=" + id +
                ", username='" + username + '\'' +
                '}';
    }
```

2.2.7.3.5　UserServiceImpl.java 文件介绍

UserServiceImpl.java 文件通过使用 JPA 操作数据库进行用户数据的增删改查。UserServiceImpl.java 文件的代码主要包括以下几个部分：

1. 引用其他类和接口。声明当前文件包含以下接口和类：
   - UserRepository 类：用于在服务类中注入并调用数据访问对象（DAO）中定义的方法。
   - User 类：用于在服务类中操作和处理用户对象。
   - UserService 类：用于在其他类中注入并调用服务类中定义的方法。
   - Autowired 注解：用于自动注入依赖对象。
   - Page 类：用于分页查询结果的封装。
   - PageRequest 类：用于创建分页请求对象。
   - Pageable 接口：用于定义分页请求的接口。
   - Sort 类：用于定义排序规则。

- Service 注解：用于将类标记为服务类。
- Transactional 注解：用于标记事务方法。
- Iterator 类：用于遍历集合。
- Optional 类：用于处理可能为空的对象。

代码如下：

```
import com.kingbase.testspringdatajpa.dao.UserRepository;
import com.kingbase.testspringdatajpa.entity.User;
import com.kingbase.testspringdatajpa.service.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.Iterator;
import java.util.Optional;
```

2. UserServiceImpl 类实现了 UserService 接口中声明的方法。

   通过依赖注入 UserRepository 实现对数据库的访问。具体实现了插入、删除、更新和查询用户数据的方法，并支持分页查询。使用 @Transactional 注解开启事务支持，确保数据库操作在事务中执行。

a. 构造 UserRepository 属性。定义一个私有的 UserRepository 属性，并在构造方法上使用 @Autowired 注解进行依赖注入。通过构造方法注入 UserRepository 依赖，使用 userRepository 属性来访问和操作用户数据。

代码如下：

```
private final UserRepository userRepository;

    @Autowired
    public UserServiceImpl(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
```

b. 插入用户数据。在 insertUser 方法接受一个 User 对象作为参数，通过调用 UserRepository 的 save 方法，将用户对象保存到数据库中。

代码如下：

```
@Override
public void insertUser(User user) { userRepository.save(user); }
```

c. 删除用户数据。在 deleteUser 方法中，通过调用 UserRepository 的 existsById 方法判断指定 id 的用户是否存在，如果存在则进行删除操作，如果不存在则直接返回。

代码如下：

```
//delete
    @Override
    public void deleteUser(Integer id) {
        if (!userRepository.existsById(id)) {
            return;
        }
    }
```

d. 修改用户数据。在 updateUser 方法中，通过调用 UserRepository 的 save 方法，将用户对象保存到数据库中，实现更新操作。同时，返回一个整数值 1 ，表示更新操作成功。另外，还根据用户名和 id 更新用户数据的方法，通过调用 UserRepository 的 updateById 方法实现更新操作，并返回更新的结果。

代码如下：

```
//update
    @Override
    public int updateUser(User user) {
        userRepository.save(user);
        return 1;
    }

    //update based on name and id
    @Override
    public int updateUserById(String name, String id) {
        return userRepository.updateById(name, id);
    }
```

e. 查询用户数据。

- a. 根据 ID 查询用户数据。在 selectUserById 方法中，通过调用 UserRepository 的 findById 方法，查询到用户数据，并将结果封装为 Optional 对象。然后，通过调用 Optional 的 orElse 方法，获取 Optional 对象中的值，将其赋值给 user 变量。最后，返回查询到的用户对象。
- b. 根据用户名查询用户数据。在 selectUserByName 方法中，通过调用 UserRepository 的 findUser 方法，根据用户名查询用户数据，并直接返回查询到的用户对象。

代码如下：

```
// select one user
    @Override
    public User selectUserById(Integer id) {
        Optional<User> optional = userRepository.findById(id);
        User user = optional.orElse(null);
```

```
        return user;
    }

    //query user based on username
    @Override
    public User selectUserByName(String username) {
        return userRepository.findUser(username);
    }
```

f. 分页查询所有用户数据。在 selectUserAll 方法中，通过创建 Sort 对象指定排序规则，创建 Pageable 对象指定分页参数，调用 UserRepository 的 findAll 方法查询用户数据，并返回一个 Page 对象。然后，通过调用 Page 对象的 iterator 方法，获取一个用户数据的迭代器。最后，返回用户数据的迭代器。

代码如下：

```
@Override
    public Iterator<User> selectUserAll(Integer pageNum, Integer pageSize) {
        //By passing parameters to this method, physical pagination can be achieved, which is very simple.
        Sort sort = Sort.by(Sort.Direction.ASC, "id");
        Pageable pageable = PageRequest.of(pageNum, pageSize, sort);
        Page<User> users = userRepository.findAll(pageable);
        Iterator<User> userIterator = users.iterator();
        return userIterator;
    }
```

2.2.7.3.6　UserService.java 文件介绍

UserService.java 文件通过使用 JPA 操作数据库进行用户数据的增删改查。UserService.java 文件的代码主要包括以下几个部分：

1. 引用其他类和接口。

   声明当前文件包含以下类：

   - User 类：用于在服务类中操作和处理用户对象。
   - Iterator 类，用于遍历集合。

   代码如下：

```
import com.kingbase.testspringdatajpa.entity.User;
import java.util.Iterator;
```

2. 定义 UserService 接口。

   通过 UserService 接口对用户数据进行插入、删除、更新和查询的方法。包含以下方法和类：

   - insertUser 方法，用于插入用户数据。

- deleteUser 方法，用于删除用户数据。
- updateUser 方法，用于更新用户数据。
- updateUserById 方法，用于根据 id 修改用户的 name 。
- selectUserById 方法，用于根据 id 查询用户数据。
- selectUserByName 方法，用于根据用户名查询用户数据。
- selectUserAll 方法，用于分页查询所有用户数据。
- User 类，用于在服务类中操作和处理用户对象。
- Iterator 类，用于遍历集合。

代码如下：

```
public interface UserService {
    //insert
    void insertUser(User user);

    //delete
    void deleteUser(Integer id);

    //update
    int updateUser(User user);

    //customize SQL and modify name based on id
    int updateUserById(String name, String id);

    //select one user
    User selectUserById(Integer id);

    //customize SQL to query users based on username
    User selectUserByName(String username);

    //query all users
    Iterator<User> selectUserAll(Integer pageNum, Integer pageSize);
}
```

2.2.7.3.7　TestSpringDataJpaApplication.java 文件介绍

TestSpringDataJpaApplication.java 文件用于启动和配置 Spring Boot 应用程序。TestSpringDataJpaApplication.java 文件的代码主要包括以下几个部分：

1. 定义类和接口。

   声明当前文件包含以下类：

   - SpringApplication 类，用于启动 Spring Boot 应用程序。

- @SpringBootApplication 注解，用于标记 Spring Boot 应用程序的入口类。

代码如下：

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

2. 定义 TestSpringDataJpaApplication 类。

   通过使用 @SpringBootApplication 注解标识 Spring Boot 应用的入口类，并在 main 方法中调用 SpringApplication.run 方法来启动应用。

代码如下：

```
@SpringBootApplication
public class TestSpringDataJpaApplication {

    public static void main(String[] args) {
        SpringApplication.run(TestSpringDataJpaApplication.class, args);
    }

}
```

2.2.7.3.8　TestSpringDataJpaApplicationTests.java 文件介绍

TestSpringDataJpaApplicationTests.java 文件用于启动和配置 Spring Boot 应用程序。

TestSpringDataJpaApplicationTests.java 文件的代码主要包括以下几个部分：

1. 引用其他类和接口。

   声明当前文件包含以下类：

   - User 类：用于在服务类中操作和处理用户对象。
   - UserService 类，用于在其他类中注入并调用服务类中定义的方法。
   - @Test 注解，用于标记测试方法。
   - Autowired 注解，用于自动注入依赖对象。
   - @SpringBootTest 注解，用于测试 Spring Boot 应用程序。
   - Iterator 类，用于遍历集合。

代码如下：

```
import com.kingbase.testspringdatajpa.entity.User;
import com.kingbase.testspringdatajpa.service.UserService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
```

```
import java.util.Iterator;
```

2. 定义 UserService 方法。

   用于测试 Spring Boot 应用程序中的 UserService 服务的各个方法。它通过注入 UserService 依赖，调用各个方法来测试插入、删除、更新和查询用户数据的功能，并打印相关结果。

   a. 通过 @Autowired 注解。将 UserService 接口的实现类自动注入到 userService 变量中。代码如下：

```
@Autowired
private UserService userService;
```

   b. 使用 contextLoads 方法测试。

- a. 插入用户数据。使用 for 循环，调用 userService 的 insertUser 方法插入 10 个用户数据。

  代码如下：

```
//insert
        for (int i = 1; i <= 10; i++) {
            userService.insertUser(new User(i, "insert" + i));
        }
```

- b. 删除用户数据。调用 userService 的 deleteUser 方法，删除 id 为 1 的用户数据。

  代码如下：

```
userService.deleteUser(1);
```

- c. 修改用户数据。调用 userService 的 updateUser 方法，更新 id 为 2 的用户数据。

  代码如下：

```
userService.updateUser(new User(2, "update"));
```

- d. 查询用户数据。
  - 调用 userService 的 selectUserById 方法，根据 id 查询用户数据，并将结果赋值给 user 变量。打印输出 user 对象。
  - 调用 userService 的 selectUserByName 方法，根据用户名查询用户数据，并将结果赋值给 userByName 变量。打印输出 userByName 对象。

  代码如下：

```
//selectUserById
User user = userService.selectUserById(2);
System.out.println("user = " + user);
//selectUserByName
```

```
User userByName = userService.selectUserByName("insert");
System.out.println("userByName = " + userByName);
```

- e. 分页查询用户数据。调用 userService 的 selectUserAll 方法，分页查询所有用户数据，并将结果赋值给 userIterator 变量。使用 forEachRemaining 方法遍历 userIterator ，并打印输出每个用户对象。

代码如下：

```
//query all users
Iterator<User> userIterator = userService.selectUserAll(0, 5);
userIterator.forEachRemaining(System.out::println);
```

2.2.7.3.9　完整的代码展示

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.7.0</version>
<relativePath/> <!-- lookup parent from repository -->
</parent>
<groupId>com.kingbase</groupId>
<artifactId>java-kingbase-springdatajpa</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>java-kingbase-springdatajpa</name>
<description>Demo project for Spring Boot</description>
<properties>
<java.version>1.8</java.version>
</properties>
<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>
<!-- alibaba druid -->
```

```
<dependency>
<groupId>com.alibaba</groupId>
<artifactId>druid</artifactId>
<version>1.2.8</version>
</dependency>
<dependency>
<groupId>cn.com.kingbase</groupId>
<artifactId>kingbase8</artifactId>
<version>9.0.0</version>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-test</artifactId>
<scope>test</scope>
</dependency>
</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
<configuration>
<excludes>
<exclude>
<groupId>org.projectlombok</groupId>
<artifactId>lombok</artifactId>
</exclude>
</excludes>
</configuration>
</plugin>
</plugins>
</build>
</project>
```

application.yml

```yml
server:
servlet:
context-path: /testspringdatajpa
port: 8890

spring:
datasource:
```

```
driver-class-name: com.kingbase8.Driver
url: jdbc:kingbase8://192.168.xx.xxx:54321/test
username: system
password: ******
type: com.alibaba.druid.pool.DruidDataSource
jpa:
database-platform: org.hibernate.dialect.Kingbase8Dialect
hibernate:
ddl-auto: update
show-sql: true
format-sql: true
open-in-view: false
```

userRepository.java

```
package com.kingbase.testspringdatajpa.dao;

@Repository("userRepository")
public interface UserRepository extends JpaRepository<User, Integer> {
//Custom repository handwritten SQL, placeholder value transfer form
@Query(value = "update test_springdatajpa_2 set username=?1 where id=?2", nativeQuery = true)
@Modifying
int updateById(String name, String id);

//SPEL expression, hql syntax
@Query("SELECT u FROM User u WHERE u.username = :username")
User findUser(@Param("username") String username);//Mapping parameter username to database field username

}
```

User.java

```
package com.kingbase.testspringdatajpa.entity;



@Entity
@Table(name = "test_springdatajpa_2")
public class User {
@Id
@GeneratedValue(strategy = GenerationType.TABLE)
// JPA provides four standard uses:
//GenerationType.TABLE: use a specific database table to save the primary key.
```

```
//GenerationType.SEQUENCE: generate primary keys based on the sequence of the underlying database, provided
that the database supports sequences.
//GenerationType.IDENTITY: the primary key is automatically generated by the database (mainly automatic
growth type)
//GenerationType.AUTO: the primary key is controlled by the program.
@Column(name = "id", nullable = false)
private Integer id;

@Column(name = "username")
private String username;

public User() {
}

public User(Integer id, String username) {
this.id = id;
this.username = username;
}

public Integer getId() {
return id;
}

public void setId(Integer id) {
this.id = id;
}

public String getUsername() {
return username;
}

public void setUsername(String username) {
this.username = username;
}

@Override
public String toString() {
return "User{" +
"id=" + id +
", username='" + username + '\'' +
'}';
}
}
```

UserServiceImpl.java

```
package com.kingbase.testspringdatajpa.service.impl;



@Transactional
@Service("userService")
public class UserServiceImpl implements UserService {
private final UserRepository userRepository;

@Autowired
public UserServiceImpl(UserRepository userRepository) {
this.userRepository = userRepository;
}
//insert
@Override
public void insertUser(User user) { userRepository.save(user); }

//delete
@Override
public void deleteUser(Integer id) {
if (!userRepository.existsById(id)) {
return;
}
}

//update
@Override
public int updateUser(User user) {
userRepository.save(user);
return 1;
}

//update based on name and id
@Override
public int updateUserById(String name, String id) {
return userRepository.updateById(name, id);
}

// select one user
@Override
public User selectUserById(Integer id) {
Optional<User> optional = userRepository.findById(id);
User user = optional.orElse(null);
return user;
```

```
}

//query user based on username
@Override
public User selectUserByName(String username) {
return userRepository.findUser(username);
}
@Override
public Iterator<User> selectUserAll(Integer pageNum, Integer pageSize) {
//By passing parameters to this method, physical pagination can be achieved, which is very simple.
Sort sort = Sort.by(Sort.Direction.ASC, "id");
Pageable pageable = PageRequest.of(pageNum, pageSize, sort);
Page<User> users = userRepository.findAll(pageable);
Iterator<User> userIterator = users.iterator();
return userIterator;
}
}
```

UserService.java

```
package com.kingbase.testspringdatajpa.service;


public interface UserService {
//insert
void insertUser(User user);

//delete
void deleteUser(Integer id);

//update
int updateUser(User user);

//customize SQL and modify name based on id
int updateUserById(String name, String id);

//select one user
User selectUserById(Integer id);

//customize SQL to query users based on username
User selectUserByName(String username);

//query all users
```

```
Iterator<User> selectUserAll(Integer pageNum, Integer pageSize);
}
```

TestSpringDataJpaApplication.java

```
package com.kingbase.testspringdatajpa;


@SpringBootApplication
public class TestSpringDataJpaApplication {

public static void main(String[] args) {
SpringApplication.run(TestSpringDataJpaApplication.class, args);
}
}
```

TestSpringDataJpaApplicationTests.java

```
package com.kingbase.testspringdatajpa;



@SpringBootTest
class TestSpringDataJpaApplicationTests {
@Autowired
private UserService userService;

@Test
void contextLoads() {
//insert
for (int i = 1; i <= 10; i++) {
userService.insertUser(new User(i, "insert" + i));
}
//delete
userService.deleteUser(1);
//update
userService.updateUser(new User(2, "update"));
//selectUserById
User user = userService.selectUserById(2);
System.out.println("user = " + user);
//selectUserByName
User userByName = userService.selectUserByName("insert");
```

```
System.out.println("userByName = " + userByName);
//query all users
Iterator<User> userIterator = userService.selectUserAll(0, 5);
userIterator.forEachRemaining(System.out::println);
}

}
```

2.2.7.4　相关文档

Jdbc 更多示例请参考 Jdbc 产品手册的程序示例。

2.3　编程接口-C/C++

2.3.1　Libkci

本文提供使用 Libkci 连接 Kingbase 数据库的连接代码示例。

2.3.1.1　前提条件

- 确保已设置基本的应用开发环境。
- 确保已安装与驱动对应版本的数据库，且数据库连接可用。
- 确保 Libkci 支持开发环境：
  - 系统架构：

    Linux：x86_64、arm、loongarch、mips、sw

    Windows：提供 64 位支持
  - 编译器：

    Linux：仅支持依赖 glibc 的编译器

    Windows：依赖 msvc120 的运行时库

满足以上条件后，可通过官网下载页的接口侧、或联系技服人员获取对应版本架构的 libkci 驱动。

2.3.1.2　操作实例

基于 Libkci 编译开发应用程序，下文以 Linux 环境使用 Libkci 连接数据库并执行简单的插入查询操作做示例说明，主要步骤如下：

2.3.1.2.1　下载并解压驱动包

解压 Libkci 对应版本的驱动包，驱动包内容如下所示：

```
Libkci
  include
     kingbase_ext.h
     libkci
        libkci_fs.h
     libkci_events.h
     libkci_fe.h
     sys_config_ext.h
     sys_config.h
  libcrypto.so.1.1
  libkci.so
  libssl.so.1.1
```

2.3.1.2.2　开发源程序

创建测试程序文件夹 test_libkci，将 Libkci 驱动包的头文件和库文件放置在指定目录，并创建源文件 test_libkci.c，示例目录结构如下：

```
test_libkci
  include
     kingbase_ext.h
     libkci
        libkci_fs.h
     libkci_events.h
     libkci_fe.h
     sys_config_ext.h
     sys_config.h
  lib
     libcrypto.so.1.1
     libkci.so
     libssl.so.1.1
  test_libkci.c
```

源码文件 test_libkci.c 中需要引用 Libkci 提供的头文件，如下所示：

```
#include <libkci_fe.h>
```

使用 KCIConnectionCreate() 创建连接，判断连接状态：

```
char connInfo[] = "host=127.0.0.1 port=54321 user=system password=123456 dbname=test";
KCIConnection *conn = NULL;
KCIResult *res = NULL;
```

```
conn = KCIConnectionCreate(connInfo);
if (KCIConnectionGetStatus(conn) != CONNECTION_OK)
{
 fprintf(stderr, "KCIConnectionCreate error:[%s]", KCIConnectionGetLastError(conn));
 if (res)
     KCIResultDealloc(res);
 KCIConnectionDestory(conn);
 exit(1);
}
```

使用 KCIStatementExecute() 执行 SQL 语句，执行完后需要手动执行 KCIResultDealloc() 销毁结果集的缓冲区：

```
KCIResult *res = NULL;

res = KCIStatementExecute(conn, "create table testQuery (id int, val1 text, val2 text, val3 text, val4 text,
val5 text, val6 text)");
KCIResultDealloc(res);
```

使用 KCIResultGetRowCount() 和 KCIResultGetColumnCount() 遍历结果集，并打印对应列数据：

```
int i,j;
KCIResult *res = NULL;

for (i=0; i<KCIResultGetRowCount(res);i++)
{
    for (j=0; j<KCIResultGetColumnCount(res); j++)
    {
        printf("| %s ", KCIResultGetColumnValue(res, i, j));
    }
    printf("\n");
}
KCIResultDealloc(res);
```

完整用例如下：

```
#include <stdio.h>
#include <stdlib.h>

#include "libkci_fe.h"


int main()
{
    int i,j;
```

```
    /* 连接串 */
    char connInfo[] = "host=127.0.0.1 port=54321 user=system password=123456 dbname=test";
    KCIConnection *conn = NULL;
    KCIResult *res = NULL;

    conn = KCIConnectionCreate(connInfo);
    if (KCIConnectionGetStatus(conn) != CONNECTION_OK)
    {
        fprintf(stderr, "KCIConnectionCreate error:[%s]", KCIConnectionGetLastError(conn));
        if (res)
            KCIResultDealloc(res);
        KCIConnectionDestory(conn);
        exit(1);
    }

    // 创建新表
    res = KCIStatementExecute(conn, "drop table if exists testQuery");
    KCIResultDealloc(res);
    res = KCIStatementExecute(conn, "create table testQuery (id int, val1 text, val2 text, val3 text, val4
text, val5 text, val6 text)");
    KCIResultDealloc(res);

    // 执行插入
    res = KCIStatementExecute(conn, "insert into testQuery values(0, '电科\', '金仓', '数据库', '000','000',
'000');insert into testQuery values(1, 'aaa','aaa','aaa','aaa','aaa','aaa');insert into testQuery values(2,
'bbb','bbb','bbb','bbb','bbb','bbb');");
    KCIResultDealloc(res);

    // 执行查询
    res = KCIStatementExecute(conn, "select * from testQuery");
    // 打印结果集
    for (i=0; i<KCIResultGetRowCount(res);i++)
    {
        for (j=0; j<KCIResultGetColumnCount(res); j++)
        {
            printf("| %s ", KCIResultGetColumnValue(res, i, j));
        }
        printf("\n");
    }
    KCIResultDealloc(res);

    KCIConnectionDestory(conn);
    return 0;
}
```

备注:

数据库连接串需要根据实际情况填写，Libkci 支持两种形式的连接串格式：

- 键值对形式

```
"host=127.0.0.1 port=54321 user=system password=123456 dbname=test"
```

- URL 形式

```
"kingbase://user:secret@host:port/dbname"
例：
"kingbase://system:123456@127.0.0.1:54321/test"
```

更多连接参数及 Libkci 接口说明请参考 Libkci 产品手册

2.3.1.2.3　编译

编写好源程序后，使用 gcc 编译，需要显示指定 Libkci 的头文件和库文件安装路径，以下是示例使用的 gcc 编译参数说明：

描述	参数
编译指定头文件路径	-I directory (directory 指定 Libkci 的头文件路径)
编译指定库文件路径	-L directory (directory 指定 Libkci 的库文件路径)
编译指定链接库	-lkci
运行时优先搜索运行库路径	-Wl,-rpath directory (directory 指定 Libkci 的库文件路径)

完整的编译命令如下，以示例的目录结构为例：

```
gcc -I ./include -L ./lib test_libkci.c -o testLibkci -lkci -Wl,-rpath,./lib
```

如果要编写 make 相关的文件，如 makefile，需要向编译参数添加如下选项：

```
CPPFLAGS += -I (Libkci 头文件所在目录)
LDFLAGS += -L (Libkci 库文件所在目录)
LIBS += -lkci
```

2.3.1.2.4　运行

编译后执行程序，命令行窗口会打印刚刚插入的数据，示例期望输出如下所示：

```
| 0 | 电科 | 金仓 | 数据库 | 000 | 000 | 000
| 1 | aaa | aaa | aaa | aaa | aaa | aaa
| 2 | bbb | bbb | bbb | bbb | bbb | bbb
```

2.3.1.3　常见问题

1. Linux 环境运行测试程序报错无法找到 libkci.so.5，报错如下：

   ```
   ./testLibkci: error while loading shared libraries: libkci.so.5: cannot open shared object file: No such file or directory
   ```

   原因：Libkci 驱动包内可能没有提供 libkci.so.5 库，使用 Libkci 编译的应用程序运行时会依赖 libkci.so.5。

   解决：将 `libkci.so` 拷贝复制一份为 libkci.so.5，或添加软链接，使应用程序运行时能正确找到 libkci.so.5 并加载依赖。

2. 驱动包 SSL 库与系统环境的 SSL 库冲突

   原因：系统环境的依赖库版本过高，或应用程序运行时错误加载了系统环境的 SSL 库。

   解决：通过 ldd 应用程序名，如 `ldd testLibkci` 查看当前环境下的依赖关系，确保运行应用程序时加载驱动包提供的 SSL 库，若仍然有 SSL 相关报错，则确定是驱动包提供的 SSL 库无法在当前环境下使用，此时请联系技服获取 Libkci 静态依赖 SSL 库的驱动包来解决 SSL 依赖冲突问题。

3. Windows 环境使用 Visual studio 编译 Debug 版本的应用程序崩溃

   原因：当前默认提供的 Windows 驱动包是 Release 版本，使用 Debug 模式编译应用程序可能造成未知问题。

   解决：请使用 Release 模式编译应用程序，如有需求，请联系技服申请 Debug 版本的 Libkci 驱动。

2.3.2　Dci

本文提供使用 Dci 连接 Kingbase 数据库的连接代码示例。

2.3.2.1　前提条件

- 确保已设置基本的应用开发环境。
- 确保已安装与驱动对应版本的数据库，且数据库连接可用。
- 确保 Dci 支持开发环境：

– 系统架构：

Linux：x86_64、arm、loongarch、mips、sw

Windows：V9 版本提供 64 位支持

– 编译器：

Linux：仅支持依赖 glibc 的编译器

Windows：依赖 msvc120 的运行时库

满足以上条件后，可通过已发布版本的数据库安装包、官网下载页的接口侧、或联系技服人员获取对应版本架构的 Dci 驱动。

2.3.2.2　Dci 和 Oci

Kingbase 提供 Dci 和 Oci 接口，Oci 兼容 Oracle 的 Oci 接口，可复用 Oralce 的 Oci 头文件。

- OCI 是 Oracle 提供的一种应用程序编程接口 (API)，允许您创建使用函数调用来访问 Oracle 数据库、并控制 SQL 语句执行和数据访问的所有阶段的应用程序。
- DCI 是 KingbaseES 提供的应用程序编程接口，该接口兼容 OCI 的功能，允许应用程序使用函数调用来访问 KingbaseES 数据库，并控制 SQL 语句执行和数据访问的所有阶段。

2.3.2.3　操作实例

基于 Dci 编译开发应用程序，下文以 Linux 环境使用 Dci 连接数据库并执行简单的插入查询操作做示例说明，主要步骤如下：

2.3.2.3.1　创建测试程序目录

下载并解压驱动包，驱动包内容如下所示：

```
Dci
  include
     dciapi.h
     dcidef.h
     dci.h
     dcitypes.h
  lib
      libcrypto.so.1.1
      libdcikdb.so
      libkci.so.5
      libssl.so.1.1
```

创建测试程序文件夹 test_dci，将 Dci 驱动包的库文件放置在创建的文件夹路径下，并创建源文件 test_dci.c 及 Dci 驱动使用的配置文 sys_service.conf，示例目录结构如下：

```
test_dci
  include
     dciapi.h
     dcidef.h
     dci.h
     dcitypes.h
  lib
     libcrypto.so.1.1
     libdcikdb.so
     libkci.so.5
     libssl.so.1.1
  sys_service.conf
  test_dci.c
```

其中，sys_service.conf 配置文件示例如下，更多参数说明请参考 Dci 产品手册的配置参数说明：

```
# DSN 数据源配置名
[KingbaseES]
# 数据库名
dbname=test
# 数据库端口
port=54321
# 数据库 IP
host=127.0.0.1
# 驱动日志生成路径及日志文件名
#DCILog=oci.log
# 驱动日志级别
#DCILogLevel=DEBUG3
# 是否自动提交
AutoCommit=1
# 客户端字符集编码，需要和数据库服务端一致
ClientEncoding=utf8
```

2.3.2.3.2　开发源程序

开发源程序 test_dci.c，以下是使用 Dci 驱动连接并操作数据库的一个简单示例：

1. 导入 Dci 头文件：

```
#include "dci.h"
```

2. 初始化 DCI 环境

   使用 `DCIEnvCreate()` 创建 Dci 环境句柄：

```
DCIEnv *envhp = NULL;
DCIEnvCreate(&envhp, DCI_DEFAULT, NULL, NULL, NULL, NULL, 0, NULL);
```

3. 创建 DCI 错误句柄

   使用 DCIHandleAlloc() 创建错误消息句柄，对应句柄类型使用 DCI_HTYPE_ERROR:

```
DCIError *errhp = NULL;
DCIHandleAlloc(envhp, (void **)&errhp, DCI_HTYPE_ERROR, 0, NULL);
```

4. 创建服务上下文，连接数据库

   使用 DCILogon() 创建错误消息句柄:

```
text *dsnName = (text *)"KingbaseES"; /* 配置文件 sys_service.conf 中配置的数据源名 */
text *userName = (text *)"SYSTEM";   /* 用户名 */
text *password = (text *)"123456";  /* 密码 */
DCISvcCtx *svchp = NULL;

DCILogon(envhp, errhp, &svchp, (text *)userName, strlen(userName), (text *)password, strlen(password),
(text*)dsnName,                        strlen(dsnName));
```

备注:

Dci 提供两种数据库连接串形式：

- DSN

  推荐使用 DSN 形式的字符串。使用该形式的连接方式，会在当前运行环境查找对应的环境变量 KINGBASE_CONFDIR，若没有对应环境变量则查找默认路径是否存在 sys_service.conf 配置文件，并加载配置文件的驱动配置。

  sys_service.conf 可通过指定系统环境变量 KINGBASE_CONFDIR 来让应用程序显示加载对应配置文件的 DSN 的配置。

  若不配置环境变量，可通过在默认路径放置 sys_service.conf 来加载对应 DSN 配置，默认搜索路径如下：

  - Windows XP 默认路径为：X: \Documents and Settings\All Users\Application Data\KingbaseES 目录。
  - Windows 7 及以上版本默认路径为：X:\ProgramData\KingbaseES 目录。
  - Linux 指定路径为：/etc 目录。

  注：

  1. 路径中 KingbaseES 文件夹需要用户自己建立；
  2. X 为系统盘符；
  3. 在 Windows 平台，需注意 Application Data 与 ProgramData 为系统隐藏文件夹。

使用 DSN 形式的连接方式的示例如下：

```
text *dsnName = (text *)"KingbaseES"; /* 配置文件 sys_service.conf 中配置的数据源名 */
text *userName = (text *)"SYSTEM";   /* 用户名 */
text *password = (text *)"123456";  /* 密码 */
DCISvcCtx *svchp = NULL;

DCILogon(envhp, errhp, &svchp, (text *)userName, strlen(userName), (text *)password,
strlen(password), (text*)dsnName,                        strlen(dsnName));
```

- 连接字符串

不推荐直接使用连接字符串的形式连接数据库。

使用该形式的连接方式，Dci 会先尝试搜索 sys_service.conf 文件，并加载名为 [Default] 的 DSN 配置；

若无对应配置文件且无对应 DSN，则加载默认配置连接数据库，同时无法修改 Dci 连接配置，导致 Dci 使用使用受限，以下是使用连接字符串形式的 Dci 默认连接配置：

非自动提交

AutoCommit=0

DDL 语句自动提交

AutoCommitDDL=1

启用批量拓展协议，优化批量操作性能

BatchInsertSize_Ext=1

开启语句级自动回滚，该配置会极大影响性能

Protocol=7.4-2

使用连接字符串的示例如下：

```
text *dsnName = (text *)"127.0.0.1:54321/test"; /* ip:port/dbname */
text *userName = (text *)"SYSTEM";   /* 用户名 */
text *password = (text *)"123456";  /* 密码 */
DCISvcCtx *svchp = NULL;

DCILogon(envhp, errhp, &svchp, (text *)userName, strlen(userName), (text *)password,
strlen(password), (text*)dsnName,                        strlen(dsnName));
```

更多参数说明请参考 Dci 产品手册的配置参数说明：

---

5. 创建 SQL 语句句柄

使用 `DCIHandleAlloc()` 创建错误消息句柄，对应句柄类型使用 DCI_HTYPE_STMT:

```
DCIStmt *stmthp = NULL;
DCIHandleAlloc(envhp, (void **)&stmthp, DCI_HTYPE_STMT, 0, NULL);
```

6. 创建测试表

使用 DCIStmtPrepare() 生成预备语句，并使用 DCIStmtExecute() 执行 SQL：

```
text *sqlCreateTb = (text *)"create table test_dci(id int, name varchar(20))";
text *sqlDropTb = (text *)"drop table if exists test_dci";

// 删表
DCIStmtPrepare(stmthp, errhp, sqlDropTb, strlen((char *)sqlDropTb), DCI_NTV_SYNTAX, DCI_DEFAULT);
DCIStmtExecute(svchp, stmthp, errhp, 0, 0, NULL, NULL, DCI_DEFAULT);

// 建表
DCIStmtPrepare(stmthp, errhp, sqlCreateTb, strlen((char *)sqlCreateTb), DCI_NTV_SYNTAX, DCI_DEFAULT);
DCIStmtExecute(svchp, stmthp, errhp, 0, 0, NULL, NULL, DCI_DEFAULT);
```

7. 插入数据

使用 DCIStmtPrepare() 生成预备语句，可使用 DCIBindByPos() 依次绑定输入参数，绑定参数后，使用 DCIStmtExecute() 执行 SQL：

```
text *sqlInsertTb = (text *)"insert into test_dci values(:1, :2)";

// 插入
DCIStmtPrepare(stmthp, errhp, sqlInsertTb, strlen((char *)sqlInsertTb), DCI_NTV_SYNTAX, DCI_DEFAULT);

// 绑定输入参数
int valueIn1[MAX_ROWS] = {1, 2};
char valueIn2[MAX_ROWS][MAX_DATABUF_LEN] = {"Kingbase1", "kikk2"};

DCIBind *bindIn1p, *bindIn2p;
DCIBindByPos(stmthp, &bindIn1p, errhp, 1, &valueIn1[0], sizeof(valueIn1[0]), SQLT_INT, NULL, NULL,
NULL, 0, NULL,                                    DCI_DEFAULT);
DCIBindByPos(stmthp, &bindIn2p, errhp, 2, valueIn2[0], sizeof(valueIn2[0]), SQLT_STR, NULL, NULL, NULL,
0, NULL,                            DCI_DEFAULT);

DCIStmtExecute(svchp, stmthp, errhp, MAX_ROWS, 0, NULL, NULL, DCI_DEFAULT);
```

8. 查询数据并打印结果集

同理插入数据的流程，使用 DCIStmtPrepare() 生成预备语句，可使用 DCIBindByPos() 依次绑定输入参数，绑定参数后，使用 DCIStmtExecute() 执行 SQL：

```
text *sqlSelectTb = (text *)"select id,name from test_dci";
```

```
// 查询
DCIStmtPrepare(stmthp, errhp, (text *)sqlSelectTb, strlen(sqlSelectTb), DCI_NTV_SYNTAX, DCI_DEFAULT);

// 绑定输出参数
DCIDefine *defnp = NULL;
int valueOut1 = 0;
char valueOut2[MAX_DATABUF_LEN]={0};
DCIDefineByPos(stmthp, &defnp, errhp, 1, &valueOut1, sizeof(valueOut1), SQLT_INT, NULL, NULL, NULL,
DCI_DEFAULT);
DCIDefineByPos(stmthp, &defnp, errhp, 2, valueOut2, sizeof(valueOut2), SQLT_STR, NULL, NULL, NULL, DCI_
DEFAULT);

DCIStmtExecute(svchp, stmthp, errhp, 0, 0, NULL, NULL, DCI_DEFAULT);
```

执行查询后，可通过 DCIStmtFetch() 获取结果集，以下是遍历结果集一次获取一行结果的示例：

```
sword status = DCI_SUCCESS;
do
{
     status = DCIStmtFetch(stmthp, errhp, 1, DCI_FETCH_NEXT, DCI_DEFAULT);
     // 剩余结果集为空则退出
     if (DCI_NO_DATA == status)
     {
            break;
     }
     printf("col1=[%d], col2=[%s]\n", valueOut1, valueOut2);
} while (DCI_SUCCESS == status);
```

9. 关闭连接并释放资源：

```
// 清理资源
DCIHandleFree(stmthp, DCI_HTYPE_STMT);
DCIHandleFree(errhp, DCI_HTYPE_ERROR);
DCILogoff(svchp, errhp);
DCIHandleFree(svchp, DCI_HTYPE_SVCCTX);
DCIHandleFree(errhp, DCI_HTYPE_ERROR);
DCIHandleFree(envhp, DCI_HTYPE_ENV);
```

以下是完整示例：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "dci.h"
```

```
#define MAX_ROWS 2
#define MAX_DATABUF_LEN 20

void check_error(DCIError *errhp, sword status) {
    if (status != DCI_SUCCESS) {
        char errbuf[512] = {0};
        sb4 errcode = 0;
        DCIErrorGet(errhp, 1, NULL, &errcode, (text *)errbuf, sizeof(errbuf), DCI_HTYPE_ERROR);
        printf("status: %d, Dci error code: %d, errbuff: %s\n", status, errcode, errbuf);
        exit(1);
    }
}

int main()
{
    text *dsnName = (text *)"KingbaseES"; /* 配置文件 sys_service.conf 中配置的数据源名 */
    text *userName = (text *)"SYSTEM";   /* 用户名 */
    text *password = (text *)"123456";  /* 密码 */

    DCIEnv *envhp = NULL;
    DCIError *errhp = NULL;
    DCISvcCtx *svchp = NULL;
    DCIStmt *stmthp = NULL;
    sword status = DCI_SUCCESS;

    text *sqlCreateTb = (text *)"create table test_dci(id int, name varchar(20))";
    text *sqlDropTb = (text *)"drop table if exists test_dci";
    text *sqlInsertTb = (text *)"insert into test_dci values(:1, :2)";
    text *sqlSelectTb = (text *)"select id,name from test_dci";


    // 初始化 DCI 环境
    status = DCIEnvCreate(&envhp, DCI_DEFAULT, NULL, NULL, NULL, NULL, 0, NULL);
    check_error(NULL, status);

    // 创建 DCI 错误句柄
    status = DCIHandleAlloc(envhp, (void **)&errhp, DCI_HTYPE_ERROR, 0, NULL);
    check_error(errhp, status);

    // 创建服务上下文
    status = DCILogon(envhp, errhp, &svchp, (text *)userName, strlen(userName), (text *)password,
strlen(password), (text*)dsnName, strlen(dsnName));
    check_error(errhp, status);

    // 创建 SQL 语句句柄
```

```
    status = DCIHandleAlloc(envhp, (void **)&stmthp, DCI_HTYPE_STMT, 0, NULL);
    check_error(errhp, status);

    // 删表
    status = DCIStmtPrepare(stmthp, errhp, sqlDropTb, strlen((char *)sqlDropTb), DCI_NTV_SYNTAX, DCI_
DEFAULT);
    check_error(errhp, status);
    status = DCIStmtExecute(svchp, stmthp, errhp, 0, 0, NULL, NULL, DCI_DEFAULT);
    check_error(errhp, status);

    // 建表
    status = DCIStmtPrepare(stmthp, errhp, sqlCreateTb, strlen((char *)sqlCreateTb), DCI_NTV_SYNTAX, DCI_
DEFAULT);
    check_error(errhp, status);
    status = DCIStmtExecute(svchp, stmthp, errhp, 0, 0, NULL, NULL, DCI_DEFAULT);
    check_error(errhp, status);

    // 插入
    status = DCIStmtPrepare(stmthp, errhp, sqlInsertTb, strlen((char *)sqlInsertTb), DCI_NTV_SYNTAX, DCI_
DEFAULT);
    check_error(errhp, status);

    // 绑定输入参数
    int valueIn1[MAX_ROWS] = {1, 2};
    char valueIn2[MAX_ROWS][MAX_DATABUF_LEN] = {"Kingbase1", "kikk2"};

    DCIBind *bindIn1p, *bindIn2p;
    status = DCIBindByPos(stmthp, &bindIn1p, errhp, 1, &valueIn1[0], sizeof(valueIn1[0]), SQLT_INT, NULL,
NULL, NULL, 0, NULL, DCI_DEFAULT);
    check_error(errhp, status);
    status = DCIBindByPos(stmthp, &bindIn2p, errhp, 2, valueIn2[0], sizeof(valueIn2[0]), SQLT_STR, NULL,
NULL, NULL, 0, NULL, DCI_DEFAULT);
    check_error(errhp, status);

    status = DCIStmtExecute(svchp, stmthp, errhp, MAX_ROWS, 0, NULL, NULL, DCI_DEFAULT);
    check_error(errhp, status);

    printf("insert ok.\n");

    status = DCITransCommit(svchp, errhp, DCI_DEFAULT);
    check_error(errhp, status);

    // 查询
    status = DCIStmtPrepare(stmthp, errhp, (text *)sqlSelectTb, strlen(sqlSelectTb), DCI_NTV_SYNTAX, DCI_
DEFAULT);
```

```
    check_error(errhp, status);

    // 绑定输出参数
    DCIDefine *defnp = NULL;
    int valueOut1 = 0;
    char valueOut2[MAX_DATABUF_LEN]={0};
    status = DCIDefineByPos(stmthp, &defnp, errhp, 1, &valueOut1, sizeof(valueOut1), SQLT_INT, NULL, NULL,
NULL, DCI_DEFAULT);
    check_error(errhp, status);
    status = DCIDefineByPos(stmthp, &defnp, errhp, 2, valueOut2, sizeof(valueOut2), SQLT_STR, NULL, NULL,
NULL, DCI_DEFAULT);
    check_error(errhp, status);

    status = DCIStmtExecute(svchp, stmthp, errhp, 0, 0, NULL, NULL, DCI_DEFAULT);
    check_error(errhp, status);
    do
    {
        status = DCIStmtFetch(stmthp, errhp, 1, DCI_FETCH_NEXT, DCI_DEFAULT);
        if (DCI_NO_DATA == status)
        {
            break;
        }
        check_error(errhp, status);
        printf("col1=[%d], col2=[%s]\n", valueOut1, valueOut2);
    } while (DCI_SUCCESS == status);


    // 清理资源
    DCIHandleFree(stmthp, DCI_HTYPE_STMT);
    DCIHandleFree(errhp, DCI_HTYPE_ERROR);
    DCILogoff(svchp, errhp);
    DCIHandleFree(svchp, DCI_HTYPE_SVCCTX);
    DCIHandleFree(errhp, DCI_HTYPE_ERROR);
    DCIHandleFree(envhp, DCI_HTYPE_ENV);

    return 0;
}
```

Dci 更多示例请参考 Dci 产品手册的程序示例。

2.3.2.3.3　编译程序

使用 gcc 编译，需要显示指定 Dci 库文件安装路径，以下是示例使用的 gcc 编译参数说明：

描述	参数
编译指定库文件路径	-L directory (directory 指定 Dci 库文件路径)
编译指定链接库	-ldcikdb
运行时优先搜索运行库路径	-Wl,-rpath directory (directory 指定 Dci 库文件路径)

完整的编译命令如下，以示例的目录结构为例：

```
gcc -I ./include -L ./lib test_dci.c -o testDci -ldcikdb -Wl,-rpath,./lib
```

2.3.2.3.4　运行程序

编译后执行程序，示例使用环境变量 KINGBASE_CONFDIR 的 sys_service.conf 的驱动配置操作数据库：

```
# 当前 sys_service.conf 路径，配置环境变量后执行程序
export KINGBASE_CONFDIR=.
./testDci
```

命令行窗口会打印刚刚插入的数据，示例期望输出如下所示：

```
insert ok.
col1=[1], col2=[Kingbase1]
col1=[2], col2=[kikk2]
```

2.3.2.4　常见问题

1. 驱动包 SSL 库与系统环境的 SSL 库冲突

   原因：系统环境的依赖库版本过高，或应用程序运行时错误加载了系统环境的 SSL 库。

   解决：通过 ldd 应用程序名，如 `ldd testDci` 查看当前环境下的依赖关系，确保运行应用程序时加载驱动包提供的 SSL 库，若仍然有 SSL 相关报错，则确定是驱动包提供的 SSL 库无法在当前环境下使用，此时请联系技服获取 Dci 静态依赖 SSL 库的驱动包来解决 SSL 依赖冲突问题。

2. Windows 环境使用 Visual studio 编译 Debug 版本的应用程序崩溃

   原因：当前默认提供的 Windows 驱动包是 Release 版本，使用 Debug 模式编译应用程序可能造成未知问题。

   解决：请使用 Release 模式编译应用程序，如有需求，请联系技服申请 Debug 版本的 Dci 驱动。

2.3.3　Odbc

本文提供使用 Odbc 连接 Kingbase 数据库的连接代码示例。

2.3.3.1　前提条件

- 确保已设置基本的应用开发环境。
- 确保已安装与驱动对应版本的数据库，且数据库连接可用。
- 确保当前支持 Odbc 开发：
  - Linux：确保已安装 unixOdbc
  - Windows：默认支持 Odbc
- 确保 Odbc 支持开发环境：
  - 系统架构：

    Linux：x86_64、arm、loongarch、mips、sw

    Windows：V9 版本提供 64 位支持
  - 编译器：

    Linux：仅支持依赖 glibc 的编译器

    Windows：依赖 msvc120 的运行时库

满足以上条件后，可通过已发布版本的数据库安装包、官网下载页的接口侧、或联系技服人员获取对应版本架构的 ODBC 驱动。

2.3.3.2　操作实例 - Linux

2.3.3.2.1　安装并部署驱动

Odbc 驱动包目录如下所示：

```
Odbc
  kdbodbcw.so
  libcrypto.so.1.1
  libkci.so.5
  libodbcinst.so.2
  libssl.so.1.1
```

1. 先确保当前环境已经安装 unixOdbc(推荐安装 unixOdbc-2.3.4)

   下载 unixOdbc 源码包，执行以下命令编译安装：

```
./configure --prefix=`pwd`/release
make
make install
```

安装后，可通过 `odbcinst --version` 查看 unixOdbc 版本：

```
unixODBC 2.3.4
```

2. 创建测试程序文件夹 test_odbc，文件夹目录如下所示：

```
test_odbc
   lib
       kdbodbcw.so
       libcrypto.so.1.1
       libkci.so.5
       libodbcinst.so.2
       libssl.so.1.1
   Makefile
   odbc.ini
   odbcinst.ini
   test_odbc.c
```

以下所有操作均在该路径下进行。

3. 配置 Odbc 数据源

新建文件 odbcinst.ini 和 odbc.ini

- odbcinst.ini 配置如下所示，生成了一个 kdbodbc test driver 的驱动配置，Driver 用于指定 Odbc 驱动路径：

```
[kdbodbc test driver]
Description = KingbaseES ODBC driver (Unicode version), for regression tests
Driver = /home/kingbase/test_odbc/lib/kdbodbcw.so
```

- odbc.ini 配置如下所示，生成了一个 DSN 名为 KingbaseES 的 Odbc 配置，Driver 指向 kdbodbc test driver 的驱动路径，该配置文件用于指定 Odbc 连接参数及配置信息：

```
[KingbaseES]
Description     = kdbodbc regression test DSN
Driver          = kdbodbc test driver
Servername      = 127.0.0.1
Port            = 54321
Username        = system
Password        = 123456
Database        = test
```

配置对应文件后，可将配置路径添加到当前命令行窗口的环境变量中，使 unixOdbc 能正确识别 Odbc 驱动配置信息。

以下是一个将当前路径下的 odbcinst.ini 和 odbc.ini 添加到环境变量的示例：

```
export ODBCSYSINI=.
export ODBCINSTINI=./odbcinst.ini
export ODBCINI=./odbc.ini
```

配置完成后，可通过 `odbcinst -j` 查看信息：

```
unixODBC 2.3.4
DRIVERS............: ./
SYSTEM DATA SOURCES: ./odbc.ini
FILE DATA SOURCES..: ./ODBCDataSources
USER DATA SOURCES..: ./odbc.ini
SQLULEN Size.......: 8
SQLLEN Size........: 8
SQLSETPOSIROW Size.: 8
```

4. 测试 Odbc 驱动能否连接到数据库

   使用 unixOdbc 的命令 `isql DSNName`，可通过 unixOdbc 直接测试驱动是否成功加载并尝试连接到数据库，示例如下：

```
$ isql KingbaseES
+---------------------------------------+
| Connected!                            |
|                                       |
| sql-statement                         |
| help [tablename]                      |
| quit                                  |
|                                       |
+---------------------------------------+
```

2.3.3.2.2　开发源程序

以下是一个插入并执行查询的具体示例。

1. 导入 unixOdbc 的头文件：

```
#include "sql.h"
#include "sqlext.h"
```

2. 初始化 Odbc 环境

   使用 `SQLAllocHandle()` 创建 SQLHENV 实例，创建成功后，使用 `SQLSetEnvAttr()` 设置刚刚创建的 SQLHENV 实例：

```
SQLHENV hEnv;
// 1. 初始化 ODBC 环境
SQLAllocHandle(SQL_HANDLE_ENV, SQL_NULL_HANDLE, &hEnv);
SQLSetEnvAttr(hEnv, SQL_ATTR_ODBC_VERSION, (SQLPOINTER)SQL_OV_ODBC3, 0);
```

3. 连接数据库

   使用 SQLAllocHandle() 创建 SQLHDBC 实例，创建成功后，使用 SQLConnect() 连接到数据库：

```
SQLHDBC hDbc;
SQLAllocHandle(SQL_HANDLE_DBC, hEnv, &hDbc);
// SQLConnect(hDbc, DSN, SQL_NTS, user, SQL_NTS, password, SQL_NTS)
SQLConnect(hDbc, (SQLCHAR*)"KingbaseES", SQL_NTS, (SQLCHAR*)"system", SQL_NTS, (SQLCHAR*)"123456", SQL_
NTS);
```

4. 创建测试表

   使用 SQLAllocHandle() 创建 SQLHSTMT 实例，用于执行 SQL 语句相关操作，建表语句不需要绑定参数，使用 SQLExecDirect() 执行 SQL：

```
SQLHSTMT hStmt;

const char *sqlDropTb = "drop table if exists test_odbc;";
const char *sqlCreateTb = "create table test_odbc (id int, name varchar(100));";

// 3. 创建表
SQLAllocHandle(SQL_HANDLE_STMT, hDbc, &hStmt);

SQLExecDirect(hStmt, (SQLCHAR*)sqlDropTb, SQL_NTS);
SQLExecDirect(hStmt, (SQLCHAR*)sqlCreateTb, SQL_NTS);
```

5. 插入数据

   使用 SQLPrepare() 创建插入的预备语句，创建成功后，使用 SQLBindParameter() 绑定参数，最后使用 SQLExecute() 执行 SQL：

```
// 4. 插入数据（绑定参数）
const char *sqlInsertTb = "insert into test_odbc values (?, ?)";
SQLINTEGER inId = 1;
SQLCHAR inName[] = "KKKingbase";

SQLPrepare(hStmt, (SQLCHAR*)sqlInsertTb, SQL_NTS);

SQLBindParameter(hStmt, 1, SQL_PARAM_INPUT, SQL_C_LONG, SQL_INTEGER, 0, 0, &inId, 0, NULL);
SQLBindParameter(hStmt, 2, SQL_PARAM_INPUT, SQL_C_CHAR, SQL_VARCHAR, 100, 0, inName, 0, NULL);

SQLExecute(hStmt);
```

6. 查询并打印结果集

   使用 SQLExecDirect() 执行查询，再使用 SQLBindCol() 绑定输出的查询结果，循环使用 SQLFetch() 遍历结果集并打印结果：

```
// 5. 执行查询
SQLRETURN ret;
const char *sqlSelectTb = "select * from test_odbc";
SQLINTEGER outId;
SQLCHAR outName[100];

SQLExecDirect(hStmt, (SQLCHAR*)sqlSelectTb, SQL_NTS);

// 绑定查询结果
SQLBindCol(hStmt, 1, SQL_C_LONG, &outId, 0, NULL);
SQLBindCol(hStmt, 2, SQL_C_CHAR, outName, sizeof(outName), NULL);

// 获取查询结果并打印
while ((ret = SQLFetch(hStmt)) != SQL_NO_DATA) {
    if (ret == SQL_SUCCESS || ret == SQL_SUCCESS_WITH_INFO) {
        printf("ID: %d, Name: %s\n", outId, outName);
    } else {
        CHECK_ERROR(ret, hStmt, SQL_HANDLE_STMT);
    }
}
```

7. 释放资源

```
// 6. 清理
SQLFreeHandle(SQL_HANDLE_STMT, hStmt);
SQLDisconnect(hDbc);
SQLFreeHandle(SQL_HANDLE_DBC, hDbc);
SQLFreeHandle(SQL_HANDLE_ENV, hEnv);
```

完整用例如下：

```
#include <stdio.h>
#include <stdlib.h>

#include "sql.h"
#include "sqlext.h"

// 定义一些宏，简化 ODBC 错误处理
#define CHECK_ERROR(ret, handle, handle_type) \
    if (ret != SQL_SUCCESS && ret != SQL_SUCCESS_WITH_INFO) { \
        SQLCHAR state[6]; \
        SQLCHAR message[1000]; \
```

```
        SQLINTEGER native_error; \
        SQLSMALLINT message_length; \
        SQLGetDiagRec(handle_type, handle, 1, state, &native_error, message, sizeof(message), &message_
length); \
        printf("ODBC Error: %s\n", message); \
        exit(1); \
    }


int main() {
    SQLHENV hEnv;
    SQLHDBC hDbc;
    SQLHSTMT hStmt;
    SQLRETURN ret;

    const char *sqlDropTb = "drop table if exists test_odbc;";
    const char *sqlCreateTb = "create table test_odbc (id int, name varchar(100));";
    const char *sqlInsertTb = "insert into test_odbc values (?, ?)";
    const char *sqlSelectTb = "select * from test_odbc";


    // 1. 初始化 ODBC 环境
    ret = SQLAllocHandle(SQL_HANDLE_ENV, SQL_NULL_HANDLE, &hEnv);
    CHECK_ERROR(ret, hEnv, SQL_HANDLE_ENV);

    ret = SQLSetEnvAttr(hEnv, SQL_ATTR_ODBC_VERSION, (SQLPOINTER)SQL_OV_ODBC3, 0);
    CHECK_ERROR(ret, hEnv, SQL_HANDLE_ENV);

    // 2. 连接数据库
    ret = SQLAllocHandle(SQL_HANDLE_DBC, hEnv, &hDbc);
    CHECK_ERROR(ret, hDbc, SQL_HANDLE_DBC);

    ret = SQLConnect(hDbc, (SQLCHAR*)"KingbaseES", SQL_NTS, (SQLCHAR*)"system", SQL_NTS, (SQLCHAR*)"123456",
SQL_NTS);
    CHECK_ERROR(ret, hDbc, SQL_HANDLE_DBC);
    printf("Connected to the database.\n");

    // 3. 创建表
    ret = SQLAllocHandle(SQL_HANDLE_STMT, hDbc, &hStmt);
    CHECK_ERROR(ret, hStmt, SQL_HANDLE_STMT);

    ret = SQLExecDirect(hStmt, (SQLCHAR*)sqlDropTb, SQL_NTS);
    CHECK_ERROR(ret, hStmt, SQL_HANDLE_STMT);

    ret = SQLExecDirect(hStmt, (SQLCHAR*)sqlCreateTb, SQL_NTS);
```

```
    CHECK_ERROR(ret, hStmt, SQL_HANDLE_STMT);
    printf("Table created successfully.\n");

    // 4. 插入数据（绑定参数）
    ret = SQLPrepare(hStmt, (SQLCHAR*)sqlInsertTb, SQL_NTS);
    CHECK_ERROR(ret, hStmt, SQL_HANDLE_STMT);

    SQLINTEGER inId = 1;
    SQLCHAR inName[] = "KKKingbase";

    ret = SQLBindParameter(hStmt, 1, SQL_PARAM_INPUT, SQL_C_LONG, SQL_INTEGER, 0, 0, &inId, 0, NULL);
    CHECK_ERROR(ret, hStmt, SQL_HANDLE_STMT);

    ret = SQLBindParameter(hStmt, 2, SQL_PARAM_INPUT, SQL_C_CHAR, SQL_VARCHAR, 100, 0, inName, 0, NULL);
    CHECK_ERROR(ret, hStmt, SQL_HANDLE_STMT);

    ret = SQLExecute(hStmt);
    CHECK_ERROR(ret, hStmt, SQL_HANDLE_STMT);
    printf("Data inserted successfully.\n");

    // 5. 执行查询
    ret = SQLExecDirect(hStmt, (SQLCHAR*)sqlSelectTb, SQL_NTS);
    CHECK_ERROR(ret, hStmt, SQL_HANDLE_STMT);

    SQLINTEGER outId;
    SQLCHAR outName[100];

    // 绑定查询结果
    ret = SQLBindCol(hStmt, 1, SQL_C_LONG, &outId, 0, NULL);
    CHECK_ERROR(ret, hStmt, SQL_HANDLE_STMT);

    ret = SQLBindCol(hStmt, 2, SQL_C_CHAR, outName, sizeof(outName), NULL);
    CHECK_ERROR(ret, hStmt, SQL_HANDLE_STMT);

    // 获取查询结果并打印
    while ((ret = SQLFetch(hStmt)) != SQL_NO_DATA) {
        if (ret == SQL_SUCCESS || ret == SQL_SUCCESS_WITH_INFO) {
            printf("ID: %d, Name: %s\n", outId, outName);
        } else {
            CHECK_ERROR(ret, hStmt, SQL_HANDLE_STMT);
        }
    }

    // 6. 清理
    SQLFreeHandle(SQL_HANDLE_STMT, hStmt);
```

```
    SQLDisconnect(hDbc);
    SQLFreeHandle(SQL_HANDLE_DBC, hDbc);
    SQLFreeHandle(SQL_HANDLE_ENV, hEnv);

    return 0;
}
```

Odbc 更多示例请参考 Odbc 产品手册的程序示例。

2.3.3.2.3　编译

使用 gcc 编译，需要显示指定 unixOdbc 库文件安装路径，以下是示例使用的 Makefile:

```
all:prog

LDFLAGS =  -L unixOdbc/lib -lodbc
CFLAGS =  -g -Wall

CPPFLAGS = -I unixOdbc/include

ODBC_CONFIG = unixOdbc/bin/odbc_config
LIBODBC := $(shell $(ODBC_CONFIG) --libs)

prog: test_odbc.c
        gcc test_odbc.c $(CPPFLAGS) $(CFLAGS) $(LDFLAGS) -o testOdbc $(LIBODBC)

clean:
        -rm testOdbc
```

备注:

Makefile 中涉及的 unixOdbc 路径需要自行修改为实际安装的 unixOdbc 路径。

编写好 Makefile 后，使用 `make` 命令编译程序：

```
$ make
```

会在当前路径下生成 testOdbc 的可执行程序。

2.3.3.2.4　运行

运行程序，程序期望输出如下：

```
$ ./testOdbc
Connected to the database.
Table created successfully.
Data inserted successfully.
ID: 1, Name: KKKingbase
```

2.3.3.3　操作实例 - Windows

Windows 默认支持 Odbc，需要配置 Odbc 数据源，配置好 Odbc 数据源后并测试能连通数据库即可，用例编写可参考 Linux 用例，本文不过多阐述，只描述如何在 Windows 下配置 Odbc 数据源并加载 Odbc 驱动。

2.3.3.3.1　安装并部署驱动

将下载后的 Windows 的 Odbc 驱动解压并放置在指定目录中，然后需要配置注册表，可手动生成.reg 文件，双击执行.reg 文件将 Kingbase 的 Odbc 添加到注册表中，示例所用的 64 位注册表文件 v9_odbc_x64.reg 如下所示：

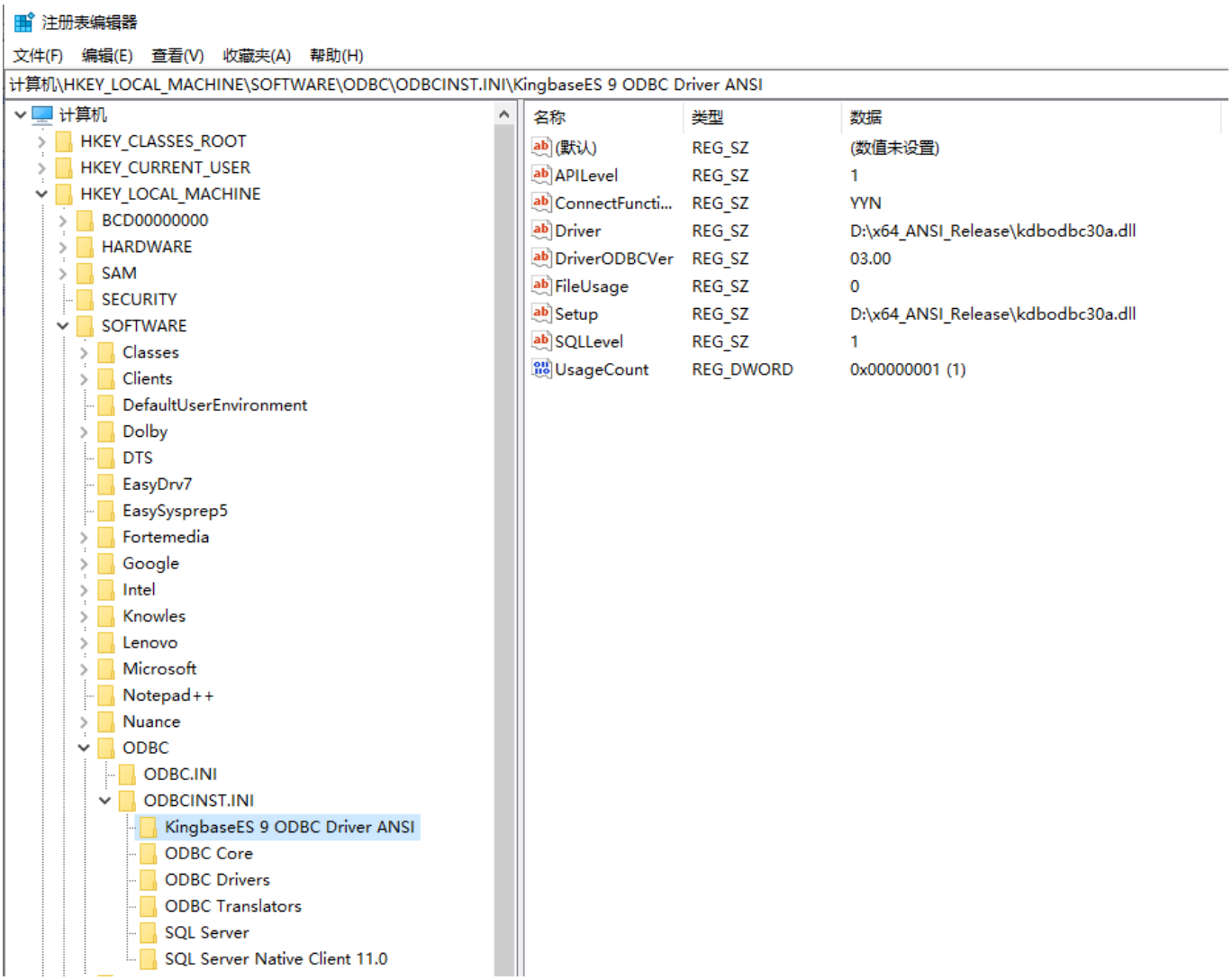
```
Windows Registry Editor Version 5.00

[HKEY_LOCAL_MACHINE\SOFTWARE\ODBC\ODBCINST.INI\ODBC Drivers]
"KingbaseES 9 ODBC Driver ANSI"="Installed"

[HKEY_LOCAL_MACHINE\SOFTWARE\ODBC\ODBCINST.INI\KingbaseES 9 ODBC Driver ANSI]
"APILevel"="1"
"ConnectFunctions"="YYN"
"Driver"="kdbodbc30a.dll"
"DriverODBCVer"="03.00"
"FileUsage"="0"
"Setup"="kdbodbc30a.dll"
"SQLLevel"="1"
"UsageCount"=dword:00000001
```

备注:

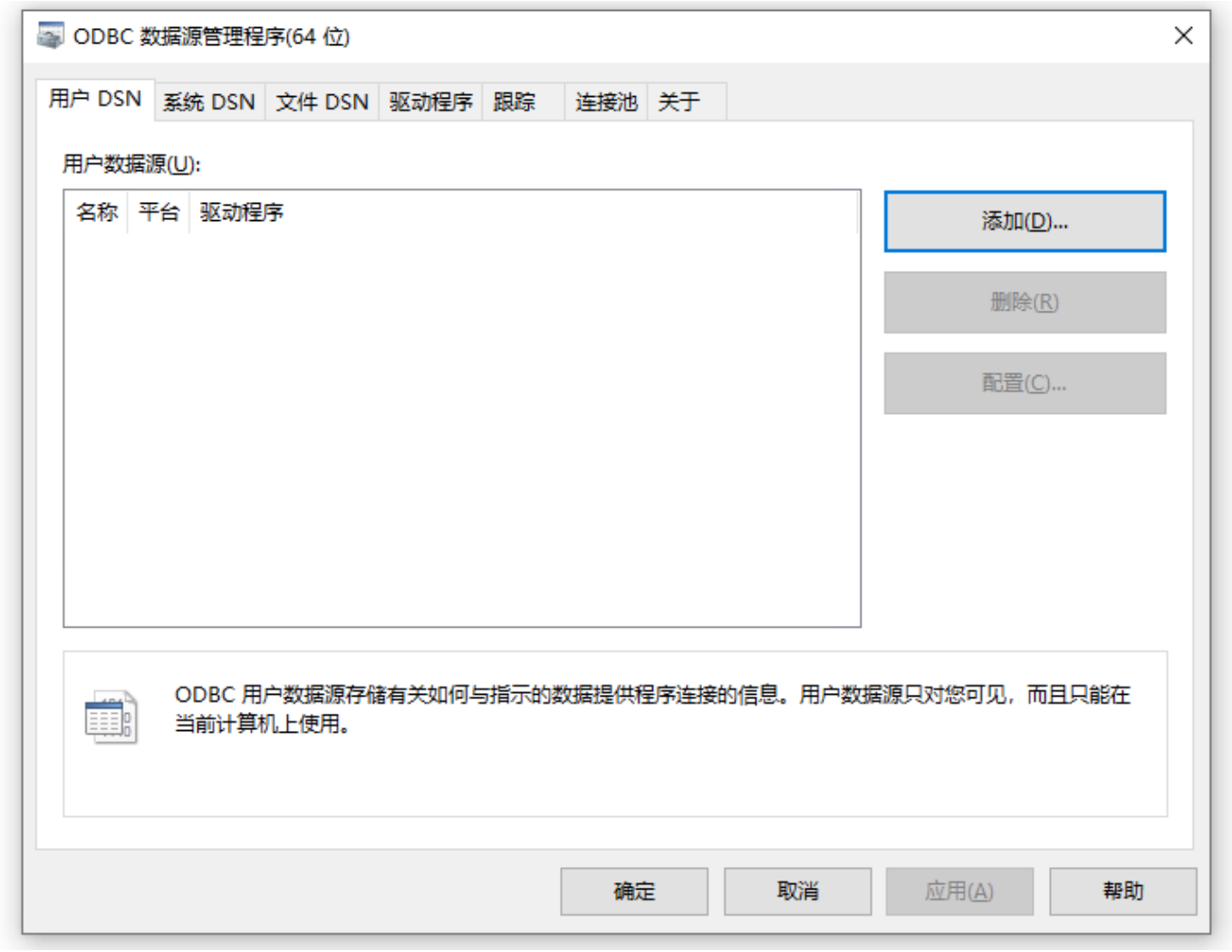
注册表文件中，“Driver”和“Setup”为 Odbc 驱动所在路径，可在此添加 Odbc 驱动全路径以便 Odbc 数据源管理器查找并加载 Odbc 驱动：

```
"Driver"="D:\\x64_ANSI_Release\\kdbodbc30a.dll"
"Setup"="D:\\x64_ANSI_Release\\kdbodbc30a.dll"
```

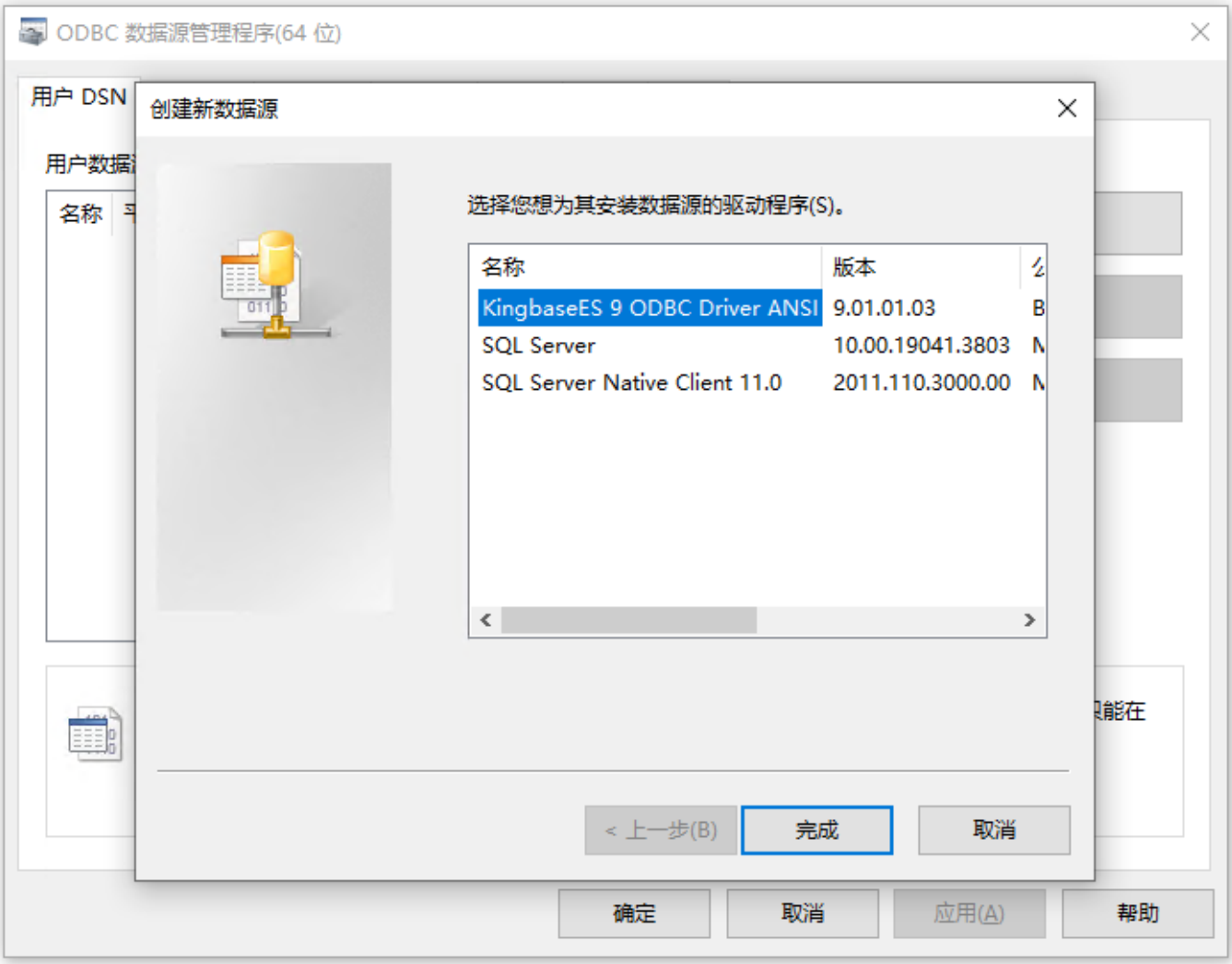
执行注册表文件后，可在注册表对应路径下查看到 Odbc 相关信息：



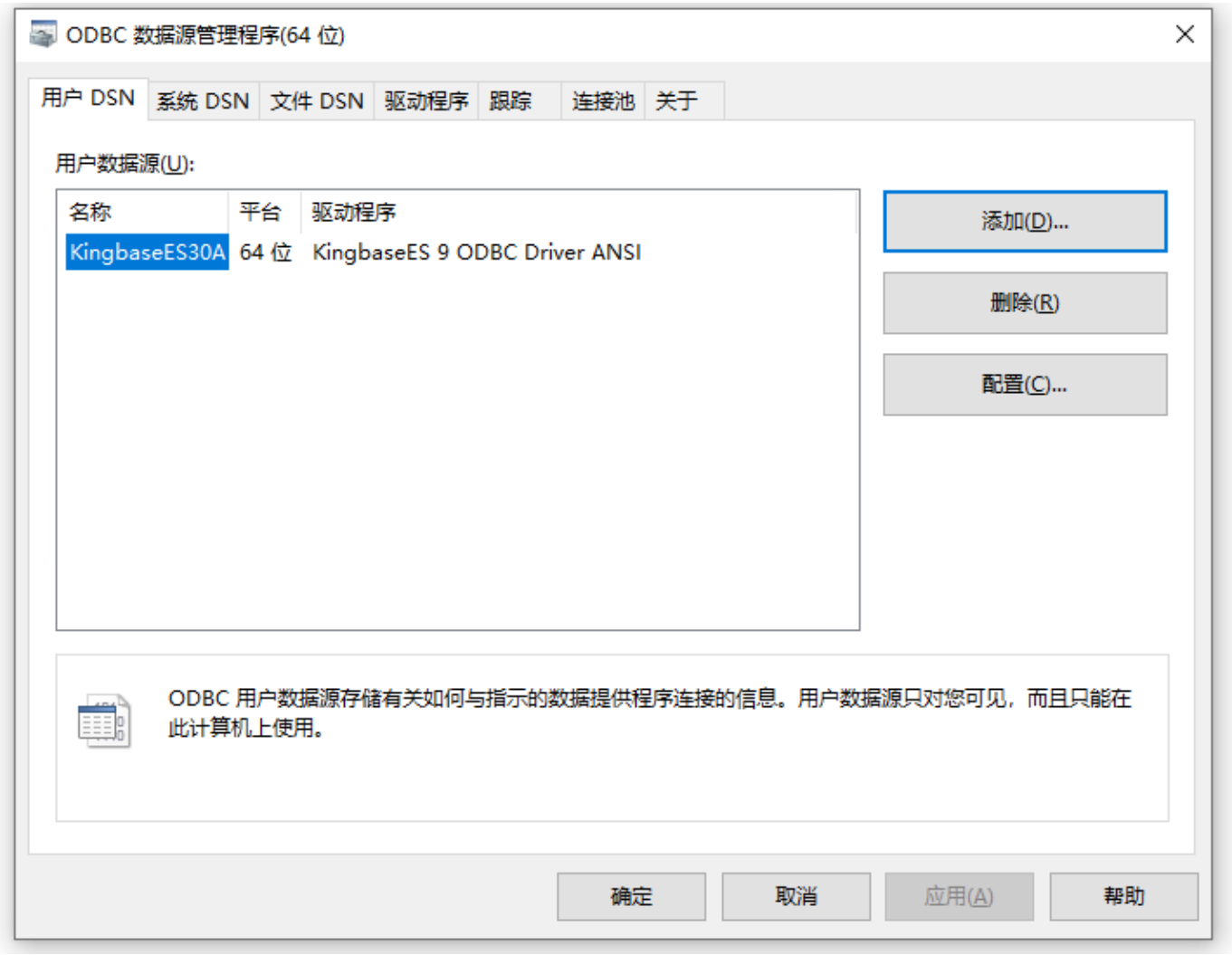
打开 Windows 自带的 Odbc 数据源管理器，点击**添加**按钮，添加 DSN：



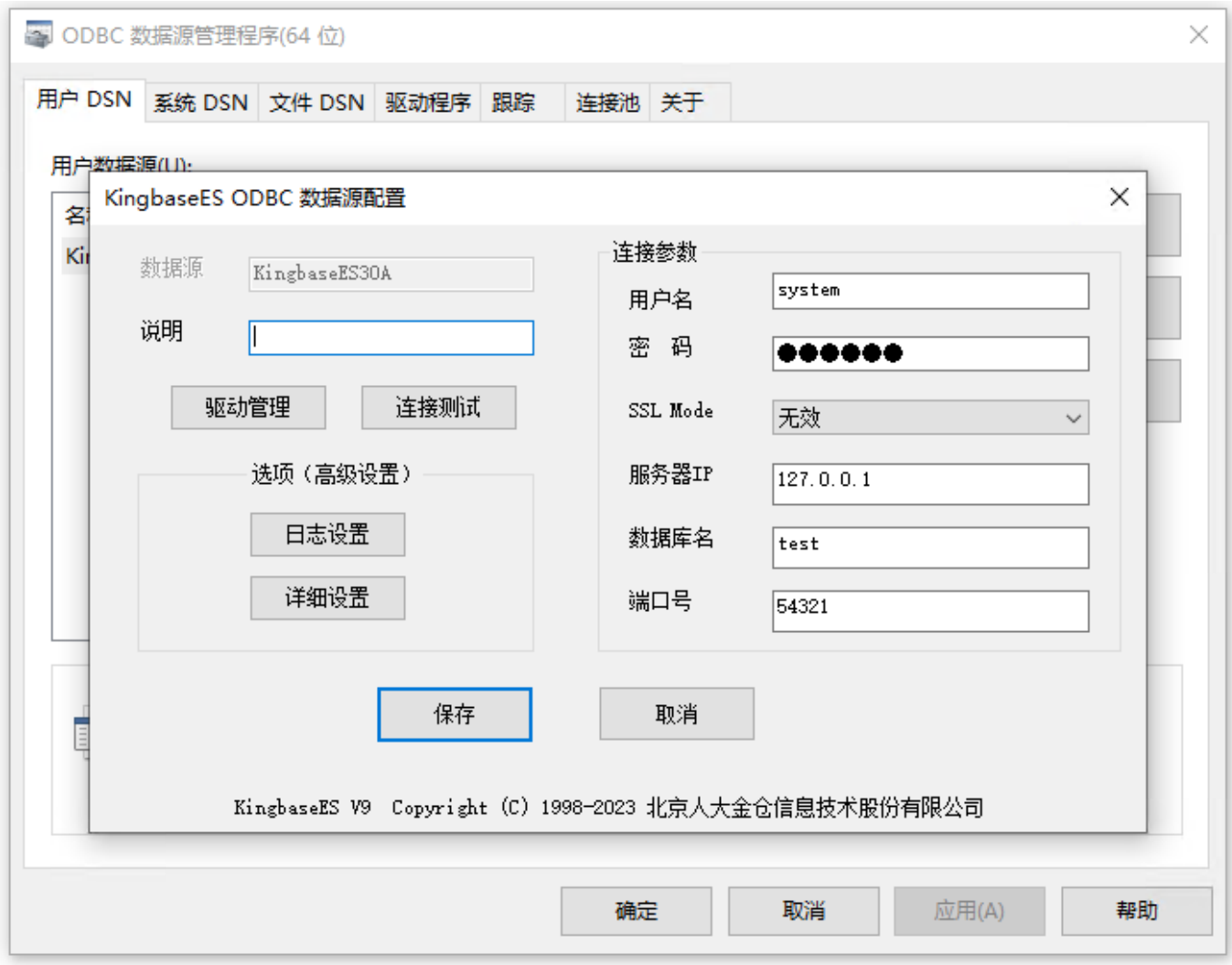
选择刚刚通过注册表添加的 KingbaseES 9 ODBC Driver ANSI ，点击**完成**：



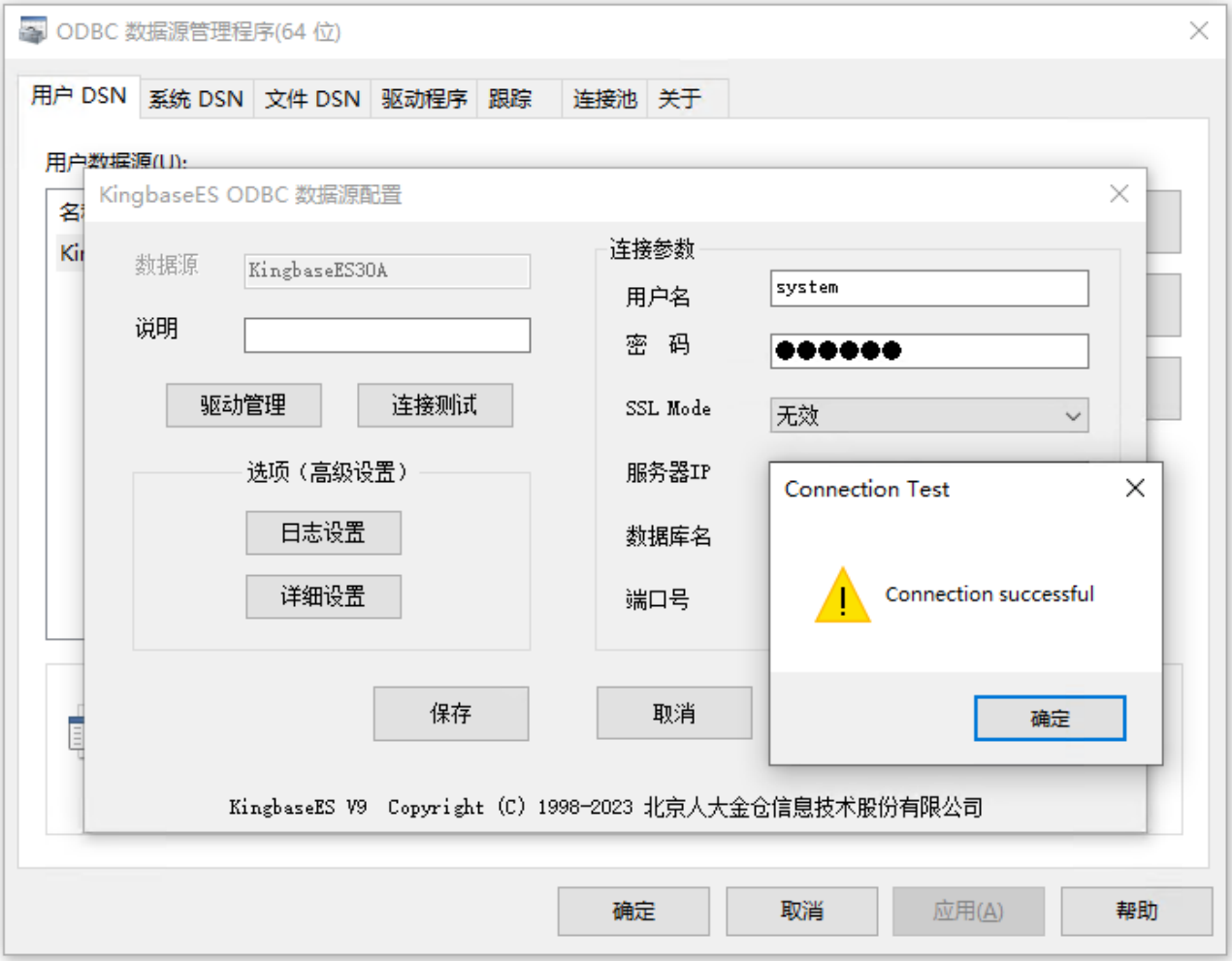
添加 DSN 成功后，对应列表会显示刚刚添加的 DSN：



配置数据源名 (应用加载 Odbc 所用的 DSN，示例所用数据源名为 KingbaseES30A，则用例调用 Odbc 时，对应 DSN 名也为 KingbaseES30A)，及数据库连接参数信息：



配置成功后，点击**连接测试**按钮，查看当前能否加载 Odbc 驱动并连接到数据库，最后点击**保存**按钮，保存配



置：

备注:

Windows 提供的 Odbc 数据源管理器是区分 32 位和 64 位 Odbc 驱动，用户需要注意数据源管理器对应位数与 Odbc 驱动位数一致，以下是不同位数的 Odbc 数据源管理器对应的启动路径：

ODBC 数据源	所在路径
64 位	C:WindowsSystem32odbcad32.exe
32 位	C:WindowsSysWOW64odbcad32.exe

2.3.3.4 常见问题

1. 驱动包 SSL 库与系统环境的 SSL 库冲突

   原因：系统环境的依赖库版本过高，或应用程序运行时错误加载了系统环境的 SSL 库。

   解决：Linux 下可通过 ldd 应用程序名，如 `ldd testOdbc` 查看当前环境下的依赖关系，确保运行应用程序时加载驱动包提供的 SSL 库，若仍然有 SSL 相关报错，则确定是驱动包提供的 SSL 库无法在当前环境下使用，此时请联系技服获取 Odbc 静态依赖 SSL 库的驱动包来解决 SSL 依赖冲突问题。

2.3.4 Oledb

本文提供使用 OLEDB 连接 Kingbase 数据库的连接代码示例。

2.3.4.1 前提条件

- 确保已设置基本的应用开发环境。
- 确保已安装与驱动对应版本的数据库，且数据库连接可用。
- 确保 OLEDB 支持开发环境：
  - 系统架构：

    Windows：提供 32 位支持

满足以上条件后，可通过联系技服人员获取对应版本架构的 OLEDB 驱动。

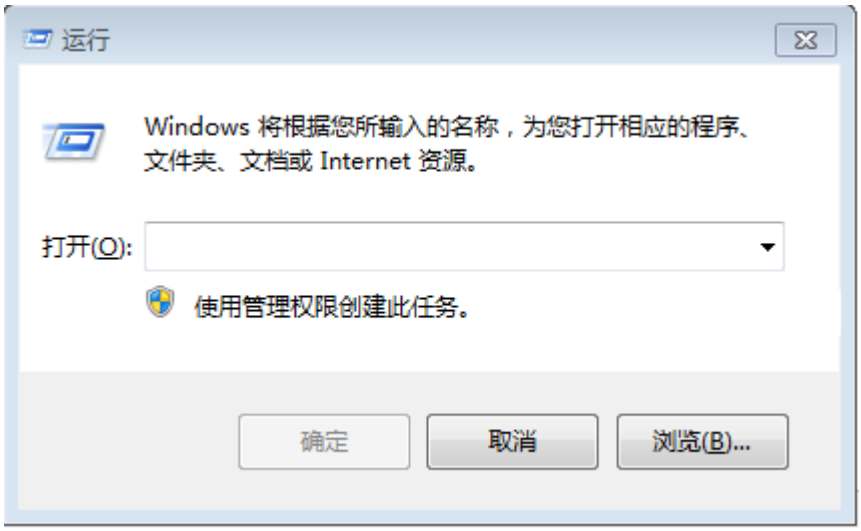
2.3.4.2 操作实例

下文以 VB.NET 环境使用 ADO(Windows 的标准件) 调用 OLEDB 连接数据库并执行简单的插入查询操作做示例说明，主要步骤如下：

1. 安装驱动程序

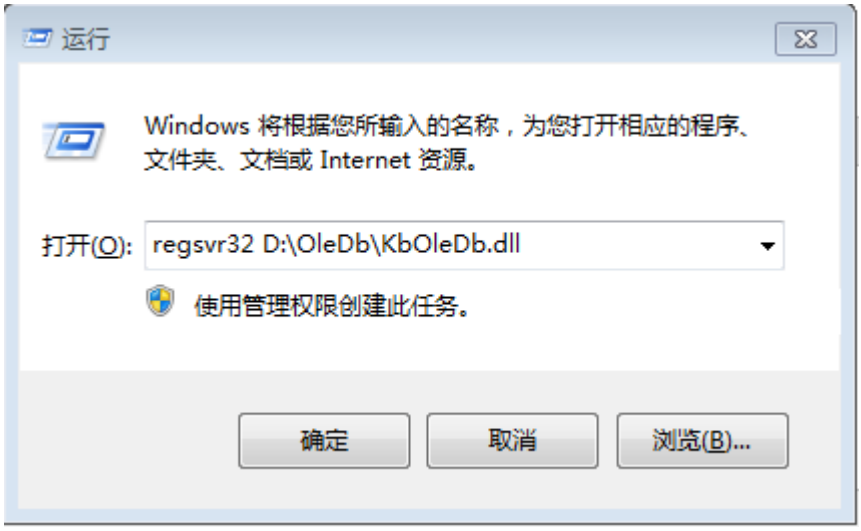
   KingbaseES OLEDB 驱动的安装是通过使用 Windows 操作系统附带的注册组件的命令程序“regsvr32”来完成，由于 KbOleDb.dll 为 32 位的驱动，因此必须是使用“regsvr32”。

   1. 使用组合键 `Win+R` ，调出 Windows 系统的“运行”对话框，如下所示。



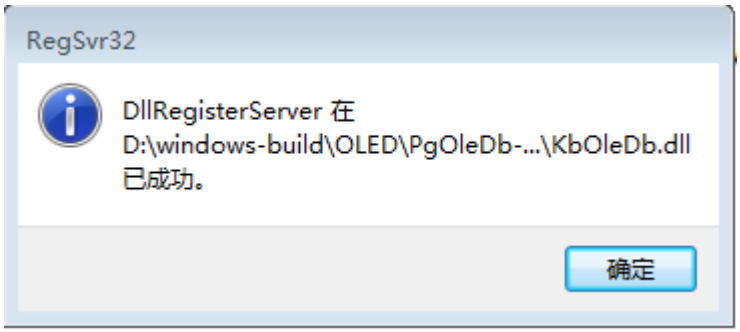
2. 在对话框中输入命令 `regsvr32 KbOleDb.dll` 。

   若没有提前将 KbOleDb.dll 与其依赖的库文件拷贝到“C: \Windows\System32”，则需要补充 KbOleDb.dll 驱动文件所在的路径，如下图所示。



3. 在将上述命令填写完整后，点击“确定”按钮进行注册。

   如果出现如下界面便表示驱动注册成功。



2. 开发源程序

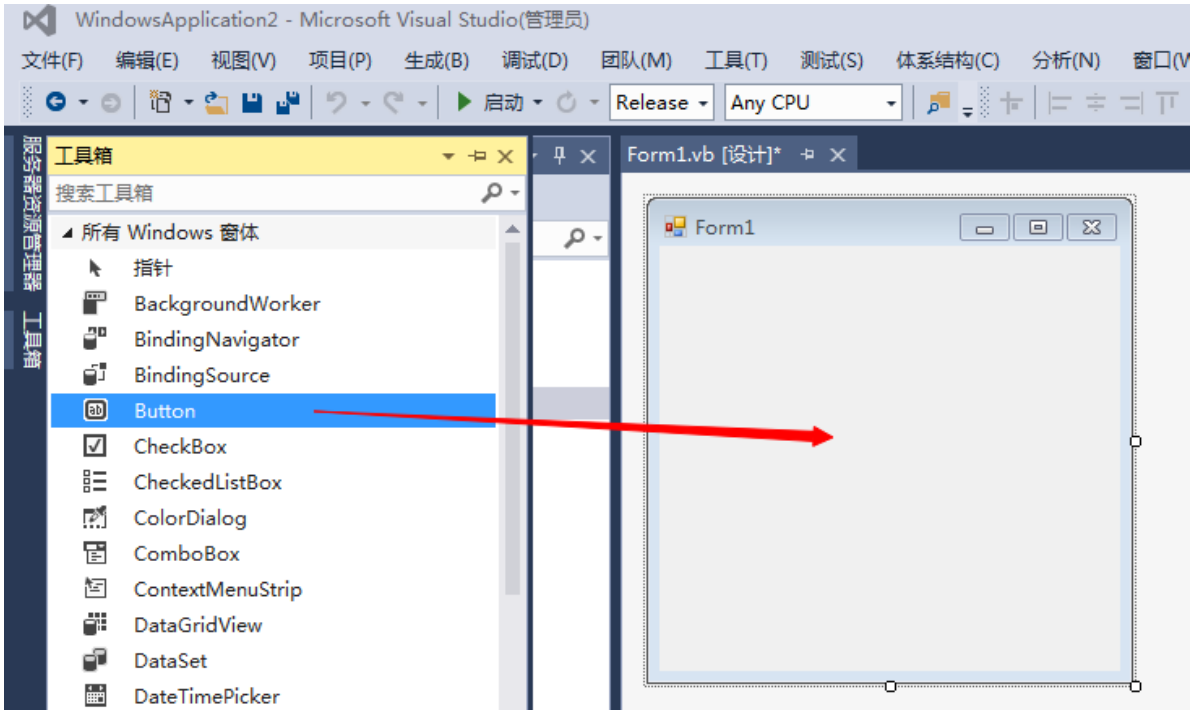
执行如下操作，使用 VisualStudio 2013 创建并开发 OLEDB 工程：

1. 创建工程

   1) 打开 Visual Studio 2013 后，点击“文件 > 新建 > 项目”，并选择如下的 VB-windows 桌面-windows 窗体应用程序。

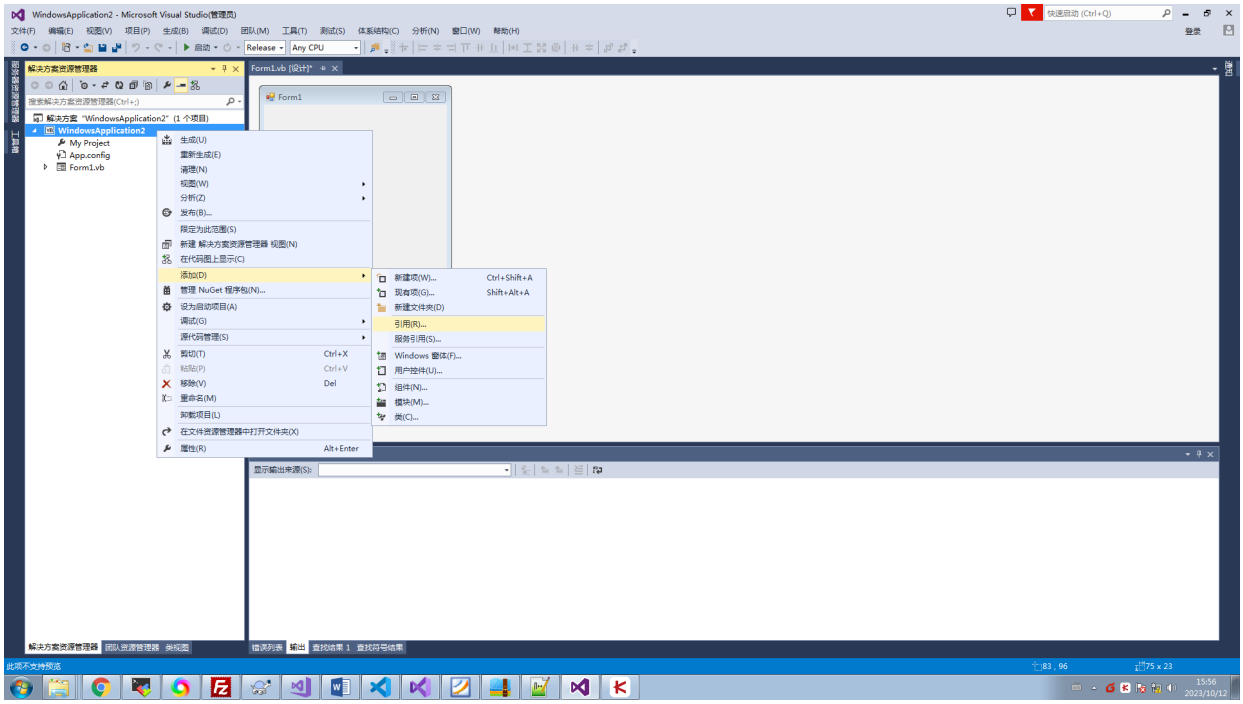
   2) 添加 button

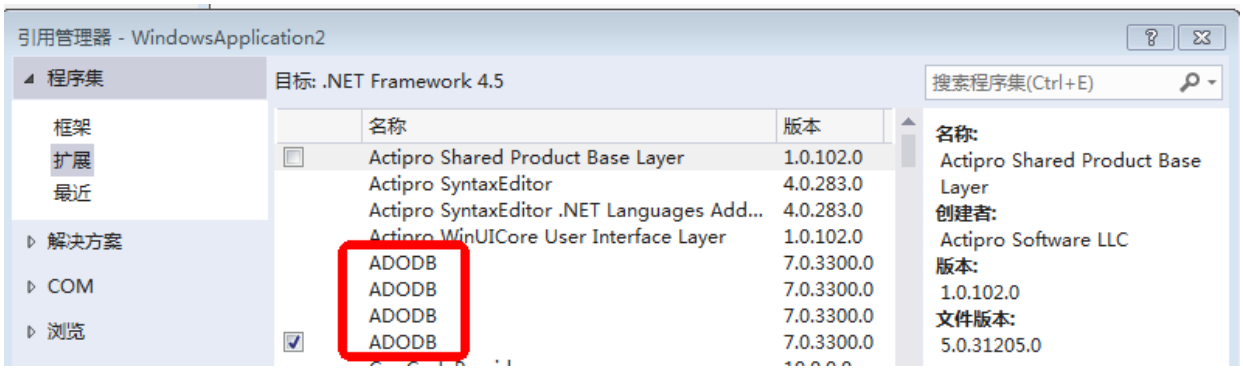
   点击左侧的“工具箱”，找到“Button”后拖拽添加到 Form 中，若没有工具箱选项可在“视图 > 工具箱”进行打开。



2. 添加引用

   右键单击项目名称，找到“添加 > 引用”后，即可打开“应用管理器”，如下图所示。





3. 应用编程

双击步骤 2 中添加的 Button 按钮打开 VB 编辑界面，将下述完整用例粘贴到 Button 响应事件的方法中。

```
'声明对象
Dim cn As ADODB.Connection
Dim rs As ADODB.Recordset
Dim cmd As ADODB.Command
Dim create As String
Dim insert1 As String
Dim insert2 As String
Dim insert3 As String
Dim insert4 As String
Dim delete As String
Dim update As String
Dim slct As String
Dim drop As String
```

```
  Dim result As String
  Dim exist As String

  '创建 sql 语句
  create = "create table test(id int, name varchar(20) not null, age int, marital boolean, salary
float);"
  insert1 = "insert into test values(1, 'Bryce', 25, false, 5000.1);"
  insert2 = "insert into test values(2, 'Judi', 23, false, 3500.5);"
  insert3 = "insert into test values(3, 'Mike', 28, true, 9000.5);"
  insert4 = "insert into test values(4, 'Peter', 28, true, 9000.5);"
  delete = "delete from test where id = 1;"
  update = "update test set salary = 4000 where id = 2;"
  slct = "select * from test;"
  drop = "drop table test;"


  '连接数据库
  MsgBox("********************  TEST OLEDB  ********************" & vbCrLf & _
  "Connect to database:" & vbCrLf & _
  "Provider=KingbaseES OLE DB Provider; Data Source=192.168.1.1,54321; User ID=system;
Password=123456; Initial Catalog=test;")

  cn = New ADODB.Connection

  cn.ConnectionString = "Provider=KingbaseES OLE DB Provider; Data Source=192.168.1.1,54321; User
ID=system; Password=123456; Initial Catalog=test;"
  cn.ConnectionTimeout = 50
  cn.CommandTimeout = 10
  cn.Open()

  '创建表并插入数据
  MsgBox("*****************************************************" & vbCrLf & _
  "Create table and make data:" & vbCrLf & _
  insert1 & vbCrLf & insert2 & vbCrLf & insert3)

  '通过开启事务的方式执行插入
  cn.BeginTrans()
  cn.Execute(create)
  cn.Execute(insert1)
  cn.Execute(insert2)
  cn.Execute(insert3)
  cn.Execute(insert4)
  cn.CommitTrans()

  '删除数据
```

```
  MsgBox("*****************************************************" & vbCrLf & _
  "Delete:" & vbCrLf & _
  delete)

  '开启事务
  cn.BeginTrans()
  cn.Execute(delete)

  '查询数据并显示
  result = "*****************************************************" & vbCrLf & _
  "Select:" & vbCrLf & _
  slct & vbCrLf & _
  "id" & vbTab & "name" & vbTab & "age" & vbTab & "marital" & vbTab & "salary" & vbCrLf & _
  "----------------------------------------" & vbCrLf
  rs = New ADODB.Recordset
  rs = cn.Execute(slct)
  While Not rs.EOF
    result = result & rs.Fields(0).Value & vbTab & rs.Fields(1).Value & vbTab & rs.Fields(2).Value
& vbTab & rs.Fields(3).Value & vbTab & rs.Fields(4).Value & vbCrLf
    rs.MoveNext()
  End While
  rs.Close()
  rs = Nothing
  MsgBox(result)

  '回滚事务
  cn.RollbackTrans()

  '修改数据
  MsgBox("*****************************************************" & vbCrLf & _
  "Update:" & vbCrLf & _
  update)
  cn.Execute(update)

  '查询回滚数据
  rs = New ADODB.Recordset
  rs.Open(slct, cn)

  '拼接提示信息并显示结果集
  result = "*****************************************************" & vbCrLf & _
  "Select:" & vbCrLf & _
  slct & vbCrLf & _
  "id" & vbTab & "name" & vbTab & "age" & vbTab & "marital" & vbTab & "salary" & vbCrLf & _
  "----------------------------------------" & vbCrLf
  While Not rs.EOF
```

```
    result = result & rs.Fields(0).Value & vbTab & rs.Fields(1).Value & vbTab & rs.Fields(2).Value
& vbTab & rs.Fields(3).Value & vbTab & rs.Fields(4).Value & vbCrLf
    rs.MoveNext()
  End While
  MsgBox(result)

  '删除表
  cn.Execute(drop)

  '断开数据库连接
  rs.Close()
  cn.Close()
  rs = Nothing
  cn = Nothing
  MsgBox("disconnct from database.")
```

替换该用例中的连接串的连接参数后，使用”F5” 按键即可运行该程序。

4. 期望结果

在应用启动后点击按钮后即可依次出现以下期望结果：

```
**************************************************
Create table and make data:
insert into test values(1, 'Bryce', 25, false, 5000.1);
insert into test values(2, 'Judi', 23, false, 3500.5);
insert into test values(3, 'Mike', 28, true, 9000.5);
```

确定

```
**************************************************
Delete:
delete from test where id = 1;
```

确定

```
**************************************************
Select:
select * from test;
id        name      age       marital   salary
----------------------------------------
2         Judi      23        False     3500.5
3         Mike      28        True      9000.5
4         Peter     28        True      9000.5
```

确定

```
**************************************************
Update:
update test set salary = 4000 where id = 2;
```

确定

```
**************************************************
Select:
select * from test;
id        name      age       marital   salary
----------------------------------------
1         Bryce     25        True      5000.1
3         Mike      28        True      9000.5
4         Peter     28        True      9000.5
2         Judi      23        False     4000
```

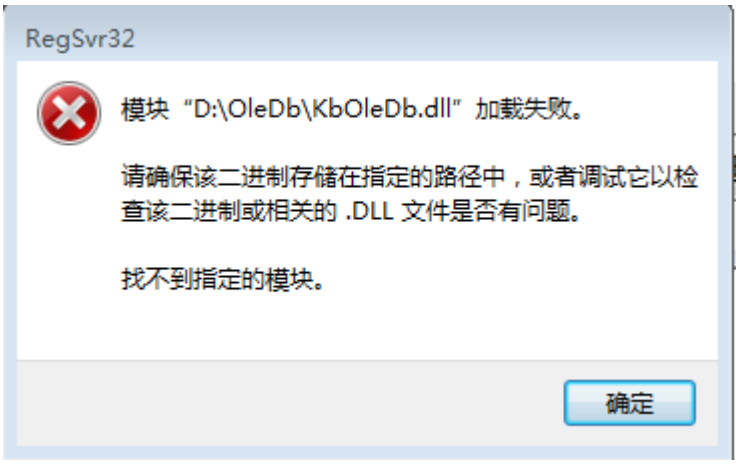
确定

2.3.4.3　常见问题

如果出现如下所示错误，可能有两种情况：

- KbOleDb.dll 驱动所在的路径有误，重新修改驱动所在路径后再进行注册即可。
- KbOleDb.dll 驱动所依赖的动态库文件与 KbOleDb.dll 驱动没有在同一路径下，将依赖的动态库复制到 KbOleDb.dll 驱动所在路径即可。

  若不知道 KbOleDb.dll 驱动依赖哪些动态库，可利用软件“depends”查看。



2.4　编程接口-Python

2.4.1　Ksycopg2

本文介绍如何通过 Python 驱动 Ksycopg2 连接和使用 Kingbase 数据库。

不同版本的 Python 环境需要使用不同版本的驱动。

2.4.1.1　前提条件

- 确保已设置基本的应用开发环境。
- 确保已安装与驱动对应版本的数据库，且数据库连接可用。
- 确保 Ksycopg2 支持开发环境：
  - 系统架构：

    Linux：x86_64、arm、loongarch、mips、sw

    Windows：提供 64 位支持
  - 编译器：

    Linux：仅支持依赖 glibc 的编译的 Python。

    Windows：依赖 msvc120 的运行时库。
  - Python 环境：

```
:::{note}

Ksycopg2 驱动需要和 Python 大版本一致，如 Python3.5 的 Ksycopg2 驱动支持 Python3.5.x 的任意小版本。
```

```
Ksycopg2 驱动区分系统架构，如 x86 架构只能使用驱动包名为 x86 的 Ksycopg2 驱动，否则无法加载。

其中，Python2.7 和 Python3.5 的 Ksycopg2 驱动包默认提供， Python2.7 版本默认提供 UCS2 版本。

其余版本按需提供，x86 和 Windows 64 位平台，可通过 `pip install ksycopg2` 尝试下载对应平台驱动。

:::
```

满足以上条件后，可通过已发布版本的数据库安装包、官网下载页的接口侧、或联系技服人员获取对应版本架构的 Ksycopg2 驱动。

2.4.1.2　操作实例

基于 Ksycopg2 编译开发应用程序，主要步骤如下：

安装部署 Python 驱动 -> 测试能否成功加载 Ksycopg2 驱动 -> 编写应用程序 -> 运行

下文将通过具体示例说明如何使用 Python 驱动连接操作数据库：

2.4.1.2.1　安装部署驱动

查看当前环境架构及 Python 版本，选用合适的 Python 驱动 Ksycopg2。

命令行执行 python -V，执行查看 Python 版本：

```
$ python -V
Python 3.5.6
```

当前环境使用的是 Python3.5，若对应架构是 x86_64，则应该选用 x86 架构的 Python3.5 版本的 Ksycopg2 驱动包安装。

注意:

Python 驱动 Ksycopg2 基于在不同系统架构不同 Python 版本间不通用，如当前开发环境为 Linux x86_64，需要使用 Python3.5.6 操作连接数据库，则用户需要安装 Linux x86_64 架构对应的 Python3.5 驱动包，否则会出现 Python 无法识别对应模块的报错。

Linux 环境的 Python 基本是 64 位的，Windows 环境的 Python 有 32 位和 64 位两种安装包，当前默认提供的 Python 驱动包都是 64 位，可以通过在 Python 命令行执行以下代码查看 Python 操作位数：

```
>>> import platform
>>> print(platform.architecture())
('64bit', 'ELF')
```

另外，Python2 是有 UCS2 和 UCS4 两个 Python 版本的，这是由于对 UNICODE 的不同编码方案导致的，主要区别在于它们可以表示的字符范围，在 Python3.3 及以上版本没有这个区分了。

故用户使用 Linux 自行编译安装 Python2.7 并使用对应 Python2.7 的 Ksycopg2 驱动包时，需要注意 UCS 版本区分，可通过在 Python 命令行执行以下代码查看 Python 是 UCS 哪个版本：

```
>>> import sys
>>> print sys.maxunicode
# 65535 -> ucs2, 1114111 -> ucs4
65535
```

将对应架构及 Python 版本的 Ksycopg2 驱动包解压，Linux 环境 Python3.5 驱动解压后的目录结构如下：

```
ksycopg2
   compat.py
   errorcodes.py
   errors.py
   extensions.py
   extras.py
   __init__.py
   _ipaddress.py
   _json.py
   _ksycopg.cpython-35m-x86_64-linux-gnu.so
   libcrypto.so.1.1
   libkci.so.5
   libssl.so.1.1
   _lru_cache.py
   pool.py
   _range.py
   sql.py
   tz.py
```

其中，_ksycopg 开头的这个库就是 Python 驱动，后缀为.py 的文件是 Ksycopg2 驱动提供的可拓展的 Python 文件，包括了一些游标工厂、批量执行、连接池等功能。

将 ksycopg2 这个文件夹拷贝到对应 Python 安装目录下的 site-packages 路径下 (site-packages 路径是 Python 默认加载模块的搜索路径)，以下是模块路径示例：

操作系统	Python 安装路径	模块安装路径
Linux	/usr/bin/python (默认系统路径)	/usr/lib64/python3.5/site-packages/
	/home/python/py-3.5 (指定路径)	/home/python/py-3.5/lib/python3.5/site-packages/
Windows	D:Python35	D:Python35Libsite-packages

安装到指定路径后，Python 命令行执行 import ksycopg2 ，查看是否有报错提示。

在确认驱动包使用无误的前提，若导入模块仍然报错，错误提示如下：

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/home/python/py-3.5/lib/python3.5/site-packages/ksycopg2/__init__.py", line 51, in <module>
    from ksycopg2._ksycopg import (                    # noqa
ImportError: libkci.so.5: cannot open shared object file: No such file or directory
```

这是由于 Ksycopg2 驱动以动态库的形式提供，Ksycopg2 会依赖 Libkci 库，而 Python 加载 Ksycopg2 时，只识别到 Ksycopg2，并未识别到 Ksycopg2 驱动的依赖库导致的错误提示。

可通过在当前命令行环境配置环境变量 LD_LIBRARY_PATH 配置 Linux 系统默认库搜索路径解决 (Ksycopg2 驱动包内会提供依赖库，若无依赖库，也可配置任意依赖库路径)：

```
# Ksycopg2 驱动包内一般会提供依赖库
export LD_LIBRARY_PATH=/home/python/py-3.5/lib/python3.5/site-packages/ksycopg2:$LD_LIBRARY_PATH
```

而 Windows 环境一般会默认识别 site-packages/ksycopg2 是否存在对应依赖库，不需要额外配置环境变量，但是要求 Windows 环境有对应 MSVC120(VS2013) **运行时库**，否则无法加载对应驱动。

若无报错，表示 Ksycopg2 驱动成功导入，可正常使用，继续下面步骤创建测试用例。

备注:

当前已发布 x86_64 和 Windows 两个版本的 Ksycopg2 驱动，用户可通过 pip install ksycopg2 尝试下载当前环境是否有支持的 Ksycopg2 驱动包。

2.4.1.2.2　开发源程序

1. 创建文件夹用于存放测试示例

   创建测试程序文件夹 test_ksycopg2，新建源码文件 test_ksycopg2.py，目录示例如下：

   test_ksycopg2
   　test_ksycopg2.py

2. 开发源程序 test_ksycopg2.py，以下是一个具体的示例：

   导入 Ksycopg2：

   ```
   import ksycopg2
   ```

   使用 `ksycopg2.connect()` 创建连接，并连接到数据库：

```
conn = ksycopg2.connect("dbname=dbname user=user password=pwd host=host port=port")
例：
conn = ksycopg2.connect("dbname=test user=system password=123456 host=127.0.0.1 port=54321")
```

备注:

Ksycopg2 提供了键值对形式和 URI 形式的连接串形式连接数据库，URI 形式的连接串是 Libkci 支持的形式，通常使用键值对的连接方式，具体说明请参考 Ksycopg2 手册连接说明。

支持的连接拓展参数完全依赖 Libkci，Ksycopg2 不会对参数进行识别辨认，故 Libkci 支持的连接参数，Ksycopg2 驱动也支持，更多连接参数说明请参考 Libkci 手册的连接参数说明。

创建连接后，使用刚刚创建的连接新建一个 Cursor 对象，用于操作数据库：

```
cur = conn.cursor()
```

使用 `cur.execute()` 执行 SQL 语句：

```
# 创建新表
cur.execute('drop table if exists test_ksy')
cur.execute('create table test_ksy(id integer, name TEXT)')

# 插入数据
cur.execute("insert into test_ksy values(%s, %s)", (1, "John"))
cur.execute("insert into test_ksy values(%s, %s)", (2, '中文测试文字'))
cur.execute("insert into test_ksy values(%s, %s)", (3, '! @#￥%……'))
```

备注:

Ksycopg2 遵循 DB API 2.0 标准，完整实现了标准对应的基本 Python 接口，对应的标准接口用法也是一致的，如 `cursor.execute()` 就是 DB API 2.0 的一个标准接口。

插入成功后，使用 `cur.execute()` 执行查询，接着使用 `cur.fetchall()` 获取结果集并打印：

```
# 执行查询，并获取结果集
cur.execute("select * from test_ksy")
rows = cur.fetchall()

# 打印结果集
print("result for query:")
for row in rows:
    print("\t", row)
```

备注:

Ksycopg2 默认获取的结果集是以列表嵌套元组的形式存放结果集的，比如上述示例中的 rows 类型为列表，rows[0] 类型为元组，即查询的结果是一个列表，列表每一项为元组。

Ksycopg2 还提供了其他形式的游标类型，比如返回字典形式的结果集，可通过扩展模块 extras.py 中的 ksycopg2.extras.Dictcursor 游标类型，这样每一行返回的结果就是一个字典，其中建是列名，值是该列的实际值。

查询结束后，需要关闭游标及连接。

以下是完整示例，示例使用 Python3 语法：

```
# -*- coding: utf-8 -*-
import ksycopg2

database = "TEST"
user = "SYSTEM"
password = "123456"
host = "127.0.0.1"
port = "54321"

try:
    conn = ksycopg2.connect(
        "dbname={} user={} password={} host={} port={}".format(database, user, password, host, port))

    cur = conn.cursor()

    cur.execute('drop table if exists test_ksy')
    cur.execute('create table test_ksy(id integer, name TEXT)')

    cur.execute("insert into test_ksy values(%s, %s)", (1, "John"))
    cur.execute("insert into test_ksy values(%s, %s)", (2, '中文测试文字'))
    cur.execute("insert into test_ksy values(%s, %s)", (3, '! @#￥%……'))

    cur.execute("select * from test_ksy")
    rows = cur.fetchall()

    print(type(rows), type(rows[0]))
    print("result for query:")
    for row in rows:
        print("\t", row)

    cur.close()
    conn.close()
except Exception as e:
```

```
    print(e)
    exit()
```

Ksycopg2 更多示例请参考 Ksycopg2 手册的程序示例。

2.4.1.2.3　运行程序

编写好测试相关代码后，使用 Python 执行即可：

```
python test_ksycopg2.py
```

示例期望输出如下所示：

```
<class 'list'> <class 'tuple'>
result for query:
         (1, 'John')
         (2, '中文测试文字')
         (3, '! @#￥%……')
```

2.4.1.3　常见问题

1. 驱动包 SSL 库与系统环境的 SSL 库冲突

   原因：系统环境的依赖库版本过高，或应用程序运行时错误加载了系统环境的 SSL 库。

   解决：通过 `ldd _ksycopg` 开头的驱动，查看当前环境下的依赖关系，确保运行应用程序时加载驱动包提供的 SSL 库，若仍然有 SSL 相关报错，则确定是驱动包提供的 SSL 库无法在当前环境下使用，此时请联系技服获取 Ksycopg2 静态依赖 SSL 库的驱动包来解决 SSL 依赖冲突问题。

2. Ksycopg2 加载失败，报错 No module named ‘ksycopg2._ksycopg’

   原因：使用与当前环境架构不符或 Python 版本不一致的驱动包，也有可能是驱动包安装路径不对，Python 无法识别。

   解决：先确保使用对应架构及对应 Python 版本的驱动包，若确认驱动包无误，则可能是 Python 未成功识别到对应 Ksycopg2 模块，可通过在 Python 强制设置模块搜索路径解决：

```
>>> import sys
>>> sys.path.insert(0, "Ksycopg2 驱动包的父目录")
```

2.5　编程接口-Go

2.5.1　Gokb

本文介绍如何通过 Go 驱动 Gokb 连接和使用 Kingbase 数据库。

2.5.1.1　前提条件

- 确保已设置基本的应用开发环境。
- 确保已安装与驱动对应版本的数据库，且数据库连接可用。

满足以上条件后，可通过官网下载页的接口侧、或联系技服人员获取对应版本的 go 驱动源码。

2.5.1.2　操作实例

搭建 go 的开发环境，并基于 go 编译开发应用程序，下文以 Linux 环境使用 Gokb 连接数据库并执行简单的插入查询操作做示例说明，主要步骤如下：

2.5.1.2.1　下载和安装 Go

1. 访问 Go 官网

访问Go 官方页面下载对应操作系统的安装包

2. 安装 Go

根据操作系统进行安装：

操作系统	描述
Windows	下载.msi 安装包并运行，按照提示完成安装
macOS	下载.pkg 文件并运行安装程序完成安装
Linux	下载.tar.gz 文件，解压并配置

以下是 Linux 的示例：

```
tar -zxvf go1.x.x.linux-amd64.tar.gz

export PATH=/go 解压后的目录/go/bin:$PATH
```

可将 export 配置在 ~/.bashrc 等文件中

3. 验证安装

在终端运行命令

```
go version
```

输出 `go version go1.x.x [os/arch]` 即表示安装成功

2.5.1.2.2　使用 GOPATH 管理 Gokb

使用 GOPATH 管理 Gokb(GOPATH 或 GOMODULE 根据需要选择一种即可)

1. 查看或配置 GOPATH

   使用命令查看当前的 GOPATH 的路径:

   ```
   go env
   ```

   也可以通过命令设置其它的路径为 GOPATH

   ```
   export GOPATH=xxx
   或
   go env -w GOPATH=xxx
   ```

   使用命令 `go env` 查看 GO111MODULE，默认为空，表示当项目下存在 go.mod 文件时通过 GOMODULE 管理，否则通过 GOPATH 管理，不同 go 版本可能存在差异，可通过命令强制指定使用 GOPATH 管理：

   ```
   export GOPATH=off
   或
   go env -w GOPATH=off
   ```

2. 配置 Gokb

   解压 go 驱动源码，将解压后的 kingbase.com/gokb 完整放置在 GOPATH 的 src 路径下，没有 src 路径时可手动创建。

   通过 shell 命令下载 Gokb 的依赖：

   ```
   go get github.com/shopspring/decimal@latest

   go get github.com/golang-sql/civil@latest
   ```

   如果使用的是 go1.17 及更高版本，`go get` 不支持在非 GOMODULE 下添加依赖，可以使用 `go install` 替换 `go get` 命令。

   下载的依赖在 $GOPATH/pkg/mod$GOPATH/pkg/mod/github.com/shopspring/decimal@v1.4.0，

   将两个依赖包不保留版本号拷贝到 $GOPATH/src$GOPATH/src/github.com/shopspring/decimal 即可。

3. 导入 Gokb(此代码在示例文件中已包含)

   在用例文件中通过 _ 方式导入 Gokb 驱动即可通过 go 的官方包 database/sql 操作数据库

```
import (

    "database/sql"// 提供数据库连接操作等 api
    "kingbase.com/gokb"// 往往不直接导入数据库驱动包，除非要使用驱动包中的特有能力
    _ "kingbase.com/gokb"// 通过_ 方式导入会调用驱动包的初始化函数，完成注册驱动
)
```

4. 编译示例并执行

   修改示例文件中的连接串，使用命令编译名为 test.go 的用例文件，得到可执行文件 test 通过./test 执行即可

```
go build test.go
```

   或可通过以下命令直接执行，此命令包含编译并执行，但是不会生成可执行文件

```
go run test.go
```

   test.go 具体示例文件见步骤四。

2.5.1.2.3　使用 GOMODULE 管理 Gokb

使用 GOMODULE 管理 Gokb(GOPATH 或 GOMODULE 根据需要选择一种即可)

1. 查看或配置 GOMODULE

   在 go 1.11 引入 GO MODULE 管理包，使用以下命令查看 GO111MODULE，默认为空，表示当项目下存在 go.mod 文件时通过 GOMODULE 管理否则通过 GOPATH 管理

```
go env
```

   不同 go 版本可能存在差异，可通过命令以下命令强制指定使用 GOMODULE 管理

```
export GOPATH=on 或 go env -w GOPATH=on
```

   在工程目录下使用以下命令初始化一个名为 test 的项目

```
go mod init test
```

   得到一个 go.mod 文件，初始内容为

```
module test
go 1.18
```

2. 配置 Gokb

使用 GOMODULE 管理依赖包时可以在联网环境下自动拉取或删除项目所需的依赖包，Gokb 尚未发布到开源网站，所以需要手动配置位置

解压 go 驱动源码，将 kingbase.com/gokb 源码放置在需要的任意位置即可

修改 go.mod，添加以下内容：

```
module test
go 1.18

require kingbase.com/gokb v1.0.0
replace kingbase.com/gokb \=\> kingbase.com/gokb 所在的绝对路径或相对路径
```

执行以下命令自动拉取项目所需要的其它依赖：

```
go mod tidy
```

该命令也会同时拉取 Gokb 所需的依赖包并指定所需的版本，不需要再手动进行管理

3. 编译用例并执行

修改示例文件中的连接串，使用命令编译名为 test.go 的用例文件，得到可执行文件 test，通过 `./test` 执行即可。

```
go build test.go
```

或可通过以下命令直接执行，此命令包含编译并执行，但是不会生成可执行文件

```
go run test.go
```

test.go 具体示例文件见步骤四

2.5.1.2.4　示例文件

```go
package main

import (
    "database/sql"          // 提供数据库连接操作等 api
    "fmt"                   // 提供格式化输出的 api
    _ "kingbase.com/gokb"   // 通过_ 方式导入会调用驱动包的初始化函数，完成注册驱动
)

//修改此处连接信息为已开启服务的 KES 的信息
const (
    host     = "127.0.0.1"
    port     = 54321
    user     = "system"
```

```
    password = "123456"
    dbname   = "test"
)

//操作的表结构
type GoTable struct {
    Num  int    `db:"num"`
    Text string `db:"text"`
    Blob []byte `db:"blob"`
    Clob string `db:"clob"`
}

//建立连接
func testConn(connInfo string) (db *sql.DB, err error) {
    db, err = sql.Open("kingbase", connInfo)
    if err != nil {
        return db, err
    }

    err = db.Ping()
    if err != nil {
        return db, err
    }
    return db,err
}

//事务操作
func testTran(db *sql.DB, sql string) (err error) {
    txn, err := db.Begin()
    defer txn.Commit()
    if err != nil {
        return err
    }

    _, err = txn.Exec(sql)
    if err != nil {
        return err
    }
    return nil
}

func testInsert(db *sql.DB, sql string) (err error) {
    err = testTran(db, sql)
    return err
}
```

```
//通过预备语句执行
func testPrepare(db *sql.DB, sql string) (err error) {
    stmt, err := db.Prepare(sql)
    defer stmt.Close()
    if err != nil {
        return err
    }
    _, err = stmt.Exec(int64(100), "VARCHAR 中文示例文字", []byte{10, 20, 30, 40, 50}, "CLOB 中文示例文字")
    if err != nil {
        return err
    }
    return nil
}

//查询操作
func testSelect(db *sql.DB, query string) (data []*GoTable, err error) {
    rows, err := db.Query(query)
    defer rows.Close()
    if err != nil {
        return nil, err
    }

    for rows.Next() {
        var row GoTable
        err = rows.Scan(&row.Num, &row.Text, &row.Blob, &row.Clob)
        if err != nil {
            return nil, err
        }

        data = append(data, &row)
    }
    return data,nil
}

func main() {
    connInfo := fmt.Sprintf("host=%s port=%d user=%s "+
        "password=%s dbname=%s sslmode=disable",
        host, port, user, password, dbname)

    db, err := testConn(connInfo)
    if err != nil {
        panic(err)
    }
    fmt.Println("Connection test success!")
```

```
    defer db.Close()

    err = testTran(db, "CREATE temp TABLE temp_golang_test (num INTEGER, text VARCHAR, blob BLOB, clob CLOB)
")
    if err != nil {
        panic(err)
    }
    fmt.Println("Transaction test success!")

    err = testInsert(db, "insert into temp_golang_test(num,text,blob,clob) values"+
        "(123456,'ABC!@#123','abcdef123456','123456!@#$%^');")
    if err != nil {
        panic(err)
    }
    fmt.Println("Insert data test success!")

    err = testPrepare(db, "insert into temp_golang_test values($1,$2,$3,$4)")
    if err != nil {
        panic(err)
    }
    fmt.Println("Prepare and execute test success!")

    data, err := testSelect(db, "select * from temp_golang_test;")
    if err != nil {
        panic(err)
    }
    fmt.Println(data[0],"\n",data[1])
    fmt.Println("Select test success!")
}
```

期望结果为

```
Connection test success!
Transaction test success!
Insert data test success!
Prepare and execute test success!
&{123456 ABC!@#123 [97 98 99 100 101 102 49 50 51 52 53 54] 123456!@#$%^}
&{100 VARCHAR 中文示例文字 [10 20 30 40 50] CLOB 中文示例文字}
Select test success!
```

2.6　编程接口-其他

2.6.1　Ado.net

本文介绍如何通过 ADO.NET 驱动 Kdbndp 连接和使用 Kingbase 数据库。

不同版本的.NET 环境需要使用不同版本的驱动。

2.6.1.1　环境准备

- 确保已设置基本的应用开发环境。
- 确保已安装与驱动对应版本的数据库，且数据库连接可用。
- 确保 ADO.NET 支持开发环境：
  - 系统架构：

    Linux：x86_64、arm、loongarch、mips、sw

    Windows：V9 版本提供 32 位和 64 位支持
  - 编译器：

    Visual Studio 安装对应的 dotnet 版本

    最新 Visual Studio 2022 支持.NET8.0

满足以上条件后，可通过Nuget 官网、已发布版本的数据库安装包、官网下载页的接口侧、或联系技服人员获取对应版本架构的 ADO.NET 驱动。

2.6.1.2　支持的.NET 版本说明

框架	nuget 包	支持模式
.net framework 2.0		pg、oracle、mysql、sqlserver
.net framework 3.5	Kdbndp_V9 3.5	pg、oracle、mysql、sqlserver
.net framework 4.0	Kdbndp_V9 4.0	pg、oracle、mysql、sqlserver
.net framework 4.5-4.8 .netstandard2.0-2.1	Kdbndp_V9 4.5	pg、oracle、mysql、sqlserver
.net framework 5.0 .netstandard2.0-2.1 .netcore3.0-3.1	Kdbndp_V9 5.0	pg、oracle、mysql、sqlserver

-续表-

框架	nuget 包	支持模式
.net framework 5.0-6.0 .netstandard2.0-2.1 .netcore3.0-3.1	Kdbndp_V9 6.0	pg、oracle、mysql、sqlserver
.net framework 5.0-7.0 .netstandard2.0-2.1 .netcore3.0-3.1	Kdbndp_V9 7.0	pg、oracle、mysql、sqlserver
.net framework 6.0-8.0 .netstandard2.0-2.1 .netcore3.0-3.1	Kdbndp_V9 8.0	pg、oracle、mysql、sqlserver
.netstandard2.0-2.1	Kdbndp_V9 2.0	pg、oracle、mysql、sqlserver
.netcore3.0-3.1	Kdbndp_V9 3.1	pg、oracle、mysql、sqlserver

2.6.1.3　支持的 ORM 框架说明

支持的 ORM 框架	支持程度	支持的数据库模式
Entity Framework 6	金仓发布方言包	pg、oracle、mysql、sqlserver
Entity Framework core	金仓发布方言包	pg、oracle、mysql、sqlserver
SqlSugar	金仓适配-深度合作	pg、oracle、mysql、sqlserver
Nhibernate	金仓发布方言包	pg、oracle、mysql、sqlserver
chole	金仓适配	pg、oracle、mysql、sqlserver
dapper	金仓适配	pg、oracle、mysql、sqlserver
freesql	金仓适配	pg、oracle、mysql、sqlserver

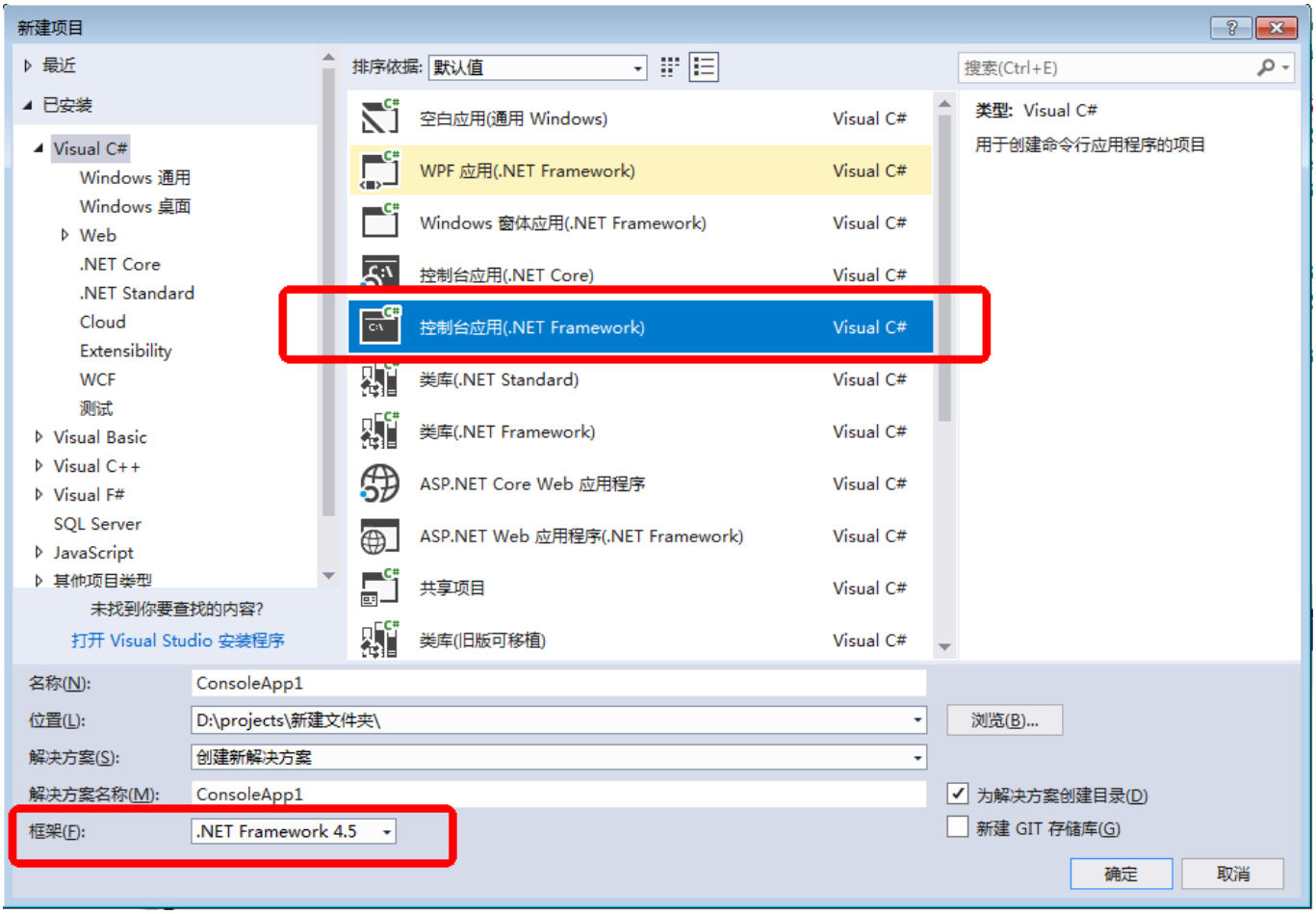
2.6.1.4　操作实例

以下以.NET4.5 为例，创建例程和连接数据库。

2.6.1.4.1　新建项目

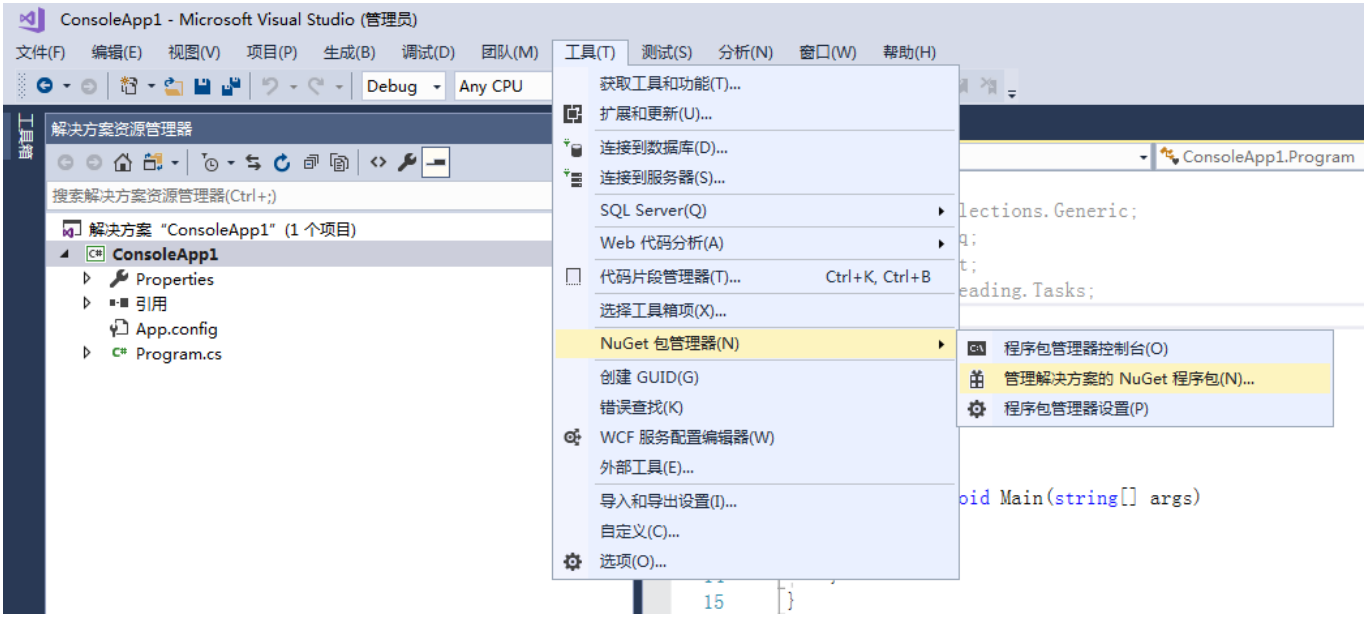
建议使用 Visual Studio 2017 以上版本新建项目。

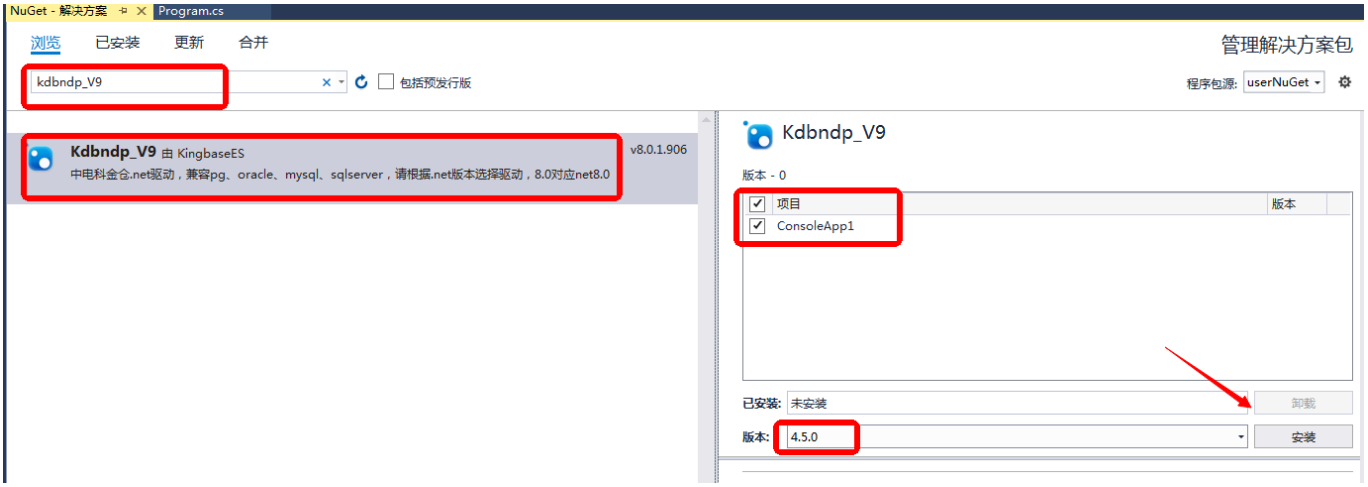
打开 VS，左上角文件-> 新建-> 项目



2.6.1.4.2　添加驱动引用

在项目中添加 NuGet 包，打开工具->Nuget 包管理器-> 管理解决方案的 NuGet 程序包，然后搜索 Kdbndp_V9，选择 4.5 版本进行安装。





2.6.1.4.3　程序编写

在生成项目中双击 Program.cs 文件，在顶部添加驱动的程序引用

```
using Kdbndp;
using KdbndpTypes;
```

在 class Program 中添加连接信息

```
namespace ConsoleApp1
{
    class Program
    {
        // 连接参数
        const string ConnectionString = "Server=127.0.0.1;User ID=system;Password=123456;Database=test;
```

```
Port=54321;Timeout=300;";
        static KdbndpConnection OpenConnection(string connectionString = null)
        {
            if (connectionString == null)
                connectionString = ConnectionString;
            var conn = new KdbndpConnection(connectionString);
            try
            {
                // 此处打开连接
                conn.Open();
            }
            catch (KingbaseException e)
            {
                throw e;
            }
            return conn;
        }
        static void Main(string[] args)
        {
        }
    }
}
```

在 class Program 中添加测试程序，进行表的创建和数据插入

```
        static void int_pbe()
        {
            using (var conn = OpenConnection())
            {
                Console.WriteLine("------插入测试------");
                int a = 123456;
                int b = 1;
                // 删除表
                string sql_drop = "drop table if exists test_int;";
                KdbndpCommand cmd1 = new KdbndpCommand(sql_drop, conn);
                Console.WriteLine(cmd1.ExecuteNonQuery());
                // 创建表
                string sql = "create table if not exists test_int(t1 int primary key);";
                KdbndpCommand cmd2 = new KdbndpCommand(sql, conn);
                Console.WriteLine(cmd2.ExecuteNonQuery());
                // 直接 SQL 插入数据
                string sql_insert = "INSERT INTO test_int VALUES (1);";
                KdbndpCommand cmd3 = new KdbndpCommand(sql_insert, conn);
                Console.WriteLine(cmd3.ExecuteNonQuery());
                // 绑定参数插入数据
```

```
            using (var cmd = new KdbndpCommand("INSERT INTO test_int VALUES (@p1)", conn))
            {
                cmd.Parameters.AddWithValue("p1", KdbndpDbType.Integer, a);
                cmd.Prepare();
                cmd.ExecuteNonQuery();
            }
        }
    }
```

添加 DataReader 查询例程

```
    static void select_test()
    {
        using (var conn = OpenConnection())
        {
            Console.WriteLine("------DataReader 查询测试------");
            // 用 DataReader 读取数据
            string sql1 = "select * from test_int";
            KdbndpCommand cmd = new KdbndpCommand(sql1, conn);
            KdbndpDataReader dr = cmd.ExecuteReader();
            if (dr.HasRows)
            {
                while (dr.Read())//检测是否有数据
                {
                    // 根据列号读取结果集，读取 int 类型可使用 GetInt32 和 GetValue 等方法
                    Console.WriteLine("GetInt32 :" + dr.GetInt32(0));
                    Console.WriteLine("GetValue :" + dr.GetValue(1));
                }
            }
        }
    }
```

添加 DataAdapter 的查询和更新例程

```
    static void Adapter_RowUpdating(object sender, KdbndpRowUpdatingEventArgs e)
    {
    }
    static void test_DataSet()
    {
        using (var conn = OpenConnection())
        {
            Console.WriteLine("------DataAdapter 查询和更新测试------");
            KdbndpDataAdapter adapter = KdbndpFactory.Instance.CreateDataAdapter() as KdbndpDataAdapter;
            adapter.SelectCommand = new KdbndpCommand("SELECT * FROM test_int;SELECT * FROM test_int;",
conn);// 可以加上 where 语句筛选数据
```

```
            // 创建 DataSet 用于存储数据
            DataSet dataSet = new DataSet("ds1");
            // 使用 Fill 方法将数据从数据库加载到数据集中
            adapter.Fill(dataSet);
            // 对数据集进行修改（例如修改某行数据）
            dataSet.Tables[0].Rows[0]["t1"] = 654321;
            // 将数据集中的更改提交到数据库
            KdbndpFactory.Instance.CreateCommandBuilder().DataAdapter = adapter;
            adapter.RowUpdating += Adapter_RowUpdating;
            // 这里会执行 INSERT、UPDATE 或 DELETE dataSet.Tables[0]
            adapter.Update(dataSet);
        }
    }
```

添加完以上例程之后，在 Main 中调用以上例程

```
    static void Main(string[] args)
    {
        int_pbe();
        select_test();
        test_DataSet();
        select_test();
    }
```

2.6.1.4.4　运行程序

F5 直接运行程序，运行结果如下所示。

2.6.1.5　常见问题

1、NuGet Kdbndp_V9 无法在当前框架安装：检查项目属性查看框架，选择对应的版本进行安装。

2、缺乏依赖：如对于.NET4.5，如果出现某些依赖项找不到，可以在 NuGet 升级最新的 Kdbndp_V9 或者手动在 NuGet 搜索缺少的依赖进行安装。

2.6.2　Pdo_kdb

本文介绍如何通过 PHP 驱动 Pdo_kdb 连接和使用 Kingbase 数据库。

不同版本的 PHP 环境需要使用不同版本的驱动。

2.6.2.1　前提条件

- 确保已设置基本的应用开发环境。
- 确保已安装与驱动对应版本的数据库，且数据库连接可用。
- 确保 Pdo_kdb 驱动包版本与 PHP 版本是否一致。

  Pdo_kdb 驱动对 PHP API 版本有严格要求，一般要求 Pdo_kdb 驱动与 PHP 版本完全一致，如 PHP8.0.28 版本最好使用 PHP8.0.28 编译出的 Pdo_kdb 驱动。

  PHP 有线程安全 (Thread Safe，简称 TS) 与非线程安全 (Non-Thread Safe，简称 NTS) 版本，Pdo_Kdb 驱动也需要与 PHP 版本是否线程安全相对应，可通过 `php -v` 查看当前环境的 PHP 版本是否线程安全，以下是 PHP8.0.28 非线程安全版本说明输出的示例：

```
$ php -v
PHP 7.2.0 (cli) (built: Aug 15 2024 16:59:37) ( NTS )
Copyright (c) 1997-2017 The PHP Group
Zend Engine v3.2.0, Copyright (c) 1998-2017 Zend Technologies
```

  以下是 Kingbase 默认提供的 Pdo_kdb 驱动包版本：

PHP 版本	是否线程安全
5.6.22	非线程安全 (NTS)
7.2.0	非线程安全 (NTS)

也可通过 Pdo_kdb 产品手册查看具体版本支持信息，若没有需要的 Pdo_kdb 驱动包，可联系技服人员申请获取对应 PHP 版本的 Pdo_kdb 驱动包。

2.6.2.2　操作实例

基于 Ksycopg2 编译开发应用程序，主要步骤如下：

安装部署 PHP 驱动 Pdo_kdb -> 测试能否成功加载 Pdo_kdb 驱动 -> 编写应用程序 -> 运行

下文将通过具体示例说明如何使用 PHP 驱动连接操作数据库：

2.6.2.2.1　下载合适 PHP 驱动

查看当前环境架构及 PHP 版本，选用合适的 PHP 驱动 Pdo_kdb 。

命令行执行 `php -v`，执行查看 PHP 版本：

```
$ php -v
PHP 7.2.0 (cli) (built: Aug 15 2024 16:59:37) ( NTS )
Copyright (c) 1997-2017 The PHP Group
Zend Engine v3.2.0, Copyright (c) 1998-2017 Zend Technologies
```

当前环境使用的是 x86_64 PHP7.2.0 非线程安全版本，需要使用对应 x86_64 架构的 PHP7.2.0 非线程安全的 Pdo_kdb 驱动包。

参考 Pdo_kdb 产品手册的更多版本的下载链接，可在浏览器输入以下 URL 下载对应版本的驱动：

```
https://kingbase.oss-cn-beijing.aliyuncs.com/KES_INTERFACE/history/PHP/all/r6/PDO/x86_64/uzts/v9_pdo_kdb_
for_php-7.2.0_x86_64_uzts.tar.gz
```

2.6.2.2.2　安装部署 PHP 驱动

一般可通过单独编译安装指定版本 PHP，也可通过集成环境安装 PHP，而 WEB 应用实际多是用的 PHP-FPM，故实际生产环境的 PHP 环境无法一言以蔽之，以下以命令行 PHP 环境及 PHP-FPM 环境作简要的 Pdo_kdb 驱动的安装部署说明。

- 命令行 PHP
  - Linux：

    命令行 PHP 环境的加载对应驱动，将 Pdo_kdb 放置在 PHP 安装路径的 `lib/php/extensions/no-debug-non-zts-20170718` 路径下，再通过 `php.ini` 配置 Pdo_kdb，最后使用 `php -m` 命令查看驱动是否成功加载，示例具体步骤如下：

    1. 解压安装

       若本地 PHP7.2.0 安装在 `/home/kingbase/php/php-7.2.0` 路径下，则 PHP 默认驱动的安装路径为 `/home/kingbase/php/php-7.2.0/lib/php/extensions/no-debug-non-zts-20170718`，否则默认为 `/usr/local/lib/php/extensions/no-debug-non-zts-20170718` 路径下，如下表所示：

安装方式	PHP 可执行文件路径	驱动安装路径
指定路径	/ home/ kingbase/ php/ php-7.2.0/bin/php	/home/kingbase/php/php-7.2.0/lib/php/extensions/no-debug-non-zts-20170718
默认路径	/usr/local/bin/php	/ usr/ local/ lib/ php/ extensions/ no-debug-non-zts-20170718

将 Pdo_kdb 驱动包解压后得到的库全部拷贝到上述驱动安装路径下即可。

备注:

PHP 拓展模块默认安装路径，可在命令行执行 `php -i | grep extension_dir` 查看对应模块拓展路径。

---

2. 配置 Pdo_kdb

   执行 `php --ini` 查看 PHP 配置文件路径：

```
$ php --ini
Configuration File (php.ini) Path: /home/kingbase/php/php-7.2.0/lib
Loaded Configuration File:         /home/kingbase/php/php-7.2.0/lib/php.ini
Scan for additional .ini files in: (none)
Additional .ini files parsed:      (none)
```

   修改 `/home/kingbase/php/php-7.2.0/lib/php.ini` 的 php.ini 文件，在 `extension=pdo;` 下方添加一行 `extension=pdo_kdb;`，让 PHP 能加载 Pdo_kdb 驱动。

   extension 也可配置全路径的形式，诸如：`extension=/home/kingbase/pdo_kdb.so;` 去加载指定路径的 Pdo_kdb 驱动。

3. 查看加载情况

   修改配置文件后，执行 `php -m` 查看 Pdo_kdb 是否加载成功，以下是加载成功的正常输出：

```
$ php -m
[PHP Modules]
...
PDO
pdo_kdb
...
```

   此时执行 `php -m` 出现报错，请参考常见问题说明。

– Windows

Windows 环境大多数 PHP 环境是直接使用的 PHP 安装包的形式，PHP 驱动包的安装路径相对固定，以下是一个 PHP 安装包的安装路径示例：

安装方式	PHP 可执行文件路径	驱动安装路径
安装包	D:\PHP_install\php-7.2.0	D:\PHP_install\php-7.2.0\ext

Pdo_kdb 驱动安装的具体步骤如下：

1. 解压安装

   将 Pdo_kdb 驱动包解压后得到的库 php_pdo_kdb.dll 拷贝到上述驱动安装路径下即可。

   需要注意的是 Windows 加载 DLL 的顺序，需要将 Pdo_kdb 驱动的依赖库 libkci.dll 及其依赖库 (即驱动包内非 Pdo_kdb 的库) 拷贝到 PHP 可执行文件路径下，对应上述表格的 `D:\PHP_install\php-7.2.0` 路径，否则 Windows 无法成功识别对应库导致 PHP 加载 Pdo_kdb 失败。

   也可在 Windows 加载 DLL 的任一默认路径放置 Pdo_kdb 驱动的依赖库，本文不对此作详细描述。

2. 配置 Pdo_kdb

   执行 `php --ini` 查看 PHP 配置文件路径：

   ```
   $ php --ini
   Configuration File (php.ini) Path: C:\Windows
   Loaded Configuration File:         C:\Windows\php.ini
   Scan for additional .ini files in: (none)
   Additional .ini files parsed:      (none)
   ```

   修改 `C:\Windows\php.ini` 的 php.ini 文件，在 `extension=pdo;` 下方添加一行 `extension=pdo_kdb;`，让 PHP 能加载 Pdo_kdb 驱动。

   extension 也可配置全路径的形式，诸如：`extension=D:\PHP_install\php_pdo_kdb.dll;` 去加载指定路径的 Pdo_kdb 驱动。

3. 查看加载情况

   修改配置文件后，执行 `php -m` 查看 Pdo_kdb 是否加载成功，以下是加载成功的正常输出：

   ```
   $ php -m
   [PHP Modules]
   ...
   PDO
   pdo_kdb
   ...
   ```

   此时执行 `php -m` 出现报错，请参考常见问题说明。

- PHP-FPM

PHP-FPM(PHP FastCGI Process Manager)，PHP FastCGI 进程管理器，用于管理 PHP 进程池的软件，用于接受 Web 服务器的请求。

PHP-FPM 主要用于 Web 应用，会有单独的进程处理 HTTP 请求。

从驱动安装部署的角度理解，用户开发环境的命令行执行 `php -m` 后发现安装了 Pdo_kdb 驱动，但并不一定在 Web 应用使用的 PHP 也安装了对应的 Pdo_kdb，因为命令行使用的 PHP 和 Web 应用使用的 PHP 可能不是同一个。

Web 应用部署的服务器多在 Linux 环境上，下文以 Linux 环境 PHP-FPM 安装部署 Pdo_kdb 作简要说明：

1. 解压安装 Pdo_kdb 驱动包

   若已知当前 Web 应用使用的 PHP 的驱动安装路径，则将 Pdo_kdb 驱动包解压放置在对应路径下。

   若不清楚，也通过当前命令行环境执行 PHP-FPM 进程，如 `php-fpm -m` 查看是否有输出，若有，可参考命令行执行 PHP 的步骤添加 Pdo_kdb 驱动。

   若不清楚 PHP 驱动安装路径，当前命令行环境也没有对应 PHP-FPM 进程，则跳过该步骤，参考下方修改配置文件步骤，修改 `extension_dir` 参数，让 PHP 加载指定路径的 Pdo_kdb 驱动。

2. 创建测试用 PHP 文件，查看当前 PHP 信息

   在当前 Web 开发环境下，浏览器可访问的 PHP 源文件开头，添加 `phpinfo();`，用于查看当前 Web 应用使用的 PHP 的详细信息：

PHP Version 5.6.40

System	Linux localhost.localdomain 3.10.0-862.el7.x86_64 #1 SMP Fri Apr 20 16:44:24 UTC 2018 x86_64
Build Date	Nov 28 2024 13:25:37
Configure Command	'./configure' '--with-apxs2'
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/usr/local/lib
Loaded Configuration File	/usr/local/lib/php.ini
Scan this dir for additional .ini files	(none)
Additional .ini files parsed	(none)
PHP API	20131106
PHP Extension	20131226
Zend Extension	220131226
Zend Extension Build	API220131226,NTS
PHP Extension Build	API20131226,NTS
Debug Build	no
Thread Safety	disabled
Zend Signal Handling	disabled
Zend Memory Manager	enabled
Zend Multibyte Support	disabled
IPv6 Support	enabled
DTrace Support	disabled
Registered PHP Streams	php, file, glob, data, http, ftp, phar
Registered Stream Socket Transports	tcp, udp, unix, udg
Registered Stream Filters	convert.iconv.*, string.rot13, string.toupper, string.tolower, string.strip_tags, convert.*, consumed, dechunk

如上图所示，可以清晰地看到 PHP 版本及对应参数信息，其中需要我们关注的是 `Loaded Configuration File` 栏对应的 php.ini 路径，有的 PHP-FPM 还会加载 `Loaded Configuration File` 下方两栏对应的额外配置文件，对应 conf.d 路径。

3. 修改配置文件

PHP 会优先读取 conf.d 路径下的额外配置文件，再读取 php.ini。

若存在 conf.d 路径的额外配置文件，需要找到该路径下的配置文件有包含 pdo 模块的文件，在 `extension=pdo;` 下方添加一行 `extension=pdo_kdb;`，让 PHP 能识别并加载 Pdo_kdb 驱动。

若不存在 conf.d 路径，则直接修改 php.ini 配置文件，在 `extension=pdo;` 下方添加一行 `extension=pdo_kdb;` 即可。

备注:

若不清楚 PHP 加载模块路径，也可修改 php.ini 配置文件的 extension_dir 参数，让 PHP 读取指定路径下的 Pdo_kdb 驱动。

---

4. 查看 PHP 信息

   修改配置文件后，浏览器刷新页面，查看 `phpinfo();` 对应的网页，`ctrl+f` 搜索是否有 pdo_kdb 加载，若没有，可按以下步骤依次排查：

   – PHP 加载依赖库失败

     可在命令行执行 `ldd pdo_kdb.so` 查看系统默认的加载路径是啥，并将 pdo_kdb 驱动的依赖库放置到系统路径下，以下是一个具体示例：

     ```
     $ ldd pdo_kdb.so
     ...
     libc.so.6 => /lib64/libc.so.6 (0x0000fffe1cde0000)
     ...
     ```

     执行 `ldd` 后，将打印当前环境变量下的默认搜索路径，如示例所示，当前系统默认搜索 `/lib64` 下的动态库。

     将 Pdo_kdb 驱动包内非 pdo_kdb.so 的所有 so 库拷贝到 `/lib64` 路径下即可。

   – PHP 读取到其他的配置文件

     若使用了集成环境，如：宝塔、phpstudy 等，对应 PHP 配置文件需要在集成环境页面 PHP 设置页修改，在 `extension=pdo;` 下方添加一行 `extension=pdo_kdb;` 即可。

   – PHP-FPM 进程需要重启

     若当前环境有常驻的 PHP-FPM 进程，则需要在修改 php.ini 配置文件后，重启 PHP-FPM 进程。

   按上述步骤排查后，刷新页面，可在网页看到 pdo_kdb 加载成功。

2.6.2.2.3　测试示例

确保 PHP 已成功加载 Pdo_kdb 驱动的前提下，此时可编写测试用例连接并操作数据库，以下是一个具体的操作示例：

创建连接：

```
$dbh = new PDO($dsn, $user, $password, array(PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION));
例:
$dbh = new PDO("kdb:host=127.0.0.1;dbname=test;port=54321;", "system", "123456", array(PDO::ATTR_ERRMODE =>
PDO::ERRMODE_EXCEPTION));
```

使用刚刚创建的 $dbh 对象执行建表语句：

```
$stmt = $dbh->prepare('drop table if exists test_kdb');
$stmt->execute();
$stmt = $dbh->prepare('create table test_kdb(id int, name varchar(20))');
$stmt->execute();
```

绑定参数，并执行插入：

```
# 插入
$name = 'kingbase';
$stmt = $dbh->prepare("insert into test_kdb values(1, :name)");
$stmt->execute(array($name));
```

插入成功后，执行查询，并打印结果集：

```
$id = 1;
$stmt = $dbh->prepare('select * from test_kdb where id=?');
$stmt->execute(array($id));
$res = $stmt->fetchAll();
var_dump($res);
```

完整示例如下：

```
<?php

$dsn = "kdb:host=127.0.0.1;dbname=test;port=54321;";
$user = 'system';
$password = '123456';
$dbh = null;

$dbh = new PDO($dsn, $user, $password, array(PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION));

$stmt = $dbh->prepare('drop table if exists test_kdb');
$stmt->execute();
$stmt = $dbh->prepare('create table test_kdb(id int, name varchar(20))');
$stmt->execute();

# 插入
$name = 'kingbase';
$stmt = $dbh->prepare("insert into test_kdb values(1, :name)");
```

```
$stmt->execute(array($name));

# 查询
$id = 1;
$stmt = $dbh->prepare('select * from test_kdb where id=?');
$stmt->execute(array($id));
$res = $stmt->fetchAll();
var_dump($res);
?>
```

示例期望输出如下：

```
array(1) {
  [0]=>
  array(4) {
    ["id"]=>
    int(1)
    [0]=>
    int(1)
    ["name"]=>
    string(8) "kingbase"
    [1]=>
    string(8) "kingbase"
  }
}
```

2.6.2.3　常见问题

1. 执行 `php -m`，报错：libkci.so.5: cannot open shared object file: No such file or directory

   原因：Pdo_kdb 驱动依赖 Libkci 库 (老版本会依赖 Libpq 库)，系统加载 Pdo_kdb 时，无法加载 Pdo_kdb 驱动的依赖库及其依赖导致。

   解决：将 Libkci 及其驱动包 (老版本为 Libpq) 放置在 `ldd pdo_kdb.so` 的系统默认搜索路径下即可。

2. 执行 `php -m`，报错：undefined symbol: pdo_parse_params

   原因：PHP 未成功加载 PDO 模块的前提下加载 Pdo_kdb 驱动导致的问题。

   解决：将 `extension=pdo_kdb` 的配置放置在 `extension=pdo` 下方即可。

3. 执行 `php -m`，报错：undefined symbol: file_globals_id

   原因：PHP 是否线程安全与 Pdo_kdb 驱动包的是否线程安全版本不一致，如：PHP 非线程安全版本使用的线程安全版本的 Pdo_kdb 驱动。

   解决：使用线程安全版本一致的 Pdo_kdb 驱动包。

4. PHP 操作数据库，表存在但无法查找对应表

   原因：可能是由于当前表及库存在在非 public 的 schema 下。

   解决：可在连接串指定 search_path 为期望搜索的 schema 解决：

```
# 如期望搜索 newschema 这个 schema 下的表
$dsn = "kdb:host=127.0.0.1;dbname=test;port=54321;options='-csearch_path=newschema'";
$dbh = new PDO($dsn, $user, $password, array(PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION));
```

5. PHP 操作数据库，出现编码问题报错

   原因：可能是由于客户端编码与数据库服务端编码不一致。

   解决：可在连接串指定客户端编码：

```
# 如期望指定客户端编码为 utf8
$dsn = "kdb:host=127.0.0.1;dbname=test;port=54321;client_encoding='utf8'";
$dbh = new PDO($dsn, $user, $password, array(PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION));
```

第3章　使用 KingbaseES

3.1　SQL 基础

本章介绍如何快速体验 KingbaseES 数据库，若需要进一步了解相关功能，请参考《数据库管理》手册。

本章包含以下内容：

- 连接和断开数据库
- 创建和管理数据库
- 创建和管理表空间
- 创建和管理模式
- 创建和管理表
- 创建和管理索引
- 创建和管理视图
- 创建和管理用户
- 启动和管理事务

3.1.1　连接和断开数据库

3.1.1.1　关于 ksql

ksql 是 KingbaseES 数据库的命令行连接工具，支持本地和远程两种连接方式。

- 本地连接: ksql 连接到同一台计算机运行的 KingbaseES 数据库。
- 远程连接: ksql 通过网络访问另一台计算机运行的 KingbaseES 数据库。

3.1.1.2　使用 ksql 连接到数据库

ksql 连接到数据库需要两步：获取服务端口、ksql 连接数据库。下述为每一步操作细节：

Step1：获取 KingbaseES 数据库服务端口

若使用默认端口号（注：KingbaseES 数据库的默认端口号为 54321），请直接执行Step2 操作。

- Windows 平台下，执行以下命令获取端口号。

```
findstr /R "^\s*port" C:\kb_install\data\kingbase.conf
```

其中 `C:\kb_install\data\kingbase.conf` 为数据库的数据文件路径，请根据实际情况进行替换。

- Linux 平台下，执行以下命令获取端口号。

```
cat /test/kb_install/data/kingbase.conf | grep ^\s*port
```

其中 `/test/kb_install/data` 为数据库的数据文件路径，请根据实际情况进行替换。

Step2：使用 ksql 连接 KingbaseES 数据库

- 指定端口号的方式连接数据库。

```
ksql -h 10.11.0.11 -p 54321 -d kingbase -U Alice
```

其中 10.11.0.11 为数据库服务器的 IP 地址，kingbase 为要连接的数据库名称，54321 是数据库服务端口，Alice 为数据库用户。若使用本地连接方式，可以省略-h 选项。

- 若 KingbaseES 数据库使用默认端口号，可以省略端口号进行连接。

```
ksql -h localhost -d kingbase -U Alice
```

其中 localhost 代表 ksql 将连接到当前服务器的 KingbaseES 数据库，kingbase 代表要连接的数据库名称，Alice 为数据库用户。

3.1.1.3　在 ksql 中断开数据库连接

可直接在 ksql 中输入 `\q` 或 QUIT 命令断开数据库连接。

提示:

关于 ksql 的更多使用方式，请参考 ksql 命令参考中的详细说明。

3.1.2　创建和管理数据库

KingbaseES 中的数据库是多个模式的集合，每个模式包含表、索引、函数等对象。

3.1.2.1　创建数据库

KingbaseES 中支持在 ksql 中输入 SQL 命令或使用 createdb 工具来创建数据库。

1、使用 CREATE DATABASE 语句创建数据库

- 语法格式

```
CREATE DATABASE db_name [options];
```

- 示例

```
--创建一个 temp_db 的数据库
CREATE DATABASE temp_db;
```

2、使用 createdb 命令行工具创建数据库

```
createdb temp_db
```

更多 createdb 的使用方法，请参考工具 createdb 章节部分。

信息:

在 KingbaseES 的 SQLServer 模式中，只支持在 master 库中创建数据库，例如，在 SQLServer 模式中使用 CREATE DATABASE 语句创建数据库：

```
--切换到 master 库下
\c master
--创建数据库
CREATE DATABASE temp_db;
```

3.1.2.2　查看数据库列表

进入数据库后可以通过 SQL 语句或 ksql 工具命令查看当前实例的数据库列表。

- 使用 SQL 语句查询数据库列表。

```
SELECT * FROM sys_database;
```

- 使用 ksql 工具的 `\l` 命令查看数据库列表。

```
\l
```

3.1.2.3　使用数据库

连接数据库后，使用 ksql 工具的 \c 命令切换数据库。

- 示例

```
--切换到 test 数据库下
\c test
```

3.1.2.4　删除数据库

KingbaseES 中支持在 ksql 中输入 SQL 命令或使用 dropdb 工具来删除数据库。

1、使用 DROP DATABASE 语句删除数据库

- 语法格式

```
DROP DATABASE [IF EXISTS] db_name;
```

- 示例

```
--删除 temp_db 数据库
DROP DATABASE temp_db;
```

2、使用 dropdb 命令行工具删除数据库

```
dropdb temp_db
```

更多 dropdb 的使用方法，请参考工具 dropdb 章节部分。

3.1.3　创建和管理表空间

表空间被用来指定数据库对象文件存储的位置，对应文件系统的一个目录。使用表空间的优点如下：

- 灵活存储，不同对象的存储可以放在不同的表空间下。
- 优化性能，将频繁访问的对象数据存放在快速存储介质的表空间上，可显著提高数据库的读写性能。

3.1.3.1　创建表空间

CREATE TABLESPACE 语句用于创建一个表空间，一般来说，仅超级用户有权限创建。

- 语法格式

```
CREATE TABLESPACE tablespace_name [options] LOCATION 'directory' [options];
```

其中，tablespace_name 是要创建的表空间的名称。directory 是要被用于表空间的目录。

- 示例

```
--创建表空间 dbspace
CREATE TABLESPACE dbspace LOCATION '/data/dbs';
```

备注:

表空间创建命令中的目录路径必须为绝对路径，并且该目录属于 KingbaseES 操作系统用户。

3.1.3.2　使用表空间

拥有表空间上的 CREATE 权限后，用户可以在表空间上创建数据库对象——表、索引等。

创建对象共有两种显式指定表空间的方法：SET default_tablespace 参数后创建对象、创建对象时指定 TABLESPACE 子句。若两种显式方式均指定，优先使用 TABLESPACE 子句的表空间值。若未显式指定表空间，数据库对象以所属数据库的表空间为默认表空间。

- 示例

```
--在指定表空间上创建表，当结果显示为 CREATE TABLE，创建成功
CREATE TABLE t1(i int) TABLESPACE dbspace ;

--设置默认表空间 dbspace 后创建表，当结果显示为 SET，设置成功
SET default_tablespace = 'dbspace';
CREATE TABLE t2(i int);
```

3.1.3.3　查询表空间

可通过以下两种方式查看当前实例中的所有表空间：

- 查询 sys_tablespace 系统视图。

```
SELECT spcname FROM sys_tablespace;
```

- 使用 ksql 工具命令查询表空间。

```
\db
```

3.1.3.4　查询表空间使用率

1. 查询表空间的当前使用情况。

```
SELECT SYS_TABLESPACE_SIZE('dbspace');
```

将返回如下信息：

```
sys_tablespace_size
---------------------
               4096
(1 row)
```

其中 4096 表示表空间的大小，单位为字节。

2. 计算表空间使用率。

表空间使用率 = SYS_TABLESPACE_SIZE / 表空间所在目录的磁盘大小。

3.1.3.5　修改表空间

ALTER TABLESPACE 语句可以修改表空间的相关属性。

- 语法格式

```
ALTER TABLESPACE name RENAME TO new_name;
```

- 示例

```
--表空间 dbspace 重命名为 newdbspace
ALTER TABLESPACE dbspace RENAME TO newdbspace;
```

3.1.3.6　删除表空间

使用 DROP TABLESPACE 语句删除表空间。

- 语法格式

```
DROP TABLESPACE [IF EXISTS] name;
```

- 示例

```
--删除 newdbspace 表空间
DROP TABLESPACE newdbspace;
```

备注:

只有表空间属主或超级用户具备执行该操作的权限。在某个表空间被删除前，要保证其中不包含任何数据库对象。

更多表空间使用，请参考《数据库管理》手册的 表空间管理。

3.1.4　创建和管理模式

模式又被称为 schema，它是用于组织和命名数据库对象（如表、视图、函数等）的逻辑容器。

模式是数据库中组织和管理数据库对象的重要机制，它提供了一种逻辑上的命名空间，具备如下优点：

- 可以避免命名冲突，允许在同一数据库中的不同模式下创建同名对象。
- 便于权限管理和隔离，每个模式分配不同的访问权限，以此控制用户或角色对特定模式中对象的访问权限，提高数据库的安全性。

3.1.4.1　创建模式

`CREATE SCHEMA` 用于在当前数据库中创建模式，创建模式时指定的模式名必须与当前数据库中任何现有模式的名称不同。

- 语法格式

```
CREATE SCHEMA [ IF NOT EXISTS ] [ schema_name ] [AUTHORIZATION owner] [options];
```

- 创建 myschema 模式时不指定 owner。

```
CREATE SCHEMA myschema;
```

提示:

不能以 `SYS_` 为前缀的 schema 名，该类 schema 为数据库系统预留的。

- 创建 myschema 模式时指定 owner 为 test 用户。

```
CREATE SCHEMA myschema AUTHORIZATION test;
```

提示:

创建模式时不指定 owner 则 owner 默认为当前用户。

3.1.4.2　查看模式列表

可通过两种方式查看当前的模式列表：

- 查询 sys_namespace 系统表。

```
SELECT * FROM sys_namespace;
```

- 使用 ksql 工具的 \dn 命令。

```
\dn
```

3.1.4.3　模式的搜索路径

搜索路径定义了系统搜索 schema 的顺序。

用户可以设置 search_path 配置参数指定寻找对象可用模式的顺序。如果在创建对象时不指定模式，则会创建在默认的模式中（根据 current_schema 函数决定）。

- 查看搜索路径。

```
SHOW SEARCH_PATH;
-----------------
 "$user", public
(1 行记录)
```

”$user” 为当前用户模式，但当前用户可能未创建模式，public 为系统默认创建的公共模式。

- 设置搜索路径。

```
--将搜索路径设置为 myschema、public，首先搜索 myschema
SET SEARCH_PATH TO myschema, public;
```

备注:

若用户函数与系统函数同名同参，则优先访问系统函数。

3.1.4.4　模式的权限控制

通过授予用户对不同模式对象的访问权限，可以实现更精细的数据库安全控制。

默认情况下，用户只能访问属于各自模式中的数据库对象。如果需要访问其他模式的对象，则该模式的所有者应该赋予它对该模式的 USAGE 权限。默认情况下，所有角色都拥有在 public 模式上的 USAGE 权限和 CREATE 权限，但是普通用户在 public 模式中创建对象是不安全的，建议撤销普通用户在 public 模式上的 CREATE 权限。

- 示例

```
--撤销所有用户在 public 模式下创建对象的权限
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
```

提示:

第一个 public 是指 public 模式，第二个 PUBLIC 是指所有用户。

以下示例将描述当前模式如何进行权限控制：

1. 查看当前模式

```
SELECT current_schema();
 current_schema
----------------
 myschema
(1 row)
```

2. 创建用户 u1，并将 myschema 的 CREATE 权限赋给用户 u1

```
CREATE USER u1;
GRANT CREATE ON SCHEMA myschema TO u1;
```

提示:

通过将模式的 CREATE 权限授予某用户，被授权用户就可以在此模式中创建对象。

---

3. 将用户 u1 对于 myschema 的 CREATE 权限收回

```
REVOKE CREATE ON SCHEMA myschema FROM u1;
```

更多创建和管理用户的方式，请参考《KingbaseES 数据库安全指南》中的 用户管理和授权 > 用户和角色管理章节。

3.1.4.5　使用模式

若要在指定模式下创建对象或者访问指定模式下的对象，需要使用模式名限定的对象名。该名称包含模式名以及对象名，他们之间用 . 号分开。

- 在 myschema 下创建 mytable 表。

```
CREATE TABLE myschema.mytable(id int, name varchar(20));
```

- 查询 myschema 下 mytable 表的数据。

```
SELECT * FROM myschema.mytable;
```

提示:

更多名称解析信息，请参考《数据库管理》手册的 模式对象的管理部分。

3.1.4.6　更改模式

使用 ALTER SCHEMA 语句可以更改模式的相关属性。

```
--将 myschema 模式更名为 myschema_new
ALTER SCHEMA myschema RENAME TO myschema_new;
```

3.1.4.7　删除模式

- 当模式下无数据库对象时，使用 DROP SCHEMA 命令进行删除。

```
--删除名为 nullschema 的空模式
DROP SCHEMA nullschema;
```

- 当模式下存在数据库对象时，还需要使用 CASCADE 关键字命令进行删除。

```
--删除 myschema 及该 schema 下的所有对象
DROP SCHEMA myschema_new CASCADE;
```

备注:

CASCADE 关键字的作用在于删除模式中的所有对象（表、函数、视图等），然后删除所模式本身。

更多模式信息，请参考各模式下 CREATE SCHEMA、ALTER SCHEMA、DROP SCHEMA 语句描述。

3.1.5　创建和管理表

表（table）是数据库中最重要的数据库对象，是数据库中数据存储的基本单位。本小节仅简单介绍表的创建、修改、结构查看、数据操作和删除。

3.1.5.1　创建表

CREATE TABLE 语句用于创建一个表，在 CREATE TABLE 语句中可以指定列、表约束等信息。

- 语法格式

```
CREATE TABLE table_name (column_name data_type,..., table_constraint);
```

- 示例

```
--创建 orders 表，并为 order_id 列定义一个主键约束以及为 customer_name 定义一个 NOT NULL 约束
CREATE TABLE orders(
 order_id INT CONSTRAINT pk_orders PRIMARY KEY,
 customer_name VARCHAR(10) NOT NULL,
 employee_id INT,
 order_date DATE
);
```

3.1.5.2　修改表结构

ALTER TABLE 语句可以修改表结构及表上的属性。

- 示例

```
--重命名 orders 表为 orders_new
ALTER TABLE orders RENAME TO orders_new;
```

具体语法信息可参考：

- ALTER TABLE(Oracle)
- ALTER TABLE(SQLServer)
- ALTER TABLE(MySQL)

3.1.5.3　查看表结构

查看表的结构可以使用 ksql 工具的 \d+ table_name 命令。

- 示例

```
--查看 orders 表的结构
\d+ orders
```

3.1.5.4　插入表数据

INSERT 命令用于向表中插入数据。

- 示例

使用 INSERT 命令增加包含所有字段的数据：

```
INSERT INTO orders VALUES(101,'RDJC',1,'1999-03-20');
```

使用 INSERT 命令增加只有部分字段的数据：

```
INSERT INTO orders(order_id, customer_name) VALUES(104,'ADJC');
```

使用 INSERT 命令增加多条数据：

```
INSERT INTO orders VALUES(103,'GDJC',4,'1989-03-20'),(105,'SDJC',1,'1999-03-20');
```

提示:

需要批量导入数据，请参考:

- COPY(Oracle)
- COPY(SQLServer)

更多 INSERT 命令的使用，请参考:

- INSERT(Oracle)
- INSERT(SQLServer)
- INSERT(MySQL)

3.1.5.5　更新表数据

UPDATE 命令用于更新表中的数据。

- 示例

使用 UPDATE 命令修改数据：

```
UPDATE orders SET employee_id = 3 WHERE order_id = 104;
```

备注:

省略 WHERE 条件将修改表的所有数据。

更多 UPDATE 命令的使用，请参考:

- UPDATE(Oracle)
- UPDATE(SQLServer)
- UPDATE(MySQL)

3.1.5.6　删除表数据

DELETE 命令用于删除表中的数据。

- 示例

使用 DELETE 命令删除数据：

```
DELETE FROM orders WHERE order_id = 103;
```

备注:

省略 WHERE 条件将删除表的所有数据。

更多 DELETE 命令的使用，请参考:

- DELETE(Oracle)
- DELETE(SQLServer)
- DELETE(MySQL)

3.1.5.7　查询表数据

SELECT 语句是用于查询表数据的 SQL 语句，查询表数据是从一个或多个表中检索数据的操作。

- 示例

使用 SELECT 语句检索表中所有数据：

```
SELECT * FROM orders;
```

使用 SELECT 语句检索表中特定列数据：

```
SELECT employee_id FROM orders;
```

使用 DISTINCT 子句消除查询结果中重复的行：

```
SELECT DISTINCT employee_id FROM orders;
```

使用 WHERE 子句筛选表中满足条件的行：

```
SELECT * FROM orders WHERE order_id > 104;
```

使用 ORDER BY 子句对查询结果进行排序：

```
SELECT * FROM orders ORDER BY order_id;
```

使用 GROUP BY 子句对查询结果进行分组：

```
SELECT COUNT(order_id), employee_id FROM orders GROUP BY employee_id;
```

更多 SELECT 语句相关的语法说明，请参见:

- SELECT(Oracle)
- SELECT(SQLServer)
- SELECT(MySQL)

更多查询表数据的方法，请参见 查询和子查询 (Oracle) 章节。

3.1.5.8　删除表

`DROP TABLE` 语句用于删除数据库中的表。

- 示例

```
--通过 DROP TABLE 语句删除 orders_new 表
DROP TABLE orders_new;
```

3.1.6　创建和管理索引

索引是一种与表相关的数据库对象，可以创建在表的一列或多列上。通过使用索引，可以让数据库不必扫描整个表，直接定位到符合条件的数据行，提高数据检索的速度。

但是，索引并不是越多越好，当表中插入、更新或删除数据时，涉及的索引也需要更新，这会造成性能开销。因此，需要对索引进行合理的创建和管理。

3.1.6.1　创建索引

KingbaseES 中，使用 `CREATE INDEX` 语句来创建索引。

- 示例

使用 `CREATE INDEX` 语句创建索引：

```
CREATE INDEX idx_employee ON orders(employee_id);
```

备注:

CREATE INDEX 创建的是非唯一索引。

使用 `CREATE INDEX` 语句创建组合索引：

```
CREATE INDEX idx_orders ON orders(order_id, order_date);
```

提示:

组合索引，也称为连接索引，是在表中的多个列上的索引。组合索引中的列应该以在检索数据的查询中效率最高的顺序出现，但在表定义中的顺序不需要与组合索引列顺序一致。

使用 CREATE UNIQUE INDEX 语句创建唯一索引：

```
CREATE UNIQUE INDEX uid_order ON orders(order_id);
```

使用 CREATE UNIQUE INDEX 语句创建唯一组合索引：

```
CREATE UNIQUE INDEX uid_orders ON orders(order_id, order_date);
```

更多 CREATE INDEX 语句相关的语法说明，请参见:

- CREATE INDEX(Oracle)
- CREATE INDEX(SQLServer)
- CREATE INDEX(MySQL)

3.1.6.2　修改索引

使用 ALTER INDEX 语句可以修改索引的相关属性信息。

- 示例

使用 ALTER INDEX 语句重命名索引名：

```
ALTER INDEX idx_employee RENAME TO idx_employeeid;
```

使用 ALTER INDEX 语句设置索引选项：

```
ALTER INDEX idx_employeeid SET(FILLFACTOR = 60);
```

提示:

FILLFACTOR 为索引的填充因子，用于指定在创建或重建索引时新页的填充程度，其值是介于 1~100 之间的整数。

更多 ALTER INDEX 语句相关的语法说明，请参见:

- ALTER INDEX(Oracle)
- ALTER INDEX(SQLServer)

3.1.6.3　查看索引

可通过下述两种方式查询数据库中的索引信息：

使用 SELECT 语句查询 sys_indexes 视图查看数据库中的所有索引：

```
SELECT * FROM sys_indexes;
```

使用 ksql 工具的 `\di` 命令查看数据库中的所有索引：

```
\di
```

更多 ksql 命令，请参考 ksql 命令参考章节。

3.1.6.4　重建索引

重建索引可以确保索引存储参数的修改完全生效，并且还能用来恢复无用或被损坏的索引。

使用 REINDEX INDEX 语句重建索引：

```
REINDEX INDEX idx_employee;
```

使用 REINDEX TABLE 语句重建表上所有索引：

```
REINDEX TABLE orders;
```

更多 REINDEX 语句相关的语法说明，请参见:

- REINDEX(Oracle)
- REINDEX(SQLServer)

3.1.6.5　删除索引

使用 `DROP INDEX` 语句删除索引：

```
DROP INDEX idx_employeeid;
```

使用 DROP INDEX 语句级联删除索引：

```
DROP INDEX idx_orders CASCADE;
```

提示:

级联删除索引是指删除索引时自动删除依赖于该索引的对象。

更多 DROP INDEX 语句相关的语法说明，请参见:

- DROP INDEX(Oracle)
- DROP INDEX(SQLServer)
- DROP INDEX(MySQL)

更多索引的概念和使用方法，请参考《数据库概念》索引章节。

3.1.7　创建和管理视图

KingbaseES 数据库中视图是一个虚拟表，不记录实际数据，只保存定义中的查询语句，当查询视图时，数据库内部执行该查询语句来获取数据。

3.1.7.1　创建视图

使用 CREATE VIEW 语句创建视图：

```
CREATE [options] VIEW name [options] AS query [options] ;
```

使用 CREATE OR REPLACE VIEW 语句创建视图：

```
CREATE OR REPLACE VIEW orders_info AS SELECT order_id, customer_name FROM orders;
```

提示:

使用 CREATE OR REPLACE VIEW 语句可以更新视图定义。

3.1.7.2　查询视图定义

查询 sys_views 视图获取自定义视图定义：

```
SELECT * FROM sys_views WHERE viewname = 'orders_info';
```

使用 ksql 命令查询视图定义：

```
\sv orders_info
```

提示:

更多 ksql 命令，请参考 ksql 命令参考。

3.1.7.3　查询视图

使用 SELECT 语句查询视图：

```
SELECT * FROM orders_info;
```

3.1.7.4　修改视图

使用 `ALTER VIEW` 语句修改视图，例如视图重命名：

```
ALTER VIEW orders_info RENAME TO orders_new;
```

3.1.7.5　删除视图

使用 `DROP VIEW` 语句删除视图：

```
DROP VIEW orders_info;
```

更多视图功能，请参考:

- CREATE VIEW(Oracle)
- CREATE VIEW(MySQL)
- CRAETE VIEW(SQLServer)
- ALTER VIEW(Oracle)
- ALTER VIEW(MySQL)
- ALTER VIEW(SQLServer)

3.1.8　创建和管理用户

KingbaseES 中的用户是拥有数据库对象和权限的实体。为了确保数据的安全性，KingbaseES 提供了多种方式来创建和管理用户。

3.1.8.1　创建用户

使用 `CREATE USER` 语句创建用户：

```
CREATE USER test WITH PASSWORD '123456';
```

提示:

只有超级用户或具有 CREATE ROLE 系统权限的用户才可以创建用户。

更多 `CREATE USER` 语句相关的语法说明，请参见:

- CREATE USER(Oracle)
- CREATE USER(MySQL)
- CREATE USER(SQLServer)

使用 createuser 命令行工具创建用户：

```
createuser rdjc
```

更多 createuser 命令的使用方法，请参考 createuser 章节。

3.1.8.2　修改用户

使用 `ALTER USER` 语句修改用户密码：

```
ALTER USER test PASSWORD '654321';
```

使用 `ALTER USER` 语句限制用户的连接数：

```
ALTER USER test CONNECTION LIMIT 1000;
```

使用 `ALTER USER` 语句增加 CREATE ROLE 权限：

```
ALTER USER test CREATE ROLE;
```

使用 `ALTER USER` 语句重命名用户名：

```
ALTER USER test RENAME TO Joe;
```

更多 ALTER USER 语句相关的语法说明，请参见:

- ALTER USER(Oracle)
- ALTER USER(MySQL)
- ALTER USER(SQLServer)

3.1.8.3　查看用户

使用 SELECT 语句查询 sys_user 视图查看用户：

```
SELECT * FROM sys_user;
```

使用 ksql 工具的 \du 命令查看用户：

```
\du
```

更多 ksql 命令，请参考 ksql 命令参考部分。

3.1.8.4　授予权限

GRANT 语句用于将对象的权限或角色的权限授予另一个角色或用户。

- 示例

使用 GRANT 语句将表的插入和更新权限授予用户：

```
GRANT INSERT, UPDATE ON orders TO Joe;
```

使用 GRANT 语句将表的查询权限授予多个用户：

```
GRANT SELECT ON orders TO rdjc, Joe;
```

更多 GRANT 语句相关的语法说明，请参见:

- GRANT(Oracle)
- GRANT(MySQL)
- GRANT(SQLServer)

3.1.8.5　撤销权限

REVOKE 语句用于从角色或用户处收回对象的权限或角色的权限。

使用 REVOKE 语句撤销用户在表上的插入和更新权限：

```
REVOKE INSERT, UPDATE ON orders FROM Joe;
```

使用 REVOKE 语句撤销多个用户在表上的查询权限：

```
REVOKE SELECT ON orders FROM rdjc, Joe;
```

更多 REVOKE 语句相关的语法说明，请参见:

- REVOKE(Oracle)
- REVOKE(MySQL)
- REVOKE(SQLServer)

3.1.8.6　删除用户

使用 `DROP USER` 语句删除用户：

```
DROP USER Joe;
```

更多 `DROP USER` 语句相关的语法说明，请参见:

- DROP USER(Oracle)
- DROP USER(MySQL)
- DROP USER(SQLServer)

使用 dropuser 命令行工具删除用户：

```
dropuser rdjc
```

更多 dropuser 的使用方法，请参考 dropuser 章节。

更多创建和管理用户的方式，请参考《KingbaseES 数据库安全指南》的用户和角色管理章节。

3.1.9　启动和管理事务

数据库中的事务是一个独立的逻辑单元，它包括了一个或多个 SQL 语句。一个事务中的语句要么全部执行，要么全不执行。

3.1.9.1　事务控制命令

- 启动事务

  用户可以使用 `START TRANSACTION` 或者 `BEGIN` 启动事务。更多启动事务的信息，请参考:

  - START TRANSACTION(Oracle)
  - START TRANSACTION(SQLServer)
  - BEGIN(Oracle)
  - BEGIN(SQLServer)

- 设置事务

  在多个事务并发执行时，系统应保证并发执行后与这些事务先后单独执行时的结果一样，因此对事务隔离级别有一定的要求。

  用户可以使用 `SET TRANSACTION` 设置事务隔离级别、事务访问模式（读/写或只读）以及可延迟模式。更多设置事务的信息，请参考:

  - SET TRANSACTION(Oracle)
  - SET TRANSACTION(SQLServer)

- 提交事务

  用户可以使用 COMMIT 或者 END 完成提交事务的功能。更多提交事务的信息，请参考：

  - END(Oracle)
  - END(SQLServer)
  - COMMIT(Oracle)
  - COMMIT TRANSACTION(SQLServer)

- 回滚事务

  回滚是在事务运行的过程中发生了某种故障，导致事务不能继续执行，最终数据库会撤销该事务内所有更新。更多回滚事务的信息，请参考:

  - ROLLBACK(Oracle)
  - ROLLBACK(SQLServer)

3.1.9.2　事务示例

1. 开启一个事务

```
BEGIN;
```

2. 向 orders 表中插入一条新的记录

```
INSERT INTO orders VALUES(107,'RDJC',5,'1999-03-20');
```

3. 如果操作都成功完成，开始提交事务

```
COMMIT;
```

4. 插入语句执行失败的语句（orders 表存在主键约束）

```
INSERT INTO orders VALUES(107,'RDJC',5,'2000-03-20');
ERROR:  duplicate key value violates unique constraint "pk_orders"
DETAIL:  Key (order_id)=(107) already exists.
```

5. 当语句执行失败时，COMMIT 与 ROLLBACK 等效，均为回滚事务

```
ROLLBACK;
```

更多事务概念，请参考《系统原理手册》中 事务 > 事务的概念章节。

版权声明

中电科金仓（北京）科技股份有限公司（简称：电科金仓）版权所有，并保留对本手册及本声明的一切权利。

未得到电科金仓的书面许可，任何人不得以任何方式或形式对本手册内的任何部分进行复制、摘录、备份、修改、传播、翻译成其他语言、将其全部或部分用于商业用途。

免责声明

本手册内容依据现有信息制作，由于产品版本升级或其他原因，其内容有可能变更。电科金仓保留在没有任何通知或者提示的情况下对手册内容进行修改的权利。

本手册仅作为使用指导，电科金仓在编写本手册时已尽力保证其内容准确可靠，但并不确保手册内容完全没有错误或遗漏，本手册中的所有信息也不构成任何明示或暗示的担保。

技术支持

- 电科金仓官方网站：http://www.kingbase.com.cn/
- 电科金仓文档中心：http://help.kingbase.com.cn/
- 全国服务热线：400-601-1188
- 电科金仓技术支持与反馈信箱：support@kingbase.com.cn

服务周期承诺

由于市场需求在不断变化，技术创新和发展的进程不断加剧，产品的版本更迭不可避免。电科金仓对于产品版本生命周期的有效管理，有助于您提前规划项目，更好地从产品服务终止上过渡。

表 1: KingbaseES 产品生命周期里程碑

关键里程碑点	定义
产品发布日期	产品正式发布版本，即 GA（general availability）版本的发布日期。
停止销售日期	正式停止销售的日期，版本停止接受订单日。该日之后，产品将不再销售。
停止功能升级日期	在该日期之后，不再提供新特性和新硬件支持。但依旧提供错误修复、安全修复、功能维护等服务。
停止功能维护日期	在该日期之后，不再维护功能，修复问题。但依旧提供安全修复等服务
停止安全维护日期	在该日期之后，不再发布补丁版本修复中高风险漏洞，仅提供有限的支持。
产品服务终止日期	停止提供产品服务和支持的日期。包括软件维护版本，缺陷修复，以及针对该产品的所有服务支持（包括服务热线和远程/现场支持）。

服务周期策略

金仓数据库管理系统 KingbaseES 产品确保以下的服务周期：

1）产品自发布之日起至产品停止功能升级（包含新特性、新硬件支持）之日止不少于 5 年。

2）产品停止功能升级之日起至产品停止功能维护（主要包括问题修复）之日止不少于 4 年。

3）产品功能维护停止之日起至产品停止安全维护（包括中高风险漏洞修复）之日止不少于 2 年。

针对金仓数据库管理系统 KingbaseES V9 产品（发布日期：2022 年 7 月 15 日），其具体服务周期安排如下：

- 产品销售日期：以产品销售合同所约定日期为准。
- 停止升级日期：自产品销售日期起计算，满 5 年之日止。
- 服务终止日期：自产品销售日期起计算，满 11 年之日止。

服务终止策略

金仓数据库管理系统 KingbaseES 产品确保在销售后，至少提供 6 年的服务支持。

备注:

电科金仓将会综合各方因素来确定产品服务终止日期。并将在实际产品服务终止日期之前至少 90 天，通过公开方式宣布产品服务终止日期。